

Proglng - Programación en Ingeniería

Generated by Doxygen 1.16.1

1 Proglng - Programación en Ingeniería	1
1.0.1 Introducción	2
1.0.2 Datos generales	2
1.0.3 Competencia de la UDA	2
1.0.4 Contenido	3
1.0.5 Entregas y evaluación	3
1.0.5.1 Ponderaciones	3
1.0.5.2 Tareas	4
1.0.5.3 Prácticas	4
1.0.5.4 Proyecto final	5
1.0.5.5 Elementos mínimos de los reportes	5
1.0.6 Repositorio y plataformas	5
1.0.7 Instalación del compilador (Windows, macOS y Linux)	6
1.0.7.1 Verificación rápida (en cualquier sistema)	6
1.0.7.2 Windows (recomendado)	6
1.0.7.3 macOS	7
1.0.7.4 Linux	7
1.0.7.5 Recomendación de editor/IDE (opcional)	7
1.0.8 Documentación del código (Doxygen)	7
1.0.8.1 Requisitos	8
1.0.8.2 Generar documentación	8
1.0.9 Estructura sugerida del repositorio	8
1.0.10 Código de ética y conducta profesional	8
1.0.11 Bibliografía	8
1.0.12 Contacto	9
1.0.13 Licencia	9
2 Directory Hierarchy	11
2.1 Directories	11
3 Data Structure Index	17
3.1 Data Structures	17
4 File Index	19
4.1 File List	19
5 Directory Documentation	23
5.1 src/20251 Directory Reference	23
5.2 src/20252 Directory Reference	24
5.3 src/20261 Directory Reference	25
5.4 src Directory Reference	25
5.5 src/20261/src Directory Reference	26
5.6 src/template Directory Reference	28

6 Data Structure Documentation	29
6.1 contacto Struct Reference	29
6.1.1 Detailed Description	29
6.1.2 Field Documentation	29
6.1.2.1 email	29
6.1.2.2 nombre	29
6.1.2.3 telefono	29
6.2 dato_s Struct Reference	29
6.2.1 Detailed Description	29
6.2.2 Field Documentation	30
6.2.2.1 a	30
6.2.2.2 b	30
6.2.2.3 c	30
6.3 dato_u Union Reference	30
6.3.1 Detailed Description	30
6.3.2 Field Documentation	30
6.3.2.1 a	30
6.3.2.2 b	30
6.3.2.3 c	30
7 File Documentation	31
7.1 README.md File Reference	31
7.2 src/20251/001_cod_prioridad.c File Reference	31
7.2.1 Function Documentation	31
7.2.1.1 main()	31
7.3 001_cod_prioridad.c	32
7.4 src/20251/002_Pokemon.c File Reference	32
7.4.1 Function Documentation	32
7.4.1.1 main()	32
7.5 002_Pokemon.c	32
7.6 src/20251/003_cuadratica.c File Reference	33
7.6.1 Function Documentation	33
7.6.1.1 main()	33
7.7 003_cuadratica.c	33
7.8 src/20251/004_rangTemp.c File Reference	34
7.8.1 Function Documentation	34
7.8.1.1 main()	34
7.9 004_rangTemp.c	35
7.10 src/20251/005_rangTemp2.c File Reference	35
7.10.1 Function Documentation	36
7.10.1.1 main()	36
7.11 005_rangTemp2.c	36

7.12 src/20251/006_valorPI.c File Reference	37
7.12.1 Function Documentation	37
7.12.1.1 main()	37
7.13 006_valorPI.c	37
7.14 src/20251/007_exp_1.c File Reference	38
7.14.1 Function Documentation	38
7.14.1.1 main()	38
7.15 007_exp_1.c	38
7.16 src/20251/008_exp_2.c File Reference	39
7.16.1 Function Documentation	39
7.16.1.1 main()	39
7.17 008_exp_2.c	40
7.18 src/20251/009_exp_3.c File Reference	40
7.18.1 Function Documentation	41
7.18.1.1 main()	41
7.19 009_exp_3.c	41
7.20 src/20251/010_Inx.c File Reference	42
7.20.1 Function Documentation	42
7.20.1.1 main()	42
7.21 010_Inx.c	43
7.22 src/20251/011_seno.c File Reference	43
7.22.1 Macro Definition Documentation	43
7.22.1.1 PI2	43
7.22.2 Function Documentation	44
7.22.2.1 main()	44
7.23 011_seno.c	44
7.24 src/20251/012_aleatorio.c File Reference	44
7.24.1 Macro Definition Documentation	45
7.24.1.1 max	45
7.24.1.2 min	45
7.24.1.3 N	45
7.24.2 Function Documentation	45
7.24.2.1 main()	45
7.25 012_aleatorio.c	46
7.26 src/20251/014_OrdBurbuja.c File Reference	46
7.26.1 Macro Definition Documentation	47
7.26.1.1 N	47
7.26.2 Function Documentation	47
7.26.2.1 main()	47
7.27 014_OrdBurbuja.c	47
7.28 src/20251/015_ParesNones.c File Reference	48
7.28.1 Macro Definition Documentation	48

7.28.1.1 N	48
7.28.2 Function Documentation	49
7.28.2.1 main()	49
7.29 015_ParesNones.c	49
7.30 src/20251/016_Seleccion.c File Reference	50
7.30.1 Macro Definition Documentation	51
7.30.1.1 N	51
7.30.2 Function Documentation	51
7.30.2.1 main()	51
7.31 016_Seleccion.c	51
7.32 src/20251/017_Insercion.c File Reference	52
7.32.1 Macro Definition Documentation	53
7.32.1.1 N	53
7.32.2 Function Documentation	53
7.32.2.1 main()	53
7.33 017_Insercion.c	53
7.34 src/20251/018_Casilleros.c File Reference	54
7.34.1 Macro Definition Documentation	55
7.34.1.1 M	55
7.34.1.2 N	55
7.34.2 Function Documentation	55
7.34.2.1 main()	55
7.35 018_Casilleros.c	56
7.36 src/20251/019_BurbujaBD.c File Reference	57
7.36.1 Macro Definition Documentation	57
7.36.1.1 N	57
7.36.2 Function Documentation	57
7.36.2.1 main()	57
7.37 019_BurbujaBD.c	58
7.38 src/20251/020_BurbujaBD.c File Reference	59
7.38.1 Macro Definition Documentation	59
7.38.1.1 N	59
7.38.2 Function Documentation	60
7.38.2.1 main()	60
7.39 020_BurbujaBD.c	60
7.40 src/20251/021_Conteno.c File Reference	61
7.40.1 Macro Definition Documentation	61
7.40.1.1 N	61
7.40.2 Function Documentation	61
7.40.2.1 main()	61
7.41 021_Conteno.c	62
7.42 src/20251/022_multiplicacion.c File Reference	63

7.42.1 Macro Definition Documentation	63
7.42.1.1 Co	63
7.42.1.2 Re	63
7.42.2 Function Documentation	64
7.42.2.1 main()	64
7.43 022_multiplicacion.c	64
7.44 src/20251/023_EliGauss.c File Reference	65
7.44.1 Macro Definition Documentation	65
7.44.1.1 NV	65
7.44.2 Function Documentation	66
7.44.2.1 main()	66
7.45 023_EliGauss.c	67
7.46 src/20251/024_EliGauss.c File Reference	68
7.46.1 Macro Definition Documentation	68
7.46.1.1 NV	68
7.46.2 Function Documentation	68
7.46.2.1 main()	68
7.47 024_EliGauss.c	69
7.48 src/20251/025_CyN.c File Reference	70
7.48.1 Macro Definition Documentation	70
7.48.1.1 N	70
7.48.2 Function Documentation	71
7.48.2.1 main()	71
7.49 025_CyN.c	71
7.50 src/20251/026_Binarios.c File Reference	72
7.50.1 Macro Definition Documentation	73
7.50.1.1 BIT	73
7.50.1.2 BIT_CLEAR	73
7.50.1.3 BIT_GET	73
7.50.1.4 BIT_SET	73
7.50.1.5 BIT_TOGGLE	73
7.50.1.6 BIT_WRITE	74
7.50.2 Function Documentation	74
7.50.2.1 main()	74
7.51 026_Binarios.c	74
7.52 src/20251/027_Paridad.c File Reference	75
7.52.1 Macro Definition Documentation	75
7.52.1.1 BIT	75
7.52.1.2 BIT_TOGGLE	76
7.52.1.3 NC	76
7.52.2 Function Documentation	76
7.52.2.1 main()	76

7.53 027_Paridad.c	76
7.54 src/20251/028_Hamming.c File Reference	77
7.54.1 Macro Definition Documentation	77
7.54.1.1 l2	77
7.54.1.2 MC	77
7.54.2 Function Documentation	77
7.54.2.1 main()	77
7.55 028_Hamming.c	78
7.56 src/20251/029_Enc.c File Reference	79
7.56.1 Macro Definition Documentation	79
7.56.1.1 MC	79
7.56.2 Function Documentation	79
7.56.2.1 main()	79
7.57 029_Enc.c	80
7.58 src/20251/030_ParEntrada.c File Reference	81
7.58.1 Function Documentation	81
7.58.1.1 main()	81
7.59 030_ParEntrada.c	82
7.60 src/20251/031_ParEntrada.c File Reference	82
7.60.1 Function Documentation	82
7.60.1.1 main()	82
7.61 031_ParEntrada.c	82
7.62 src/20251/032_ParEntrada.c File Reference	83
7.62.1 Function Documentation	83
7.62.1.1 main()	83
7.62.1.2 str2float()	84
7.62.1.3 str2int()	84
7.63 032_ParEntrada.c	85
7.64 src/20251/033_Arreglo1D.c File Reference	85
7.64.1 Macro Definition Documentation	86
7.64.1.1 max	86
7.64.1.2 min	86
7.64.1.3 N	86
7.64.2 Function Documentation	86
7.64.2.1 main()	86
7.65 033_Arreglo1D.c	87
7.66 src/20251/Ejemplo_035.c File Reference	87
7.66.1 Function Documentation	88
7.66.1.1 main()	88
7.66.1.2 media()	88
7.66.1.3 sumar()	89
7.67 Ejemplo_035.c	90

7.68 src/20251/Ejemplo_036.c File Reference	90
7.68.1 Function Documentation	91
7.68.1.1 main()	91
7.69 Ejemplo_036.c	92
7.70 src/20251/Ejemplo_037.c File Reference	92
7.70.1 Function Documentation	93
7.70.1.1 main()	93
7.71 Ejemplo_037.c	94
7.72 src/20252/Ejemplo041.c File Reference	96
7.72.1 Function Documentation	96
7.72.1.1 main()	96
7.73 Ejemplo041.c	97
7.74 src/20252/Ejemplo042.c File Reference	97
7.74.1 Macro Definition Documentation	97
7.74.1.1 N	97
7.74.2 Function Documentation	98
7.74.2.1 main()	98
7.75 Ejemplo042.c	98
7.76 src/20252/Ejemplo043.c File Reference	98
7.76.1 Macro Definition Documentation	99
7.76.1.1 N	99
7.76.2 Function Documentation	99
7.76.2.1 main()	99
7.77 Ejemplo043.c	100
7.78 src/20252/Ejemplo044.c File Reference	100
7.78.1 Macro Definition Documentation	101
7.78.1.1 N	101
7.78.2 Function Documentation	101
7.78.2.1 main()	101
7.79 Ejemplo044.c	101
7.80 src/20252/Ejemplo045.c File Reference	102
7.80.1 Macro Definition Documentation	102
7.80.1.1 N	102
7.80.2 Function Documentation	102
7.80.2.1 main()	102
7.81 Ejemplo045.c	103
7.82 src/20252/Ejemplo001.c File Reference	105
7.82.1 Function Documentation	105
7.82.1.1 main()	105
7.83 Ejemplo001.c	105
7.84 src/20261/src/Ejemplo001.c File Reference	106
7.84.1 Detailed Description	106

7.84.2 Function Documentation	106
7.84.2.1 main()	106
7.85 Ejemplo001.c	107
7.86 src/20252/Ejemplo002.c File Reference	107
7.86.1 Function Documentation	107
7.86.1.1 main()	107
7.87 Ejemplo002.c	108
7.88 src/20261/src/Ejemplo002.c File Reference	108
7.88.1 Detailed Description	108
7.88.2 Function Documentation	109
7.88.2.1 main()	109
7.89 Ejemplo002.c	110
7.90 src/20252/Ejemplo003.c File Reference	111
7.90.1 Function Documentation	111
7.90.1.1 main()	111
7.91 Ejemplo003.c	111
7.92 src/20261/src/Ejemplo003.c File Reference	112
7.92.1 Detailed Description	112
7.92.2 Function Documentation	113
7.92.2.1 main()	113
7.93 Ejemplo003.c	114
7.94 src/20252/Ejemplo004.c File Reference	115
7.94.1 Function Documentation	115
7.94.1.1 main()	115
7.95 Ejemplo004.c	115
7.96 src/20261/src/Ejemplo004.c File Reference	116
7.96.1 Detailed Description	116
7.96.2 Function Documentation	117
7.96.2.1 main()	117
7.97 Ejemplo004.c	117
7.98 src/20252/Ejemplo005.c File Reference	118
7.98.1 Function Documentation	118
7.98.1.1 main()	118
7.99 Ejemplo005.c	119
7.100 src/20261/src/Ejemplo005.c File Reference	119
7.100.1 Detailed Description	120
7.100.2 Function Documentation	120
7.100.2.1 main()	120
7.101 Ejemplo005.c	121
7.102 src/20252/Ejemplo006.c File Reference	122
7.102.1 Function Documentation	122
7.102.1.1 main()	122

7.103 Ejemplo006.c	122
7.104 src/20261/src/Ejemplo006.c File Reference	123
7.104.1 Detailed Description	123
7.104.2 Function Documentation	124
7.104.2.1 main()	124
7.105 Ejemplo006.c	125
7.106 src/20252/Ejemplo007.c File Reference	127
7.106.1 Function Documentation	127
7.106.1.1 main()	127
7.107 Ejemplo007.c	128
7.108 src/20261/src/Ejemplo007.c File Reference	128
7.108.1 Detailed Description	129
7.108.2 Function Documentation	129
7.108.2.1 main()	129
7.109 Ejemplo007.c	130
7.110 src/20252/Ejemplo008.c File Reference	131
7.110.1 Function Documentation	131
7.110.1.1 main()	131
7.111 Ejemplo008.c	132
7.112 src/20261/src/Ejemplo008.c File Reference	132
7.112.1 Detailed Description	132
7.112.2 Function Documentation	133
7.112.2.1 main()	133
7.113 Ejemplo008.c	133
7.114 src/20252/Ejemplo009.c File Reference	134
7.114.1 Function Documentation	134
7.114.1.1 main()	134
7.115 Ejemplo009.c	135
7.116 src/20261/src/Ejemplo009.c File Reference	135
7.116.1 Detailed Description	136
7.116.2 Function Documentation	136
7.116.2.1 main()	136
7.117 Ejemplo009.c	137
7.118 src/20252/Ejemplo010.c File Reference	138
7.118.1 Function Documentation	138
7.118.1.1 main()	138
7.119 Ejemplo010.c	139
7.120 src/20261/src/Ejemplo010.c File Reference	139
7.120.1 Detailed Description	140
7.120.2 Function Documentation	140
7.120.2.1 main()	140
7.121 Ejemplo010.c	141

7.122 src/20252/Ejemplo011.c File Reference	141
7.122.1 Function Documentation	142
7.122.1.1 main()	142
7.123 Ejemplo011.c	142
7.124 src/20261/src/Ejemplo011.c File Reference	143
7.124.1 Detailed Description	143
7.124.2 Function Documentation	144
7.124.2.1 main()	144
7.125 Ejemplo011.c	145
7.126 src/20252/Ejemplo012.c File Reference	145
7.126.1 Function Documentation	146
7.126.1.1 main()	146
7.127 Ejemplo012.c	147
7.128 src/20261/src/Ejemplo012.c File Reference	147
7.128.1 Detailed Description	148
7.128.2 Function Documentation	148
7.128.2.1 main()	148
7.129 Ejemplo012.c	149
7.130 src/20252/Ejemplo013.c File Reference	150
7.130.1 Macro Definition Documentation	150
7.130.1.1 N	150
7.130.2 Function Documentation	151
7.130.2.1 main()	151
7.131 Ejemplo013.c	151
7.132 src/20261/src/Ejemplo013.c File Reference	152
7.132.1 Detailed Description	153
7.132.2 Function Documentation	153
7.132.2.1 main()	153
7.133 Ejemplo013.c	154
7.134 src/20252/Ejemplo014.c File Reference	155
7.134.1 Macro Definition Documentation	155
7.134.1.1 N	155
7.134.2 Function Documentation	155
7.134.2.1 main()	155
7.135 Ejemplo014.c	156
7.136 src/20261/src/Ejemplo014.c File Reference	157
7.136.1 Detailed Description	158
7.136.2 Function Documentation	158
7.136.2.1 main()	158
7.137 Ejemplo014.c	159
7.138 src/20252/Ejemplo015.c File Reference	160
7.138.1 Macro Definition Documentation	160

7.138.1.1 N	160
7.138.2 Function Documentation	160
7.138.2.1 main()	160
7.139 Ejemplo015.c	161
7.140 src/20261/src/Ejemplo015.c File Reference	162
7.140.1 Detailed Description	162
7.140.2 Function Documentation	163
7.140.2.1 main()	163
7.141 Ejemplo015.c	164
7.142 src/20252/Ejemplo016.c File Reference	164
7.142.1 Macro Definition Documentation	165
7.142.1.1 N	165
7.142.2 Function Documentation	165
7.142.2.1 main()	165
7.143 Ejemplo016.c	166
7.144 src/20261/src/Ejemplo016.c File Reference	167
7.144.1 Detailed Description	167
7.144.2 Function Documentation	168
7.144.2.1 main()	168
7.145 Ejemplo016.c	168
7.146 src/20252/Ejemplo017.c File Reference	169
7.146.1 Macro Definition Documentation	170
7.146.1.1 N	170
7.146.2 Function Documentation	170
7.146.2.1 main()	170
7.147 Ejemplo017.c	171
7.148 src/20261/src/Ejemplo017.c File Reference	172
7.148.1 Detailed Description	173
7.148.2 Function Documentation	173
7.148.2.1 main()	173
7.149 Ejemplo017.c	175
7.150 src/20252/Ejemplo018.c File Reference	176
7.150.1 Macro Definition Documentation	176
7.150.1.1 N	176
7.150.2 Function Documentation	176
7.150.2.1 main()	176
7.151 Ejemplo018.c	177
7.152 src/20261/src/Ejemplo018.c File Reference	178
7.152.1 Detailed Description	179
7.152.2 Macro Definition Documentation	179
7.152.2.1 N	179
7.152.3 Function Documentation	179

7.152.3.1 main()	179
7.153 Ejemplo018.c	180
7.154 src/20252/Ejemplo019.c File Reference	181
7.154.1 Macro Definition Documentation	181
7.154.1.1 N	181
7.154.2 Function Documentation	182
7.154.2.1 main()	182
7.155 Ejemplo019.c	182
7.156 src/20261/src/Ejemplo019.c File Reference	183
7.156.1 Detailed Description	184
7.156.2 Macro Definition Documentation	185
7.156.2.1 N	185
7.156.3 Function Documentation	185
7.156.3.1 main()	185
7.157 Ejemplo019.c	186
7.158 src/20252/Ejemplo020.c File Reference	187
7.158.1 Macro Definition Documentation	187
7.158.1.1 M	187
7.158.1.2 N	187
7.158.2 Function Documentation	187
7.158.2.1 main()	187
7.159 Ejemplo020.c	188
7.160 src/20261/src/Ejemplo020.c File Reference	188
7.160.1 Detailed Description	189
7.160.2 Macro Definition Documentation	190
7.160.2.1 N	190
7.160.3 Function Documentation	190
7.160.3.1 main()	190
7.161 Ejemplo020.c	191
7.162 src/20252/Ejemplo021.c File Reference	192
7.162.1 Macro Definition Documentation	192
7.162.1.1 M	192
7.162.1.2 N	193
7.162.2 Function Documentation	193
7.162.2.1 main()	193
7.163 Ejemplo021.c	193
7.164 src/20261/src/Ejemplo021.c File Reference	194
7.164.1 Detailed Description	194
7.164.2 Macro Definition Documentation	195
7.164.2.1 N	195
7.164.3 Function Documentation	195
7.164.3.1 main()	195

7.165 Ejemplo021.c	196
7.166 src/20252/Ejemplo022.c File Reference	198
7.166.1 Macro Definition Documentation	198
7.166.1.1 N	198
7.166.2 Function Documentation	198
7.166.2.1 main()	198
7.167 Ejemplo022.c	199
7.168 src/20261/src/Ejemplo022.c File Reference	200
7.168.1 Detailed Description	200
7.168.2 Macro Definition Documentation	201
7.168.2.1 N	201
7.168.3 Function Documentation	201
7.168.3.1 main()	201
7.169 Ejemplo022.c	202
7.170 src/20252/Ejemplo023.c File Reference	203
7.170.1 Macro Definition Documentation	204
7.170.1.1 N	204
7.170.2 Function Documentation	204
7.170.2.1 main()	204
7.171 Ejemplo023.c	205
7.172 src/20261/src/Ejemplo023.c File Reference	206
7.172.1 Detailed Description	206
7.172.2 Macro Definition Documentation	207
7.172.2.1 N	207
7.172.3 Function Documentation	208
7.172.3.1 main()	208
7.173 Ejemplo023.c	210
7.174 src/20252/Ejemplo024.c File Reference	212
7.174.1 Macro Definition Documentation	213
7.174.1.1 BIT	213
7.174.1.2 BIT_CLEAR	213
7.174.1.3 BIT_GET	213
7.174.1.4 BIT_SET	213
7.174.1.5 BIT_TOGGLE	213
7.174.1.6 BIT_WRITE	213
7.174.1.7 ES_PAR	213
7.174.2 Function Documentation	214
7.174.2.1 main()	214
7.175 Ejemplo024.c	214
7.176 src/20261/src/Ejemplo024.c File Reference	214
7.176.1 Detailed Description	215
7.176.2 Macro Definition Documentation	216

7.176.2.1 N	216
7.176.3 Function Documentation	216
7.176.3.1 main()	216
7.177 Ejemplo024.c	217
7.178 src/20252/Ejemplo025.c File Reference	218
7.178.1 Macro Definition Documentation	219
7.178.1.1 BIT	219
7.178.1.2 BIT_CLEAR	219
7.178.1.3 BIT_GET	219
7.178.1.4 BIT_SET	220
7.178.1.5 BIT_TOGGLE	220
7.178.1.6 BIT_WRITE	220
7.178.1.7 ES_PAR	220
7.178.1.8 N	220
7.178.2 Function Documentation	220
7.178.2.1 main()	220
7.179 Ejemplo025.c	221
7.180 src/20261/src/Ejemplo025.c File Reference	221
7.180.1 Detailed Description	222
7.180.2 Macro Definition Documentation	223
7.180.2.1 N	223
7.180.3 Function Documentation	223
7.180.3.1 main()	223
7.181 Ejemplo025.c	224
7.182 src/20252/Ejemplo026.c File Reference	225
7.182.1 Macro Definition Documentation	226
7.182.1.1 BIT	226
7.182.1.2 BIT_CLEAR	226
7.182.1.3 BIT_GET	226
7.182.1.4 BIT_SET	226
7.182.1.5 BIT_TOGGLE	226
7.182.1.6 BIT_WRITE	226
7.182.1.7 ES_PAR	227
7.182.1.8 N	227
7.182.2 Function Documentation	227
7.182.2.1 main()	227
7.183 Ejemplo026.c	228
7.184 src/20261/src/Ejemplo026.c File Reference	229
7.184.1 Detailed Description	229
7.184.2 Macro Definition Documentation	230
7.184.2.1 mx	230
7.184.2.2 N	230

7.184.3 Function Documentation	230
7.184.3.1 main()	230
7.185 Ejemplo026.c	231
7.186 src/20252/Ejemplo027.c File Reference	233
7.186.1 Function Documentation	233
7.186.1.1 main()	233
7.187 Ejemplo027.c	233
7.188 src/20261/src/Ejemplo027.c File Reference	234
7.188.1 Detailed Description	234
7.188.2 Macro Definition Documentation	235
7.188.2.1 mx	235
7.188.2.2 N	235
7.188.3 Function Documentation	235
7.188.3.1 main()	235
7.189 Ejemplo027.c	236
7.190 src/20252/Ejemplo028.c File Reference	237
7.190.1 Macro Definition Documentation	238
7.190.1.1 N	238
7.190.1.2 NC	238
7.190.2 Function Documentation	238
7.190.2.1 main()	238
7.191 Ejemplo028.c	239
7.192 src/20261/src/Ejemplo028.c File Reference	240
7.192.1 Detailed Description	241
7.192.2 Macro Definition Documentation	241
7.192.2.1 M	241
7.192.2.2 N	241
7.192.3 Function Documentation	241
7.192.3.1 main()	241
7.193 Ejemplo028.c	242
7.194 src/20252/Ejemplo029.c File Reference	243
7.194.1 Function Documentation	243
7.194.1.1 exp1()	243
7.194.1.2 exp3()	244
7.194.1.3 exp4()	245
7.194.1.4 exp5()	245
7.194.1.5 factorial()	246
7.194.1.6 factorial2()	247
7.194.1.7 fct()	247
7.194.1.8 main()	248
7.194.1.9 potencia()	249
7.194.1.10 potencia2()	249

7.195 Ejemplo029.c	250
7.196 src/20261/src/Ejemplo029.c File Reference	252
7.196.1 Detailed Description	252
7.196.2 Macro Definition Documentation	252
7.196.2.1 M	252
7.196.2.2 N	253
7.196.3 Function Documentation	253
7.196.3.1 main()	253
7.197 Ejemplo029.c	254
7.198 src/20252/Ejemplo030.c File Reference	255
7.198.1 Function Documentation	255
7.198.1.1 buscar()	255
7.198.1.2 main()	256
7.199 Ejemplo030.c	256
7.200 src/20261/src/Ejemplo030.c File Reference	257
7.200.1 Detailed Description	257
7.200.2 Macro Definition Documentation	258
7.200.2.1 N	258
7.200.3 Function Documentation	258
7.200.3.1 main()	258
7.201 Ejemplo030.c	259
7.202 src/20252/Ejemplo031.c File Reference	260
7.202.1 Function Documentation	261
7.202.1.1 main()	261
7.203 Ejemplo031.c	261
7.204 src/20261/src/Ejemplo031.c File Reference	261
7.204.1 Detailed Description	262
7.204.2 Macro Definition Documentation	262
7.204.2.1 N	262
7.204.3 Function Documentation	263
7.204.3.1 main()	263
7.205 Ejemplo031.c	264
7.206 src/20252/Ejemplo032.c File Reference	266
7.206.1 Function Documentation	267
7.206.1.1 main()	267
7.207 Ejemplo032.c	267
7.208 src/20261/src/Ejemplo032.c File Reference	267
7.208.1 Detailed Description	268
7.208.2 Macro Definition Documentation	269
7.208.2.1 BIT	269
7.208.2.2 BIT_CLEAR	269
7.208.2.3 BIT_GET	269

7.208.2.4 BIT_SET	269
7.208.2.5 BIT_TOGGLE	269
7.208.2.6 BIT_WRITE	269
7.208.3 Function Documentation	269
7.208.3.1 main()	269
7.209 Ejemplo032.c	271
7.210 src/20252/Ejemplo033.c File Reference	272
7.210.1 Macro Definition Documentation	272
7.210.1.1 N	272
7.210.2 Function Documentation	272
7.210.2.1 asen()	272
7.210.2.2 main()	273
7.210.2.3 validar()	274
7.210.2.4 validar2()	275
7.211 Ejemplo033.c	275
7.212 src/20261/src/Ejemplo033.c File Reference	276
7.212.1 Detailed Description	277
7.212.2 Macro Definition Documentation	277
7.212.2.1 BIT	277
7.212.2.2 BIT_GET	277
7.212.2.3 BIT_TOGGLE_BYTE	278
7.212.2.4 N	278
7.212.3 Function Documentation	278
7.212.3.1 main()	278
7.213 Ejemplo033.c	279
7.214 src/20252/Ejemplo034.c File Reference	280
7.214.1 Macro Definition Documentation	280
7.214.1.1 NC	280
7.214.1.2 NE	280
7.214.1.3 NM	280
7.214.2 Function Documentation	280
7.214.2.1 main()	280
7.215 Ejemplo034.c	281
7.216 src/20261/src/Ejemplo034.c File Reference	282
7.216.1 Detailed Description	282
7.216.2 Macro Definition Documentation	283
7.216.2.1 BIT	283
7.216.2.2 BIT_GET	283
7.216.2.3 BIT_SET	283
7.216.2.4 BIT_TOGGLE	283
7.216.2.5 IS_POW2_POS	283
7.216.2.6 N	283

7.216.2.7 P	283
7.216.3 Function Documentation	284
7.216.3.1 main()	284
7.217 Ejemplo034.c	286
7.218 src/20261/src/Ejemplo034_2.c File Reference	288
7.218.1 Detailed Description	289
7.218.2 Macro Definition Documentation	289
7.218.2.1 BIT	289
7.218.2.2 BIT_GET	289
7.218.2.3 BIT_SET	290
7.218.2.4 BIT_TOGGLE	290
7.218.2.5 IS_POW2_POS	290
7.218.2.6 N	290
7.218.2.7 P	290
7.218.3 Function Documentation	290
7.218.3.1 main()	290
7.219 Ejemplo034_2.c	292
7.220 src/20252/Ejemplo035.c File Reference	295
7.220.1 Function Documentation	295
7.220.1.1 main()	295
7.221 Ejemplo035.c	295
7.222 src/20261/src/Ejemplo035.c File Reference	296
7.222.1 Detailed Description	296
7.222.2 Macro Definition Documentation	297
7.222.2.1 BIT	297
7.222.2.2 BIT_CLEAR	297
7.222.2.3 BIT_GET	297
7.222.2.4 BIT_SET	298
7.222.2.5 N	298
7.222.3 Function Documentation	298
7.222.3.1 main()	298
7.223 Ejemplo035.c	299
7.224 src/20252/Ejemplo036.c File Reference	301
7.224.1 Function Documentation	301
7.224.1.1 main()	301
7.225 Ejemplo036.c	302
7.226 src/20261/src/Ejemplo036.c File Reference	302
7.226.1 Detailed Description	303
7.226.2 Function Documentation	304
7.226.2.1 exp1()	304
7.226.2.2 exp3()	305
7.226.2.3 exp4()	306

7.226.2.4 facto()	307
7.226.2.5 fct()	308
7.226.2.6 main()	309
7.226.2.7 pot()	310
7.226.2.8 pot2()	311
7.227 Ejemplo036.c	312
7.228 src/20252/Ejemplo037.c File Reference	313
7.228.1 Function Documentation	314
7.228.1.1 main()	314
7.229 Ejemplo037.c	314
7.230 src/20261/src/Ejemplo037.c File Reference	314
7.230.1 Detailed Description	315
7.230.2 Function Documentation	316
7.230.2.1 main()	316
7.230.2.2 OddD()	316
7.230.2.3 SoddD()	317
7.231 Ejemplo037.c	318
7.232 src/20252/Ejemplo038.c File Reference	319
7.232.1 Function Documentation	320
7.232.1.1 main()	320
7.233 Ejemplo038.c	320
7.234 src/20261/src/Ejemplo038.c File Reference	321
7.234.1 Detailed Description	321
7.234.2 Function Documentation	321
7.234.2.1 main()	321
7.235 Ejemplo038.c	322
7.236 src/20252/Ejemplo039.c File Reference	322
7.236.1 Function Documentation	322
7.236.1.1 capturar()	322
7.236.1.2 crearMat()	323
7.236.1.3 imprimir()	323
7.236.1.4 main()	324
7.236.1.5 multiplicar()	325
7.237 Ejemplo039.c	326
7.238 src/20261/src/Ejemplo039.c File Reference	327
7.238.1 Detailed Description	328
7.238.2 Function Documentation	328
7.238.2.1 main()	328
7.239 Ejemplo039.c	328
7.240 src/20252/Ejemplo040.c File Reference	329
7.240.1 Function Documentation	329
7.240.1.1 capturar()	329

7.240.1.2 capturarM()	330
7.240.1.3 crearMat()	330
7.240.1.4 crearMatC()	331
7.240.1.5 crearMatD()	331
7.240.1.6 imprimir()	332
7.240.1.7 imprimirM()	332
7.240.1.8 liberar()	332
7.240.1.9 main()	333
7.240.1.10 multiplicar()	334
7.240.1.11 multiplicarM()	335
7.241 Ejemplo040.c	336
7.242 src/20261/src/Ejemplo040.c File Reference	338
7.242.1 Detailed Description	338
7.242.2 Macro Definition Documentation	339
7.242.2.1 N	339
7.242.3 Function Documentation	339
7.242.3.1 main()	339
7.243 Ejemplo040.c	340
7.244 src/template/Template.c File Reference	341
7.244.1 Function Documentation	341
7.244.1.1 main()	341
7.245 Template.c	341

Chapter 1

ProgIng - Programación en Ingeniería

Programación en Ingeniería (IILI06104)

Licenciatura en Ingeniería en Comunicaciones y Electrónica (IS75LI0303)

Licenciatura en Ingeniería en Mecatrónica (IS75LI0403)

Licenciatura en Ingeniería Sistemas Computacionales (IS75LI0502)

Licenciatura en Ingeniería de Datos e Inteligencia Artificial (IS75LI0801)

[Explorar la documentación »](#)

[Ver Demo](#) · [Reportar Bug](#) · [Requiere Modificaciones](#)

Tabla de contenidos

- Introducción
- Datos generales
- Competencia de la UDA
- Contenido
- Entregas y evaluación
 - Tareas
 - Prácticas
 - Proyecto final
 - Elementos mínimos de los reportes
- Repositorio y plataformas
- Documentación del código (Doxygen)
- Estructura sugerida del repositorio
- Código de ética y conducta profesional
- Bibliografía
- [Contacto](#)
- Licencia

1.0.1 Introducción

Esta Unidad de Aprendizaje (UDA) desarrolla bases sólidas de programación **en C** para resolver problemas de ingeniería con énfasis en:

- Estructuras de datos y algoritmos,
- Memoria dinámica y punteros,
- Manejo eficiente de archivos (texto y binarios),
- Buenas prácticas de programación estructurada,
- Trabajo profesional y ético.

Nota: Este repositorio concentra materiales, código, guías y entregables de la UDA.

1.0.2 Datos generales

- **UDA:** Programación en Ingeniería
- **Clave:** IILI06104
- **Periodo:** Enero 2026
- **Días:** Lunes y Jueves
- **Horario:** 12:00–14:00
- **Aula:** B101
- **Créditos:** 6 (150 horas totales: 72 con profesor/a + 78 autónomas)
- **Prerrequisito normativo:** Ninguno
- **Prerrequisito recomendable:** Fundamentos de los Sistemas de Información (IILI06061)

1.0.3 Competencia de la UDA

Diseña e implementa programas computacionales en C para abordar problemas complejos en ingeniería de datos e inteligencia artificial, empleando estructuras de datos avanzadas, manipulación de memoria dinámica y manejo eficiente de archivos, con apego a buenas prácticas, ética profesional y metodologías de programación estructurada.

1.0.4 Contenido

El contenido se organiza en los siguientes ejes:

1. **Estructura de un programa en C**
2. **Variables**
 - Reglas para identificadores
 - Tipos de datos
 - Alcance
3. **Operadores básicos**
 - Aritméticos, comparación, lógicos, binarios
 - Ejemplos
4. **Sentencias de control**
 - Decisiones (if/switch), ciclos (for/while/do-while), anidamientos
5. **Arreglos**
 - 1D (inicialización, longitud, búsqueda, ordenamiento: fuerza bruta, inserción, conteo)
 - Cadenas (longitud, carácter nulo, copiado, concatenado, búsqueda, conversión)
 - 2D y multidimensionales
6. **Manipulación de bits**
7. **Funciones**
 - Prototipos, paso por valor/referencia, `main(argc, argv)`, recursividad
8. **Estructuras y uniones**
9. **Punteros**
 - Aritmética, punteros a arreglos/cadenas/estructuras/funciones
 - Memoria dinámica
 - Punteros a punteros
 - Listas enlazadas (simples y dobles), árboles binarios
10. **Manejo de archivos**
 - Conceptos, flujo, posicionamiento
 - Texto y binarios (lectura/escritura/búsqueda/actualización/temporales)
11. **Directivas de preprocesamiento**
 - Macros, depuración
12. **Tópicos avanzados**
 - Unicode, números complejos, hilos (threads)

1.0.5 Entregas y evaluación

1.0.5.1 Ponderaciones

Elemento	Porcentaje
Tareas	8%

Elemento	Porcentaje
Exámenes rápidos	8%
Primera evaluación (1–4)	8%
Primera práctica	10%
Segunda evaluación (5–9)	8%
Segunda práctica	10%
Tercera evaluación (10–13)	8%
Tercera práctica	10%
Código del proyecto final	10%
Presentación del proyecto final	10%
Reporte del proyecto final	10%
Total	100%

1.0.5.2 Tareas

- **Plataformas:** Microsoft Teams y CSAcademy
- **Entrega por correo:** mibarram@gmail.com
- **Asunto:** PI 2026 1 T## NUA Apellidos
- **Formato:** reporte **PDF** con diagramas de flujo y simulaciones (cuando aplique)
- **Restricción:** no entregar ejecutables; **solo** código fuente y reporte
- **Fecha de entrega:** 1 semana (a partir de la asignación)

1.0.5.3 Prácticas

- **Plataforma:** Microsoft Teams
- **Entrega por correo:** mibarram@gmail.com
- **Asunto:** PI 2026 1 P# NUA Apellidos
- **Modalidad:** equipo (máximo 3 integrantes)
- **Formato:** reporte **PDF** con diagramas de flujo, simulaciones, **tablas de resultados**
- **Restricción:** no ejecutables; **solo** código fuente y reporte
- **Fecha de entrega:** acordada en clase

1.0.5.4 Proyecto final

- **Plataforma:** Microsoft Teams
- **Entrega por correo:** mibarram@gmail.com
- **Asunto:** PI 2026 1 Py ## NUA Apellidos
- **Modalidad:** equipo (máximo 3 integrantes)
- **Entregables:**
 - reporte **PDF** (diagramas de flujo, simulaciones, tablas y análisis),
 - **código fuente**,
 - **presentación**,
 - sesión de **preguntas y respuestas**.

1.0.5.5 Elementos mínimos de los reportes

1. **Introducción**
2. **Objetivos**
3. **Procedimiento (Algoritmo + Estructura de datos)**
4. **Diagrama de flujo**
5. **Resultados y análisis**
6. **Tablas comparativas** (si aplica)
7. **Conclusiones**
8. **Bibliografía**

1.0.6 Repositorio y plataformas

- **Repositorio oficial (GitHub):** <https://github.com/ibarram/ProgIng/>
- **Microsoft Teams (código de equipo):** co3pdww

- **Repositorio/Materiales (Dropbox):** disponible en el grupo (liga compartida)
- **Comunidad (Facebook):** grupo de apoyo (liga compartida en Teams)

Plataformas recomendadas para práctica y competencia:

- Repl.it, HackerRank, CSAcademy, OmegaUp, Project Euler, Codeforces, Codingame

Software sugerido:

- Dev-C++, MinGW, Code::Blocks, Visual Studio Community, Visual Studio Code, Eclipse, GNU Emacs, NetBeans

1.0.7 Instalación del compilador (Windows, macOS y Linux)

En la UDA trabajaremos principalmente con **C**. Necesitas un compilador (recomendado: **GCC** o **Clang**) y una terminal.

1.0.7.1 Verificación rápida (en cualquier sistema)

Después de instalar, verifica:

```
gcc --version
g++ --version
clang --version
```

Prueba de compilación:

```
# Compilar
gcc hello.c -o hello

# Ejecutar (Linux/macOS)
./hello

# Ejecutar (Windows)
hello.exe
```

Si usas bibliotecas o `math.h`, suele requerirse `-lm`: `gcc main.c -o main -lm`

1.0.7.2 Windows (recomendado)

Opción A: MSYS2 (recomendada por estabilidad y paquetes)

1. Instala MSYS2 (método estándar para tener terminal + paquetes).
2. Abre la terminal **MSYS2 UCRT64** o **MSYS2 MINGW64**.
3. Actualiza e instala el toolchain:

```
pacman -Syu
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

1. Agrega al **PATH** de Windows la carpeta de binarios (por ejemplo, `... \msys64\ucrt64\bin`).
2. Verifica con `gcc --version`.

Opción B: MinGW-w64 (simple)

- Instala MinGW-w64, agrega `bin` al **PATH** y verifica con `gcc --version`.

Opción C: WSL (Windows Subsystem for Linux)

- Si ya usas WSL (Ubuntu/Debian), sigue las instrucciones de Linux (Debian/Ubuntu).

1.0.7.3 macOS

Opción A: Xcode Command Line Tools (recomendada)

1. Instala las herramientas de línea de comandos:

```
xcode-select --install
```

1. Verifica con:

```
clang --version
gcc --version
```

En macOS, `gcc` normalmente apunta a **Clang** (es normal). Puedes compilar con `clang` o `gcc`.

Opción B: Homebrew + GCC (opcional)

1. Instala Homebrew (si lo usas en tu equipo).
2. Instala GCC:

```
brew install gcc
```

1. Compila usando `gcc-<versión>` (por ejemplo `gcc-14`):

```
gcc-14 hello.c -o hello
```

1.0.7.4 Linux

En la mayoría de distribuciones, GCC/Clang se instalan desde el gestor de paquetes.

Debian/Ubuntu

```
sudo apt update
sudo apt install -y build-essential gdb
```

Fedora

```
sudo dnf install -y gcc gcc-c++ make gdb
```

openSUSE (Leap/Tumbleweed)

```
sudo zypper refresh
sudo zypper install -y gcc gcc-c++ make gdb
```

Arch/Manjaro

```
sudo pacman -Syu --needed base-devel gcc gdb
```

1.0.7.5 Recomendación de editor/IDE (opcional)

- **Visual Studio Code** + extensiones de C/C++ (Microsoft) + CMake Tools (si aplica)
- **Code::Blocks** o **CLion** (si cuentas con licencia)
- En Linux/macOS: `terminal` + `make` es suficiente para iniciar

Software sugerido:

- Dev-C++, MinGW, Code::Blocks, Visual Studio Community, Visual Studio Code, Eclipse, GNU Emacs, NetBeans

1.0.8 Documentación del código (Doxygen)

Este repositorio incluye ejemplos en C documentados ([HTML](#) y [PDF](#)) con comentarios estilo **Doxygen**. La documentación se genera en:

- `doc/doxygen/html/index.html`
- `doc/doxygen/latex/refman.pdf`

1.0.8.1 Requisitos

- **Doxygen**
- **Graphviz** (opcional, pero recomendado para CALL_GRAPH / CALLER_GRAPH)

Instalación típica:

- **openSUSE:** `sudo zypper install doxygen graphviz texlive-scheme-medium latexmk`
- **Ubuntu/Debian:** `sudo apt-get update && sudo apt-get install doxygen graphviz texlive-scheme-medium latexmk`
- **macOS (Homebrew):** `brew install doxygen graphviz MacTeX`

1.0.8.2 Generar documentación

```
make docs
make docs-pdf
```

1.0.9 Estructura sugerida del repositorio

```
.
├── doc/
│   ├── pdf/           # Temario, consideraciones, guías
│   ├── slide/         # Presentaciones (PDF)
│   ├── img/           # Imágenes (escudo, figuras)
│   └── markdown/      # Notas y guías (Markdown)
├── src/
│   ├── 20251/         # Ejemplos desarrollados en la UDA para el periodo E-J 2025
│   ├── 20251/         # Ejemplos desarrollados en la UDA para el periodo A-D 2025
│   ├── 20251/         # Ejemplos desarrollados en la UDA para el periodo E-J 2026
│   ├── tools/         # Scripts de apoyo (opcional)
│   └── templates/     # Plantillas de reporte y estilos
├── data/              # Archivos de entrada/salida
└── LICENSE
```

1.0.10 Código de ética y conducta profesional

- El código debe ser **original** y debidamente **referenciado**.
- Se penaliza el **plagio** (copiar-pegar sin atribución) y la **suplantación**.
- Se fomenta la colaboración en equipo **cuando esté permitida**, respetando las reglas de autoría.
- El uso de herramientas de apoyo (incluida IA) debe reflejarse con transparencia en el **reporte** (qué se usó y cómo).

1.0.11 Bibliografía

- Gazi, O. (2024). *Modern C Programming* (1st Ed.). Springer Cham.
- Horton, I. (2013). *Beginning C* (5th Ed.). Apress.
- Chavan, S. (2017). *C Recipes* (1st Ed.). Apress.
- Toppo, N. & Dewan, H. (2013). *Pointers in C* (1st Ed.). Apress.
- Goyal, A. (2013). *Moving from C to C++* (1st Ed.). Apress.
- Kalicharan, N. (2017). *Advanced Topics in C* (1st Ed.). Apress.
- Erickson, J. (2019). *Algorithms* (1st Ed.). University of Illinois.

1.0.12 Contacto

Dr. M.-A. Ibarra-Manzano - DICIS-UG - ORCID: 0000-0003-4317-0248 - SCOPUS:↔
15837259000

Unidad de Aprendizaje Link: [Programación en Ingeniería](#)

1.0.13 Licencia

Este repositorio se distribuye bajo **GPL-3.0**. Consulta el archivo `LICENSE`.

Chapter 2

Directory Hierarchy

2.1 Directories

20251	23
001_cod_prioridad.c	31
002_Pokemon.c	32
003_cuadratica.c	33
004_rangTemp.c	34
005_rangTemp2.c	35
006_valorPI.c	37
007_exp_1.c	38
008_exp_2.c	39
009_exp_3.c	40
010_Inx.c	42
011_seno.c	43
012_aleatorio.c	44
014_OrdBurbuja.c	46
015_ParesNones.c	48
016_Seleccion.c	50
017_Insercion.c	52
018_Casilleros.c	54
019_BurbujaBD.c	57
020_BurbujaBD.c	59
021_Conteno.c	61
022_multiplicacion.c	63
023_EliGauss.c	65
024_EliGauss.c	68
025_CyN.c	70
026_Binarios.c	72
027_Paridad.c	75
028_Hamming.c	77
029_Enc.c	79
030_ParEntrada.c	81
031_ParEntrada.c	82
032_ParEntrada.c	83
033_Arreglo1D.c	85
Ejemplo_035.c	87
Ejemplo_036.c	90
Ejemplo_037.c	92
20252	24
Ejemplo001.c	105
Ejemplo002.c	107
Ejemplo003.c	111
Ejemplo004.c	115

Ejemplo005.c	118
Ejemplo006.c	122
Ejemplo007.c	127
Ejemplo008.c	131
Ejemplo009.c	134
Ejemplo010.c	138
Ejemplo011.c	141
Ejemplo012.c	145
Ejemplo013.c	150
Ejemplo014.c	155
Ejemplo015.c	160
Ejemplo016.c	164
Ejemplo017.c	169
Ejemplo018.c	176
Ejemplo019.c	181
Ejemplo020.c	187
Ejemplo021.c	192
Ejemplo022.c	198
Ejemplo023.c	203
Ejemplo024.c	212
Ejemplo025.c	218
Ejemplo026.c	225
Ejemplo027.c	233
Ejemplo028.c	237
Ejemplo029.c	243
Ejemplo030.c	255
Ejemplo031.c	260
Ejemplo032.c	266
Ejemplo033.c	272
Ejemplo034.c	280
Ejemplo035.c	295
Ejemplo036.c	301
Ejemplo037.c	313
Ejemplo038.c	319
Ejemplo039.c	322
Ejemplo040.c	329
Ejemplo041.c	96
Ejemplo042.c	97
Ejemplo043.c	98
Ejemplo044.c	100
Ejemplo045.c	102
20261	25
src	26
Ejemplo001.c	106
Ejemplo002.c	108
Ejemplo003.c	112
Ejemplo004.c	116
Ejemplo005.c	119
Ejemplo006.c	123
Ejemplo007.c	128
Ejemplo008.c	132
Ejemplo009.c	135
Ejemplo010.c	139
Ejemplo011.c	143
Ejemplo012.c	147
Ejemplo013.c	152
Ejemplo014.c	157
Ejemplo015.c	162

Ejemplo016.c	167
Ejemplo017.c	172
Ejemplo018.c	178
Ejemplo019.c	183
Ejemplo020.c	188
Ejemplo021.c	194
Ejemplo022.c	200
Ejemplo023.c	206
Ejemplo024.c	214
Ejemplo025.c	221
Ejemplo026.c	229
Ejemplo027.c	234
Ejemplo028.c	240
Ejemplo029.c	252
Ejemplo030.c	257
Ejemplo031.c	261
Ejemplo032.c	267
Ejemplo033.c	276
Ejemplo034.c	282
Ejemplo034_2.c	288
Ejemplo035.c	296
Ejemplo036.c	302
Ejemplo037.c	314
Ejemplo038.c	321
Ejemplo039.c	327
Ejemplo040.c	338
src	25
20251	23
001_cod_prioridad.c	31
002_Pokemon.c	32
003_cuadratica.c	33
004_rangTemp.c	34
005_rangTemp2.c	35
006_valorPI.c	37
007_exp_1.c	38
008_exp_2.c	39
009_exp_3.c	40
010_lnx.c	42
011_seno.c	43
012_aleatorio.c	44
014_OrdBurbuja.c	46
015_ParesNones.c	48
016_Seleccion.c	50
017_Insercion.c	52
018_Casilleros.c	54
019_BurbujaBD.c	57
020_BurbujaBD.c	59
021_Conteno.c	61
022_multiplicacion.c	63
023_EliGauss.c	65
024_EliGauss.c	68
025_CyN.c	70
026_Binarios.c	72
027_Paridad.c	75
028_Hamming.c	77
029_Enc.c	79
030_ParEntrada.c	81
031_ParEntrada.c	82

032_ParEntrada.c	83
033_Arreglo1D.c	85
Ejemplo_035.c	87
Ejemplo_036.c	90
Ejemplo_037.c	92
20252	24
Ejemplo001.c	105
Ejemplo002.c	107
Ejemplo003.c	111
Ejemplo004.c	115
Ejemplo005.c	118
Ejemplo006.c	122
Ejemplo007.c	127
Ejemplo008.c	131
Ejemplo009.c	134
Ejemplo010.c	138
Ejemplo011.c	141
Ejemplo012.c	145
Ejemplo013.c	150
Ejemplo014.c	155
Ejemplo015.c	160
Ejemplo016.c	164
Ejemplo017.c	169
Ejemplo018.c	176
Ejemplo019.c	181
Ejemplo020.c	187
Ejemplo021.c	192
Ejemplo022.c	198
Ejemplo023.c	203
Ejemplo024.c	212
Ejemplo025.c	218
Ejemplo026.c	225
Ejemplo027.c	233
Ejemplo028.c	237
Ejemplo029.c	243
Ejemplo030.c	255
Ejemplo031.c	260
Ejemplo032.c	266
Ejemplo033.c	272
Ejemplo034.c	280
Ejemplo035.c	295
Ejemplo036.c	301
Ejemplo037.c	313
Ejemplo038.c	319
Ejemplo039.c	322
Ejemplo040.c	329
Ejemplo041.c	96
Ejemplo042.c	97
Ejemplo043.c	98
Ejemplo044.c	100
Ejemplo045.c	102
20261	25
src	26
Ejemplo001.c	106
Ejemplo002.c	108
Ejemplo003.c	112
Ejemplo004.c	116
Ejemplo005.c	119

Ejemplo006.c	123
Ejemplo007.c	128
Ejemplo008.c	132
Ejemplo009.c	135
Ejemplo010.c	139
Ejemplo011.c	143
Ejemplo012.c	147
Ejemplo013.c	152
Ejemplo014.c	157
Ejemplo015.c	162
Ejemplo016.c	167
Ejemplo017.c	172
Ejemplo018.c	178
Ejemplo019.c	183
Ejemplo020.c	188
Ejemplo021.c	194
Ejemplo022.c	200
Ejemplo023.c	206
Ejemplo024.c	214
Ejemplo025.c	221
Ejemplo026.c	229
Ejemplo027.c	234
Ejemplo028.c	240
Ejemplo029.c	252
Ejemplo030.c	257
Ejemplo031.c	261
Ejemplo032.c	267
Ejemplo033.c	276
Ejemplo034.c	282
Ejemplo034_2.c	288
Ejemplo035.c	296
Ejemplo036.c	302
Ejemplo037.c	314
Ejemplo038.c	321
Ejemplo039.c	327
Ejemplo040.c	338
template	28
Template.c	341
src	26
Ejemplo001.c	106
Ejemplo002.c	108
Ejemplo003.c	112
Ejemplo004.c	116
Ejemplo005.c	119
Ejemplo006.c	123
Ejemplo007.c	128
Ejemplo008.c	132
Ejemplo009.c	135
Ejemplo010.c	139
Ejemplo011.c	143
Ejemplo012.c	147
Ejemplo013.c	152
Ejemplo014.c	157
Ejemplo015.c	162
Ejemplo016.c	167
Ejemplo017.c	172
Ejemplo018.c	178
Ejemplo019.c	183

Ejemplo020.c	188
Ejemplo021.c	194
Ejemplo022.c	200
Ejemplo023.c	206
Ejemplo024.c	214
Ejemplo025.c	221
Ejemplo026.c	229
Ejemplo027.c	234
Ejemplo028.c	240
Ejemplo029.c	252
Ejemplo030.c	257
Ejemplo031.c	261
Ejemplo032.c	267
Ejemplo033.c	276
Ejemplo034.c	282
Ejemplo034_2.c	288
Ejemplo035.c	296
Ejemplo036.c	302
Ejemplo037.c	314
Ejemplo038.c	321
Ejemplo039.c	327
Ejemplo040.c	338
template	28
Template.c	341

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

contacto	29
dato_s	29
dato_u	30

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

src/20251/001_cod_prioridad.c	31
src/20251/002_Pokemon.c	32
src/20251/003_cuadratica.c	33
src/20251/004_rangTemp.c	34
src/20251/005_rangTemp2.c	35
src/20251/006_valorPl.c	37
src/20251/007_exp_1.c	38
src/20251/008_exp_2.c	39
src/20251/009_exp_3.c	40
src/20251/010_Inx.c	42
src/20251/011_seno.c	43
src/20251/012_aleatorio.c	44
src/20251/014_OrdBurbuja.c	46
src/20251/015_ParesNones.c	48
src/20251/016_Seleccion.c	50
src/20251/017_Insercion.c	52
src/20251/018_Casilleros.c	54
src/20251/019_BurbujaBD.c	57
src/20251/020_BurbujaBD.c	59
src/20251/021_Conteno.c	61
src/20251/022_multiplicacion.c	63
src/20251/023_EliGauss.c	65
src/20251/024_EliGauss.c	68
src/20251/025_CyN.c	70
src/20251/026_Binarios.c	72
src/20251/027_Paridad.c	75
src/20251/028_Hamming.c	77
src/20251/029_Enc.c	79
src/20251/030_ParEntrada.c	81
src/20251/031_ParEntrada.c	82
src/20251/032_ParEntrada.c	83
src/20251/033_Arreglo1D.c	85
src/20251/Ejemplo_035.c	87
src/20251/Ejemplo_036.c	90
src/20251/Ejemplo_037.c	92
src/20252/Ejemplo001.c	105
src/20252/Ejemplo002.c	107
src/20252/Ejemplo003.c	111
src/20252/Ejemplo004.c	115
src/20252/Ejemplo005.c	118

src/20252/Ejemplo006.c	122
src/20252/Ejemplo007.c	127
src/20252/Ejemplo008.c	131
src/20252/Ejemplo009.c	134
src/20252/Ejemplo010.c	138
src/20252/Ejemplo011.c	141
src/20252/Ejemplo012.c	145
src/20252/Ejemplo013.c	150
src/20252/Ejemplo014.c	155
src/20252/Ejemplo015.c	160
src/20252/Ejemplo016.c	164
src/20252/Ejemplo017.c	169
src/20252/Ejemplo018.c	176
src/20252/Ejemplo019.c	181
src/20252/Ejemplo020.c	187
src/20252/Ejemplo021.c	192
src/20252/Ejemplo022.c	198
src/20252/Ejemplo023.c	203
src/20252/Ejemplo024.c	212
src/20252/Ejemplo025.c	218
src/20252/Ejemplo026.c	225
src/20252/Ejemplo027.c	233
src/20252/Ejemplo028.c	237
src/20252/Ejemplo029.c	243
src/20252/Ejemplo030.c	255
src/20252/Ejemplo031.c	260
src/20252/Ejemplo032.c	266
src/20252/Ejemplo033.c	272
src/20252/Ejemplo034.c	280
src/20252/Ejemplo035.c	295
src/20252/Ejemplo036.c	301
src/20252/Ejemplo037.c	313
src/20252/Ejemplo038.c	319
src/20252/Ejemplo039.c	322
src/20252/Ejemplo040.c	329
src/20252/Ejemplo041.c	96
src/20252/Ejemplo042.c	97
src/20252/Ejemplo043.c	98
src/20252/Ejemplo044.c	100
src/20252/Ejemplo045.c	102
src/20261/src/Ejemplo001.c	
Hola mundo mínimo: imprime un mensaje en consola	106
src/20261/src/Ejemplo002.c	
Codificador de prioridad de interrupción para 4 dispositivos (D1..D4)	108
src/20261/src/Ejemplo003.c	
Máximo número de Pokémon que pueden evolucionarse con venta de no evolucionados	112
src/20261/src/Ejemplo004.c	
Mínimo y máximo posible de niños en equipo rojo tras exactamente K cambios	116
src/20261/src/Ejemplo005.c	
Solución de la ecuación cuadrática con raíces reales o complejas	119
src/20261/src/Ejemplo006.c	
Etiquetado de una temperatura T usando 4 umbrales ($T_1 < T_2 < T_3 < T_4$) y 5 categorías	123
src/20261/src/Ejemplo007.c	
Cálculo del entero i tal que $1+3+5+\dots+(2i-1) \geq n$ (relación con $\sqrt{n}$)	128
src/20261/src/Ejemplo008.c	
Aproximación de e mediante la serie de Leibniz	132
src/20261/src/Ejemplo009.c	
Aproximación de $\exp(x)$ con serie de Taylor (cálculo directo de potencias y factorial)	135

src/20261/src/Ejemplo010.c	
Aproximación de $\exp(x)$ con Taylor usando producto incremental dentro del término	139
src/20261/src/Ejemplo011.c	
Aproximación de $\exp(x)$ con Taylor usando recurrencia entre términos (forma eficiente)	143
src/20261/src/Ejemplo012.c	
Aproximación de $\ln(x)$ con serie tipo atanh calculando potencias desde cero	147
src/20261/src/Ejemplo013.c	
Aproximación de $\ln(x)$ con serie tipo atanh usando recurrencia entre términos	152
src/20261/src/Ejemplo014.c	
Aproximación de $\sin(x)$ con serie de Taylor calculando potencia y factorial desde cero	157
src/20261/src/Ejemplo015.c	
Aproximación de $\sin(x)$ con Taylor usando producto por término (sin factorial entero explícito)	162
src/20261/src/Ejemplo016.c	
Aproximación de $\sin(x)$ con Taylor usando recurrencia entre términos ($O(n)$)	167
src/20261/src/Ejemplo017.c	
Aproximación de $\arcsin(x)$ mediante serie de Taylor (Maclaurin) para $ x < 1$	172
src/20261/src/Ejemplo018.c	
Generación de un arreglo de enteros aleatorios en el rango [min, max]	178
src/20261/src/Ejemplo019.c	
Generación y ordenamiento de un arreglo de enteros aleatorios (orden ascendente)	183
src/20261/src/Ejemplo020.c	
Genera un arreglo de enteros aleatorios y lo ordena con Odd–Even Sort (Brick Sort)	188
src/20261/src/Ejemplo021.c	
Genera un arreglo de flotantes aleatorios y lo ordena con Selection Sort	194
src/20261/src/Ejemplo022.c	
Genera un arreglo de flotantes aleatorios y lo ordena con Insertion Sort	200
src/20261/src/Ejemplo023.c	
Ordenamiento tipo Bucket Sort (distribución por casilleros) + ordenamiento interno por selección	206
src/20261/src/Ejemplo024.c	
Bucket Sort (distribución por casilleros) + ordenamiento interno por selección (versión compacta)	214
src/20261/src/Ejemplo025.c	
Ordenamiento alternado por selección: mínimo a la izquierda, máximo a la derecha (floats)	221
src/20261/src/Ejemplo026.c	
Ordenamiento por conteo (Counting Sort) para enteros en el rango [0, max]	229
src/20261/src/Ejemplo027.c	
Ordenamiento por conteo (Counting Sort) para enteros en el rango [min, max]	234
src/20261/src/Ejemplo028.c	
Resta de matrices: $C = A - B$ para matrices reales de tamaño $n \times m$	240
src/20261/src/Ejemplo029.c	
Multiplicación de matrices: $C = A * B$ con verificación de dimensiones	252
src/20261/src/Ejemplo030.c	
Resolución de un sistema lineal $Ax=b$ por eliminación de Gauss (sin pivoteo) y sustitución hacia atrás	257
src/20261/src/Ejemplo031.c	
Generación de datos aleatorios y transformación por centrado/normalización (min–max o z-score)	261
src/20261/src/Ejemplo032.c	
Demostración de macros para manipulación de bits en un entero	267
src/20261/src/Ejemplo033.c	
Cálculo de paridad XOR de un mensaje y simulación de error (flip) en un bit aleatorio	276
src/20261/src/Ejemplo034.c	
Inserción de paridades (Hamming) en posiciones potencia de 2 y detección de error de 1 bit	282
src/20261/src/Ejemplo034_2.c	
Inserción de bytes de paridad en posiciones potencia de 2 (Hamming por bytes) y detección de un error de 1 bit	288
src/20261/src/Ejemplo035.c	
Cifrado/descifrado XOR sobre letras (A–Z, a–z) usando una clave numérica	296

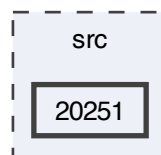
src/20261/src/ Ejemplo036.c	
Comparación de implementaciones de potencia, factorial y aproximación de $\exp(x)$	302
src/20261/src/ Ejemplo037.c	
Calcula la suma de OddD(k) en un intervalo [A,B] usando recursión	314
src/20261/src/ Ejemplo038.c	
Imprime la cantidad de argumentos y lista argv[i]	321
src/20261/src/ Ejemplo039.c	
Suma dos números reales recibidos por línea de comandos	327
src/20261/src/ Ejemplo040.c	
Genera n números aleatorios en [min,max] y calcula la media	338
src/template/ Template.c	341

Chapter 5

Directory Documentation

5.1 src/20251 Directory Reference

Directory dependency graph for 20251:



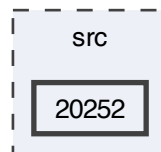
Files

- file [001_cod_prioridad.c](#)
- file [002_Pokemon.c](#)
- file [003_cuadratica.c](#)
- file [004_rangTemp.c](#)
- file [005_rangTemp2.c](#)
- file [006_valorPI.c](#)
- file [007_exp_1.c](#)
- file [008_exp_2.c](#)
- file [009_exp_3.c](#)
- file [010_Inx.c](#)
- file [011_seno.c](#)
- file [012_aleatorio.c](#)
- file [014_OrdBurbuja.c](#)
- file [015_ParesNones.c](#)
- file [016_Seleccion.c](#)
- file [017_Insercion.c](#)
- file [018_Casilleros.c](#)
- file [019_BurbujaBD.c](#)
- file [020_BurbujaBD.c](#)
- file [021_Conteno.c](#)
- file [022_multiplicacion.c](#)

- file [023_EliGauss.c](#)
- file [024_EliGauss.c](#)
- file [025_CyN.c](#)
- file [026_Binarios.c](#)
- file [027_Paridad.c](#)
- file [028_Hamming.c](#)
- file [029_Enc.c](#)
- file [030_ParEntrada.c](#)
- file [031_ParEntrada.c](#)
- file [032_ParEntrada.c](#)
- file [033_Arreglo1D.c](#)
- file [Ejemplo_035.c](#)
- file [Ejemplo_036.c](#)
- file [Ejemplo_037.c](#)

5.2 src/20252 Directory Reference

Directory dependency graph for 20252:



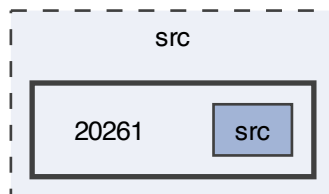
Files

- file [Ejemplo001.c](#)
- file [Ejemplo002.c](#)
- file [Ejemplo003.c](#)
- file [Ejemplo004.c](#)
- file [Ejemplo005.c](#)
- file [Ejemplo006.c](#)
- file [Ejemplo007.c](#)
- file [Ejemplo008.c](#)
- file [Ejemplo009.c](#)
- file [Ejemplo010.c](#)
- file [Ejemplo011.c](#)
- file [Ejemplo012.c](#)
- file [Ejemplo013.c](#)
- file [Ejemplo014.c](#)
- file [Ejemplo015.c](#)
- file [Ejemplo016.c](#)
- file [Ejemplo017.c](#)
- file [Ejemplo018.c](#)
- file [Ejemplo019.c](#)
- file [Ejemplo020.c](#)

- file [Ejemplo021.c](#)
- file [Ejemplo022.c](#)
- file [Ejemplo023.c](#)
- file [Ejemplo024.c](#)
- file [Ejemplo025.c](#)
- file [Ejemplo026.c](#)
- file [Ejemplo027.c](#)
- file [Ejemplo028.c](#)
- file [Ejemplo029.c](#)
- file [Ejemplo030.c](#)
- file [Ejemplo031.c](#)
- file [Ejemplo032.c](#)
- file [Ejemplo033.c](#)
- file [Ejemplo034.c](#)
- file [Ejemplo035.c](#)
- file [Ejemplo036.c](#)
- file [Ejemplo037.c](#)
- file [Ejemplo038.c](#)
- file [Ejemplo039.c](#)
- file [Ejemplo040.c](#)
- file [Ejemplo041.c](#)
- file [Ejemplo042.c](#)
- file [Ejemplo043.c](#)
- file [Ejemplo044.c](#)
- file [Ejemplo045.c](#)

5.3 src/20261 Directory Reference

Directory dependency graph for 20261:



Directories

- directory [src](#)

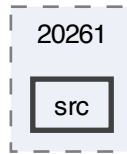
5.4 src Directory Reference

Directories

- directory [20251](#)
- directory [20252](#)
- directory [20261](#)
- directory [template](#)

5.5 src/20261/src Directory Reference

Directory dependency graph for src:



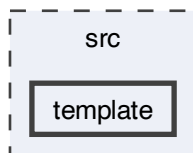
Files

- file [Ejemplo001.c](#)
Hola mundo mínimo: imprime un mensaje en consola.
- file [Ejemplo002.c](#)
Codificador de prioridad de interrupción para 4 dispositivos (D1..D4).
- file [Ejemplo003.c](#)
Máximo número de Pokémon que pueden evolucionarse con venta de no evolucionados.
- file [Ejemplo004.c](#)
Mínimo y máximo posible de niños en equipo rojo tras exactamente K cambios.
- file [Ejemplo005.c](#)
Solución de la ecuación cuadrática con raíces reales o complejas.
- file [Ejemplo006.c](#)
Etiquetado de una temperatura T usando 4 umbrales ($T1 < T2 < T3 < T4$) y 5 categorías.
- file [Ejemplo007.c](#)
Cálculo del entero i tal que $1+3+5+\dots+(2i-1) \geq n$ (relación con $\text{sqrt}(n)$).
- file [Ejemplo008.c](#)
Aproximación de π mediante la serie de Leibniz.
- file [Ejemplo009.c](#)
Aproximación de $\exp(x)$ con serie de Taylor (cálculo directo de potencias y factorial).
- file [Ejemplo010.c](#)
Aproximación de $\exp(x)$ con Taylor usando producto incremental dentro del término.
- file [Ejemplo011.c](#)
Aproximación de $\exp(x)$ con Taylor usando recurrencia entre términos (forma eficiente).
- file [Ejemplo012.c](#)
Aproximación de $\ln(x)$ con serie tipo atanh calculando potencias desde cero.
- file [Ejemplo013.c](#)
Aproximación de $\ln(x)$ con serie tipo atanh usando recurrencia entre términos.
- file [Ejemplo014.c](#)
Aproximación de $\sin(x)$ con serie de Taylor calculando potencia y factorial desde cero.
- file [Ejemplo015.c](#)
Aproximación de $\sin(x)$ con Taylor usando producto por término (sin factorial entero explícito).
- file [Ejemplo016.c](#)
Aproximación de $\sin(x)$ con Taylor usando recurrencia entre términos ($O(n)$).
- file [Ejemplo017.c](#)

- Aproximación de $\arcsin(x)$ mediante serie de Taylor (Maclaurin) para $|x| < 1$.*
 - file [Ejemplo018.c](#)
- Generación de un arreglo de enteros aleatorios en el rango $[min, max]$.*
 - file [Ejemplo019.c](#)
- Generación y ordenamiento de un arreglo de enteros aleatorios (orden ascendente).*
 - file [Ejemplo020.c](#)
- Genera un arreglo de enteros aleatorios y lo ordena con Odd–Even Sort (Brick Sort).*
 - file [Ejemplo021.c](#)
- Genera un arreglo de flotantes aleatorios y lo ordena con Selection Sort.*
 - file [Ejemplo022.c](#)
- Genera un arreglo de flotantes aleatorios y lo ordena con Insertion Sort.*
 - file [Ejemplo023.c](#)
- Ordenamiento tipo Bucket Sort (distribución por casilleros) + ordenamiento interno por selección.*
 - file [Ejemplo024.c](#)
- Bucket Sort (distribución por casilleros) + ordenamiento interno por selección (versión compacta).*
 - file [Ejemplo025.c](#)
- Ordenamiento alternado por selección: mínimo a la izquierda, máximo a la derecha (floats).*
 - file [Ejemplo026.c](#)
- Ordenamiento por conteo (Counting Sort) para enteros en el rango $[0, max]$.*
 - file [Ejemplo027.c](#)
- Ordenamiento por conteo (Counting Sort) para enteros en el rango $[min, max]$.*
 - file [Ejemplo028.c](#)
- Resta de matrices: $C = A - B$ para matrices reales de tamaño $n \times m$.*
 - file [Ejemplo029.c](#)
- Multiplicación de matrices: $C = A * B$ con verificación de dimensiones.*
 - file [Ejemplo030.c](#)
- Resolución de un sistema lineal $Ax=b$ por eliminación de Gauss (sin pivoteo) y sustitución hacia atrás.*
 - file [Ejemplo031.c](#)
- Generación de datos aleatorios y transformación por centrado/normalización (min–max o z-score).*
 - file [Ejemplo032.c](#)
- Demostración de macros para manipulación de bits en un entero.*
 - file [Ejemplo033.c](#)
- Cálculo de paridad XOR de un mensaje y simulación de error (flip) en un bit aleatorio.*
 - file [Ejemplo034.c](#)
- Inserción de paridades (Hamming) en posiciones potencia de 2 y detección de error de 1 bit.*
 - file [Ejemplo034_2.c](#)
- Inserción de bytes de paridad en posiciones potencia de 2 (Hamming por bytes) y detección de un error de 1 bit.*
 - file [Ejemplo035.c](#)
- Cifrado/descifrado XOR sobre letras (A–Z, a–z) usando una clave numérica.*
 - file [Ejemplo036.c](#)
- Comparación de implementaciones de potencia, factorial y aproximación de $\exp(x)$.*
 - file [Ejemplo037.c](#)
- Calcula la suma de OddD(k) en un intervalo $[A,B]$ usando recursión.*
 - file [Ejemplo038.c](#)
- Imprime la cantidad de argumentos y lista $argv[i]$.*
 - file [Ejemplo039.c](#)
- Suma dos números reales recibidos por línea de comandos.*
 - file [Ejemplo040.c](#)
- Genera n números aleatorios en $[min,max]$ y calcula la media.*
 - file [Ejemplo040.c](#)

5.6 src/template Directory Reference

Directory dependency graph for template:



Files

- file [Template.c](#)

Chapter 6

Data Structure Documentation

6.1 contacto Struct Reference

Data Fields

- char [nombre](#) [NM]
- long int [telefono](#)
- char [email](#) [NE]

6.1.1 Detailed Description

Definition at line 7 of file [Ejemplo034.c](#).

6.1.2 Field Documentation

6.1.2.1 email

```
char contacto::email[NE]
```

Definition at line 11 of file [Ejemplo034.c](#).

6.1.2.2 nombre

```
char contacto::nombre[NM]
```

Definition at line 9 of file [Ejemplo034.c](#).

6.1.2.3 telefono

```
long int contacto::telefono
```

Definition at line 10 of file [Ejemplo034.c](#).

The documentation for this struct was generated from the following file:

- [src/20252/Ejemplo034.c](#)

6.2 dato_s Struct Reference

Data Fields

- char [a](#)
- int [b](#)
- float [c](#)

6.2.1 Detailed Description

Definition at line 3 of file [Ejemplo035.c](#).

6.2.2 Field Documentation

6.2.2.1 a

```
char dato_s::a
```

Definition at line 4 of file [Ejemplo035.c](#).

6.2.2.2 b

```
int dato_s::b
```

Definition at line 5 of file [Ejemplo035.c](#).

6.2.2.3 c

```
float dato_s::c
```

Definition at line 6 of file [Ejemplo035.c](#).

The documentation for this struct was generated from the following file:

- [src/20252/Ejemplo035.c](#)

6.3 dato_u Union Reference

Data Fields

- char [a](#)
- int [b](#)
- float [c](#)

6.3.1 Detailed Description

Definition at line 9 of file [Ejemplo035.c](#).

6.3.2 Field Documentation

6.3.2.1 a

```
char dato_u::a
```

Definition at line 10 of file [Ejemplo035.c](#).

6.3.2.2 b

```
int dato_u::b
```

Definition at line 11 of file [Ejemplo035.c](#).

6.3.2.3 c

```
float dato_u::c
```

Definition at line 12 of file [Ejemplo035.c](#).

The documentation for this union was generated from the following file:

- [src/20252/Ejemplo035.c](#)

Chapter 7

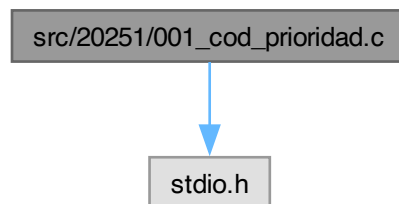
File Documentation

7.1 README.md File Reference

7.2 src/20251/001_cod_prioridad.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 001_cod_prioridad.c:



Functions

- `int main (int argc, char *argv[])`

7.2.1 Function Documentation

7.2.1.1 main()

```
int main (
    int argc,
    char * argv[])
Definition at line 3 of file 001_cod_prioridad.c.
00004 {
00005     int D4, D3, D2, D1, sal, b0, b1, b2;
00006     printf("Ingrese el estado: ");
00007     scanf("%d %d %d %d", &D4, &D3, &D2, &D1);
00008     b0 = D1|(~D2&D3);
00009     b1 = ~D1&(D2|D3);
00010     b2 = ~D1&~D2&~D3&D4;
00011     sal = 4*b2+2*b1+b0;
00012     printf("Salida = %d (%d%d%d)\n", sal, b2, b1, b0);
00013     return 0;
00014 }
```

7.3 001_cod_prioridad.c

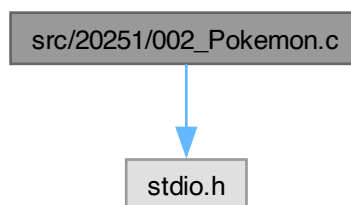
[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int D4, D3, D2, D1, sal, b0, b1, b2;
00006     printf("Ingrese el estado: ");
00007     scanf("%d %d %d %d", &D4, &D3, &D2, &D1);
00008     b0 = D1|(~D2&D3);
00009     b1 = ~D1&(D2|D3);
00010     b2 = ~D1&~D2&~D3&D4;
00011     sal = 4*b2+2*b1+b0;
00012     printf("Salida = %d (%d%d%d)\n", sal, b2, b1, b0);
00013     return 0;
00014 }
```

7.4 src/20251/002_Pokemon.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 002_Pokemon.c:



Functions

- int [main](#) (int argc, char *argv[])

7.4.1 Function Documentation

7.4.1.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 3 of file [002_Pokemon.c](#).

```
00004 {
00005     int N, M, X, Y, S;
00006     printf("Ingrese los valores: ");
00007     scanf("%d %d %d %d", &N, &M, &X, &Y);
00008     S = (N*Y+M)/(X+Y);
00009     printf("Resultado = %d\n", N*X<M?N:S);
00010     return 0;
00011 }
```

7.5 002_Pokemon.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
```

```

00003 int main(int argc, char *argv[])
00004 {
00005     int N, M, X, Y, S;
00006     printf("Ingresa los valores: ");
00007     scanf("%d %d %d %d", &N, &M, &X, &Y);
00008     S = (N*Y+M)/(X+Y);
00009     printf("Resultado = %d\n", N*X<M?N:S);
00010     return 0;
00011 }

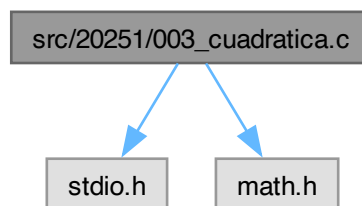
```

7.6 src/20251/003_cuadratica.c File Reference

```
#include <stdio.h>
```

```
#include <math.h>
```

Include dependency graph for 003_cuadratica.c:



Functions

- int `main` (int argc, char *argv[])

7.6.1 Function Documentation

7.6.1.1 main()

```

int main (
    int argc,
    char * argv[] )

```

Definition at line 4 of file 003_cuadratica.c.

```

00005 {
00006     float a, b, c, x1r, x1i, x2r, x2i, r;
00007     printf("Ingresa el cuadratico: ");
00008     scanf("%f", &a);
00009     printf("Ingresa el lineal: ");
00010     scanf("%f", &b);
00011     printf("Ingresa el independiente: ");
00012     scanf("%f", &c);
00013     r = b*b-4*a*c;
00014     x1r = (-b+(r>=0?sqrt(r):0))/(2*a);
00015     x1i = r>=0?0:sqrt(-r)/(2*a);
00016     x2r = (-b+(r>=0?-sqrt(r):0))/(2*a);
00017     x2i = r>=0?0:-sqrt(-r)/(2*a);
00018     printf("X1 = %f%fi\n", x1r, x1i);
00019     printf("X2 = %f%fi\n", x2r, x2i);
00020     return 0;
00021 }

```

7.7 003_cuadratica.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
```

```

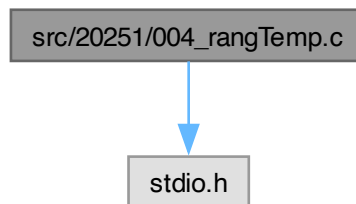
00002 #include <math.h>
00003
00004 int main(int argc, char *argv[])
00005 {
00006     float a, b, c, x1r, x1i, x2r, x2i, r;
00007     printf("Ingresa el cuadrático: ");
00008     scanf("%f", &a);
00009     printf("Ingresa el lineal: ");
00010     scanf("%f", &b);
00011     printf("Ingresa el independiente: ");
00012     scanf("%f", &c);
00013     r = b*b-4*a*c;
00014     x1r = (-b+(r>=0?sqrt(r):0))/(2*a);
00015     x1i = r>=0?0:sqrt(-r)/(2*a);
00016     x2r = (-b+(r>=0?-sqrt(r):0))/(2*a);
00017     x2i = r>=0?0:-sqrt(-r)/(2*a);
00018     printf("X1 = %f%+fi\n", x1r, x1i);
00019     printf("X2 = %f%+fi\n", x2r, x2i);
00020     return 0;
00021 }

```

7.8 src/20251/004_rangTemp.c File Reference

#include <stdio.h>

Include dependency graph for 004_rangTemp.c:



Functions

- int `main` (int argc, char *argv[])

7.8.1 Function Documentation

7.8.1.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 3 of file [004_rangTemp.c](#).

```

00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     printf("Ingresa los rangos de temperatura: ");
00008     scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00009     printf("Ingresa la temperatura: ");
00010     scanf("%f", &T);
00011     B = ((T4>T)<<3)|((T3>T)<<2)|((T2>T)<<1)|(T1>T);
00012     printf("%d\n", B);
00013     switch(B)
00014     {
00015     case 15:
00016         printf("Muy Baja\n");
00017         break;
00018     case 14:
00019         printf("Baja\n");

```



```

00020         break;
00021     case 12:
00022         printf("Templada\n");
00023         break;
00024     case 0:
00025         printf("Muy Alta\n");
00026         break;
00027     default:
00028         printf("Alta");
00029     }
00030     return 0;
00031 }

```

7.9 004_rangTemp.c

[Go to the documentation of this file.](#)

```

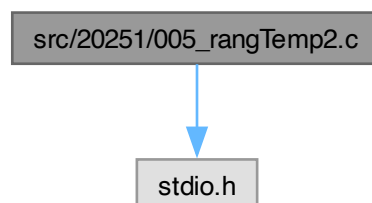
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     printf("Ingrese los rangos de temperatura: ");
00008     scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00009     printf("Ingrese la temperatura: ");
00010     scanf("%f", &T);
00011     B = ((T4>T)<3) | ((T3>T)<2) | ((T2>T)<1) | (T1>T);
00012     printf("%d\n", B);
00013     switch(B)
00014     {
00015     case 15:
00016         printf("Muy Baja\n");
00017         break;
00018     case 14:
00019         printf("Baja\n");
00020         break;
00021     case 12:
00022         printf("Templada\n");
00023         break;
00024     case 0:
00025         printf("Muy Alta\n");
00026         break;
00027     default:
00028         printf("Alta");
00029     }
00030     return 0;
00031 }

```

7.10 src/20251/005_rangTemp2.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 005_rangTemp2.c:



Functions

- int [main](#) (int argc, char *argv[])

7.10.1 Function Documentation

7.10.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file 005_rangTemp2.c.

```
00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     printf("Ingrese los rangos de temperatura: ");
00008     scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00009     printf("Ingrese la temperatura: ");
00010     scanf("%f", &T);
00011     B = ((T4>T)<<3)|((T3>T)<<2)|((T2>T)<<1)|(T1>T);
00012     printf("%d\n", B);
00013     switch(B)
00014     {
00015     case 15:
00016         printf("Muy ");
00017     case 14:
00018         printf("Baja\n");
00019         break;
00020     case 12:
00021         printf("Templada\n");
00022         break;
00023     case 0:
00024         printf("Muy ");
00025     default:
00026         printf("Alta\n");
00027     }
00028     return 0;
00029 }
```

7.11 005_rangTemp2.c

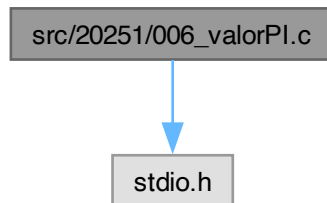
[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     printf("Ingrese los rangos de temperatura: ");
00008     scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00009     printf("Ingrese la temperatura: ");
00010     scanf("%f", &T);
00011     B = ((T4>T)<<3)|((T3>T)<<2)|((T2>T)<<1)|(T1>T);
00012     printf("%d\n", B);
00013     switch(B)
00014     {
00015     case 15:
00016         printf("Muy ");
00017     case 14:
00018         printf("Baja\n");
00019         break;
00020     case 12:
00021         printf("Templada\n");
00022         break;
00023     case 0:
00024         printf("Muy ");
00025     default:
00026         printf("Alta\n");
00027     }
00028     return 0;
00029 }
```

7.12 src/20251/006_valorPI.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 006_valorPI.c:



Functions

- `int main (int argc, char *argv[])`

7.12.1 Function Documentation

7.12.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [006_valorPI.c](#).

```

00004 {
00005     float pi;
00006     int i, n, den, sg;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<0);
00011     for(i=0, pi=1; i<n; i++)
00012     {
00013         den = 2*i+3;
00014         sg = 2*(i%2)-1;
00015         pi+=(sg/(float)den);
00016     }
00017     pi*=4;
00018     printf("PI = %f\n", pi);
00019     return 0;
00020 }
```

7.13 006_valorPI.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float pi;
00006     int i, n, den, sg;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<0);
00011     for(i=0, pi=1; i<n; i++)
00012     {
00013         den = 2*i+3;
00014         sg = 2*(i%2)-1;
```

```

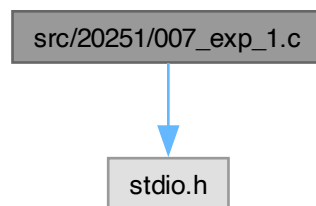
00015         pi+=(sg/(float)den);
00016     }
00017     pi*=4;
00018     printf("PI = %f\n", pi);
00019     return 0;
00020 }

```

7.14 src/20251/007_exp_1.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 007_exp_1.c:



Functions

- int [main](#) (int argc, char *argv[])

7.14.1 Function Documentation

7.14.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file 007_exp_1.c.

```

00004 {
00005     float ex, x, num;
00006     long int i, j, n, den;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0; i<n; i++)
00014     {
00015         for(j=0, num=1, den=1; j<i; j++)
00016         {
00017             num*=x;
00018             // for(j=0, den=1; j<i; j++)
00019                 den*=(j+1); //den=den*(j+1);
00020         }
00021         ex += num/den;
00022     }
00023     printf("exp(%f) = %f\n", x, ex);
00024     return 0;
00025 }

```

7.15 007_exp_1.c

[Go to the documentation of this file.](#)

```

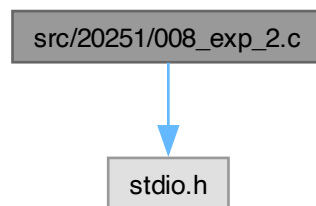
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float ex, x, num;
00006     long int i, j, n, den;
00007     printf("Ingresa el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingresa el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0; i<n; i++)
00014     {
00015         for(j=0, num=1, den=1; j<i; j++)
00016         {
00017             num*=x;
00018             // for(j=0, den=1; j<i; j++)
00019             den*=(j+1); //den=den*(j+1);
00020         }
00021         ex += num/den;
00022     }
00023     printf("exp(%f) = %f\n", x, ex);
00024     return 0;
00025 }

```

7.16 src/20251/008_exp_2.c File Reference

#include <stdio.h>

Include dependency graph for 008_exp_2.c:



Functions

- int [main](#) (int argc, char *argv[])

7.16.1 Function Documentation

7.16.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [008_exp_2.c](#).

```

00004 {
00005     float ex, x, fct;
00006     long int i, j, n;
00007     printf("Ingresa el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingresa el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0; i<n; i++)
00014     {

```

```

00015         for(j=0, fct=1; j<i; j++)
00016             fct *= x/(j+1);
00017         ex += fct;
00018     }
00019     printf("exp(%f) = %f\n", x, ex);
00020     return 0;
00021 }

```

Here is the call graph for this function:



7.17 008_exp_2.c

[Go to the documentation of this file.](#)

```

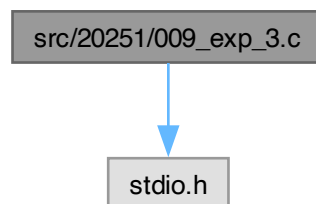
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float ex, x, fct;
00006     long int i, j, n;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0; i<n; i++)
00014     {
00015         for(j=0, fct=1; j<i; j++)
00016             fct *= x/(j+1);
00017         ex += fct;
00018     }
00019     printf("exp(%f) = %f\n", x, ex);
00020     return 0;
00021 }

```

7.18 src/20251/009_exp_3.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 009_exp_3.c:



Functions

- `int main (int argc, char *argv[])`

7.18.1 Function Documentation

7.18.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file 009_exp_3.c.

```
00004 {
00005     float ex, x, fct;
00006     long int i, n;
00007     printf("Ingresa el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingresa el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0, fct=1; i<n; i++)
00014     {
00015         ex += fct;
00016         fct *= x/(i+1);
00017     }
00018     printf("exp(%f) = %f\n", x, ex);
00019     return 0;
00020 }
```

Here is the call graph for this function:



7.19 009_exp_3.c

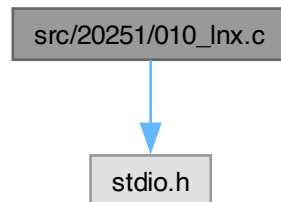
[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float ex, x, fct;
00006     long int i, n;
00007     printf("Ingresa el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingresa el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0, fct=1; i<n; i++)
00014     {
00015         ex += fct;
00016         fct *= x/(i+1);
00017     }
00018     printf("exp(%f) = %f\n", x, ex);
00019     return 0;
00020 }
```

7.20 src/20251/010_Inx.c File Reference

#include <stdio.h>

Include dependency graph for 010_Inx.c:



Functions

- int [main](#) (int argc, char *argv[])

7.20.1 Function Documentation

7.20.1.1 main()

```

int main (
    int argc,
    char * argv[])
Definition at line 3 of file 010_Inx.c.
00004 {
00005     float lnx, x, num, fct, px;
00006     int i, den, n;
00007     do{
00008         printf("Intese el valor de x: ");
00009         scanf("%f", &x);
00010     }while(x<=0);
00011     do{
00012         printf("Ingrese el numero de terminos: ");
00013         scanf("%d", &n);
00014     }while(n<=0);
00015     for(i=0, px=(x-1)/(x+1), fct=px, lnx=0; i<n; i++)
00016     {
00017         lnx += fct;
00018         fct*= (2*i+1)*(px*px)/(2*i+3);
00019     }
00020     lnx*=2;
00021     printf("ln(%f) = %f\n", x, lnx);
00022     return 0;
00023 }
00024 }
  
```

Here is the call graph for this function:



7.21 010_Inx.c

[Go to the documentation of this file.](#)

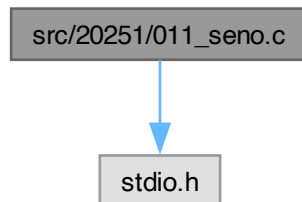
```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float lnx, x, num, fct, px;
00006     int i, den, n;
00007     do{
00008         printf("Intese el valor de x: ");
00009         scanf("%f", &x);
00010     }while(x<=0);
00011     do{
00012         printf("Ingrese el numero de terminos: ");
00013         scanf("%d", &n);
00014     }while(n<=0);
00015     for(i=0, px=(x-1)/(x+1), fct=px, lnx=0; i<n; i++)
00016     {
00017         lnx += fct;
00018         fct*= (2*i+1)*(px*px)/(2*i+3);
00019     }
00020     lnx*=2;
00021     printf("ln(%f) = %f\n", x, lnx);
00022     return 0;
00023 }
00024 }
```

7.22 src/20251/011_seno.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 011_seno.c:



Macros

- #define [PI2](#) 2*3.1415926535897932

Functions

- int [main](#) (int argc, char *argv[])

7.22.1 Macro Definition Documentation

7.22.1.1 PI2

```
#define PI2 2*3.1415926535897932
```

Definition at line 3 of file [011_seno.c](#).

7.22.2 Function Documentation

7.22.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 5 of file 011_seno.c.

```
00006 {
00007     float x, sx, fct;
00008     int n, i;
00009     printf("Ingrese el valor de x: ");
00010     scanf("%f", &x);
00011     x-=((int) (x/(PI2)))*PI2;
00012     do{
00013         printf("Ingrese el numero de terminos: ");
00014         scanf("%d", &n);
00015     }while(n<1);
00016     for(i=0, sx=0, fct=x; i<n; i++)
00017     {
00018         sx += (1-2*(i%2))*fct;
00019         fct *= (x/(2*i+2)) * (x/(2*i+3));
00020     }
00021     printf("sin(%f) = %f\n", x, sx);
00022     return 0;
00023 }
```

Here is the call graph for this function:



7.23 011_seno.c

[Go to the documentation of this file.](#)

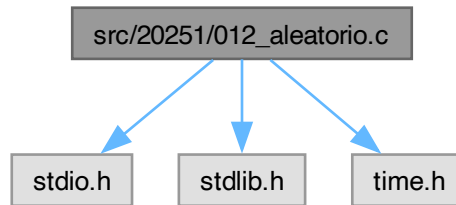
```
00001 #include <stdio.h>
00002
00003 #define PI2 2*3.1415926535897932
00004
00005 int main(int argc, char *argv[])
00006 {
00007     float x, sx, fct;
00008     int n, i;
00009     printf("Ingrese el valor de x: ");
00010     scanf("%f", &x);
00011     x-=((int) (x/(PI2)))*PI2;
00012     do{
00013         printf("Ingrese el numero de terminos: ");
00014         scanf("%d", &n);
00015     }while(n<1);
00016     for(i=0, sx=0, fct=x; i<n; i++)
00017     {
00018         sx += (1-2*(i%2))*fct;
00019         fct *= (x/(2*i+2)) * (x/(2*i+3));
00020     }
00021     printf("sin(%f) = %f\n", x, sx);
00022     return 0;
00023 }
```

7.24 src/20251/012_aleatorio.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <time.h>
```

Include dependency graph for 012_aleatorio.c:



Macros

- `#define N 100`
- `#define max 20`
- `#define min -10`

Functions

- `int main (int argc, char *argv[])`

7.24.1 Macro Definition Documentation

7.24.1.1 max

```
#define max 20
```

Definition at line 6 of file 012_aleatorio.c.

7.24.1.2 min

```
#define min -10
```

Definition at line 7 of file 012_aleatorio.c.

7.24.1.3 N

```
#define N 100
```

Definition at line 5 of file 012_aleatorio.c.

7.24.2 Function Documentation

7.24.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 9 of file 012_aleatorio.c.

```

00010 {
00011     int A[N], i;
00012     float X[N];
00013     srand(time(NULL));
00014     for(i=0; i<N; i++)
00015     {
00016         A[i] = rand()%(max-min)+min;
00017         X[i] = (max-min)*((float)rand())/RAND_MAX+min;
  
```

```

00018         printf("A[%d] = %d\n", i, A[i]);
00019         printf("X[%d] = %f\n", i, X[i]);
00020     }
00021     return 0;
00022 }

```

7.25 012_aleatorio.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 100
00006 #define max 20
00007 #define min -10
00008
00009 int main(int argc, char *argv[])
00010 {
00011     int A[N], i;
00012     float X[N];
00013     srand(time(NULL));
00014     for(i=0; i<N; i++)
00015     {
00016         A[i] = rand()%(max-min)+min;
00017         X[i] = (max-min)*((float)rand())/RAND_MAX+min;
00018         printf("A[%d] = %d\n", i, A[i]);
00019         printf("X[%d] = %f\n", i, X[i]);
00020     }
00021     return 0;
00022 }

```

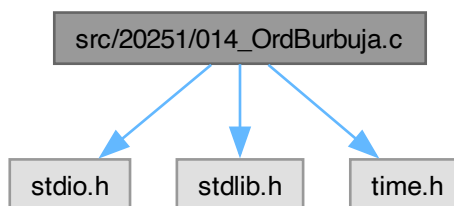
7.26 src/20251/014_OrdBurbuja.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 014_OrdBurbuja.c:



Macros

- #define N 1000000

Functions

- int [main](#) (int argc, char *argv[])

7.26.1 Macro Definition Documentation

7.26.1.1 N

#define N 1000000

Definition at line 5 of file 014_OrdBurbuja.c.

7.26.2 Function Documentation

7.26.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file 014_OrdBurbuja.c.

```
00008 {
00009     long int n, i, j;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=0; i<n-1; i++)
00034         for(j=i+1; j<n; j++)
00035             if(X[i]>X[j])
00036             {
00037                 X[i]*=X[j];
00038                 X[j]=X[i]/X[j];
00039                 X[i]/=X[j];
00040             }
00041     printf("Ordenados.\n");
00042     for(i=0; i<n; i++)
00043         printf("X[%ld] = %f\n", i+1, X[i]);
00044     return 0;
00045 }
```

7.27 014_OrdBurbuja.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
```

```

00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=0; i<n-1; i++)
00034         for(j=i+1; j<n; j++)
00035             if(X[i]>X[j])
00036             {
00037                 X[i]*=X[j];
00038                 X[j]=X[i]/X[j];
00039                 X[i]/=X[j];
00040             }
00041     printf("Ordenados.\n");
00042     for(i=0; i<n; i++)
00043         printf("X[%ld] = %f\n", i+1, X[i]);
00044     return 0;
00045 }

```

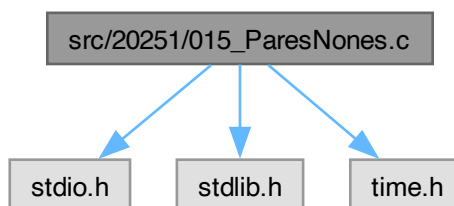
7.28 src/20251/015_ParesNones.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 015_ParesNones.c:



Macros

- #define [N](#) 1000000

Functions

- int [main](#) (int argc, char *argv[])

7.28.1 Macro Definition Documentation

7.28.1.1 [N](#)

```
#define N 1000000
```

Definition at line 5 of file [015_ParesNones.c](#).

7.28.2 Function Documentation

7.28.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file 015_ParesNones.c.

```
00008 {
00009     long int n, i, j;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while (n<2 || n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(j=0; j<n; j++)
00034     {
00035         for(i=j%2; i<(n+j%2)/2; i++)
00036             if(X[2*i-j%2]>X[2*i+(j+1)%2])
00037             {
00038                 X[2*i-j%2]*=X[2*i+(j+1)%2];
00039                 X[2*i+(j+1)%2]=X[2*i-j%2]/X[2*i+(j+1)%2];
00040                 X[2*i-j%2]/=X[2*i+(j+1)%2];
00041             }
00042     /*     for(i=0; i<n/2; i++)
00043             if(X[2*i]>X[2*i+1])
00044             {
00045                 X[2*i]*=X[2*i+1];
00046                 X[2*i+1]=X[2*i+1]/X[2*i];
00047                 X[2*i]/=X[2*i+1];
00048             }
00049     for(i=1; i<(n+1)/2; i++)
00050             if(X[2*i-1]>X[2*i])
00051             {
00052                 X[2*i-1]*=X[2*i];
00053                 X[2*i]=X[2*i-1]/X[2*i];
00054                 X[2*i-1]/=X[2*i];
00055             }*/
00056     }
00057     printf("Ordenados.\n");
00058     for(i=0; i<n; i++)
00059         printf("X[%ld] = %f\n", i+1, X[i]);
00060     return 0;
00061 }
00062 }
```

7.29 015_ParesNones.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
```

```

00015     }while (n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(j=0; j<n; j++)
00034     {
00035         for(i=j%2; i<(n+j%2)/2; i++)
00036             if(X[2*i-j%2]>X[2*i+(j+1)%2])
00037             {
00038                 X[2*i-j%2]*=X[2*i+(j+1)%2];
00039                 X[2*i+(j+1)%2]=X[2*i-j%2]/X[2*i+(j+1)%2];
00040                 X[2*i-j%2]/=X[2*i+(j+1)%2];
00041             }
00042     /*     for(i=0; i<n/2; i++)
00043             if(X[2*i]>X[2*i+1])
00044             {
00045                 X[2*i]*=X[2*i+1];
00046                 X[2*i+1]=X[2*i+1]/X[2*i];
00047                 X[2*i]/=X[2*i+1];
00048             }
00049             for(i=1; i<(n+1)/2; i++)
00050                 if(X[2*i-1]>X[2*i])
00051                 {
00052                     X[2*i-1]*=X[2*i];
00053                     X[2*i]=X[2*i-1]/X[2*i];
00054                     X[2*i-1]/=X[2*i];
00055                 }*/
00056     }
00057     printf("Ordenados.\n");
00058     for(i=0; i<n; i++)
00059         printf("X[%ld] = %f\n", i+1, X[i]);
00060     return 0;
00061 }
00062 }

```

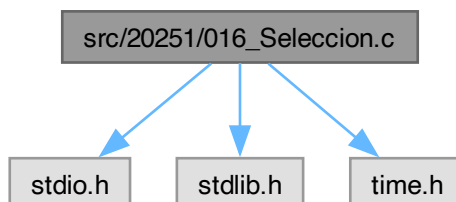
7.30 src/20251/016_Seleccion.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 016_Seleccion.c:



Macros

- #define N 1000000

Functions

- `int main` (`int argc`, `char *argv[]`)

7.30.1 Macro Definition Documentation

7.30.1.1 N

```
#define N 1000000
```

Definition at line 5 of file [016_Seleccion.c](#).

7.30.2 Function Documentation

7.30.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file [016_Seleccion.c](#).

```
00008 {
00009     long int n, i, j, aux_i;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while (n<2 || n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=0; i<n-1; i++)
00034     {
00035         for(j=i+1, aux_i=i; j<n; j++)
00036             if(X[j]<X[aux_i])
00037                 aux_i = j;
00038         if(i!=aux_i)
00039         {
00040             X[i] *= X[aux_i];
00041             X[aux_i] = X[i]/X[aux_i];
00042             X[i] /= X[aux_i];
00043         }
00044     }
00045     printf("Ordenados.\n");
00046     for(i=0; i<n; i++)
00047         printf("X[%ld] = %f\n", i+1, X[i]);
00048     return 0;
00049 }
00050 }
```

7.31 016_Seleccion.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j, aux_i;
```

```

00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=0; i<n-1; i++)
00034     {
00035         for(j=i+1, aux_i=i; j<n; j++)
00036             if(X[j]<X[aux_i])
00037                 aux_i = j;
00038         if(i!=aux_i)
00039         {
00040             X[i] *= X[aux_i];
00041             X[aux_i] = X[i]/X[aux_i];
00042             X[i] /= X[aux_i];
00043         }
00044     }
00045
00046     printf("Ordenados.\n");
00047     for(i=0; i<n; i++)
00048         printf("X[%ld] = %f\n", i+1, X[i]);
00049     return 0;
00050 }

```

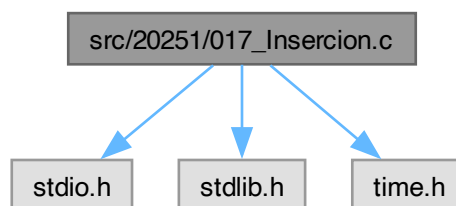
7.32 src/20251/017_Insercion.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 017_Insercion.c:



Macros

- #define `N` 1000000

Functions

- int `main` (int argc, char *argv[])

7.32.1 Macro Definition Documentation

7.32.1.1 N

#define N 1000000

Definition at line 5 of file 017_Insercion.c.

7.32.2 Function Documentation

7.32.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file 017_Insercion.c.

```
00008 {
00009     long int n, i, j;
00010     float X[N], max, min, aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=1; i<n; i++)
00034     {
00035         aux = X[i];
00036         j = i-1;
00037         while(X[j]>aux)
00038         {
00039             X[j+1] = X[j];
00040             j--;
00041             if(j<0)
00042                 break;
00043         }
00044         X[j+1]=aux;
00045     }
00046     printf("Ordenados.\n");
00047     for(i=0; i<n; i++)
00048         printf("X[%ld] = %f\n", i+1, X[i]);
00049     return 0;
00050 }
00051 }
```

7.33 017_Insercion.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j;
00010     float X[N], max, min, aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
```

```

00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=1; i<n; i++)
00034     {
00035         aux = X[i];
00036         j = i-1;
00037         while (X[j]>aux)
00038         {
00039             X[j+1] = X[j];
00040             j--;
00041             if(j<0)
00042                 break;
00043         }
00044         X[j+1]=aux;
00045     }
00046     printf("Ordenados.\n");
00047     for(i=0; i<n; i++)
00048         printf("X[%ld] = %f\n", i+1, X[i]);
00049     return 0;
00050 }
00051 }

```

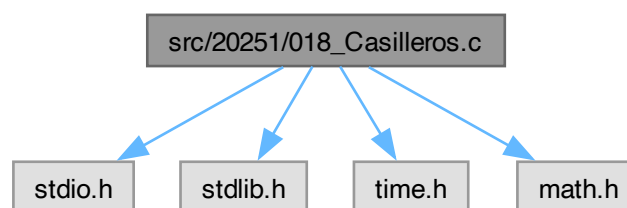
7.34 src/20251/018_Casilleros.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

```

Include dependency graph for 018_Casilleros.c:



Macros

- #define **N** 1000000
- #define **M** 1000

Functions

- int **main** (int argc, char *argv[])

7.34.1 Macro Definition Documentation

7.34.1.1 M

```
#define M 1000
```

Definition at line 7 of file [018_Casilleros.c](#).

7.34.1.2 N

```
#define N 1000000
```

Definition at line 6 of file [018_Casilleros.c](#).

7.34.2 Function Documentation

7.34.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 9 of file [018_Casilleros.c](#).

```
00010 {
00011     long int n, m, i, j, k, car[M], ind[M], ind2;
00012     float X[N], max, min, aux, rg[M], rng_int, Y[N];
00013     srand(time(NULL));
00014     do{
00015         printf("Ingrese el numero de elementos: ");
00016         scanf("%ld", &n);
00017     }while(n<2||n>N);
00018     printf("Ingrese le valor maximo: ");
00019     scanf("%f", &max);
00020     printf("Ingrese el valor minimo: ");
00021     scanf("%f", &min);
00022     if(max<min)
00023     {
00024         max*=min;
00025         min=max/min;
00026         max/=min;
00027     }
00028     printf("[%f, %f]\n", min, max);
00029     printf("Desordenados.\n");
00030     for(i=0; i<n; i++)
00031     {
00032         X[i] = ((max-min)*rand())/RAND_MAX+min;
00033         printf("X[%ld] = %f\n", i+1, X[i]);
00034     }
00035     m = sqrt(n);
00036     printf("Casilleros: %ld\n", m);
00037     rng_int = ceil((max-min)/m);
00038     rg[0] = min;
00039     car[0] = 0;
00040     for(i=1; i<m+1; i++)
00041     {
00042         rg[i] = rg[i-1]+rng_int;
00043         car[i] = 0;
00044     }
00045     for(i=0; i<n; i++)
00046         car[(int)((X[i]-min)/rng_int)]++;
00047     ind[0]=0;
00048     for(i=1; i<m; i++)
00049         ind[i]=ind[i-1]+car[i-1];
00050     for(i=0; i<m; i++)
00051         printf("#[%f, %f] = %ld\t%ld\n", rg[i], rg[i+1], car[i], ind[i]);
00052     for(i=0; i<n; i++)
00053     {
00054         ind2 = (int)((X[i]-min)/rng_int);
00055         Y[ind[ind2]] = X[i];
00056         ind[ind2]++;
00057     }
00058     printf("Buket.\n");
00059     for(i=0; i<n; i++)
00060         printf("Y[%ld] = %f\n", i+1, Y[i]);
00061     for(k=0, ind2 = 0; k<m; k++)
00062     {
00063         for(i=ind2; i<(ind2+car[k]-1) ; i++)
00064             for(j=i+1; j<(ind2+car[k]); j++)
00065                 if(Y[i]>Y[j])
00066                 {
00067                     Y[i]*=Y[j];
00068                     Y[j]=Y[i]/Y[j];
```

```

00069             Y[i]/=Y[j];
00070         }
00071         ind2 += car[k];
00072     }
00073
00074
00075     printf("Ordenados.\n");
00076     for(i=0; i<n; i++)
00077         printf("Y[%ld] = %f\n", i+1, Y[i]);
00078     return 0;
00079 }

```

7.35 018_Casilleros.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004 #include <math.h>
00005
00006 #define N 1000000
00007 #define M 1000
00008
00009 int main(int argc, char *argv[])
00010 {
00011     long int n, m, i, j, k, car[M], ind[M], ind2;
00012     float X[N], max, min, aux, rg[M], rng_int, Y[N];
00013     srand(time(NULL));
00014     do{
00015         printf("Ingrese el numero de elementos: ");
00016         scanf("%ld", &n);
00017     }while(n<2||n>N);
00018     printf("Ingrese le valor maximo: ");
00019     scanf("%f", &max);
00020     printf("Ingrese el valor minimo: ");
00021     scanf("%f", &min);
00022     if(max<min)
00023     {
00024         max*=min;
00025         min=max/min;
00026         max/=min;
00027     }
00028     printf("[%f, %f]\n", min, max);
00029     printf("Desordenados.\n");
00030     for(i=0; i<n; i++)
00031     {
00032         X[i] = ((max-min)*rand())/RAND_MAX+min;
00033         printf("X[%ld] = %f\n", i+1, X[i]);
00034     }
00035     m = sqrt(n);
00036     printf("Casilleros: %ld\n", m);
00037     rng_int = ceil((max-min)/m);
00038     rg[0] = min;
00039     car[0] = 0;
00040     for(i=1; i<m+1; i++)
00041     {
00042         rg[i] = rg[i-1]+rng_int;
00043         car[i] = 0;
00044     }
00045     for(i=0; i<n; i++)
00046         car[(int)((X[i]-min)/rng_int)]++;
00047     ind[0]=0;
00048     for(i=1; i<m; i++)
00049         ind[i]=ind[i-1]+car[i-1];
00050     for(i=0; i<m; i++)
00051         printf("#[%f, %f] = %ld\t%ld\n", rg[i], rg[i+1], car[i], ind[i]);
00052     for(i=0; i<n; i++)
00053     {
00054         ind2 = (int)((X[i]-min)/rng_int);
00055         Y[ind[ind2]] = X[i];
00056         ind[ind2]++;
00057     }
00058     printf("Buket.\n");
00059     for(i=0; i<n; i++)
00060         printf("Y[%ld] = %f\n", i+1, Y[i]);
00061     for(k=0, ind2 = 0; k<m; k++)
00062     {
00063         for(i=ind2; i<(ind2+car[k]-1) ; i++)
00064             for(j=i+1; j<(ind2+car[k]); j++)
00065                 if(Y[i]>Y[j])
00066                 {
00067                     Y[i]*=Y[j];
00068                     Y[j]=Y[i]/Y[j];
00069                     Y[i]/=Y[j];

```

```

00070         }
00071         ind2 += car[k];
00072     }
00073
00074
00075     printf("Ordenados.\n");
00076     for(i=0; i<n; i++)
00077         printf("Y[%ld] = %f\n", i+1, Y[i]);
00078     return 0;
00079 }

```

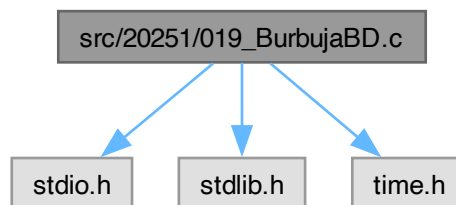
7.36 src/20251/019_BurbujaBD.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 019_BurbujaBD.c:



Macros

- `#define N 1000000`

Functions

- `int main (int argc, char *argv[])`

7.36.1 Macro Definition Documentation

7.36.1.1 N

```
#define N 1000000
```

Definition at line 5 of file 019_BurbujaBD.c.

7.36.2 Function Documentation

7.36.2.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 7 of file 019_BurbujaBD.c.

```

00008 {
00009     long int n, i, j, k;
00010     float X[N], max, min, aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);

```

```

00015     }while (n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033
00034     for(i=0, j=n-1; i<(n+1)/2; i++, j--)
00035     {
00036         for(k=i+1; k<n-i; k++)
00037         {
00038             if(X[i]>X[k])
00039             {
00040                 aux = X[i];
00041                 X[i] = X[k];
00042                 X[k] = aux;
00043             }
00044         }
00045         for(k=j-1; k>-1+i; k--)
00046         {
00047             if(X[j]<X[k])
00048             {
00049                 aux = X[j];
00050                 X[j] = X[k];
00051                 X[k] = aux;
00052             }
00053         }
00054     }
00055
00056     printf("Ordenados.\n");
00057     for(i=0; i<n; i++)
00058         printf("X[%ld] = %f\n", i+1, X[i]);
00059     return 0;
00060 }

```

7.37 019_BurbujaBD.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j, k;
00010     float X[N], max, min, aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while (n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033
00034     for(i=0, j=n-1; i<(n+1)/2; i++, j--)

```



```

00035     {
00036         for(k=i+1; k<n-i; k++)
00037         {
00038             if(X[i]>X[k])
00039             {
00040                 aux = X[i];
00041                 X[i] = X[k];
00042                 X[k] = aux;
00043             }
00044         }
00045         for(k=j-1; k>-1+i; k--)
00046         {
00047             if(X[j]<X[k])
00048             {
00049                 aux = X[j];
00050                 X[j] = X[k];
00051                 X[k] = aux;
00052             }
00053         }
00054     }
00055
00056     printf("Ordenados.\n");
00057     for(i=0; i<n; i++)
00058         printf("X[%ld] = %f\n", i+1, X[i]);
00059     return 0;
00060 }

```

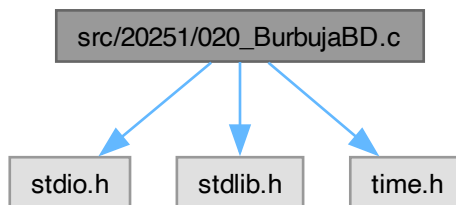
7.38 src/20251/020_BurbujaBD.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 020_BurbujaBD.c:



Macros

- `#define N 1000000`

Functions

- `int main (int argc, char *argv[ ])`

7.38.1 Macro Definition Documentation

7.38.1.1 N

```
#define N 1000000
```

Definition at line 5 of file 020_BurbujaBD.c.

7.38.2 Function Documentation

7.38.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file [020_BurbujaBD.c](#).

```
00008 {
00009     long int n, i, j, k;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for (i=0; i<n-1; i++)
00034         for(j=(i%2?n-1-(i+1)/2:i/2+1); (i%2?j>i/2:j<(n-i/2)); i%2?j--:j++)
00035         {
00036             k = i%2?n-(i+1)/2:i/2;
00037             if(i%2?X[k]<X[j]:X[k]>X[j])
00038             {
00039                 X[k]*=X[j];
00040                 X[j]=X[k]/X[j];
00041                 X[k]/=X[j];
00042             }
00043         }
00044     printf("Ordenados.\n");
00045     for(i=0; i<n; i++)
00046         printf("X[%ld] = %f\n", i+1, X[i]);
00047     return 0;
00048 }
00049 }
```

7.39 020_BurbujaBD.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j, k;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
```

```

00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033
00034     for (i=0; i<n-1; i++)
00035         for(j=(i%2?n-1-(i+1)/2:i/2+1); (i%2?j>i/2:j<(n-i/2)); i%2?j--:j++)
00036         {
00037             k = i%2?n-(i+1)/2:i/2;
00038             if(i%2?X[k]<X[j]:X[k]>X[j])
00039             {
00040                 X[k]*=X[j];
00041                 X[j]=X[k]/X[j];
00042                 X[k]/=X[j];
00043             }
00044         }
00045     printf("Ordenados.\n");
00046     for(i=0; i<n; i++)
00047         printf("X[%ld] = %f\n", i+1, X[i]);
00048     return 0;
00049 }

```

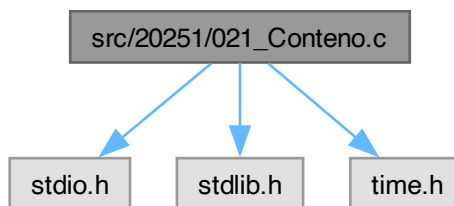
7.40 src/20251/021_Conteno.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 021_Conteno.c:



Macros

- `#define N 1000000`

Functions

- `int main (int argc, char *argv[])`

7.40.1 Macro Definition Documentation

7.40.1.1 N

```
#define N 1000000
```

Definition at line 5 of file 021_Conteno.c.

7.40.2 Function Documentation

7.40.2.1 main()

```
int main (
```

```

    int argc,
    char * argv[])

```

Definition at line 7 of file [021_Conteno.c](#).

```

00008 {
00009     int n, min, max, X[N], i, j, rg;
00010     do{
00011         printf("Ingrese el numero de elementos: ");
00012         scanf("%d", &n);
00013     }while(n<1||n>N);
00014     printf("Ingrese el valor maximo: ");
00015     scanf("%d", &max);
00016     printf("Ingrese el valor minimo: ");
00017     scanf("%d", &min);
00018     if(min>max)
00019     {
00020         max^=min;
00021         min^=max;
00022         max^=min;
00023     }
00024     rg = max-min;
00025     srand(time(NULL));
00026     for(i=0; i<n; i++)
00027         X[i] = (rand()%(max-min+1))+min;
00028     printf("Desordenado.\n");
00029     for(i=0; i<n; i++)
00030         printf("X[%d] = %d\n", i+1, X[i]);
00031     int h[rg+1];
00032     for(i=0; i<rg+1; i++)
00033         h[i] = 0;
00034     for(i=0; i<n; i++)
00035         h[X[i]-min]++;
00036     printf("Histograma\n");
00037     for(i=0; i<rg+1; i++)
00038         printf("h[%d] = %d\n", min+i, h[i]);
00039     for(i=0, j=0; i<n; )
00040     {
00041         while(!h[j])
00042             j++;
00043         while(h[j])
00044         {
00045             h[j]--;
00046             X[i++] = j+min;
00047         }
00048     }
00049     printf("Ordenado.\n");
00050     for(i=0; i<n; i++)
00051         printf("X[%d] = %d\n", i+1, X[i]);
00052     return 0;
00053 }
00054 }

```

7.41 021_Conteno.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     int n, min, max, X[N], i, j, rg;
00010     do{
00011         printf("Ingrese el numero de elementos: ");
00012         scanf("%d", &n);
00013     }while(n<1||n>N);
00014     printf("Ingrese el valor maximo: ");
00015     scanf("%d", &max);
00016     printf("Ingrese el valor minimo: ");
00017     scanf("%d", &min);
00018     if(min>max)
00019     {
00020         max^=min;
00021         min^=max;
00022         max^=min;
00023     }
00024     rg = max-min;
00025     srand(time(NULL));
00026     for(i=0; i<n; i++)
00027         X[i] = (rand()%(max-min+1))+min;
00028     printf("Desordenado.\n");

```

```

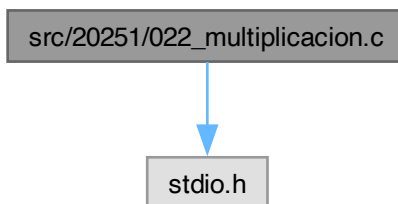
00029     for(i=0; i<n; i++)
00030         printf("X[%d] = %d\n", i+1, X[i]);
00031     int h[rg+1];
00032     for(i=0; i<rg+1; i++)
00033         h[i] = 0;
00034     for(i=0; i<n; i++)
00035         h[X[i]-min]++;
00036     printf("Histograma\n");
00037     for(i=0; i<rg+1; i++)
00038         printf("h[%d] = %d\n", min+i, h[i]);
00039     for(i=0, j=0; i<n; )
00040     {
00041         while(!h[j])
00042             j++;
00043         while(h[j])
00044         {
00045             h[j]--;
00046             X[i++] = j+min;
00047         }
00048     }
00049     printf("Ordenado.\n");
00050     for(i=0; i<n; i++)
00051         printf("X[%d] = %d\n", i+1, X[i]);
00052     return 0;
00053 }
00054 }

```

7.42 src/20251/022_multiplicacion.c File Reference

#include <stdio.h>

Include dependency graph for 022_multiplicacion.c:



Macros

- #define [Re](#) 100
- #define [Co](#) 100

Functions

- int [main](#) (int argc, char *argv[])

7.42.1 Macro Definition Documentation

7.42.1.1 Co

#define Co 100

Definition at line 4 of file [022_multiplicacion.c](#).

7.42.1.2 Re

#define Re 100

Definition at line 3 of file [022_multiplicacion.c](#).

7.42.2 Function Documentation

7.42.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 6 of file [022_multiplicacion.c](#).

```
00007 {
00008     int rA, cA, rB, cB, i, j, k;
00009     float A[Re][Co], B[Re][Co], C[Re][Co];
00010     do{
00011         printf("Ingrese el numero de renglones de A: ");
00012         scanf("%d", &rA);
00013     }while(rA<1||rA>Re);
00014     do{
00015         printf("Ingrese el numero de columnas de A: ");
00016         scanf("%d", &cA);
00017     }while(cA<1||cA>Co);
00018     for(i=0; i<cA; i++)
00019         for(j=0; j<rA; j++)
00020         {
00021             printf("A[%d][%d] = ", i+1, j+1);
00022             while(!scanf("%f", &A[i][j]));
00023         }
00024     do{
00025         printf("Ingrese el numero de renglones de B: ");
00026         scanf("%d", &rB);
00027     }while(rB<1||rB>Re);
00028     do{
00029         printf("Ingrese el numero de columnas de B: ");
00030         scanf("%d", &cB);
00031     }while(cB<1||cB>Co);
00032     for(i=0; i<cB; i++)
00033         for(j=0; j<rB; j++)
00034         {
00035             printf("B[%d][%d] = ", i+1, j+1);
00036             while(!scanf("%f", &B[i][j]));
00037         }
00038     if(cA!=rB)
00039     {
00040         printf("Error: La multiplicacion no puede realizarse\n");
00041         return 1;
00042     }
00043     for(i=0; i<rA; i++)
00044         for(j=0; j<cB; j++)
00045             for(k=0; C[i][j]=0; k<cA; k++)
00046                 C[i][j]+=A[i][k]*B[k][j];
00047     for(i=0; i<rA; i++)
00048         for(j=0; j<cB; j++)
00049             printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00050     return 0;
00051 }
```

7.43 022_multiplicacion.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 #define Re 100
00004 #define Co 100
00005
00006 int main(int argc, char *argv[])
00007 {
00008     int rA, cA, rB, cB, i, j, k;
00009     float A[Re][Co], B[Re][Co], C[Re][Co];
00010     do{
00011         printf("Ingrese el numero de renglones de A: ");
00012         scanf("%d", &rA);
00013     }while(rA<1||rA>Re);
00014     do{
00015         printf("Ingrese el numero de columnas de A: ");
00016         scanf("%d", &cA);
00017     }while(cA<1||cA>Co);
00018     for(i=0; i<cA; i++)
00019         for(j=0; j<rA; j++)
00020         {
00021             printf("A[%d][%d] = ", i+1, j+1);
00022             while(!scanf("%f", &A[i][j]));
00023         }
00024     do{
```

```

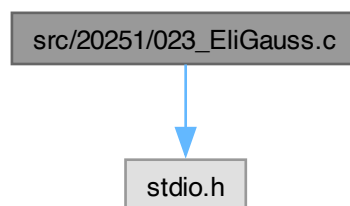
00025     printf("Ingrese el numero de renglones de B: ");
00026     scanf("%d", &rB);
00027 }while (rB<1||rB>Re);
00028 do{
00029     printf("Ingrese el numero de columnas de B: ");
00030     scanf("%d", &cB);
00031 }while (cB<1||cB>Co);
00032 for(i=0; i<cB; i++)
00033     for(j=0; j<rB; j++)
00034     {
00035         printf("B[%d][%d] = ", i+1, j+1);
00036         while(!scanf("%f", &B[i][j]));
00037     }
00038 if(cA!=rB)
00039 {
00040     printf("Error: La multiplicacion no puede realizarse\n");
00041     return 1;
00042 }
00043 for(i=0; i<rA; i++)
00044     for(j=0; j<cB; j++)
00045         for(k=0; C[i][j]=0; k<cA; k++)
00046             C[i][j]+=A[i][k]*B[k][j];
00047 for(i=0; i<rA; i++)
00048     for(j=0; j<cB; j++)
00049         printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00050 return 0;
00051 }

```

7.44 src/20251/023_EliGauss.c File Reference

#include <stdio.h>

Include dependency graph for 023_EliGauss.c:



Macros

- #define [NV](#) 100

Functions

- int [main](#) (int argc, char *argv[])

7.44.1 Macro Definition Documentation

7.44.1.1 NV

#define NV 100

Definition at line 3 of file [023_EliGauss.c](#).

7.44.2 Function Documentation

7.44.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 5 of file [023_EliGauss.c](#).

```
00006 {
00007     int n, i, j, k;
00008     float A[NV][NV], b[NV], fct, x[NV];
00009     do{
00010         printf("Ingrese el numero de variables: ");
00011         scanf("%d", &n);
00012     }while(n<1 || n>NV);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {
00017             printf("A[%d][%d] = ", i+1, j+1);
00018             scanf("%f", &A[i][j]);
00019         }
00020         printf("b[%d] = ", i+1);
00021         scanf("%f", &b[i]);
00022     }
00023     printf("---\n");
00024     for(i=0; i<n; i++)
00025     {
00026         for(j=0; j<n; j++)
00027             printf("%.2f\t", A[i][j]);
00028         printf("%.2f\n", b[i]);
00029     }
00030     for(i=1; i<n; i++)
00031     {
00032         fct = A[i][0]/A[0][0];
00033         for(j=0; j<n; j++)
00034             A[i][j] -= (fct*A[0][j]);
00035         b[i] -= fct*b[0];
00036     }
00037     printf("---\n");
00038     for(i=0; i<n; i++)
00039     {
00040         for(j=0; j<n; j++)
00041             printf("%.2f\t", A[i][j]);
00042         printf("%.2f\n", b[i]);
00043     }
00044     for(i=2; i<n; i++)
00045     {
00046         fct = A[i][1]/A[1][1];
00047         for(j=0; j<n; j++)
00048             A[i][j] -= (fct*A[1][j]);
00049         b[i] -= fct*b[1];
00050     }
00051     printf("---\n");
00052     for(i=0; i<n; i++)
00053     {
00054         for(j=0; j<n; j++)
00055             printf("%.2f\t", A[i][j]);
00056         printf("%.2f\n", b[i]);
00057     }
00058     for(i=0; i<n; i++)
00059         x[i] = 0;
00060     for(i=n-1; i>=0; i--)
00061     {
00062         x[i] = b[i];
00063         for(j=0; j<n; j++)
00064             if(i!=j)
00065                 x[i] -= A[i][j]*x[j];
00066         x[i] /= A[i][i];
00067         printf("x[%d] = %f\n", i+1, x[i]);
00068     }
00069     return 0;
00070 }
```


Here is the call graph for this function:



7.45 023_EliGauss.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define NV 100
00004
00005 int main(int argc, char *argv[])
00006 {
00007     int n, i, j, k;
00008     float A[NV][NV], b[NV], fct, x[NV];
00009     do{
00010         printf("Ingrese el numero de variables: ");
00011         scanf("%d", &n);
00012     }while(n<1||n>NV);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {
00017             printf("A[%d][%d] = ", i+1, j+1);
00018             scanf("%f", &A[i][j]);
00019         }
00020         printf("b[%d] = ", i+1);
00021         scanf("%f", &b[i]);
00022     }
00023     printf("---\n");
00024     for(i=0; i<n; i++)
00025     {
00026         for(j=0; j<n; j++)
00027             printf("%.2f\t", A[i][j]);
00028         printf("%.2f\n", b[i]);
00029     }
00030     for(i=1; i<n; i++)
00031     {
00032         fct = A[i][0]/A[0][0];
00033         for(j=0; j<n; j++)
00034             A[i][j] -= (fct*A[0][j]);
00035         b[i] -= fct*b[0];
00036     }
00037     printf("---\n");
00038     for(i=0; i<n; i++)
00039     {
00040         for(j=0; j<n; j++)
00041             printf("%.2f\t", A[i][j]);
00042         printf("%.2f\n", b[i]);
00043     }
00044     for(i=2; i<n; i++)
00045     {
00046         fct = A[i][1]/A[1][1];
00047         for(j=0; j<n; j++)
00048             A[i][j] -= (fct*A[1][j]);
00049         b[i] -= fct*b[1];
00050     }
00051     printf("---\n");
00052     for(i=0; i<n; i++)
00053     {
00054         for(j=0; j<n; j++)
00055             printf("%.2f\t", A[i][j]);
00056         printf("%.2f\n", b[i]);
00057     }
00058     for(i=0; i<n; i++)
00059         x[i] = 0;
00060     for(i=n-1; i>-1; i--)
00061     {

```

```

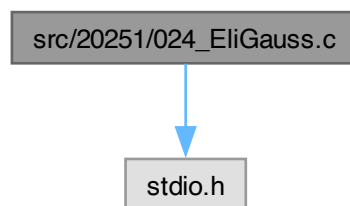
00062         x[i]=b[i];
00063         for(j=0; j<n; j++)
00064             if(i!=j)
00065                 x[i]-=A[i][j]*x[j];
00066         x[i]/=A[i][i];
00067         printf("x[%d] = %f\n", i+1, x[i]);
00068     }
00069     return 0;
00070 }

```

7.46 src/20251/024_EliGauss.c File Reference

#include <stdio.h>

Include dependency graph for 024_EliGauss.c:



Macros

- #define [NV](#) 100

Functions

- int [main](#) (int argc, char *argv[])

7.46.1 Macro Definition Documentation

7.46.1.1 NV

#define NV 100

Definition at line 3 of file [024_EliGauss.c](#).

7.46.2 Function Documentation

7.46.2.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 5 of file [024_EliGauss.c](#).

```

00006 {
00007     int n, i, j, k;
00008     float A[NV][NV], b[NV], fct, x[NV];
00009     do{
00010         printf("Ingrese el numero de variables: ");
00011         scanf("%d", &n);
00012     }while(n<1||n>NV);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {

```

```

00017         printf("A[%d][%d] = ", i+1, j+1);
00018         scanf("%f", &A[i][j]);
00019     }
00020     printf("b[%d] = ", i+1);
00021     scanf("%f", &b[i]);
00022 }
00023 printf("---\n");
00024 for(i=0; i<n; i++)
00025 {
00026     for(j=0; j<n; j++)
00027         printf("%.2f\t", A[i][j]);
00028     printf("%.2f\n", b[i]);
00029 }
00030 for(i=0; i<n-1; i++)
00031     for(j=i+1; j<n; j++)
00032     {
00033         fct = A[j][i]/A[i][i];
00034         for(k=0; k<n; k++)
00035             A[j][k] -= (fct*A[i][k]);
00036         b[j] -= fct*b[i];
00037     }
00038
00039 for(i=0; i<n; i++)
00040     x[i] = 0;
00041 for(i=n-1; i>-1; i--)
00042 {
00043     x[i]=b[i];
00044     for(j=0; j<n; j++)
00045         if(i!=j)
00046             x[i]-=A[i][j]*x[j];
00047     x[i]/=A[i][i];
00048     printf("x[%d] = %f\n", i+1, x[i]);
00049 }
00050 return 0;
00051 }

```

Here is the call graph for this function:



7.47 024_EliGauss.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define NV 100
00004
00005 int main(int argc, char *argv[])
00006 {
00007     int n, i, j, k;
00008     float A[NV][NV], b[NV], fct, x[NV];
00009     do{
00010         printf("Ingrese el numero de variables: ");
00011         scanf("%d", &n);
00012     }while(n<1||n>NV);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {
00017             printf("A[%d][%d] = ", i+1, j+1);
00018             scanf("%f", &A[i][j]);
00019         }
00020         printf("b[%d] = ", i+1);
00021         scanf("%f", &b[i]);
00022     }
00023     printf("---\n");
00024     for(i=0; i<n; i++)
00025     {

```

```

00026         for(j=0; j<n; j++)
00027             printf("%.2f\t", A[i][j]);
00028         printf("%.2f\n", b[i]);
00029     }
00030     for(i=0; i<n-1; i++)
00031         for(j=i+1; j<n; j++)
00032         {
00033             fct = A[j][i]/A[i][i];
00034             for(k=0; k<n; k++)
00035                 A[j][k] -= (fct*A[i][k]);
00036             b[j] -= fct*b[i];
00037         }
00038
00039     for(i=0; i<n; i++)
00040         x[i] = 0;
00041     for(i=n-1; i>=0; i--)
00042     {
00043         x[i]=b[i];
00044         for(j=0; j<n; j++)
00045             if(i!=j)
00046                 x[i]-=A[i][j]*x[j];
00047         x[i]/=A[i][i];
00048         printf("x[%d] = %f\n", i+1, x[i]);
00049     }
00050     return 0;
00051 }

```

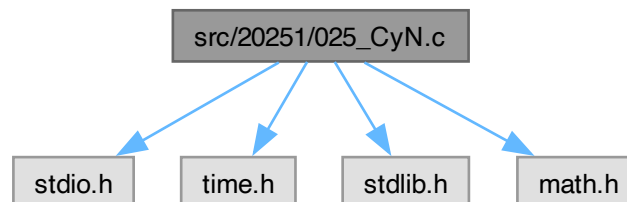
7.48 src/20251/025_CyN.c File Reference

```

#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include <math.h>

```

Include dependency graph for 025_CyN.c:



Macros

- `#define N 10000`

Functions

- `int main (int argc, char *argv[ ])`

7.48.1 Macro Definition Documentation

7.48.1.1 N

```
#define N 10000
```

Definition at line 6 of file 025_CyN.c.

7.48.2 Function Documentation

7.48.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 8 of file 025_CyN.c.

```
00009 {
00010     float X[N], min, max, xmin, xmax, xmed, xvar, xran;
00011     int n, i, flag, flag_e, flag_n, flag_c;
00012     do{
00013         printf("Ingrese el numero de muestra: ");
00014         scanf("%d", &n);
00015     }while(n<1);
00016     printf("Ingrese el valor minimo: ");
00017     scanf("%f", &min);
00018     printf("Ingrese le valor maximo: ");
00019     scanf("%f", &max);
00020     if(min>max)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     srand(time(NULL));
00027     for(i=0; i<n; i++)
00028     {
00029         X[i] = ((max-min)*rand())/RAND_MAX+min;
00030         printf("X[%d] = %f\n", i+1, X[i]);
00031     }
00032     printf("Normalización: ");
00033     scanf("%d", &flag_n);
00034     printf("Centrado: ");
00035     scanf("%d", &flag_c);
00036     printf("Estadístico: ");
00037     scanf("%d", &flag_e);
00038     flag = ((flag_c<1)|(flag_n)<1)|(flag_e<1);
00039     printf("flag = %d\n", flag);
00040     for(xmin=X[0], xmax=X[0], xmed=X[0], xvar=X[0]*X[0], i=1; i<n; i++)
00041     {
00042         if(X[i]<xmin)
00043             xmin = X[i];
00044         if(X[i]>xmax)
00045             xmax = X[i];
00046         xmed += X[i];
00047         xvar += X[i]*X[i];
00048     }
00049     xmed/=n;
00050     xvar/=n;
00051     xvar-=(xmed*xmed);
00052     xran=xmax-xmin;
00053     if((flag&1)|(flag&4))
00054         for(i=0; i<n; i++)
00055             X[i]-=(flag&4?xmed:xmin);
00056     if((flag&2)|(flag&8))
00057         for(i=0; i<n; i++)
00058             X[i]/=(flag&8?xvar:xran);
00059     printf("Maximo: %f\nMinimo: %f\nMedia: %f\nVarianza: %f\n",
00060         xmax, xmin, xmed, xvar);
00061     for(i=0; i<n; i++)
00062         printf("X[%d] = %f\n", i+1, X[i]);
00063     return 0;
00064 }
```

7.49 025_CyN.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <time.h>
00003 #include <stdlib.h>
00004 #include <math.h>
00005
00006 #define N 10000
00007
00008 int main(int argc, char *argv[])
00009 {
00010     float X[N], min, max, xmin, xmax, xmed, xvar, xran;
00011     int n, i, flag, flag_e, flag_n, flag_c;
00012     do{
00013         printf("Ingrese el numero de muestra: ");
```

```

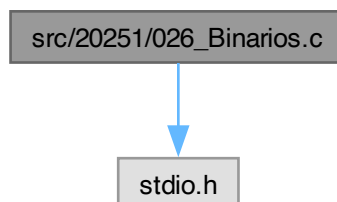
00014     scanf("%d", &n);
00015 }while(n<1);
00016 printf("Ingrese el valor minimo: ");
00017 scanf("%f", &min);
00018 printf("Ingrese le valor maximo: ");
00019 scanf("%f", &max);
00020 if(min>max)
00021 {
00022     max*=min;
00023     min=max/min;
00024     max/=min;
00025 }
00026 srand(time(NULL));
00027 for(i=0; i<n; i++)
00028 {
00029     X[i] = ((max-min)*rand())/RAND_MAX+min;
00030     printf("X[%d] = %f\n", i+1, X[i]);
00031 }
00032 printf("Normalización: ");
00033 scanf("%d", &flag_n);
00034 printf("Centrado: ");
00035 scanf("%d", &flag_c);
00036 printf("Estadístico: ");
00037 scanf("%d", &flag_e);
00038 flag = ((flag_c<1)|(flag_n<1)|(flag_e<1));
00039 printf("flag = %d\n", flag);
00040 for(xmin=X[0], xmax=X[0], xmed=X[0], xvar=X[0]*X[0], i=1; i<n; i++)
00041 {
00042     if(X[i]<xmin)
00043         xmin = X[i];
00044     if(X[i]>xmax)
00045         xmax = X[i];
00046     xmed += X[i];
00047     xvar += X[i]*X[i];
00048 }
00049 xmed/=n;
00050 xvar/=n;
00051 xvar-=xmed*xmed;
00052 xran=xmax-xmin;
00053 if((flag&1)|(flag&4))
00054     for(i=0; i<n; i++)
00055         X[i]-=(flag&4?xmed:xmin);
00056 if((flag&2)|(flag&8))
00057     for(i=0; i<n; i++)
00058         X[i]/=(flag&8?xvar:xran);
00059 printf("Maximo: %f\nMinimo: %f\nMedia: %f\nVarianza: %f\n",
00060        xmax, xmin, xmed, xvar);
00061 for(i=0; i<n; i++)
00062     printf("X[%d] = %f\n", i+1, X[i]);
00063 return 0;
00064 }

```

7.50 src/20251/026_Binarios.c File Reference

#include <stdio.h>

Include dependency graph for 026_Binarios.c:



Macros

- #define `BIT(n)`

- `#define BIT_GET(x, n)`
- `#define BIT_SET(x, n)`
- `#define BIT_CLEAR(x, n)`
- `#define BIT_TOGGLE(x, n)`
- `#define BIT_WRITE(x, n, v)`

Functions

- `int main (int argc, char *argv[])`

7.50.1 Macro Definition Documentation

7.50.1.1 BIT

```
#define BIT(  
    n)
```

Value:

`(1<<(n))`

Definition at line 3 of file [026_Binarios.c](#).

7.50.1.2 BIT_CLEAR

```
#define BIT_CLEAR(  
    x,  
    n)
```

Value:

`((x) &= ~BIT(n))`

Definition at line 6 of file [026_Binarios.c](#).

7.50.1.3 BIT_GET

```
#define BIT_GET(  
    x,  
    n)
```

Value:

`((x) & BIT(n))`

Definition at line 4 of file [026_Binarios.c](#).

7.50.1.4 BIT_SET

```
#define BIT_SET(  
    x,  
    n)
```

Value:

`((x) |= BIT(n))`

Definition at line 5 of file [026_Binarios.c](#).

7.50.1.5 BIT_TOGGLE

```
#define BIT_TOGGLE(  
    x,  
    n)
```

Value:

`((x) ^= BIT(n))`

Definition at line 7 of file [026_Binarios.c](#).

7.50.1.6 BIT_WRITE

```
#define BIT_WRITE(  
    x,  
    n,  
    v)
```

Value:

```
((v)?BIT_SET(x,n):BIT_CLEAR(x,n))
```

Definition at line 8 of file [026_Binarios.c](#).

7.50.2 Function Documentation

7.50.2.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 10 of file [026_Binarios.c](#).

```
00011 {  
00012     int i, n, nb;  
00013     int xi;  
00014     char x, xb;  
00015     printf("x = ");  
00016     scanf("%d", &xi);  
00017     x = (char)xi;  
00018     n = 8*sizeof(x);  
00019     for(i=n; i>0; i--)  
00020         printf("%d", BIT_GET(x,i-1)?1:0);  
00021     printf("\n");  
00022     do{  
00023         printf("Ingrese el numero de bit: ");  
00024         scanf("%d", &nb);  
00025     }while(nb<-1|nb>=n);  
00026     printf("%d\n", BIT_SET(x, nb));  
00027     do{  
00028         printf("Ingrese el numero de bit: ");  
00029         scanf("%d", &nb);  
00030     }while(nb<-1|nb>=n);  
00031     printf("%d\n", BIT_CLEAR(x, nb));  
00032     do{  
00033         printf("Ingrese el numero de bit: ");  
00034         scanf("%d", &nb);  
00035     }while(nb<-1|nb>=n);  
00036     printf("%d\n", BIT_TOGGLE(x, nb));  
00037     do{  
00038         printf("Ingrese el numero de bit: ");  
00039         scanf("%d", &nb);  
00040     }while(nb<-1|nb>=n);  
00041     printf("Ingrese el valor del bit: ");  
00042     scanf("%d", &xi);  
00043     xb = (char)xi;  
00044     printf("%d\n", BIT_WRITE(x, nb, xb));  
00045     return 0;  
00046 }
```

7.51 026_Binarios.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>  
00002  
00003 #define BIT(n)                (1<<(n))  
00004 #define BIT_GET(x,n)          ((x)&BIT(n))  
00005 #define BIT_SET(x,n)           ((x) |= BIT(n))  
00006 #define BIT_CLEAR(x,n)         ((x) &= ~BIT(n))  
00007 #define BIT_TOGGLE(x,n)        ((x) ^= BIT(n))  
00008 #define BIT_WRITE(x,n,v)       ((v)?BIT_SET(x,n):BIT_CLEAR(x,n))  
00009  
00010 int main(int argc, char *argv[])  
00011 {  
00012     int i, n, nb;  
00013     int xi;  
00014     char x, xb;  
00015     printf("x = ");  
00016     scanf("%d", &xi);  
00017     x = (char)xi;  
00018     n = 8*sizeof(x);
```



```

00019     for(i=n; i>0; i--)
00020         printf("%d", BIT_GET(x,i-1)?1:0);
00021     printf("\n");
00022     do{
00023         printf("Ingrese el numero de bit: ");
00024         scanf("%d", &nb);
00025     }while(nb<-1|nb>=n);
00026     printf("%d\n", BIT_SET(x, nb));
00027     do{
00028         printf("Ingrese el numero de bit: ");
00029         scanf("%d", &nb);
00030     }while(nb<-1|nb>=n);
00031     printf("%d\n", BIT_CLEAR(x, nb));
00032     do{
00033         printf("Ingrese el numero de bit: ");
00034         scanf("%d", &nb);
00035     }while(nb<-1|nb>=n);
00036     printf("%d\n", BIT_TOGGLE(x, nb));
00037     do{
00038         printf("Ingrese el numero de bit: ");
00039         scanf("%d", &nb);
00040     }while(nb<-1|nb>=n);
00041     printf("Ingrese el valor del bit: ");
00042     scanf("%d", &xi);
00043     xb = (char)xi;
00044     printf("%d\n", BIT_WRITE(x, nb, xb));
00045     return 0;
00046 }

```

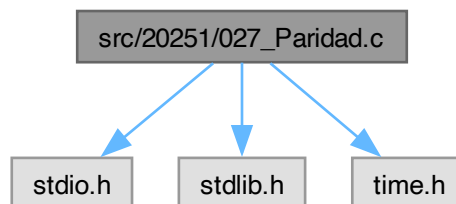
7.52 src/20251/027_Paridad.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 027_Paridad.c:



Macros

- #define `BIT(n)`
- #define `BIT_TOGGLE(x, n)`
- #define `NC 100`

Functions

- int `main` (int argc, char *argv[])

7.52.1 Macro Definition Documentation

7.52.1.1 BIT

```

#define BIT(
    n)

```

Value:`(1<<(n))`Definition at line 5 of file [027_Paridad.c](#).**7.52.1.2 BIT_TOGGLE**

```
#define BIT_TOGGLE(
    x,
    n)
```

Value:`((x) ^= BIT(n))`Definition at line 6 of file [027_Paridad.c](#).**7.52.1.3 NC**

```
#define NC 100
```

Definition at line 7 of file [027_Paridad.c](#).**7.52.2 Function Documentation****7.52.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 9 of file [027_Paridad.c](#).

```
00010 {
00011     char msg[NC+1], p, v;
00012     int i, nc, _nc, _nb;
00013     srand(time(NULL));
00014     i=0;
00015     do{
00016         msg[i++] = getc(stdin);
00017     }while(msg[i-1]!=10&& i<NC);
00018     nc = i-1;
00019     msg[nc] = '\0';
00020     for(i=0, p=0; i<nc; i++)
00021         p^=msg[i];
00022     for(i=0, v=p; i<nc; i++)
00023         v^=msg[i];
00024     printf("MSG: %s(%d)-%c-%d\n", msg, nc, p, (int )v);
00025     _nc = rand()%nc;
00026     _nb = rand()% (8*sizeof(char));
00027     BIT_TOGGLE(msg[_nc], _nb);
00028     for(i=0, v=p; i<nc; i++)
00029         v^=msg[i];
00030     printf("MSG: %s(%d)-%c-%u\n", msg, nc, p, (unsigned int)v);
00031     return 0;
00032 }
```

7.53 027_Paridad.c[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define BIT(n)          (1<<(n))
00006 #define BIT_TOGGLE(x,n) ((x) ^= BIT(n))
00007 #define NC              100
00008
00009 int main(int argc, char *argv[])
00010 {
00011     char msg[NC+1], p, v;
00012     int i, nc, _nc, _nb;
00013     srand(time(NULL));
00014     i=0;
00015     do{
00016         msg[i++] = getc(stdin);
00017     }while(msg[i-1]!=10&& i<NC);
00018     nc = i-1;
00019     msg[nc] = '\0';
```

```

00020     for(i=0, p=0; i<nc; i++)
00021         p^=msg[i];
00022     for(i=0, v=p; i<nc; i++)
00023         v^=msg[i];
00024     printf("MSG: %s(%d)-%c-%d\n", msg, nc, p, (int )v);
00025     _nc = rand()%nc;
00026     _nb = rand()%(8*sizeof(char));
00027     BIT_TOGGLE(msg[_nc], _nb);
00028     for(i=0, v=p; i<nc; i++)
00029         v^=msg[i];
00030     printf("MSG: %s(%d)-%c-%u\n", msg, nc, p, (unsigned int)v);
00031     return 0;
00032 }

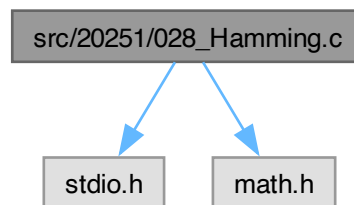
```

7.54 src/20251/028_Hamming.c File Reference

```
#include <stdio.h>
```

```
#include <math.h>
```

Include dependency graph for 028_Hamming.c:



Macros

- `#define MC 2048`
- `#define l2 log(2)`

Functions

- `int main (int argc, char *argv[ ])`

7.54.1 Macro Definition Documentation

7.54.1.1 l2

```
#define l2 log(2)
```

Definition at line 5 of file 028_Hamming.c.

7.54.1.2 MC

```
#define MC 2048
```

Definition at line 4 of file 028_Hamming.c.

7.54.2 Function Documentation

7.54.2.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 7 of file [028_Hamming.c](#).

```

00008 {
00009     char str[MC], strP[MC];
00010     int i, j, k, nc, nP;
00011     i=0;
00012     do{
00013         str[i++] = getc(stdin);
00014     }while(str[i-1] != 10);
00015     str[i-1] = '\0';
00016     nc = i-1;
00017     nP = log(nc+log(nc)/12+1)/12+1;
00018     printf("%s(%d, %d)\n", str, nc, nP);
00019     for(i=0, j=1, k=0; i<nc+nP; i++)
00020     {
00021         if((i+1) != j)
00022             strP[i] = str[k++];
00023         else
00024         {
00025             strP[i] = 0;
00026             j<<=1;
00027         }
00028     }
00029     for(i=0, j=1; i<nP; i++)
00030     {
00031         for(k=j; k<=(nc+nP); k++)
00032             if(((k-j)%(2*j))<j)
00033                 strP[j-1]^=strP[k-1];
00034         // printf("%d\t", ((k-j)%(2*j))<j);
00035         // printf("\n");
00036         j<<=1;
00037     }
00038     strP[nc+nP] = '\0';
00039     for(i=0; i<nc+nP; i++)
00040         printf("%d\n", strP[i]);
00041     printf("%s\n", strP);
00042     return 0;
00043 }

```

7.55 028_Hamming.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <math.h>
00003
00004 #define MC 2048
00005 #define L2 log(2)
00006
00007 int main(int argc, char *argv[])
00008 {
00009     char str[MC], strP[MC];
00010     int i, j, k, nc, nP;
00011     i=0;
00012     do{
00013         str[i++] = getc(stdin);
00014     }while(str[i-1] != 10);
00015     str[i-1] = '\0';
00016     nc = i-1;
00017     nP = log(nc+log(nc)/12+1)/12+1;
00018     printf("%s(%d, %d)\n", str, nc, nP);
00019     for(i=0, j=1, k=0; i<nc+nP; i++)
00020     {
00021         if((i+1) != j)
00022             strP[i] = str[k++];
00023         else
00024         {
00025             strP[i] = 0;
00026             j<<=1;
00027         }
00028     }
00029     for(i=0, j=1; i<nP; i++)
00030     {
00031         for(k=j; k<=(nc+nP); k++)
00032             if(((k-j)%(2*j))<j)
00033                 strP[j-1]^=strP[k-1];
00034         // printf("%d\t", ((k-j)%(2*j))<j);
00035         // printf("\n");
00036         j<<=1;
00037     }
00038     strP[nc+nP] = '\0';
00039     for(i=0; i<nc+nP; i++)
00040         printf("%d\n", strP[i]);
00041     printf("%s\n", strP);
00042     return 0;

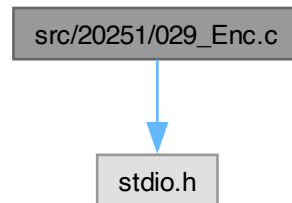
```

```
00043 }
```

7.56 src/20251/029_Enc.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 029_Enc.c:



Macros

- `#define MC 2048`

Functions

- `int main (int argc, char *argv[])`

7.56.1 Macro Definition Documentation

7.56.1.1 MC

```
#define MC 2048
```

Definition at line 3 of file 029_Enc.c.

7.56.2 Function Documentation

7.56.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 5 of file 029_Enc.c.

```
00006 {
00007     char str[MC], strE[MC], strD[MC], aux[MC];
00008     int i, nc, a, ai, b, n;
00009     a = 17;
00010     // ai = 49;
00011     b = 21;
00012     n = 52;
00013     i=0;
00014     do{
00015         str[i++] = getc(stdin);
00016     }while(str[i-1] != '\0');
00017     str[i-1] = '\0';
00018     nc = i-1;
00019     printf("MSG: %s\n", str);
00020     for(i=0; i<nc; i++)
00021         printf("%d ", str[i]);
00022     printf("\n");
00023     for(i=0; i<nc; i++)
00024     {
```

```

00025         aux[i] = str[i]-'A'-6*(str[i]>='a');
00026         printf("%d ", aux[i]);
00027     }
00028     printf("\n");
00029     for(i=0; i<nc; i++)
00030     {
00031         aux[i] = (a*((int)aux[i])+b)%n;
00032         printf("%d ", aux[i]);
00033     }
00034     printf("\n");
00035     for(i=0; i<nc; i++)
00036         strE[i] = aux[i]+'A'+6*(aux[i]>(n/2-1));
00037     strE[nc]='\0';
00038     printf("%s\n", strE);
00039     for(i=0; i<nc; i++)
00040     {
00041         aux[i] = strE[i]-'A'-6*(strE[i]>='a');
00042         printf("%d ", aux[i]);
00043     }
00044     printf("\n");
00045     ai=1;
00046     while(((ai*a)%n)!=1)
00047         ai++;
00048     printf("ai = %d\n", ai);
00049     for(i=0; i<nc; i++)
00050     {
00051         aux[i] = (ai*((aux[i]<b)*n+aux[i]-b))%n;
00052         printf("%d ", aux[i]);
00053     }
00054     printf("\n");
00055     for(i=0; i<nc; i++)
00056     {
00057         strD[i] = aux[i]+'A'+6*(aux[i]>25);
00058         printf("%d ", strD[i]);
00059     }
00060     strD[nc] = '\0';
00061     printf("\n");
00062     printf("%s\n", strD);
00063     return 0;
00064 }

```

7.57 029_Enc.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define MC 2048
00004
00005 int main(int argc, char *argv[])
00006 {
00007     char str[MC], strE[MC], strD[MC], aux[MC];
00008     int i, nc, a, ai, b, n;
00009     a = 17;
00010     // ai = 49;
00011     b = 21;
00012     n = 52;
00013     i=0;
00014     do{
00015         str[i++] = getc(stdin);
00016     }while(str[i-1]!=10);
00017     str[i-1] = '\0';
00018     nc = i-1;
00019     printf("MSG: %s\n", str);
00020     for(i=0; i<nc; i++)
00021         printf("%d ", str[i]);
00022     printf("\n");
00023     for(i=0; i<nc; i++)
00024     {
00025         aux[i] = str[i]-'A'-6*(str[i]>='a');
00026         printf("%d ", aux[i]);
00027     }
00028     printf("\n");
00029     for(i=0; i<nc; i++)
00030     {
00031         aux[i] = (a*((int)aux[i])+b)%n;
00032         printf("%d ", aux[i]);
00033     }
00034     printf("\n");
00035     for(i=0; i<nc; i++)
00036         strE[i] = aux[i]+'A'+6*(aux[i]>(n/2-1));
00037     strE[nc]='\0';
00038     printf("%s\n", strE);
00039     for(i=0; i<nc; i++)
00040     {

```

```

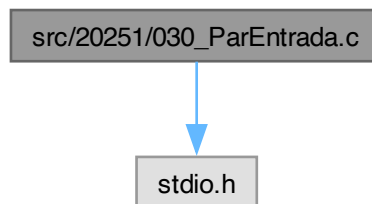
00041     aux[i] = strE[i]-'A'-6*(strE[i]>='a');
00042     printf("%d ", aux[i]);
00043 }
00044 printf("\n");
00045 ai=1;
00046 while((ai*a)%n!=1)
00047     ai++;
00048 printf("ai = %d\n", ai);
00049 for(i=0; i<nc; i++)
00050 {
00051     aux[i] = (ai*((aux[i]<b)*n+aux[i]-b))%n;
00052     printf("%d ", aux[i]);
00053 }
00054 printf("\n");
00055 for(i=0; i<nc; i++)
00056 {
00057     strD[i] = aux[i]+'A'+6*(aux[i]>25);
00058     printf("%d ", strD[i]);
00059 }
00060 strD[nc] = '\0';
00061 printf("\n");
00062 printf("%s\n", strD);
00063 return 0;
00064 }

```

7.58 src/20251/030_ParEntrada.c File Reference

#include <stdio.h>

Include dependency graph for 030_ParEntrada.c:



Functions

- int [main](#) (int argc, char *argv[])

7.58.1 Function Documentation

7.58.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [030_ParEntrada.c](#).

```

00004 {
00005     int i;
00006     printf("Numero de parametros: %d\n", argc);
00007     for(i=0; i<argc; i++)
00008         printf("%d. %s\n", i+1, argv[i]);
00009     return 0;
00010 }

```

7.59 030_ParEntrada.c

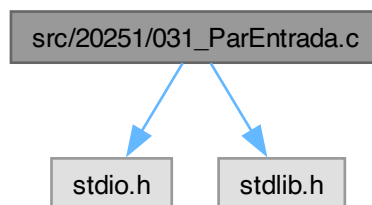
[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int i;
00006     printf("Numero de parametros: %d\n", argc);
00007     for(i=0; i<argc; i++)
00008         printf("%d. %s\n", i+1, argv[i]);
00009     return 0;
00010 }
```

7.60 src/20251/031_ParEntrada.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for 031_ParEntrada.c:



Functions

- int [main](#) (int argc, char *argv[])

7.60.1 Function Documentation

7.60.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 4 of file [031_ParEntrada.c](#).

```
00005 {
00006     int i, a, b, s;
00007     if(argc!=3)
00008     {
00009         printf("Uso: %s A B\n", argv[0]);
00010         return 1;
00011     }
00012     a = atoi(argv[1]);
00013     b = atoi(argv[2]);
00014     s = a + b;
00015     printf("%d+%d=%d\n", a, b, s);
00016     return 0;
00017 }
```

7.61 031_ParEntrada.c

[Go to the documentation of this file.](#)


```

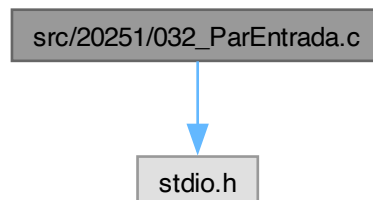
00001 #include<stdio.h>
00002 #include<stdlib.h>
00003
00004 int main(int argc, char *argv[])
00005 {
00006     int i, a, b, s;
00007     if(argc!=3)
00008     {
00009         printf("Uso: %s A B\n", argv[0]);
00010         return 1;
00011     }
00012     a = atoi(argv[1]);
00013     b = atoi(argv[2]);
00014     s = a + b;
00015     printf("%d+%d=%d\n", a, b, s);
00016     return 0;
00017 }

```

7.62 src/20251/032_ParEntrada.c File Reference

#include <stdio.h>

Include dependency graph for 032_ParEntrada.c:



Functions

- int [str2int](#) (char str[])
- float [str2float](#) (char str[])
- int [main](#) (int argc, char *argv[])

7.62.1 Function Documentation

7.62.1.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 34 of file [032_ParEntrada.c](#).

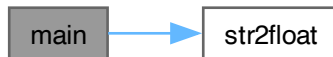
```

00035 {
00036     int i;
00037     float s, d;
00038     if(argc==1)
00039     {
00040         printf("Uso: %s A B\n", argv[0]);
00041         return 1;
00042     }
00043     for(i=1, s=0; i<argc; i++)
00044     {
00045         d = str2float(argv[i]);
00046         s += d;
00047         printf("%d-> %f\t%f\n", i, d, s);
00048     }
00049     printf("Suma = %f\n", s);
00050     return 0;

```

```
00051 }
```

Here is the call graph for this function:



7.62.1.2 str2float()

```
float str2float (
    char str[])
```

Definition at line 15 of file [032_ParEntrada.c](#).

```

00016 {
00017     float num = 0;
00018     int i = str[0]=='-'?1:0, p10=10;
00019     while(str[i]>='0' && str[i]<='9')
00020     {
00021         num *= 10;
00022         num += (str[i++] - '0');
00023     }
00024     if(str[i++] != '.')
00025         return str[0]=='-'?-num:num;
00026     while(str[i]>='0' && str[i]<='9')
00027     {
00028         num += (str[i++] - '0') / ((float)p10);
00029         p10 *= 10;
00030     }
00031     return str[0]=='-'?-num:num;
00032 }
```

Here is the caller graph for this function:



7.62.1.3 str2int()

```
int str2int (
    char str[])
```

Definition at line 3 of file [032_ParEntrada.c](#).

```

00004 {
00005     int num = 0;
00006     int i = str[0]=='?'?1:0;
00007     while(str[i]>='0' && str[i]<='9')
00008     {
00009         num *= 10;
00010         num += (str[i++] - '0');
00011     }
00012     return str[0]=='?'-num:num;
00013 }
```

7.63 032_ParEntrada.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002
00003 int str2int(char str[])
00004 {
00005     int num = 0;
00006     int i = str[0]=='-'?1:0;
00007     while(str[i]>='0' && str[i]<='9')
00008     {
00009         num *= 10;
00010         num += (str[i++]-'0');
00011     }
00012     return str[0]=='-'?-num:num;
00013 }
00014
00015 float str2float(char str[])
00016 {
00017     float num = 0;
00018     int i = str[0]=='-'?1:0, p10=10;
00019     while(str[i]>='0' && str[i]<='9')
00020     {
00021         num *= 10;
00022         num += (str[i++]-'0');
00023     }
00024     if(str[i++]!='.')
00025         return str[0]=='-'?-num:num;
00026     while(str[i]>='0' && str[i]<='9')
00027     {
00028         num += (str[i++]-'0')/((float)p10);
00029         p10 *= 10;
00030     }
00031     return str[0]=='-'?-num:num;
00032 }
00033
00034 int main(int argc, char *argv[])
00035 {
00036     int i;
00037     float s, d;
00038     if(argc==1)
00039     {
00040         printf("Uso: %s A B\n", argv[0]);
00041         return 1;
00042     }
00043     for(i=1, s=0; i<argc; i++)
00044     {
00045         d = str2float(argv[i]);
00046         s += d;
00047         printf("%d-> %f\t%f\n", i, d, s);
00048     }
00049     printf("Suma = %f\n", s);
00050     return 0;
00051 }

```

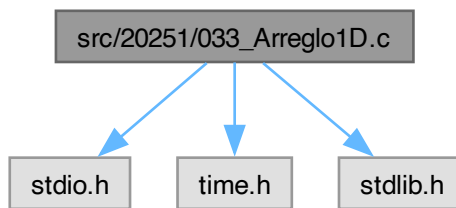
7.64 src/20251/033_Arreglo1D.c File Reference

```

#include <stdio.h>
#include <time.h>
#include <stdlib.h>

```

Include dependency graph for 033_Arreglo1D.c:



Macros

- `#define N 60`
- `#define min 1.5`
- `#define max 2.1`

Functions

- `int main (int argc, char *argv[])`

7.64.1 Macro Definition Documentation

7.64.1.1 max

```
#define max 2.1
```

Definition at line 7 of file [033_Arreglo1D.c](#).

7.64.1.2 min

```
#define min 1.5
```

Definition at line 6 of file [033_Arreglo1D.c](#).

7.64.1.3 N

```
#define N 60
```

Definition at line 5 of file [033_Arreglo1D.c](#).

7.64.2 Function Documentation

7.64.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 9 of file [033_Arreglo1D.c](#).

```
00010 {
00011     float X[N];
00012     float med, var;
00013     int i, n;
00014     srand(time(NULL));
00015     do{
00016         printf("Ingrese el numero de estudiantes: ");
00017         scanf("%d", &n);
00018     }while (n<1 || n>N);
00019     for(i=0, med=0, var=0; i<n; i++)
```

```

00020      {
00021          X[i] = (max-min)*rand()/RAND_MAX+min;
00022          med += X[i];
00023          var += (X[i]*X[i]);
00024          printf("X[%d] = %.3f\n", i+1, X[i]);
00025      }
00026      med /= n;
00027      var /= n;
00028      var -= (med*med);
00029      printf("Media = %.3f\n", med);
00030      printf("Varianza = %.3f\n", var);
00031      return 0;
00032 }

```

7.65 033_Arreglo1D.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<time.h>
00003 #include<stdlib.h>
00004
00005 #define N 60
00006 #define min 1.5
00007 #define max 2.1
00008
00009 int main(int argc, char *argv[])
00010 {
00011     float X[N];
00012     float med, var;
00013     int i, n;
00014     srand(time(NULL));
00015     do{
00016         printf("Ingrese el numero de estudiantes: ");
00017         scanf("%d", &n);
00018     }while(n<1||n>N);
00019     for(i=0, med=0, var=0; i<n; i++)
00020     {
00021         X[i] = (max-min)*rand()/RAND_MAX+min;
00022         med += X[i];
00023         var += (X[i]*X[i]);
00024         printf("X[%d] = %.3f\n", i+1, X[i]);
00025     }
00026     med /= n;
00027     var /= n;
00028     var -= (med*med);
00029     printf("Media = %.3f\n", med);
00030     printf("Varianza = %.3f\n", var);
00031     return 0;
00032 }

```

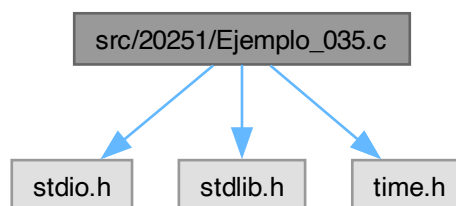
7.66 src/20251/Ejemplo_035.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo_035.c:



Functions

- float [sumar](#) (float *X, int n)
- float [media](#) (float *X, int n)
- int [main](#) (void)

7.66.1 Function Documentation

7.66.1.1 main()

```
int main (
    void )
```

Definition at line 15 of file [Ejemplo_035.c](#).

```
00016 {
00017     int n, m, i;
00018     float *A;
00019     srand(time(NULL));
00020     do{
00021         printf("Ingrese el numero de elementos: ");
00022         scanf("%d", &n);
00023     }while(n<1);
00024     do{
00025         printf("Ingrese el numero de muestras: ");
00026         scanf("%d", &m);
00027     }while(m<1||m>n);
00028     A = (float*)malloc(n*sizeof(float));
00029     if(A==NULL)
00030         return 1;
00031     for(i=0; i<n; i++)
00032     {
00033         A[i] = (1.0*rand())/RAND_MAX;
00034         printf("A[%d] = %f\t%p\n", i+1, A[i], A+i);
00035     }
00036     for(i=0; i<n-m; i++)
00037         printf("u(%d) = %f\n", i+1, media(A+i, m));
00038     free(A);
00039     return 0;
00040 }
```

Here is the call graph for this function:



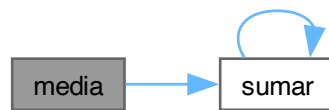
7.66.1.2 media()

```
float media (
    float * X,
    int n)
```

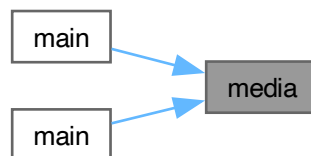
Definition at line 10 of file [Ejemplo_035.c](#).

```
00011 {
00012     return sumar(X, n)/n;
00013 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



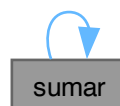
7.66.1.3 sumar()

```
float sumar (  
    float * X,  
    int n)
```

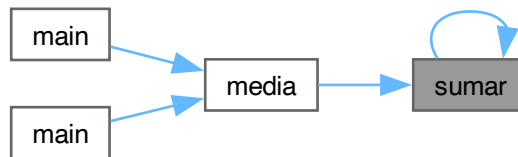
Definition at line 5 of file [Ejemplo_035.c](#).

```
00006 {  
00007     return n?X[0]+sumar(X+1,n-1):0;  
00008 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.67 Ejemplo_035.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 float sumar(float *X, int n)
00006 {
00007     return n?X[0]+sumar(X+1,n-1):0;
00008 }
00009
00010 float media(float *X, int n)
00011 {
00012     return sumar(X, n)/n;
00013 }
00014
00015 int main(void)
00016 {
00017     int n, m, i;
00018     float *A;
00019     srand(time(NULL));
00020     do{
00021         printf("Ingrese el numero de elementos: ");
00022         scanf("%d", &n);
00023     }while(n<1);
00024     do{
00025         printf("Ingrese el numero de muestras: ");
00026         scanf("%d", &m);
00027     }while(m<1||m>n);
00028     A = (float*)malloc(n*sizeof(float));
00029     if(A==NULL)
00030         return 1;
00031     for(i=0; i<n; i++)
00032     {
00033         A[i] = (1.0*rand())/RAND_MAX;
00034         printf("A[%d] = %f\t%p\n", i+1, A[i], A+i);
00035     }
00036     for(i=0; i<n-m; i++)
00037         printf("u(%d) = %f\n", i+1, media(A+i, m));
00038     free(A);
00039     return 0;
00040 }

```

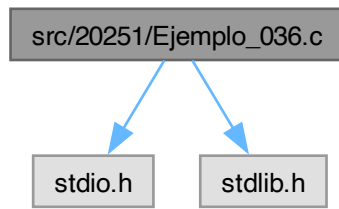
7.68 src/20251/Ejemplo_036.c File Reference

```

#include <stdio.h>
#include <stdlib.h>

```


Include dependency graph for Ejemplo_036.c:



Functions

- int [main](#) (void)

7.68.1 Function Documentation

7.68.1.1 main()

```
int main (
    void )
```

Definition at line 4 of file [Ejemplo_036.c](#).

```

00005 {
00006     float **A, **B, **C, *pA, *pC;
00007     int nA, mA, nB, mB, i, j, k;
00008     do{
00009         printf("Ingrese el numero de columnas de A: ");
00010         scanf("%d", &nA);
00011         printf("Ingrese el numro de renglones de A: ");
00012         scanf("%d", &mA);
00013     }while(nA<1||mA<1);
00014     do{
00015         printf("Ingrese el numero de columnas de B: ");
00016         scanf("%d", &nB);
00017         printf("Ingrese el numro de renglones de B: ");
00018         scanf("%d", &mB);
00019     }while(nB<1||mB<1||nB!=mA);
00020     pA = (float*)malloc(nA*mA*sizeof(float));
00021     if(pA==NULL)
00022         return 1;
00023     A = (float**)malloc(nA*sizeof(float*));
00024     if(A==NULL)
00025     {
00026         free(pA);
00027         return 2;
00028     }
00029     for(i=0; i<nA; i++)
00030         A[i] = pA+i*mA;
00031     B = (float**)malloc(nB*sizeof(float*));
00032     if(B==NULL)
00033     {
00034         free(pA);
00035         free(A);
00036         return 3;
00037     }
00038     for(i=0; i<nB; i++)
00039     {
00040         B[i] = (float*)malloc(mB*sizeof(float));
00041         if(B[i]== NULL)
00042         {
00043             free(pA);
00044             free(A);
00045             for(--i; i>-1; i--)
00046                 free(B[i]);
00047             free(B);
00048             return 4;
00049         }

```

```

00050     }
00051     free(B);
00052     free(pA);
00053     free(A);
00054     return 0;
00055 }

```

7.69 Ejemplo_036.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003
00004 int main(void)
00005 {
00006     float **A, **B, **C, *pA, *pC;
00007     int nA, mA, nB, mB, i, j, k;
00008     do{
00009         printf("Ingrese el numero de columnas de A: ");
00010         scanf("%d", &nA);
00011         printf("Ingrese el numro de renglones de A: ");
00012         scanf("%d", &mA);
00013     }while(nA<1||mA<1);
00014     do{
00015         printf("Ingrese el numero de columnas de B: ");
00016         scanf("%d", &nB);
00017         printf("Ingrese el numro de renglones de B: ");
00018         scanf("%d", &mB);
00019     }while(nB<1||mB<1||nB!=mA);
00020     pA = (float*)malloc(nA*mA*sizeof(float));
00021     if(pA==NULL)
00022         return 1;
00023     A = (float**)malloc(nA*sizeof(float*));
00024     if(A==NULL)
00025     {
00026         free(pA);
00027         return 2;
00028     }
00029     for(i=0; i<nA; i++)
00030         A[i] = pA+i*mA;
00031     B = (float**)malloc(nB*sizeof(float*));
00032     if(B==NULL)
00033     {
00034         free(pA);
00035         free(A);
00036         return 3;
00037     }
00038     for(i=0; i<nB; i++)
00039     {
00040         B[i] = (float*)malloc(mB*sizeof(float));
00041         if(B[i]== NULL)
00042         {
00043             free(pA);
00044             free(A);
00045             for(--i; i>-1; i--)
00046                 free(B[i]);
00047             free(B);
00048             return 4;
00049         }
00050     }
00051     free(B);
00052     free(pA);
00053     free(A);
00054     return 0;
00055 }

```

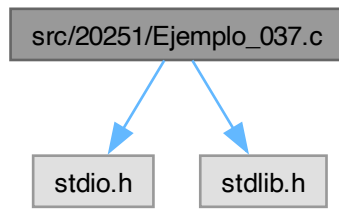
7.70 src/20251/Ejemplo_037.c File Reference

```

#include <stdio.h>
#include <stdlib.h>

```

Include dependency graph for Ejemplo_037.c:



Functions

- int [main](#) (void)

7.70.1 Function Documentation

7.70.1.1 main()

```
int main (
    void )
```

Definition at line 13 of file [Ejemplo_037.c](#).

```

00014 {
00015     float **A, **B, **C, *pA, *pC;
00016     int nA, mA, nB, mB, i, j, k;
00017     FILE *fp;
00018     do{
00019         printf("Ingrese el numero de columnas de A: ");
00020         scanf("%d", &nA);
00021         printf("Ingrese el numro de renglones de A: ");
00022         scanf("%d", &mA);
00023     }while(nA<1||mA<1);
00024     do{
00025         printf("Ingrese el numero de columnas de B: ");
00026         scanf("%d", &nB);
00027         printf("Ingrese el numro de renglones de B: ");
00028         scanf("%d", &mB);
00029     }while(nB<1||mB<1||nB!=mA);
00030     pA = (float*)malloc(nA*mA*sizeof(float));
00031     if(pA==NULL)
00032         return 1;
00033     A = (float**)malloc(nA*sizeof(float*));
00034     if(A==NULL)
00035     {
00036         free(pA);
00037         return 2;
00038     }
00039     for(i=0; i<nA; i++)
00040         A[i] = pA+i*mA;
00041     B = (float**)malloc(nB*sizeof(float*));
00042     if(B==NULL)
00043     {
00044         free(pA);
00045         free(A);
00046         return 3;
00047     }
00048     for(i=0; i<nB; i++)
00049     {
00050         B[i] = (float*)malloc(mB*sizeof(float));
00051         if(B[i]== NULL)
00052         {
00053             free(pA);
00054             free(A);
00055             for(--i; i>-1; i--)
00056                 free(B[i]);
00057             free(B);
00058             return 4;

```

```

00059     }
00060 }
00061 pC = (float*)malloc(nA*mB*sizeof(float));
00062 if(pC==NULL)
00063 {
00064     free(pA);
00065     free(A);
00066     for(i=0; i<nB; i++)
00067         free(B[i]);
00068     free(B);
00069     return 5;
00070 }
00071 C = (float**)malloc(nA*sizeof(float*));
00072 if(C==NULL)
00073 {
00074     free(pA);
00075     free(A);
00076     for(i=0; i<nB; i++)
00077         free(B[i]);
00078     free(B);
00079     free(pC);
00080     return 6;
00081 }
00082 for(i=0; i<nA; i++)
00083     C[i] = pC+i*mB;
00084 for(i=0; i<nA; i++)
00085     for(j=0; j<mA; j++)
00086     {
00087         printf("A[%d][%d] = ", i+1, j+1);
00088         scanf("%f", A[i]+j);
00089     }
00090 for(i=0; i<nB; i++)
00091     for(j=0; j<mB; j++)
00092     {
00093         printf("B[%d][%d] = ", i+1, j+1);
00094         scanf("%f", B[i]+j);
00095     }
00096 for(i=0; i<nA; i++)
00097     for(j=0; j<mB; j++)
00098         for(k=0; C[i][j]=0; k<nB; k++)
00099             C[i][j] += A[i][k]*B[k][j];
00100 for(i=0; i<nA; i++)
00101     for(j=0; j<mB; j++)
00102         printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00103 fp = fopen("Matriz.txt", "w");
00104 if(fp==NULL)
00105 {
00106     free(pC);
00107     free(C);
00108     for(i=0; i<nB; i++)
00109         free(B[i]);
00110     free(B);
00111     free(pA);
00112     free(A);
00113     return 7;
00114 }
00115 for(i=0; i<nA; i++)
00116     for(j=0; j<mB; j++)
00117         fprintf(fp, "%f ", C[i][j]);
00118 fclose(fp);
00119 free(pC);
00120 free(C);
00121 for(i=0; i<nB; i++)
00122     free(B[i]);
00123 free(B);
00124 free(pA);
00125 free(A);
00126 return 0;
00127 }

```

7.71 Ejemplo_037.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003
00004 /* Multiplicacion de matrices */
00005 /* A: mA x nA */
00006 /* B: mB x nB */
00007 /* C: mA x nB */
00008 /* A matriz ordenada */
00009 /* B matriz dispersa */
00010 /* C matriz ordenada */
00011 /* fp archivo de salida de C */

```

```

00012
00013 int main(void)
00014 {
00015     float **A, **B, **C, *pA, *pC;
00016     int nA, mA, nB, mB, i, j, k;
00017     FILE *fp;
00018     do{
00019         printf("Ingrese el numero de columnas de A: ");
00020         scanf("%d", &nA);
00021         printf("Ingrese el numro de renglones de A: ");
00022         scanf("%d", &mA);
00023     }while(nA<1||mA<1);
00024     do{
00025         printf("Ingrese el numero de columnas de B: ");
00026         scanf("%d", &nB);
00027         printf("Ingrese el numro de renglones de B: ");
00028         scanf("%d", &mB);
00029     }while(nB<1||mB<1||nB!=mA);
00030     pA = (float*)malloc(nA*mA*sizeof(float));
00031     if(pA==NULL)
00032         return 1;
00033     A = (float**)malloc(nA*sizeof(float*));
00034     if(A==NULL)
00035     {
00036         free(pA);
00037         return 2;
00038     }
00039     for(i=0; i<nA; i++)
00040         A[i] = pA+i*mA;
00041     B = (float**)malloc(nB*sizeof(float*));
00042     if(B==NULL)
00043     {
00044         free(pA);
00045         free(A);
00046         return 3;
00047     }
00048     for(i=0; i<nB; i++)
00049     {
00050         B[i] = (float*)malloc(mB*sizeof(float));
00051         if(B[i]== NULL)
00052         {
00053             free(pA);
00054             free(A);
00055             for(--i; i>-1; i--)
00056                 free(B[i]);
00057             free(B);
00058             return 4;
00059         }
00060     }
00061     pC = (float*)malloc(nA*mB*sizeof(float));
00062     if(pC==NULL)
00063     {
00064         free(pA);
00065         free(A);
00066         for(i=0; i<nB; i++)
00067             free(B[i]);
00068         free(B);
00069         return 5;
00070     }
00071     C = (float**)malloc(nA*sizeof(float*));
00072     if(C==NULL)
00073     {
00074         free(pA);
00075         free(A);
00076         for(i=0; i<nB; i++)
00077             free(B[i]);
00078         free(B);
00079         free(pC);
00080         return 6;
00081     }
00082     for(i=0; i<nA; i++)
00083         C[i] = pC+i*mB;
00084     for(i=0; i<nA; i++)
00085         for(j=0; j<mA; j++)
00086         {
00087             printf("A[%d][%d] = ", i+1, j+1);
00088             scanf("%f", A[i]+j);
00089         }
00090     for(i=0; i<nB; i++)
00091         for(j=0; j<mB; j++)
00092         {
00093             printf("B[%d][%d] = ", i+1, j+1);
00094             scanf("%f", B[i]+j);
00095         }
00096     for(i=0; i<nA; i++)
00097         for(j=0; j<mB; j++)
00098             for(k=0; C[i][j]=0; k<nB; k++)

```

```

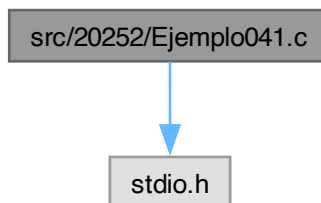
00099     C[i][j] += A[i][k]*B[k][j];
00100     for(i=0; i<nA; i++)
00101         for(j=0; j<mB; j++)
00102             printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00103     fp = fopen("Matriz.txt", "w");
00104     if(fp==NULL)
00105     {
00106         free(pC);
00107         free(C);
00108         for(i=0; i<nB; i++)
00109             free(B[i]);
00110         free(B);
00111         free(pA);
00112         free(A);
00113         return 7;
00114     }
00115     for(i=0; i<nA; i++)
00116         for(j=0; j<mB; j++)
00117             fprintf(fp, "%f ", C[i][j]);
00118     fclose(fp);
00119     free(pC);
00120     free(C);
00121     for(i=0; i<nB; i++)
00122         free(B[i]);
00123     free(B);
00124     free(pA);
00125     free(A);
00126     return 0;
00127 }

```

7.72 src/20252/Ejemplo041.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo041.c:



Functions

- int [main](#) (int argc, char *argv[])

7.72.1 Function Documentation

7.72.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo041.c](#).

```

00004 {
00005     int i, n;
00006     FILE *fp;
00007     printf("Ingrese el valor de n: ");
00008     scanf("%d", &n);
00009     fp = fopen("Prueba.txt", "wt");
00010     if(fp==NULL)

```

```

00011         return 1;
00012     for (i=0; i<n; i++)
00013         fprintf(fp, "%d\n", i+1);
00014     fclose(fp);
00015     return 0;
00016 }

```

7.73 Ejemplo041.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int i, n;
00006     FILE *fp;
00007     printf("Ingrese el valor de n: ");
00008     scanf("%d", &n);
00009     fp = fopen("Prueba.txt", "wt");
00010     if (fp==NULL)
00011         return 1;
00012     for (i=0; i<n; i++)
00013         fprintf(fp, "%d\n", i+1);
00014     fclose(fp);
00015     return 0;
00016 }

```

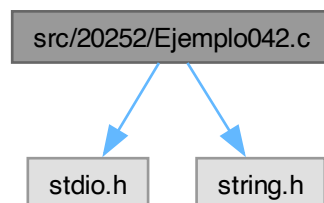
7.74 src/20252/Ejemplo042.c File Reference

```

#include <stdio.h>
#include <string.h>

```

Include dependency graph for Ejemplo042.c:



Macros

- #define [N](#) 50

Functions

- int [main](#) (int argc, char *argv[])

7.74.1 Macro Definition Documentation

7.74.1.1 N

```
#define N 50
```

Definition at line 4 of file [Ejemplo042.c](#).

7.74.2 Function Documentation

7.74.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 6 of file [Ejemplo042.c](#).

```
00007 {
00008     int i, n;
00009     FILE *fp;
00010     char filename[N];
00011     printf("Ingrese el valor de n: ");
00012     scanf("%d", &n);
00013     if(argc==2)
00014         strcpy(filename, argv[1]);
00015     else
00016         strcpy(filename, "Prueba.txt");
00017     fp = fopen(filename, "wt");
00018     if(fp==NULL)
00019         return 1;
00020     for(i=0; i<n; i++)
00021         fprintf(fp, "%d\n", i+1);
00022     fclose(fp);
00023     return 0;
00024 }
```

7.75 Ejemplo042.c

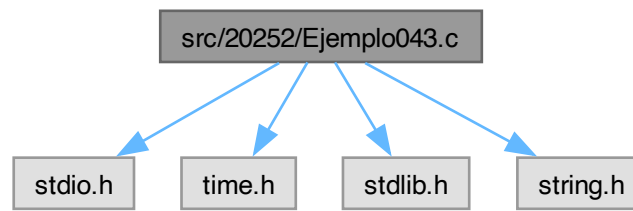
[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <string.h>
00003
00004 #define N 50
00005
00006 int main(int argc, char *argv[])
00007 {
00008     int i, n;
00009     FILE *fp;
00010     char filename[N];
00011     printf("Ingrese el valor de n: ");
00012     scanf("%d", &n);
00013     if(argc==2)
00014         strcpy(filename, argv[1]);
00015     else
00016         strcpy(filename, "Prueba.txt");
00017     fp = fopen(filename, "wt");
00018     if(fp==NULL)
00019         return 1;
00020     for(i=0; i<n; i++)
00021         fprintf(fp, "%d\n", i+1);
00022     fclose(fp);
00023     return 0;
00024 }
```

7.76 src/20252/Ejemplo043.c File Reference

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include <string.h>
```


Include dependency graph for Ejemplo043.c:



Macros

- `#define N 50`

Functions

- `int main (int argc, char *argv[ ])`

7.76.1 Macro Definition Documentation

7.76.1.1 N

`#define N 50`

Definition at line 6 of file [Ejemplo043.c](#).

7.76.2 Function Documentation

7.76.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 8 of file [Ejemplo043.c](#).

```
00009 {
00010     int i, n;
00011     double x;
00012     FILE *fp;
00013     char filename[N];
00014     do{
00015         printf("Ingrese el valor de n: ");
00016         scanf("%d", &n);
00017     }while(n<1);
00018     if(argc==2)
00019         strcpy(filename, argv[1]);
00020     else
00021         strcpy(filename, "Prueba.txt");
00022     fp = fopen(filename, "wt");
00023     if(fp==NULL)
00024         return 1;
00025     srand(time(NULL));
00026     fprintf(fp, "%d\n", n);
00027     for(i=0; i<n; i++)
00028     {
00029         x = (1.0*rand())/RAND_MAX;
00030         fprintf(fp, "%d\t%lf\n", i+1, x);
00031         printf("x[%d] = %lf\n", i+1, x);
00032     }
00033     fclose(fp);
00034     return 0;
00035 }
```

7.77 Ejemplo043.c

[Go to the documentation of this file.](#)

```

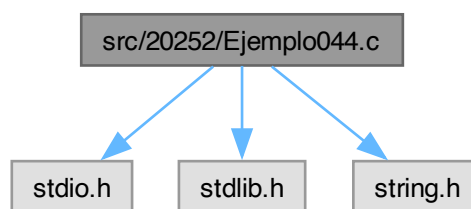
00001 #include <stdio.h>
00002 #include <time.h>
00003 #include <stdlib.h>
00004 #include <string.h>
00005
00006 #define N 50
00007
00008 int main(int argc, char *argv[])
00009 {
00010     int i, n;
00011     double x;
00012     FILE *fp;
00013     char filename[N];
00014     do{
00015         printf("Ingrese el valor de n: ");
00016         scanf("%d", &n);
00017     }while(n<1);
00018     if(argc==2)
00019         strcpy(filename, argv[1]);
00020     else
00021         strcpy(filename, "Prueba.txt");
00022     fp = fopen(filename, "wt");
00023     if(fp==NULL)
00024         return 1;
00025     srand(time(NULL));
00026     fprintf(fp, "%d\n", n);
00027     for(i=0; i<n; i++)
00028     {
00029         x = (1.0*rand())/RAND_MAX;
00030         fprintf(fp, "%d\t%lf\n", i+1, x);
00031         printf("x[%d] = %lf\n", i+1, x);
00032     }
00033     fclose(fp);
00034     return 0;
00035 }
```

7.78 src/20252/Ejemplo044.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Include dependency graph for Ejemplo044.c:



Macros

- `#define N 50`

Functions

- `int main (int argc, char *argv[])`

7.78.1 Macro Definition Documentation

7.78.1.1 N

#define N 50

Definition at line 5 of file [Ejemplo044.c](#).

7.78.2 Function Documentation

7.78.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file [Ejemplo044.c](#).

```
00008 {
00009     int i, n, c;
00010     double *x, m;
00011     FILE *fp;
00012     char filename[N];
00013     if(argc==2)
00014         strcpy(filename, argv[1]);
00015     else
00016         strcpy(filename, "Prueba.txt");
00017     fp = fopen(filename, "rt");
00018     if(fp==NULL)
00019         return 1;
00020     fscanf(fp, "%d\n", &n);
00021     printf("n = %d\n", n);
00022     x = (double*)malloc(n*sizeof(double));
00023     if(x==NULL)
00024     {
00025         fclose(fp);
00026         return 2;
00027     }
00028     for(i=0, m=0; i<n; i++)
00029     {
00030         //     fscanf(fp, "%d\t%lf\n", &c, &x[i]);
00031         fscanf(fp, "%d\t%lf\n", &c, &x[i]);
00032         printf("x[%d] = %lf\n", c, x[i]);
00033         m+=x[i];
00034     }
00035     m/=n;
00036     printf("Media = %lf\n", m);
00037     fclose(fp);
00038     return 0;
00039 }
```

7.79 Ejemplo044.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <string.h>
00004
00005 #define N 50
00006
00007 int main(int argc, char *argv[])
00008 {
00009     int i, n, c;
00010     double *x, m;
00011     FILE *fp;
00012     char filename[N];
00013     if(argc==2)
00014         strcpy(filename, argv[1]);
00015     else
00016         strcpy(filename, "Prueba.txt");
00017     fp = fopen(filename, "rt");
00018     if(fp==NULL)
00019         return 1;
00020     fscanf(fp, "%d\n", &n);
00021     printf("n = %d\n", n);
00022     x = (double*)malloc(n*sizeof(double));
00023     if(x==NULL)
00024     {
00025         fclose(fp);
00026         return 2;
00027     }
```

```

00028     for(i=0, m=0; i<n; i++)
00029     {
00030 //         fscanf(fp, "%d\t%lf\n", &c, &x[i]);
00031         fscanf(fp, "%d\t%lf\n", &c, &x[i]);
00032         printf("x[%d] = %lf\n", c, x[i]);
00033         m+=x[i];
00034     }
00035     m/=n;
00036     printf("Media = %lf\n", m);
00037     fclose(fp);
00038     return 0;
00039 }

```

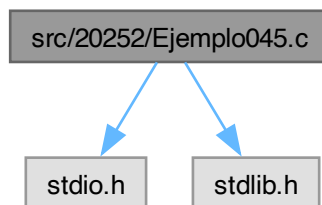
7.80 src/20252/Ejemplo045.c File Reference

```

#include <stdio.h>
#include <stdlib.h>

```

Include dependency graph for Ejemplo045.c:



Macros

- `#define N 50`

Functions

- `int main (int argc, char *argv[ ])`

7.80.1 Macro Definition Documentation

7.80.1.1 N

```
#define N 50
```

Definition at line 4 of file [Ejemplo045.c](#).

7.80.2 Function Documentation

7.80.2.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 6 of file [Ejemplo045.c](#).

```

00007 {
00008     char filename[N] = "full_data.csv";
00009     char *str;
00010     char c;
00011     int i, j, nc, nl, *ln, *Ceremony, *Year;
00012     FILE *fp;

```

```

00013     fp = fopen(filename, "rt");
00014     if (fp==NULL)
00015     {
00016         printf("Error al leer el archivo.\n");
00017         return 1;
00018     }
00019     //fscanf(fp, "%s", str);
00020     /*
00021     for(i=0; i<10; i++)
00022     {
00023         fgets(str, NC-1, fp);
00024         printf("%s", str);
00025     }
00026     */
00027     j=0;
00028     i=0;
00029     while((c=fgetc(fp))!=EOF)
00030     {
00031         j++;
00032         if(c==10)
00033             i++;
00034     }
00035     nl = i;
00036     nc = j;
00037     printf("Num L: %d\n", nl);
00038     printf("Num C: %d\n", nc);
00039     str = (char*)malloc((nc+1)*sizeof(char));
00040     if(str==NULL)
00041     {
00042         fclose(fp);
00043         return 1;
00044     }
00045     ln = (int*)calloc(nl, sizeof(int));
00046     if(ln==NULL)
00047     {
00048         fclose(fp);
00049         free(str);
00050         return 2;
00051     }
00052     fseek(fp, 0, SEEK_SET);
00053     for(i=0, j=1; i<nc; i++)
00054     {
00055         str[i] = getc(fp);
00056         if(str[i]==10)
00057             ln[j++] = i+1;
00058     }
00059     str[i] = '\0';
00060     fclose(fp);
00061     for(i=1; i<nl; i++)
00062         str[ln[i]-1] = '\0';
00063     Ceremony = (int*)malloc((nl-1)*sizeof(int));
00064     if(Ceremony==NULL)
00065     {
00066         free(ln);
00067         free(str);
00068         return 3;
00069     }
00070     Year = (int*)malloc((nl-1)*sizeof(int));
00071     if(Year==NULL)
00072     {
00073         free(Ceremony);
00074         free(ln);
00075         free(str);
00076         return 3;
00077     }
00078     for(i=1; i<nl; i++)
00079     {
00080         Ceremony[i-1] = atoi(str+ln[i]);
00081         j=0;
00082         while(str[ln[i]+j]!=9)
00083             j++;
00084         Year[i-1] = atoi(str+ln[i]+j+1);
00085         printf("%d. %d\t%d\n", i, Ceremony[i-1], Year[i-1]);
00086     }
00087     free(Year);
00088     free(Ceremony);
00089     free(ln);
00090     free(str);
00091     return 0;
00092 }

```

7.81 Ejemplo045.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 #define N 50
00005
00006 int main(int argc, char *argv[])
00007 {
00008     char filename[N] = "full_data.csv";
00009     char *str;
00010     char c;
00011     int i, j, nc, nl, *ln, *Ceremony, *Year;
00012     FILE *fp;
00013     fp = fopen(filename, "rt");
00014     if (fp==NULL)
00015     {
00016         printf("Error al leer el archivo.\n");
00017         return 1;
00018     }
00019     //fscanf(fp, "%s", str);
00020     /*
00021     for(i=0; i<10; i++)
00022     {
00023         fgets(str, NC-1, fp);
00024         printf("%s", str);
00025     }
00026     */
00027     j=0;
00028     i=0;
00029     while((c=fgetc(fp))!=EOF)
00030     {
00031         j++;
00032         if(c==10)
00033             i++;
00034     }
00035     nl = i;
00036     nc = j;
00037     printf("Num L: %d\n", nl);
00038     printf("Num C: %d\n", nc);
00039     str = (char*)malloc((nc+1)*sizeof(char));
00040     if(str==NULL)
00041     {
00042         fclose(fp);
00043         return 1;
00044     }
00045     ln = (int*)calloc(nl, sizeof(int));
00046     if(ln==NULL)
00047     {
00048         fclose(fp);
00049         free(str);
00050         return 2;
00051     }
00052     fseek(fp, 0, SEEK_SET);
00053     for(i=0, j=1; i<nc; i++)
00054     {
00055         str[i] = getc(fp);
00056         if(str[i]==10)
00057             ln[j++] = i+1;
00058     }
00059     str[i] = '\0';
00060     fclose(fp);
00061     for(i=1; i<nl; i++)
00062         str[ln[i]-1] = '\0';
00063     Ceremony = (int*)malloc((nl-1)*sizeof(int));
00064     if(Ceremony==NULL)
00065     {
00066         free(ln);
00067         free(str);
00068         return 3;
00069     }
00070     Year = (int*)malloc((nl-1)*sizeof(int));
00071     if(Year==NULL)
00072     {
00073         free(Ceremony);
00074         free(ln);
00075         free(str);
00076         return 3;
00077     }
00078     for(i=1; i<nl; i++)
00079     {
00080         Ceremony[i-1] = atoi(str+ln[i]);
00081         j=0;
00082         while(str[ln[i]+j]!=9)
00083             j++;
00084         Year[i-1] = atoi(str+ln[i]+j+1);
00085         printf("%d. %d\t%d\n", i, Ceremony[i-1], Year[i-1]);
00086     }
00087     free(Year);

```

```

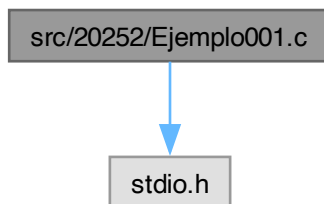
00088     free(Ceremony);
00089     free(ln);
00090     free(str);
00091     return 0;
00092 }

```

7.82 src/20252/Ejemplo001.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo001.c:



Functions

- int [main](#) (int argc, char *argv[])

7.82.1 Function Documentation

7.82.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo001.c](#).

```

00004 {
00005     int a = 10, b = -250, n = 4;
00006     printf("%d\t%d\n", a, b);
00007     printf("%d\t%d\n", a, b);
00008     printf("%05d\t%05d\n", a, b);
00009     printf("%0*d\t%0*d\n", 5, a, 5, b);
00010     printf("%0*d\t%0*d\n", n, a, n, b);
00011     return 0;
00012 }

```

7.83 Ejemplo001.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int a = 10, b = -250, n = 4;
00006     printf("%d\t%d\n", a, b);
00007     printf("%d\t%d\n", a, b);
00008     printf("%05d\t%05d\n", a, b);
00009     printf("%0*d\t%0*d\n", 5, a, 5, b);
00010     printf("%0*d\t%0*d\n", n, a, n, b);
00011     return 0;
00012 }

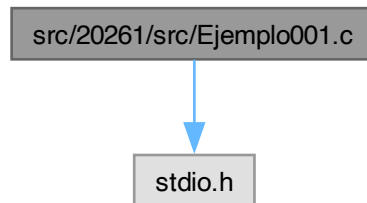
```

7.84 src/20261/src/Ejemplo001.c File Reference

Hola mundo mínimo: imprime un mensaje en consola.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo001.c:



Functions

- int [main](#) (int argc, char *argv[])

Punto de entrada del programa.

7.84.1 Detailed Description

Hola mundo mínimo: imprime un mensaje en consola.

Programa introductorio en C para verificar compilación y ejecución.

Entrada

Ninguna.

Salida

Imprime la cadena "Universidad de Guanajuato" seguida de salto de línea.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Note

Compilación sugerida:

```
gcc Ejemplo001.c -o Ejemplo001
./Ejemplo001
```

Definition in file [Ejemplo001.c](#).

7.84.2 Function Documentation

7.84.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
Punto de entrada del programa.
```

Parameters

<i>argc</i>	Cantidad de argumentos de línea de comandos.
-------------	--

<code>argv</code>	Arreglo de cadenas con argumentos.
-------------------	------------------------------------

Returns

0 si termina correctamente.

Definition at line 33 of file [Ejemplo001.c](#).

```
00034 {  
00035     (void)argc;  
00036     (void)argv;  
00037  
00038     printf("Universidad de Guanajuato\n");  
00039     return 0;  
00040 }
```

7.85 Ejemplo001.c

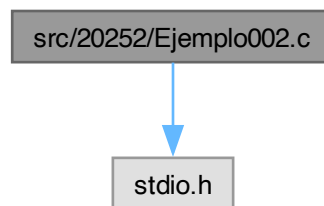
[Go to the documentation of this file.](#)

```
00001  
00024  
00025 #include <stdio.h>  
00026  
00033 int main(int argc, char *argv[])  
00034 {  
00035     (void)argc;  
00036     (void)argv;  
00037  
00038     printf("Universidad de Guanajuato\n");  
00039     return 0;  
00040 }
```

7.86 src/20252/Ejemplo002.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo002.c:



Functions

- int [main](#) (int argc, char *argv[])

7.86.1 Function Documentation

7.86.1.1 main()

```
int main (  
    int argc,  
    char * argv[ ])
```

Definition at line 3 of file [Ejemplo002.c](#).

```
00004 {
00005     int N, M, X, Y, T;
00006     scanf("%d %d %d %d", &N, &M, &X, &Y);
00007     T = (Y*N+M) / (X+Y);
00008     T = (T>N?N:T);
00009     printf("%d\n", T);
00010     return 0;
00011 }
```

7.87 Ejemplo002.c

[Go to the documentation of this file.](#)

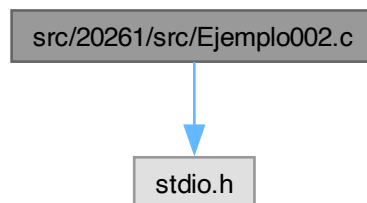
```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int N, M, X, Y, T;
00006     scanf("%d %d %d %d", &N, &M, &X, &Y);
00007     T = (Y*N+M) / (X+Y);
00008     T = (T>N?N:T);
00009     printf("%d\n", T);
00010     return 0;
00011 }
```

7.88 src/20261/src/Ejemplo002.c File Reference

Codificador de prioridad de interrupción para 4 dispositivos (D1..D4).

```
#include <stdio.h>
```

Include dependency graph for Ejemplo002.c:



Functions

- int [main](#) (int argc, char *argv[])

Punto de entrada. Implementa el codificador de prioridad 4→3 bits.

7.88.1 Detailed Description

Codificador de prioridad de interrupción para 4 dispositivos (D1..D4).

Se reciben 4 solicitudes de interrupción (una por dispositivo). La prioridad se asigna de manera que el dispositivo con número más bajo tiene mayor prioridad: D1 > D2 > D3 > D4.

La salida es el número del dispositivo seleccionado codificado en binario (3 bits):

- Ninguno -> 0 (000)
- D1 -> 1 (001)
- D2 -> 2 (010)

- D3 -> 3 (011)
- D4 -> 4 (100)

Entrada

Cuatro enteros (0/1) en este orden:

D4 D3 D2 D1

Salida

Un renglón con:

- el valor decimal seleccionado (0..4)
- y los bits de salida (b2 b1 b0) solo como depuración.

Precondiciones

Las entradas deben ser booleanas (0 o 1) para que el comportamiento sea el esperado.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Definition in file [Ejemplo002.c](#).

7.88.2 Function Documentation

7.88.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Implementa el codificador de prioridad 4→3 bits.

Parameters

<i>argc</i>	Cantidad de argumentos de línea de comandos (no se usa).
<i>argv</i>	Argumentos de línea de comandos (no se usa).

Returns

0 si finaliza correctamente; 1 si falla la lectura.

Definition at line 43 of file [Ejemplo002.c](#).

```
00044 {
00045     (void)argc;
00046     (void)argv;
00047
00048     /* Entradas: solicitud de interrupción por dispositivo (0/1). */
00049     int D4, D3, D2, D1;
00050
00051     /* Bits de salida: b2 b1 b0 (representan 0..4). */
00052     int b2, b1, b0;
00053
00054     /* Salida decimal equivalente a (b2 b1 b0). */
00055     int salida;
00056
00057     /* Se leen las entradas en el orden: D4 D3 D2 D1. */
00058     if (scanf("%d %d %d %d", &D4, &D3, &D2, &D1) != 4) {
00059         printf("Error: se esperaban 4 valores (D4 D3 D2 D1).\n");
00060         return 1;
00061     }
00062
00063     /*
00064         Lógica de prioridad (D1 mayor prioridad):
```

```

00065
00066     Para "habilitar" un dispositivo de menor prioridad, se exige que NO haya
00067     dispositivos de mayor prioridad activos.
00068
00069     b2 debe ser 1 solo cuando gana D4 (100):
00070     b2 = D4 · ¬D3 · ¬D2 · ¬D1
00071 */
00072 b2 = D4 && (!D3) && (!D2) && (!D1);
00073
00074 /*
00075     b1 es 1 cuando la salida es 2 (010) o 3 (011):
00076     - Para 3: D3 · ¬D2 · ¬D1
00077     - Para 2: D2 · ¬D1
00078     => b1 = (D3 · ¬D2 · ¬D1) + (D2 · ¬D1)
00079 */
00080 b1 = (D3 && (!D2) && (!D1)) || (D2 && (!D1));
00081
00082 /*
00083     b0 es 1 cuando la salida es 1 (001) o 3 (011):
00084     - Para 1: D1
00085     - Para 3: D3 · ¬D2 · ¬D1
00086     => b0 = D1 + (D3 · ¬D2 · ¬D1)
00087 */
00088 b0 = (D1) || (D3 && (!D2) && (!D1));
00089
00090 /* Convertir (b2 b1 b0) a decimal: salida = b2*4 + b1*2 + b0. */
00091 salida = (b2 << 2) | (b1 << 1) | b0;
00092
00093 printf("Salida = %d (%ld %ld %ld)\n", salida, b2, b1, b0);
00094 return 0;
00095 }

```

7.89 Ejemplo002.c

[Go to the documentation of this file.](#)

```

00001
00034
00035 #include <stdio.h>
00036
00043 int main(int argc, char *argv[])
00044 {
00045     (void)argc;
00046     (void)argv;
00047
00048     /* Entradas: solicitud de interrupción por dispositivo (0/1). */
00049     int D4, D3, D2, D1;
00050
00051     /* Bits de salida: b2 b1 b0 (representan 0..4). */
00052     int b2, b1, b0;
00053
00054     /* Salida decimal equivalente a (b2 b1 b0). */
00055     int salida;
00056
00057     /* Se leen las entradas en el orden: D4 D3 D2 D1. */
00058     if (scanf("%d %d %d %d", &D4, &D3, &D2, &D1) != 4) {
00059         printf("Error: se esperaban 4 valores (D4 D3 D2 D1).\n");
00060         return 1;
00061     }
00062
00063     /*
00064         Lógica de prioridad (D1 mayor prioridad):
00065
00066         Para "habilitar" un dispositivo de menor prioridad, se exige que NO haya
00067         dispositivos de mayor prioridad activos.
00068
00069         b2 debe ser 1 solo cuando gana D4 (100):
00070         b2 = D4 · ¬D3 · ¬D2 · ¬D1
00071     */
00072     b2 = D4 && (!D3) && (!D2) && (!D1);
00073
00074     /*
00075         b1 es 1 cuando la salida es 2 (010) o 3 (011):
00076         - Para 3: D3 · ¬D2 · ¬D1
00077         - Para 2: D2 · ¬D1
00078         => b1 = (D3 · ¬D2 · ¬D1) + (D2 · ¬D1)
00079     */
00080     b1 = (D3 && (!D2) && (!D1)) || (D2 && (!D1));
00081
00082     /*
00083         b0 es 1 cuando la salida es 1 (001) o 3 (011):
00084         - Para 1: D1
00085         - Para 3: D3 · ¬D2 · ¬D1
00086         => b0 = D1 + (D3 · ¬D2 · ¬D1)
00087     */

```

```

00088     b0 = (D1) || (D3 && (!D2) && (!D1));
00089
00090     /* Convertir (b2 b1 b0) a decimal: salida = b2*4 + b1*2 + b0. */
00091     salida = (b2 << 2) | (b1 << 1) | b0;
00092
00093     printf("Salida = %d (%1d %1d %1d)\n", salida, b2, b1, b0);
00094     return 0;
00095 }

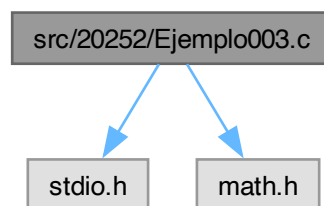
```

7.90 src/20252/Ejemplo003.c File Reference

```
#include <stdio.h>
```

```
#include <math.h>
```

Include dependency graph for Ejemplo003.c:



Functions

- `int main (int argc, char *argv[ ])`

7.90.1 Function Documentation

7.90.1.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 4 of file [Ejemplo003.c](#).

```

00005 {
00006     float a, b, c, x1r, x2r, x1i, x2i, r;
00007     printf("Ingrese el valor de a: ");
00008     scanf("%f", &a);
00009     printf("Ingrese el valor de b: ");
00010     scanf("%f", &b);
00011     printf("Ingrese el valor de c: ");
00012     scanf("%f", &c);
00013     r = b*b-4*a*c;
00014     x1r=(-b+(r>=0?sqrt(r):0))/(2*a);
00015     x2r=(-b-(r>=0?sqrt(r):0))/(2*a);
00016     x1i=(r>=0?0:sqrt(-r)/(2*a));
00017     x2i=-x1i;
00018     printf("x1=%.2f%+.2fi\n", x1r, x1i);
00019     printf("x2=%.2f%+.2fi\n", x2r, x2i);
00020     return 0;
00021 }

```

7.91 Ejemplo003.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<math.h>

```

```

00003
00004 int main(int argc, char*argv[])
00005 {
00006     float a, b, c, x1r, x2r, x1i, x2i, r;
00007     printf("Ingresa el valor de a: ");
00008     scanf("%f", &a);
00009     printf("Ingresa el valor de b: ");
00010     scanf("%f", &b);
00011     printf("Ingresa el valor de c: ");
00012     scanf("%f", &c);
00013     r = b*b-4*a*c;
00014     x1r=(-b+(r>=0?sqrt(r):0))/(2*a);
00015     x2r=(-b-(r>=0?sqrt(r):0))/(2*a);
00016     x1i=(r>=0?0:sqrt(-r)/(2*a));
00017     x2i=-x1i;
00018     printf("x1=%.2f%.2fi\n", x1r, x1i);
00019     printf("x2=%.2f%.2fi\n", x2r, x2i);
00020     return 0;
00021 }

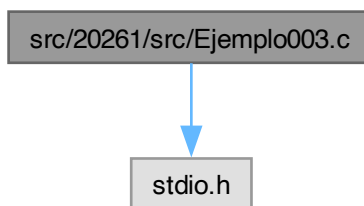
```

7.92 src/20261/src/Ejemplo003.c File Reference

Máximo número de Pokémon que pueden evolucionarse con venta de no evolucionados.

#include <stdio.h>

Include dependency graph for Ejemplo003.c:



Functions

- int `main` (int argc, char *argv[])

Punto de entrada. Calcula el máximo de evoluciones.

7.92.1 Detailed Description

Máximo número de Pokémon que pueden evolucionarse con venta de no evolucionados.

Se tienen N Pokémon y M barras de dulce. Evolucionar un Pokémon cuesta X dulces. Vender un Pokémon NO evolucionado produce Y dulces. No se puede vender un Pokémon evolucionado. Se pide el máximo número de evoluciones posibles.

Este código calcula una cota óptima usando la desigualdad: $M + (N - P) \cdot Y \geq P \cdot X$ que se transforma en: $P \leq (M + N \cdot Y) / (X + Y)$ y finalmente $P = \min(N, \text{floor}((M + N \cdot Y) / (X + Y)))$.

Entrada

El enunciado indica:

N M X Y

Warning

Este programa lee en el orden:

`M N X Y`

(intercambia N y M respecto al enunciado). Si vas a usar la entrada como está definida en el problema, ajusta el `scanf` o el orden de variables.

Salida

Un entero: máximo número de Pokémon que se pueden evolucionar.

Restricciones

1 N, M, X, Y 1e9.

Note

Con límites de 1e9, productos como $N*Y$ pueden exceder 32 bits. Conviene usar tipos de 64 bits (long long) si se requiere robustez numérica.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Definition in file [Ejemplo003.c](#).

7.92.2 Function Documentation

7.92.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula el máximo de evoluciones.

Parameters

<code>argc</code>	No usado.
<code>argv</code>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 52 of file [Ejemplo003.c](#).

```
00053 {
00054     (void)argc;
00055     (void)argv;
00056
00057     /*
00058      * VARIABLES (según como ESTE código está leyendo la entrada):
00059      * M = cantidad de Pokémon capturados (enunciado lo llama N)
00060      * N = cantidad de barras de dulce (candies) (enunciado lo llama M)
00061      * X = costo en dulces para evolucionar 1 Pokémon
00062      * Y = dulces que se ganan al vender 1 Pokémon (NO evolucionado)
00063      * P = número máximo de Pokémon que se pueden evolucionar (respuesta)
00064      */
00065     int M, N, X, Y, P;
00066
00067     /* Lee: M N X Y (nota: el enunciado usualmente pide N M X Y). */
00068     scanf("%d %d %d %d", &M, &N, &X, &Y);
00069
00070     /*
00071      * Si evolucionamos P Pokémon y vendemos los (M - P) restantes:
00072      * dulces disponibles = N + (M - P)*Y
00073      * Requisito para evolucionar P:
```

```

00074         N + (M - P)*Y >= P*X
00075         => N + M*Y >= P*(X + Y)
00076         => P <= floor((N + M*Y)/(X + Y))
00077
00078         Además, P no puede ser mayor que M (no puedes evolucionar más Pokémon de los que tienes).
00079     */
00080     P = (N + Y * M) / (X + Y);
00081
00082     /* Imprime min(P, M). */
00083     printf("%d\n", P > M ? M : P);
00084     return 0;
00085 }

```

7.93 Ejemplo003.c

[Go to the documentation of this file.](#)

```

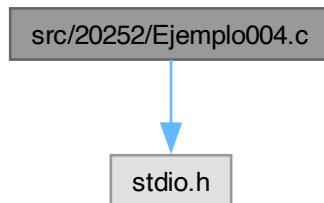
00001
00043
00044 #include <stdio.h>
00045
00052 int main(int argc, char *argv[])
00053 {
00054     (void)argc;
00055     (void)argv;
00056
00057     /*
00058     VARIABLES (según como ESTE código está leyendo la entrada):
00059     M = cantidad de Pokémon capturados (enunciado lo llama N)
00060     N = cantidad de barras de dulce (candies) (enunciado lo llama M)
00061     X = costo en dulces para evolucionar 1 Pokémon
00062     Y = dulces que se ganan al vender 1 Pokémon (NO evolucionado)
00063     P = número máximo de Pokémon que se pueden evolucionar (respuesta)
00064     */
00065     int M, N, X, Y, P;
00066
00067     /* Lee: M N X Y (nota: el enunciado usualmente pide N M X Y). */
00068     scanf("%d %d %d %d", &M, &N, &X, &Y);
00069
00070     /*
00071     Si evolucionamos P Pokémon y vendemos los (M - P) restantes:
00072     dulces disponibles = N + (M - P)*Y
00073     Requisito para evolucionar P:
00074     N + (M - P)*Y >= P*X
00075     => N + M*Y >= P*(X + Y)
00076     => P <= floor((N + M*Y)/(X + Y))
00077
00078     Además, P no puede ser mayor que M (no puedes evolucionar más Pokémon de los que tienes).
00079     */
00080     P = (N + Y * M) / (X + Y);
00081
00082     /* Imprime min(P, M). */
00083     printf("%d\n", P > M ? M : P);
00084     return 0;
00085 }

```


7.94 src/20252/Ejemplo004.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo004.c:



Functions

- int [main](#) (int argc, char *argv[])

7.94.1 Function Documentation

7.94.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo004.c](#).

```
00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     scanf("%f %f %f %f %f", &T1, &T2, &T3, &T4, &T);
00008     B = ((T4>T)<<3) | ((T3>T)<<2) | ((T2>T)<<1) | (T1>T);
00009     switch(B)
00010     {
00011     case 15:
00012         printf("Muy ");
00013     case 14:
00014         printf("Baja");
00015         break;
00016     case 12:
00017         printf("Templada");
00018         break;
00019     case 0:
00020         printf("Muy ");
00021     default:
00022         printf("Alta");
00023     }
00024     printf(".\n");
00025     return 0;
00026 }
```

7.95 Ejemplo004.c

[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     scanf("%f %f %f %f %f", &T1, &T2, &T3, &T4, &T);
00008     B = ((T4>T)<<3) | ((T3>T)<<2) | ((T2>T)<<1) | (T1>T);
```

```

00009     switch (B)
00010     {
00011     case 15:
00012         printf("Muy ");
00013     case 14:
00014         printf("Baja");
00015         break;
00016     case 12:
00017         printf("Templada");
00018         break;
00019     case 0:
00020         printf("Muy ");
00021     default:
00022         printf("Alta");
00023     }
00024     printf(".\n");
00025     return 0;
00026 }

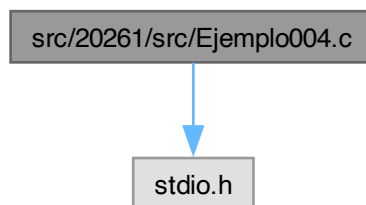
```

7.96 src/20261/src/Ejemplo004.c File Reference

Mínimo y máximo posible de niños en equipo rojo tras exactamente K cambios.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo004.c:



Functions

- int [main](#) (int argc, char *argv[])

Punto de entrada. Calcula mínimo y máximo tamaño del equipo rojo.

7.96.1 Detailed Description

Mínimo y máximo posible de niños en equipo rojo tras exactamente K cambios.

N niños se dividen en dos equipos: R en rojo y N-R en azul. Exactamente K niños se cambian de equipo (cada niño como máximo una vez). Se pide:

- mínimo posible de niños en rojo al final
- máximo posible de niños en rojo al final

Modelado: Sea x = # que cambian de rojo->azul. Entonces $K-x$ cambian de azul->rojo. Rojo final: $R_{\text{final}} = R - x + (K - x) = R + K - 2x$ con restricciones: $0 \leq x \leq R$ $0 \leq K-x \leq N-R$

Entrada

$N \ R \ K$

Salida

Dos enteros: mínimo y máximo posibles en rojo.

Restricciones

1 N 1e5, 0 R, K N.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Definition in file [Ejemplo004.c](#).

7.96.2 Function Documentation**7.96.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula mínimo y máximo tamaño del equipo rojo.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 42 of file [Ejemplo004.c](#).

```
00043 {
00044     (void)argc;
00045     (void)argv;
00046
00047     int N, R, K, Rmn, Rmx;
00048
00049     scanf("%d %d %d", &N, &R, &K);
00050
00051     /*
00052      * Mínimo:
00053      * Se logra maximizando x (tantos rojos como sea posible se van a azul).
00054      * x_max = min(K, R)
00055      * Rmn = R + K - 2*x_max
00056      * Esto equivale a |R - K|.
00057      * El código implementa |R-K| sin if usando el signo:
00058      * (2*(R>=K)-1) es +1 si R>=K, -1 si R<K.
00059      */
00060     Rmn = (2*(R >= K) - 1) * (R - K);
00061
00062     /*
00063      * Máximo:
00064      * Se logra minimizando x (que se vayan lo menos posible del rojo).
00065      * x_min = max(0, K - (N - R))
00066      * Si K <= (N - R): Rmx = R + K
00067      * Si K > (N - R): Rmx = 2N - (R + K)
00068      * El código evita if usando booleanos.
00069      */
00070     Rmx = ((N - R) < K) * 2 * N
00071           + (2 * ((N - R) >= K) - 1) * (R + K);
00072
00073     printf("%d\t%d\n", Rmn, Rmx);
00074     return 0;
00075 }
```

7.97 Ejemplo004.c

[Go to the documentation of this file.](#)

```
00001
00033
00034 #include <stdio.h>
00035
```

```

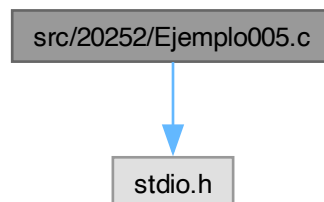
00042 int main(int argc, char *argv[])
00043 {
00044     (void)argc;
00045     (void)argv;
00046
00047     int N, R, K, Rmn, Rmx;
00048
00049     scanf("%d %d %d", &N, &R, &K);
00050
00051     /*
00052     Mínimo:
00053     Se logra maximizando x (tantos rojos como sea posible se van a azul).
00054     x_max = min(K, R)
00055     Rmn = R + K - 2*x_max
00056     Esto equivale a |R - K|.
00057     El código implementa |R-K| sin if usando el signo:
00058     (2*(R>=K)-1) es +1 si R>=K, -1 si R<K.
00059     */
00060     Rmn = (2*(R >= K) - 1) * (R - K);
00061
00062     /*
00063     Máximo:
00064     Se logra minimizando x (que se vayan lo menos posible del rojo).
00065     x_min = max(0, K - (N - R))
00066     Si K <= (N - R): Rmx = R + K
00067     Si K > (N - R): Rmx = 2N - (R + K)
00068     El código evita if usando booleanos.
00069     */
00070     Rmx = ((N - R) < K) * 2 * N
00071           + (2 * ((N - R) >= K) - 1) * (R + K);
00072
00073     printf("%d\t%d\n", Rmn, Rmx);
00074     return 0;
00075 }

```

7.98 src/20252/Ejemplo005.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo005.c:



Functions

- int [main](#) (int argc, char *argv[])

7.98.1 Function Documentation

7.98.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo005.c](#).

```

00004 {
00005     // s es el signo

```

```

00006 // d es el denominador
00007 // n es el numero de terminos
00008 // i es el contador
00009 // pi es el valor de pi
00010 int n, i, s, d;
00011 float pi;
00012 do{
00013     printf("Ingrese el numero de terminos: ");
00014     scanf("%d", &n);
00015 }while(n<=0);
00016 for(i=0, pi=0; i<n; i++)
00017 {
00018     s = 1-2*(i%2);
00019     d = 2*i+1;
00020     pi+=(s*1.0/d);
00021 }
00022 pi*=4;
00023 printf("PI = %f\n", pi);
00024 return 0;
00025 }

```

7.99 Ejemplo005.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     // s es el signo
00006     // d es el denominador
00007     // n es el numero de terminos
00008     // i es el contador
00009     // pi es el valor de pi
00010     int n, i, s, d;
00011     float pi;
00012     do{
00013         printf("Ingrese el numero de terminos: ");
00014         scanf("%d", &n);
00015     }while(n<=0);
00016     for(i=0, pi=0; i<n; i++)
00017     {
00018         s = 1-2*(i%2);
00019         d = 2*i+1;
00020         pi+=(s*1.0/d);
00021     }
00022     pi*=4;
00023     printf("PI = %f\n", pi);
00024     return 0;
00025 }

```

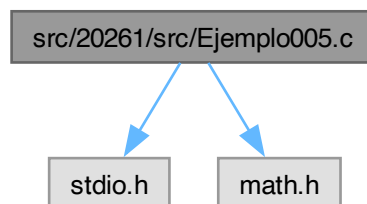
7.100 src/20261/src/Ejemplo005.c File Reference

Solución de la ecuación cuadrática con raíces reales o complejas.

```
#include <stdio.h>
```

```
#include <math.h>
```

Include dependency graph for Ejemplo005.c:



Functions

- `int main (int argc, char *argv[ ])`

Punto de entrada. Calcula y muestra raíces (reales o complejas).

7.100.1 Detailed Description

Solución de la ecuación cuadrática con raíces reales o complejas.

Resuelve: $ax^2 + bx + c = 0$ usando el discriminante: $\Delta = b^2 - 4ac$

Casos:

- $\Delta > 0$: dos raíces reales distintas
- $\Delta = 0$: una raíz real doble
- $\Delta < 0$: dos raíces complejas conjugadas

Para $\Delta < 0$: $\text{sqrt}() = i*\text{sqrt}(-)$ y las raíces se expresan como: $x = (-b \pm i*\text{sqrt}(-)) / (2a)$

Entrada

Tres valores float, leídos por consola (a, b, c).

Salida

Imprime las dos raíces en forma: $x = \text{real} + \text{imag}*i$

Warning

Si $a = 0$ la ecuación no es cuadrática; este programa divide entre $(2a)$.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Definition in file [Ejemplo005.c](#).

7.100.2 Function Documentation

7.100.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Punto de entrada. Calcula y muestra raíces (reales o complejas).

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 44 of file [Ejemplo005.c](#).

```
00045 {
00046     (void)argc;
00047     (void)argv;
00048
00049     float a, b, c;           /* coeficientes */
00050     float r;                 /* discriminante */
00051     float x1r, x1i, x2r, x2i; /* partes real/imag de cada raíz */
```

```

00052
00053     printf("Ingrese el termino cuadratico: ");
00054     scanf("%f", &a);
00055
00056     printf("Ingrese el termino lineal: ");
00057     scanf("%f", &b);
00058
00059     printf("Ingrese el termino independiente: ");
00060     scanf("%f", &c);
00061
00062     /* = b^2 - 4ac */
00063     r = b*b - 4*a*c;
00064
00065     /*
00066         Para >= 0:
00067         x1 = (-b + sqrt()) / (2a), parte imaginaria 0
00068         x2 = (-b - sqrt()) / (2a), parte imaginaria 0
00069
00070         Para < 0:
00071         parte real: (-b)/(2a)
00072         parte imag: ± sqrt(-)/(2a)
00073     */
00074     x1r = (-b + (r >= 0 ? sqrt(r) : 0)) / (2*a);
00075     x1i = (r >= 0 ? 0 : sqrt(-r) / (2*a));
00076
00077     x2r = (-b - (r >= 0 ? sqrt(r) : 0)) / (2*a);
00078     x2i = (r >= 0 ? 0 : -x1i); /* conjugada */
00079
00080     printf("x1 = %.4f%+.4fi\n", x1r, x1i);
00081     printf("x2 = %.4f%+.4fi\n", x2r, x2i);
00082
00083     return 0;
00084 }

```

7.101 Ejemplo005.c

[Go to the documentation of this file.](#)

```

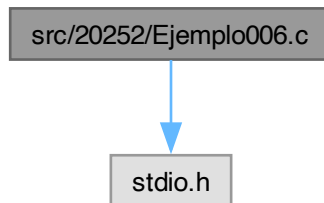
00001
00034
00035 #include <stdio.h>
00036 #include <math.h> /* sqrt() */
00037
00044 int main(int argc, char *argv[])
00045 {
00046     (void)argc;
00047     (void)argv;
00048
00049     float a, b, c; /* coeficientes */
00050     float r; /* discriminante */
00051     float x1r, x1i, x2r, x2i; /* partes real/imag de cada raíz */
00052
00053     printf("Ingrese el termino cuadratico: ");
00054     scanf("%f", &a);
00055
00056     printf("Ingrese el termino lineal: ");
00057     scanf("%f", &b);
00058
00059     printf("Ingrese el termino independiente: ");
00060     scanf("%f", &c);
00061
00062     /* = b^2 - 4ac */
00063     r = b*b - 4*a*c;
00064
00065     /*
00066         Para >= 0:
00067         x1 = (-b + sqrt()) / (2a), parte imaginaria 0
00068         x2 = (-b - sqrt()) / (2a), parte imaginaria 0
00069
00070         Para < 0:
00071         parte real: (-b)/(2a)
00072         parte imag: ± sqrt(-)/(2a)
00073     */
00074     x1r = (-b + (r >= 0 ? sqrt(r) : 0)) / (2*a);
00075     x1i = (r >= 0 ? 0 : sqrt(-r) / (2*a));
00076
00077     x2r = (-b - (r >= 0 ? sqrt(r) : 0)) / (2*a);
00078     x2i = (r >= 0 ? 0 : -x1i); /* conjugada */
00079
00080     printf("x1 = %.4f%+.4fi\n", x1r, x1i);
00081     printf("x2 = %.4f%+.4fi\n", x2r, x2i);
00082
00083     return 0;
00084 }

```

7.102 src/20252/Ejemplo006.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo006.c:



Functions

- int [main](#) (int argc, char *argv[])

7.102.1 Function Documentation

7.102.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo006.c](#).

```
00004 {
00005     int n, i, den;
00006     float x, ex, num;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%d", &n);
00012     }while(n<1);
00013     for(i=0, ex=0, num=1, den=1; i<n; i++)
00014     {
00015         ex+=(num/den);
00016         num*=x;
00017         den*=i+1;
00018     }
00019     printf("exp(%f) = %f\n", x, ex);
00020     return 0;
00021 }
```

7.103 Ejemplo006.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int n, i, den;
00006     float x, ex, num;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%d", &n);
00012     }while(n<1);
00013     for(i=0, ex=0, num=1, den=1; i<n; i++)
```



```

00014      {
00015          ex+=(num/den);
00016          num*=x;
00017          den*= (i+1);
00018      }
00019      printf("exp(%f) = %f\n", x, ex);
00020      return 0;
00021  }

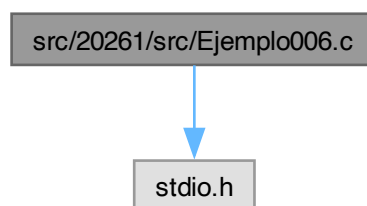
```

7.104 src/20261/src/Ejemplo006.c File Reference

Etiquetado de una temperatura T usando 4 umbrales ($T_1 < T_2 < T_3 < T_4$) y 5 categorías.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo006.c:



Functions

- int `main` (int argc, char *argv[])

Punto de entrada. Clasifica la temperatura T en 5 regiones usando umbrales.

7.104.1 Detailed Description

Etiquetado de una temperatura T usando 4 umbrales ($T_1 < T_2 < T_3 < T_4$) y 5 categorías.

El programa lee cuatro umbrales de temperatura (T_1 , T_2 , T_3 , T_4) y luego una temperatura T que se desea clasificar. Se generan 5 regiones:

- 1) $T < T_1$ -> "Muy baja"
- 2) $T_1 \leq T < T_2$ -> "Baja"
- 3) $T_2 \leq T < T_3$ -> "Templada"
- 4) $T_3 \leq T < T_4$ -> "Alta"
- 5) $T \geq T_4$ -> "Muy alta"

La clasificación se implementa con 4 banderas booleanas ($B_1..B_4$) y se empaquetan en un código de 4 bits ($B_4 \leftarrow B_3B_2B_1$) para seleccionar el mensaje mediante switch().

Entrada

Dos lecturas desde stdin:

```

T1 T2 T3 T4
T

```

Salida

Imprime un código de depuración B (y sus bits) y una etiqueta textual: "muy baja", "baja", "templada", "alta" o "muy alta".

Precondiciones

- Se asume el orden: $T_1 < T_2 < T_3 < T_4$.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Note

Si los umbrales no están ordenados, pueden aparecer códigos no contemplados y el programa podría no imprimir etiqueta (cae en default).

Definition in file [Ejemplo006.c](#).

7.104.2 Function Documentation

7.104.2.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Punto de entrada. Clasifica la temperatura T en 5 regiones usando umbrales.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si termina correctamente.

Definition at line 48 of file [Ejemplo006.c](#).

```
00049 {  
00050     (void)argc;  
00051     (void)argv;  
00052  
00053     /*  
00054      T1, T2, T3, T4: Umbrales de temperatura que delimitan 5 regiones.  
00055      - T1: umbral inferior (más frío)  
00056      - T4: umbral superior (más caliente)  
00057      - Se asume el orden: T1 < T2 < T3 < T4  
00058  
00059      T: temperatura a clasificar (etiquetar).  
00060     */  
00061     float T1, T2, T3, T4, T;  
00062  
00063     /*  
00064      B1..B4 son banderas (booleanos como enteros 0/1) que indican si T es menor  
00065      que cada umbral, usando comparaciones del tipo (Ti > T).  
00066  
00067      Ejemplo:  
00068      B1 = (T1 > T)  -> 1 si T es menor que T1  
00069                      0 si T es mayor o igual que T1  
00070     */  
00071     int B1, B2, B3, B4;  
00072  
00073     /*  
00074      B será un "código" de 4 bits (B4 B3 B2 B1) que identifica en qué intervalo cae T.  
00075      Luego se usa switch(B) para imprimir la etiqueta.  
00076     */  
00077     int B;  
00078  
00079     /* --- Lectura de umbrales --- */  
00080     printf("Ingrese los rangos de temperaturas: ");  
00081     scanf("%f %f %f %f", &T1, &T2, &T3, &T4);  
00082  
00083     /* --- Lectura de la temperatura a evaluar --- */  
00084     printf("Ingrese la temperatura: ");  
00085     scanf("%f", &T);  
00086  
00087     /*  
00088      Se calcula cada bandera:  
00089      - B1 = 1 si T < T1  
00090      - B2 = 1 si T < T2  
00091      - B3 = 1 si T < T3
```

```

00092     - B4 = 1 si T < T4
00093
00094     (Usa Ti > T, que es equivalente a T < Ti)
00095     */
00096     B1 = (T1 > T);
00097     B2 = (T2 > T);
00098     B3 = (T3 > T);
00099     B4 = (T4 > T);
00100
00101     /*
00102     Empaquetar (B4 B3 B2 B1) en un entero B usando desplazamientos:
00103         B4 « 3  coloca B4 en el bit 3 (valor 8)
00104         B3 « 2  coloca B3 en el bit 2 (valor 4)
00105         B2 « 1  coloca B2 en el bit 1 (valor 2)
00106         B1      queda en el bit 0 (valor 1)
00107
00108     Resultado:
00109         B = (B4*8) + (B3*4) + (B2*2) + (B1*1)
00110     */
00111     B = (B4 « 3) | (B3 « 2) | (B2 « 1) | (B1);
00112
00113     /* Depuración: imprime B y sus bits B4B3B2B1 */
00114     printf("%d (%d%d%d%d)\n", B, B4, B3, B2, B1);
00115
00116     printf("La temperatura es ");
00117
00118     /*
00119     Para T1<T2<T3<T4, los códigos típicos son:
00120         T < T1      -> 1111 (15) -> "muy baja"
00121         T1<=T<T2    -> 1110 (14) -> "baja"
00122         T2<=T<T3    -> 1100 (12) -> "templada"
00123         T3<=T<T4    -> 1000 ( 8) -> "alta"
00124         T>=T4       -> 0000 ( 0) -> "muy alta"
00125     */
00126     switch (B)
00127     {
00128     case 15:
00129         printf("muy ");
00130         /* fall-through intencional: completa con "baja" */
00131     case 14:
00132         printf("baja\n");
00133         break;
00134
00135     case 12:
00136         printf("templada\n");
00137         break;
00138
00139     case 0:
00140         printf("muy ");
00141         /* fall-through intencional: completa con "alta" */
00142     case 8:
00143         printf("alta\n");
00144         break;
00145
00146     default:
00147         /* Código no contemplado (p.ej. umbrales desordenados) */
00148         break;
00149     }
00150
00151     printf("\n");
00152     return 0;
00153 }

```

7.105 Ejemplo006.c

[Go to the documentation of this file.](#)

```

00001
00039
00040 #include <stdio.h>
00041
00048 int main(int argc, char *argv[])
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     /*
00054     T1, T2, T3, T4: Umbrales de temperatura que delimitan 5 regiones.
00055         - T1: umbral inferior (más frío)
00056         - T4: umbral superior (más caliente)
00057         - Se asume el orden: T1 < T2 < T3 < T4
00058
00059     T: temperatura a clasificar (etiquetar).
00060     */
00061     float T1, T2, T3, T4, T;

```

```

00062
00063 /*
00064 B1..B4 son banderas (booleanos como enteros 0/1) que indican si T es menor
00065 que cada umbral, usando comparaciones del tipo (Ti > T).
00066
00067 Ejemplo:
00068     B1 = (T1 > T)  -> 1 si T es menor que T1
00069                  0 si T es mayor o igual que T1
00070 */
00071 int B1, B2, B3, B4;
00072
00073 /*
00074 B será un "código" de 4 bits (B4 B3 B2 B1) que identifica en qué intervalo cae T.
00075 Luego se usa switch(B) para imprimir la etiqueta.
00076 */
00077 int B;
00078
00079 /* --- Lectura de umbrales --- */
00080 printf("Ingrese los rangos de temperaturas: ");
00081 scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00082
00083 /* --- Lectura de la temperatura a evaluar --- */
00084 printf("Ingrese la temperatura: ");
00085 scanf("%f", &T);
00086
00087 /*
00088 Se calcula cada bandera:
00089 - B1 = 1 si T < T1
00090 - B2 = 1 si T < T2
00091 - B3 = 1 si T < T3
00092 - B4 = 1 si T < T4
00093
00094 (Usa Ti > T, que es equivalente a T < Ti)
00095 */
00096 B1 = (T1 > T);
00097 B2 = (T2 > T);
00098 B3 = (T3 > T);
00099 B4 = (T4 > T);
00100
00101 /*
00102 Empaquetar (B4 B3 B2 B1) en un entero B usando desplazamientos:
00103     B4 << 3  coloca B4 en el bit 3 (valor 8)
00104     B3 << 2  coloca B3 en el bit 2 (valor 4)
00105     B2 << 1  coloca B2 en el bit 1 (valor 2)
00106     B1       queda en el bit 0 (valor 1)
00107
00108 Resultado:
00109     B = (B4*8) + (B3*4) + (B2*2) + (B1*1)
00110 */
00111 B = (B4 << 3) | (B3 << 2) | (B2 << 1) | (B1);
00112
00113 /* Depuración: imprime B y sus bits B4B3B2B1 */
00114 printf("%d (%d%d%d%d)\n", B, B4, B3, B2, B1);
00115
00116 printf("La temperatura es ");
00117
00118 /*
00119 Para T1<T2<T3<T4, los códigos típicos son:
00120     T < T1      -> 1111 (15) -> "muy baja"
00121     T1<=T<T2    -> 1110 (14) -> "baja"
00122     T2<=T<T3    -> 1100 (12) -> "templada"
00123     T3<=T<T4    -> 1000 ( 8) -> "alta"
00124     T>=T4       -> 0000 ( 0) -> "muy alta"
00125 */
00126 switch (B)
00127 {
00128     case 15:
00129         printf("muy ");
00130         /* fall-through intencional: completa con "baja" */
00131     case 14:
00132         printf("baja\n");
00133         break;
00134
00135     case 12:
00136         printf("templada\n");
00137         break;
00138
00139     case 0:
00140         printf("muy ");
00141         /* fall-through intencional: completa con "alta" */
00142     case 8:
00143         printf("alta\n");
00144         break;
00145
00146     default:
00147         /* Código no contemplado (p.ej. umbrales desordenados) */
00148         break;

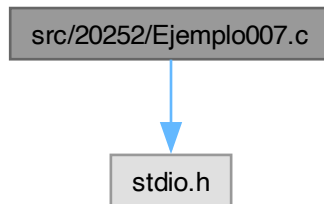
```

```
00149     }
00150
00151     printf("\n");
00152     return 0;
00153 }
```

7.106 src/20252/Ejemplo007.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo007.c:



Functions

- int [main](#) (int argc, char *argv[])

7.106.1 Function Documentation

7.106.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo007.c](#).

```
00004 {
00005     int n, i;
00006     float x, ex, fct;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%d", &n);
00012     }while(n<1);
00013     for(i=0, ex=0, fct=1; i<n; i++)
00014     {
00015         ex+=fct;
00016         fct *= x/(i+1);
00017     }
00018     printf("exp(%f) = %f\n", x, ex);
00019     return 0;
00020 }
```

Here is the call graph for this function:



7.107 Ejemplo007.c

[Go to the documentation of this file.](#)

```

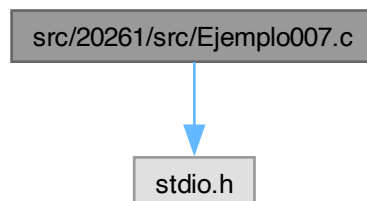
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int n, i;
00006     float x, ex, fct;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%d", &n);
00012     }while(n<1);
00013     for(i=0, ex=0, fct=1; i<n; i++)
00014     {
00015         ex+=fct;
00016         fct *= x/(i+1);
00017     }
00018     printf("exp(%f) = %f\n", x, ex);
00019     return 0;
00020 }
  
```

7.108 src/20261/src/Ejemplo007.c File Reference

Cálculo del entero i tal que $1+3+5+\dots+(2i-1) \geq n$ (relación con $\text{sqrt}(n)$).

#include <stdio.h>

Include dependency graph for Ejemplo007.c:



Functions

- int `main` (int argc, char *argv[])

Punto de entrada. Calcula i restando impares consecutivos.

7.108.1 Detailed Description

Cálculo del entero i tal que $1+3+5+\dots+(2i-1) \geq n$ (relación con $\text{sqrt}(n)$).

El programa usa la identidad: $1 + 3 + 5 + \dots + (2i-1) = i^2$

Restando números impares consecutivos a rn (inicialmente $rn=n$), el contador i se incrementa hasta que $rn \leq 0$.

Interpretación:

- Si n es un cuadrado perfecto ($n = m^2$), entonces al final $i = m$ y rn llega a 0.
- Si n NO es cuadrado perfecto, el algoritmo termina con $i = \text{ceil}(\text{sqrt}(n))$.

Entrada

Un entero $n > 0$.

Salida

Imprime una línea con el formato:

$i^2 = n$

Warning

El texto impreso sugiere igualdad exacta, pero si n no es cuadrado perfecto, $i^2 \neq n$. El valor i corresponde a $\text{ceil}(\text{sqrt}(n))$.

Complejidad

Tiempo: $O(\text{sqrt}(n))$. Memoria: $O(1)$.

Definition in file [Ejemplo007.c](#).

7.108.2 Function Documentation

7.108.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula i restando impares consecutivos.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 41 of file [Ejemplo007.c](#).

```
00042 {
00043     (void)argc;
00044     (void)argv;
00045
00046     int n;    /* número de entrada */
00047     int rn;   /* "residuo" que se va reduciendo restando impares */
00048     int i;    /* contador de impares restados */
00049
00050     /* Validar que n sea positivo */
00051     do {
00052         printf("Ingrese el valor de n: ");
00053         scanf("%d", &n);
00054     } while (n <= 0);
00055
00056     rn = n;
```

```

00057     i = 0;
00058
00059     /*
00060      En cada iteración se resta el siguiente impar:
00061      1, 3, 5, 7, ... que corresponde a (2*i+1) cuando i inicia en 0.
00062      Después de k iteraciones:
00063      rn = n - (1+3+...+(2k-1)) = n - k^2
00064     */
00065     while (rn > 0)
00066     {
00067         rn -= (2*i + 1); /* resta impar actual */
00068         i++;           /* incrementa contador */
00069     }
00070
00071     /* Nota: i será sqrt(n) si n es cuadrado perfecto; si no, i = ceil(sqrt(n)) */
00072     printf("%d^2 = %d\n", i, n);
00073
00074     return 0;
00075 }

```

7.109 Ejemplo007.c

[Go to the documentation of this file.](#)

```

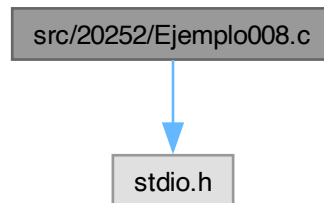
00001
00032
00033 #include <stdio.h>
00034
00041 int main(int argc, char *argv[])
00042 {
00043     (void)argc;
00044     (void)argv;
00045
00046     int n; /* número de entrada */
00047     int rn; /* "residuo" que se va reduciendo restando impares */
00048     int i; /* contador de impares restados */
00049
00050     /* Validar que n sea positivo */
00051     do {
00052         printf("Ingrese el valor de n: ");
00053         scanf("%d", &n);
00054     } while (n <= 0);
00055
00056     rn = n;
00057     i = 0;
00058
00059     /*
00060      En cada iteración se resta el siguiente impar:
00061      1, 3, 5, 7, ... que corresponde a (2*i+1) cuando i inicia en 0.
00062      Después de k iteraciones:
00063      rn = n - (1+3+...+(2k-1)) = n - k^2
00064     */
00065     while (rn > 0)
00066     {
00067         rn -= (2*i + 1); /* resta impar actual */
00068         i++;           /* incrementa contador */
00069     }
00070
00071     /* Nota: i será sqrt(n) si n es cuadrado perfecto; si no, i = ceil(sqrt(n)) */
00072     printf("%d^2 = %d\n", i, n);
00073
00074     return 0;
00075 }

```


7.110 src/20252/Ejemplo008.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo008.c:



Functions

- int [main](#) (int argc, char *argv[])

7.110.1 Function Documentation

7.110.1.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 3 of file [Ejemplo008.c](#).

```
00004 {  
00005     int i, n;  
00006     float x, lnx, fct, num;  
00007     do{  
00008         printf("Ingrese el numero de terminos: ");  
00009         scanf("%d", &n);  
00010     }while(n<1);  
00011     do{  
00012         printf("Ingrese l valor de x: ");  
00013         scanf("%f", &x);  
00014     }while(x<=0);  
00015     for(i=0, fct=(x-1)/(x+1), num=fct, lnx=0; i<n; i++)  
00016     {  
00017         lnx+=(num/(2*i+1));  
00018         num*=(fct*fct);  
00019     }  
00020     lnx*=2;  
00021     printf("ln(%f) = %f\n", x, lnx);  
00022     return 0;  
00023 }
```

Here is the call graph for this function:



7.111 Ejemplo008.c

[Go to the documentation of this file.](#)

```

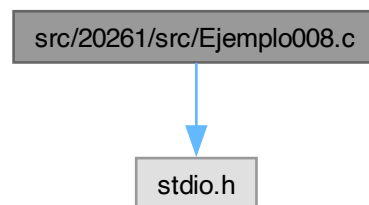
00001 #include<stdio.h>
00002
00003 int main(int argc, char*argv[])
00004 {
00005     int i, n;
00006     float x, lnx, fct, num;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<1);
00011     do{
00012         printf("Ingrese l valor de x: ");
00013         scanf("%f", &x);
00014     }while(x<=0);
00015     for(i=0, fct=(x-1)/(x+1), num=fct, lnx=0; i<n; i++)
00016     {
00017         lnx+=(num/(2*i+1));
00018         num*=(fct*fct);
00019     }
00020     lnx*=2;
00021     printf("ln(%f) = %f\n", x, lnx);
00022     return 0;
00023 }
```

7.112 src/20261/src/Ejemplo008.c File Reference

Aproximación de mediante la serie de Leibniz.

#include <stdio.h>

Include dependency graph for Ejemplo008.c:



Functions

- int [main](#)(int argc, char *argv[])

Punto de entrada. Calcula una aproximación de con n términos.

7.112.1 Detailed Description

Aproximación de mediante la serie de Leibniz.

Usa la serie: $\frac{1}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$

Con n términos: $4 * \sum_{i=0}^{n-1} (-1)^i / (2i+1)$

Entrada

Un entero $n \geq 1$ (número de términos).

Salida

Imprime el valor aproximado:

PI = <valor>

Complejidad

Tiempo: $O(n)$. Memoria: $O(1)$.

Note

La convergencia es lenta: para obtener muchos decimales se requieren muchos términos.

Definition in file [Ejemplo008.c](#).

7.112.2 Function Documentation**7.112.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula una aproximación de π con n términos.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 36 of file [Ejemplo008.c](#).

```
00037 {
00038     (void)argc;
00039     (void)argv;
00040
00041     float pi;      /* acumulador para  $\pi$  */
00042     int i, n;      /* i: índice; n: términos */
00043     int den;       /* denominador (2i+1) */
00044     int signo;     /* +1 o -1 alternando */
00045
00046     /* Validar n >= 1 */
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     /*
00053      * Inicialización:
00054      * i = 0
00055      * pi = 0 (acumulador de  $\pi$  antes de multiplicar por 4)
00056      */
00057     for (i = 0, pi = 0; i < n; i++)
00058     {
00059         den = 2*i + 1;      /* 1, 3, 5, 7, ... */
00060         signo = 1 - 2*(i % 2); /* i par -> +1, i impar -> -1 */
00061         pi += (signo * 1.0f / den);
00062     }
00063
00064     pi *= 4; /*  $\pi = 4 * (\text{pi})$  */
00065     printf("PI = %f\n", pi);
00066
00067     return 0;
00068 }
```

7.113 Ejemplo008.c

[Go to the documentation of this file.](#)

```

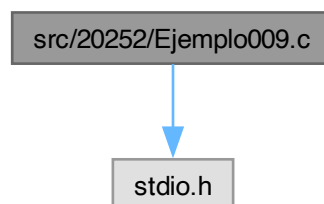
00001
00027
00028 #include <stdio.h>
00029
00036 int main(int argc, char *argv[])
00037 {
00038     (void)argc;
00039     (void)argv;
00040
00041     float pi;        /* acumulador para */
00042     int i, n;        /* i: índice; n: términos */
00043     int den;         /* denominador (2i+1) */
00044     int signo;       /* +1 o -1 alternando */
00045
00046     /* Validar n >= 1 */
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     /*
00053      Inicialización:
00054      i = 0
00055      pi = 0 (acumulador de /4 antes de multiplicar por 4)
00056     */
00057     for (i = 0, pi = 0; i < n; i++)
00058     {
00059         den = 2*i + 1;        /* 1, 3, 5, 7, ... */
00060         signo = 1 - 2*(i % 2); /* i par -> +1, i impar -> -1 */
00061         pi += (signo * 1.0f / den);
00062     }
00063
00064     pi *= 4; /* = 4*(/4) */
00065     printf("PI = %f\n", pi);
00066
00067     return 0;
00068 }

```

7.114 src/20252/Ejemplo009.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo009.c:



Functions

- int `main` (int argc, char *argv[])

7.114.1 Function Documentation

7.114.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo009.c](#).

```

00004 {
00005     int i, n, s;
00006     long den;
00007     float x, sx, num;
00008     do{
00009         printf("Ingrese el numero de terminos: ");
00010         scanf("%d", &n);
00011     }while(n<1);
00012     printf("Ingrese el valor de x: ");
00013     scanf("%f", &x);
00014     for(i=0, num=x, den=1, sx=0; i<n; i++)
00015     {
00016         s = 1-2*(i%2);
00017         sx += (s*num/den);
00018         //printf("%d. %d %f %ld %f\n", i, s, num, den, sx);
00019         num*=(x*x);
00020         den*=( (2*i+2)*(2*i+3) );
00021     }
00022     printf("sin(%f) = %f\n", x, sx);
00023     return 0;
00024 }
```

7.115 Ejemplo009.c

[Go to the documentation of this file.](#)

```

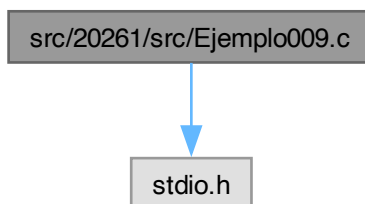
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int i, n, s;
00006     long den;
00007     float x, sx, num;
00008     do{
00009         printf("Ingrese el numero de terminos: ");
00010         scanf("%d", &n);
00011     }while(n<1);
00012     printf("Ingrese el valor de x: ");
00013     scanf("%f", &x);
00014     for(i=0, num=x, den=1, sx=0; i<n; i++)
00015     {
00016         s = 1-2*(i%2);
00017         sx += (s*num/den);
00018         //printf("%d. %d %f %ld %f\n", i, s, num, den, sx);
00019         num*=(x*x);
00020         den*=( (2*i+2)*(2*i+3) );
00021     }
00022     printf("sin(%f) = %f\n", x, sx);
00023     return 0;
00024 }
```

7.116 src/20261/src/Ejemplo009.c File Reference

Aproximación de $\exp(x)$ con serie de Taylor (cálculo directo de potencias y factorial).

#include <stdio.h>

Include dependency graph for Ejemplo009.c:



Functions

- `int main (int argc, char *argv[ ])`

Punto de entrada. Calcula $\exp(x)$ con Taylor y cálculo “desde cero” de cada término.

7.116.1 Detailed Description

Aproximación de $\exp(x)$ con serie de Taylor (cálculo directo de potencias y factorial).

Usa la expansión: $\exp(x) = \sum_{i=0}^{\infty} x^i / i!$

Este programa aproxima con n términos: $\exp(x) \approx \sum_{i=0}^{n-1} x^i / i!$

Para cada término i , calcula:

- $\text{num} = x^i$ (multiplicando x repetidamente)
- $\text{den} = i!$ (multiplicando $1*2*\dots*i$)

Entrada

- Un entero $n \geq 1$ (número de términos).
- Un real x (float).

Salida

Imprime:

`exp(x) = <aprox>`

Complejidad

Tiempo: $O(n^2)$ (por el doble ciclo). Memoria: $O(1)$.

Warning

El factorial crece muy rápido; 'den' (long int) puede desbordarse para i moderados. También puede perderse precisión por usar float.

Definition in file [Ejemplo009.c](#).

7.116.2 Function Documentation

7.116.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula $\exp(x)$ con Taylor y cálculo “desde cero” de cada término.

Parameters

<code>argc</code>	No usado.
<code>argv</code>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 42 of file [Ejemplo009.c](#).

```

00043 {
00044     (void)argc;
00045     (void)argv;
00046
00047     int n, i, j;
00048     long int den; /* factorial i! (puede desbordar para i grandes) */
00049     float x, num, ex; /* num = x^i, ex = acumulador */
00050
00051     /* Leer n >= 1 */
00052     do {
00053         printf("Ingrese el numero de terminos: ");
00054         scanf("%d", &n);
00055     } while (n < 1);
00056
00057     /* Leer x */
00058     printf("Ingrese el valor de x: ");
00059     scanf("%f", &x);
00060
00061     /*
00062      ex acumula la suma:
00063      ex = sum_{i=0}^{n-1} x^i / i!
00064     */
00065     for (i = 0, ex = 0; i < n; i++)
00066     {
00067         /*
00068          Para el término i:
00069          num = x^i
00070          den = i!
00071
00072          Se reinician en 1 y se multiplican i veces.
00073         */
00074         for (j = 0, num = 1, den = 1; j < i; j++)
00075         {
00076             num *= x; /* num = x^(j+1) al avanzar */
00077             den *= (j + 1); /* den = (j+1)! al avanzar */
00078         }
00079
00080         ex += (num / den);
00081     }
00082
00083     printf("exp(%f) = %f\n", x, ex);
00084     return 0;
00085 }

```

7.117 Ejemplo009.c

[Go to the documentation of this file.](#)

```

00001
00033
00034 #include <stdio.h>
00035
00042 int main(int argc, char *argv[])
00043 {
00044     (void)argc;
00045     (void)argv;
00046
00047     int n, i, j;
00048     long int den; /* factorial i! (puede desbordar para i grandes) */
00049     float x, num, ex; /* num = x^i, ex = acumulador */
00050
00051     /* Leer n >= 1 */
00052     do {
00053         printf("Ingrese el numero de terminos: ");
00054         scanf("%d", &n);
00055     } while (n < 1);
00056
00057     /* Leer x */
00058     printf("Ingrese el valor de x: ");
00059     scanf("%f", &x);
00060
00061     /*
00062      ex acumula la suma:
00063      ex = sum_{i=0}^{n-1} x^i / i!
00064     */
00065     for (i = 0, ex = 0; i < n; i++)
00066     {
00067         /*
00068          Para el término i:

```

```

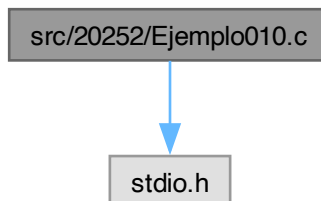
00069         num = x^i
00070         den = i!
00071
00072         Se reinician en 1 y se multiplican i veces.
00073     */
00074     for (j = 0, num = 1, den = 1; j < i; j++)
00075     {
00076         num *= x;          /* num = x^(j+1) al avanzar */
00077         den *= (j + 1);    /* den = (j+1)! al avanzar */
00078     }
00079
00080     ex += (num / den);
00081 }
00082
00083 printf("exp(%f) = %f\n", x, ex);
00084 return 0;
00085 }

```

7.118 src/20252/Ejemplo010.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo010.c:



Functions

- int [main](#) (int argc, char *argv[])

7.118.1 Function Documentation

7.118.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

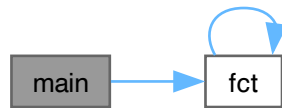
Definition at line 3 of file [Ejemplo010.c](#).

```

00004 {
00005     int i, n, s;
00006     float x, sx, fct;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<1);
00011     printf("Ingrese el valor de x: ");
00012     scanf("%f", &x);
00013     for(i=0, fct=x, sx=0; i<n; i++)
00014     {
00015         s = 1-2*(i%2);
00016         sx += (s*fct);
00017         fct *= (x/(2*i+2)) * (x/(2*i+3));
00018     }
00019     printf("sin(%f) = %f\n", x, sx);
00020     return 0;
00021 }

```


Here is the call graph for this function:



7.119 Ejemplo010.c

[Go to the documentation of this file.](#)

```

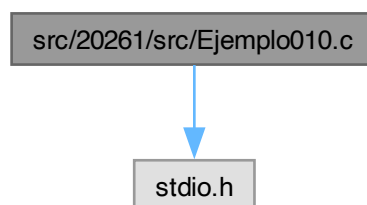
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int i, n, s;
00006     float x, sx, fct;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<1);
00011     printf("Ingrese el valor de x: ");
00012     scanf("%f", &x);
00013     for(i=0, fct=x, sx=0; i<n; i++)
00014     {
00015         s = 1-2*(i%2);
00016         sx += (s*fct);
00017         fct *= (x/(2*i+2)) * (x/(2*i+3));
00018     }
00019     printf("sin(%f) = %f\n", x, sx);
00020     return 0;
00021 }
  
```

7.120 src/20261/src/Ejemplo010.c File Reference

Aproximación de $\exp(x)$ con Taylor usando producto incremental dentro del término.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo010.c:



Functions

- int `main` (int argc, char *argv[])

Punto de entrada. Calcula $\exp(x)$ con Taylor y producto por término.

7.120.1 Detailed Description

Aproximación de $\exp(x)$ con Taylor usando producto incremental dentro del término.

Serie: $\exp(x) = \sum_{i=0}^{\infty} \frac{x^i}{i!}$

Este código calcula cada término i como: $fct = \frac{x^{i-1}}{(j+1)}$ ($x / (j+1)$) lo cual equivale exactamente a: $fct = \frac{x^i}{i!}$

Entrada

- Un entero $n \geq 1$ (términos).
- Un real x (float).

Salida

Imprime:

`exp(x) = <aprox>`

Complejidad

Tiempo: $O(n^2)$ (por el doble ciclo). Memoria: $O(1)$.

Note

Esta forma evita manejar factorial explícito como entero, pero sigue siendo $O(n^2)$.

Definition in file [Ejemplo010.c](#).

7.120.2 Function Documentation

7.120.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula $\exp(x)$ con Taylor y producto por término.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 39 of file [Ejemplo010.c](#).

```
00040 {
00041     (void)argc;
00042     (void)argv;
00043
00044     int n, i, j;
00045     float x, fct, ex;
00046
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     printf("Ingrese el valor de x: ");
00053     scanf("%f", &x);
00054
00055     /*
00056     Para cada i:
00057         fct = x^i / i! calculado como producto:
00058         fct = (x/1)*(x/2)*...*(x/i)
00059     Luego:
00060         ex += fct
00061     */
```

```

00062     for (i = 0, ex = 0; i < n; i++)
00063     {
00064         for (j = 0, fct = 1; j < i; j++)
00065             fct *= (x / (j + 1)); /* multiplica por x/(j+1) */
00066
00067         ex += fct;
00068     }
00069
00070     printf("exp(%f) = %f\n", x, ex);
00071     return 0;
00072 }

```

Here is the call graph for this function:



7.121 Ejemplo010.c

[Go to the documentation of this file.](#)

```

00001
00030
00031 #include <stdio.h>
00032
00039 int main(int argc, char *argv[])
00040 {
00041     (void)argc;
00042     (void)argv;
00043
00044     int n, i, j;
00045     float x, fct, ex;
00046
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     printf("Ingrese el valor de x: ");
00053     scanf("%f", &x);
00054
00055     /*
00056     Para cada i:
00057         fct = x^i / i! calculado como producto:
00058         fct = (x/1)*(x/2)*...*(x/i)
00059     Luego:
00060         ex += fct
00061     */
00062     for (i = 0, ex = 0; i < n; i++)
00063     {
00064         for (j = 0, fct = 1; j < i; j++)
00065             fct *= (x / (j + 1)); /* multiplica por x/(j+1) */
00066
00067         ex += fct;
00068     }
00069
00070     printf("exp(%f) = %f\n", x, ex);
00071     return 0;
00072 }

```

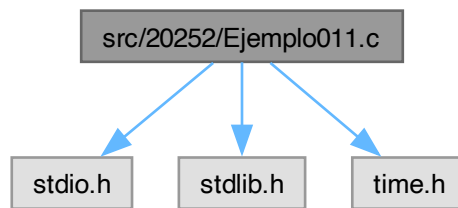
7.122 src/20252/Ejemplo011.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo011.c:



Functions

- `int main (int argc, char *argv[])`

7.122.1 Function Documentation

7.122.1.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 5 of file [Ejemplo011.c](#).

```
00006 {  
00007     int i, n, max, min, x, aux;  
00008     srand(time(NULL));  
00009     do{  
00010         printf("Ingrese el numero de elementos: ");  
00011         scanf("%d", &n);  
00012     }while(n<1);  
00013     printf("Ingrese el valor maximo: ");  
00014     scanf("%d", &max);  
00015     printf("Ingrese el valor minimo: ");  
00016     scanf("%d", &min);  
00017     if(min>max)  
00018     {  
00019         aux = max;  
00020         max = min;  
00021         min = aux;  
00022     }  
00023     for(i=0; i<n; i++)  
00024     {  
00025         x = rand()%(max-min+1)+min;  
00026         printf("%d. %d\n", i+1, x);  
00027     }  
00028     return 0;  
00029 }
```

7.123 Ejemplo011.c

[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>  
00002 #include<stdlib.h>  
00003 #include<time.h>  
00004  
00005 int main(int argc, char*argv[])  
00006 {  
00007     int i, n, max, min, x, aux;  
00008     srand(time(NULL));  
00009     do{  
00010         printf("Ingrese el numero de elementos: ");  
00011         scanf("%d", &n);  
00012     }while(n<1);
```

```

00013     printf("Ingrese el valor maximo: ");
00014     scanf("%d", &max);
00015     printf("Ingrese el valor minimo: ");
00016     scanf("%d", &min);
00017     if(min>max)
00018     {
00019         aux = max;
00020         max = min;
00021         min = aux;
00022     }
00023     for(i=0; i<n; i++)
00024     {
00025         x = rand()%(max-min+1)+min;
00026         printf("%d. %d\n", i+1, x);
00027     }
00028     return 0;
00029 }

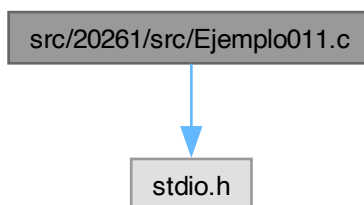
```

7.124 src/20261/src/Ejemplo011.c File Reference

Aproximación de $\exp(x)$ con Taylor usando recurrencia entre términos (forma eficiente).

#include <stdio.h>

Include dependency graph for Ejemplo011.c:



Functions

- int `main` (int argc, char *argv[])

Punto de entrada. Calcula $\exp(x)$ con Taylor usando recurrencia $O(n)$.

7.124.1 Detailed Description

Aproximación de $\exp(x)$ con Taylor usando recurrencia entre términos (forma eficiente).

Serie: $\exp(x) = \sum_{i=0}^{\infty} x^i / i!$

Define el término: $t_i = x^i / i!$

Recurrencia: $t_0 = 1$ $t_{i+1} = t_i * (x / (i+1))$

Con esto se calcula la suma con un solo ciclo: $ex = t_0 + t_1 + \dots + t_{n-1}$

Entrada

- Un entero $n \geq 1$ (términos).
- Un real x (float).

Salida

Imprime:

`exp(x) = <aprox>`

Complejidad

Tiempo: $O(n)$. Memoria: $O(1)$.

Note

Esta es la versión más eficiente de (009–011) porque evita recalcular potencias/factoriales.

Definition in file [Ejemplo011.c](#).

7.124.2 Function Documentation**7.124.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula $\exp(x)$ con Taylor usando recurrencia $O(n)$.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

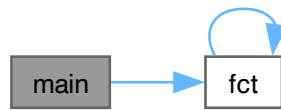
Returns

0 si finaliza correctamente.

Definition at line 44 of file [Ejemplo011.c](#).

```
00045 {
00046     (void)argc;
00047     (void)argv;
00048
00049     int n, i;
00050     float x, fct, ex;
00051
00052     do {
00053         printf("Ingrese el numero de terminos: ");
00054         scanf("%d", &n);
00055     } while (n < 1);
00056
00057     printf("Ingrese el valor de x: ");
00058     scanf("%f", &x);
00059
00060     /*
00061     Inicialización:
00062         fct = t_0 = 1
00063         ex = 0
00064
00065     En cada iteración i:
00066         ex += fct      (acumula t_i)
00067         fct *= x/(i+1) (convierte t_i en t_{i+1})
00068     */
00069     for (i = 0, ex = 0, fct = 1; i < n; i++)
00070     {
00071         ex += fct;
00072         fct *= (x / (i + 1));
00073     }
00074
00075     printf("exp(%f) = %f\n", x, ex);
00076     return 0;
00077 }
```

Here is the call graph for this function:



7.125 Ejemplo011.c

[Go to the documentation of this file.](#)

```

00001
00035
00036 #include <stdio.h>
00037
00044 int main(int argc, char *argv[])
00045 {
00046     (void)argc;
00047     (void)argv;
00048
00049     int n, i;
00050     float x, fct, ex;
00051
00052     do {
00053         printf("Ingrese el numero de terminos: ");
00054         scanf("%d", &n);
00055     } while (n < 1);
00056
00057     printf("Ingrese el valor de x: ");
00058     scanf("%f", &x);
00059
00060     /*
00061     Inicialización:
00062     fct = t_0 = 1
00063     ex = 0
00064
00065     En cada iteración i:
00066     ex += fct      (acumula t_i)
00067     fct *= x/(i+1) (convierte t_i en t_{i+1})
00068     */
00069     for (i = 0, ex = 0, fct = 1; i < n; i++)
00070     {
00071         ex += fct;
00072         fct *= (x / (i + 1));
00073     }
00074
00075     printf("exp(%f) = %f\n", x, ex);
00076     return 0;
00077 }

```

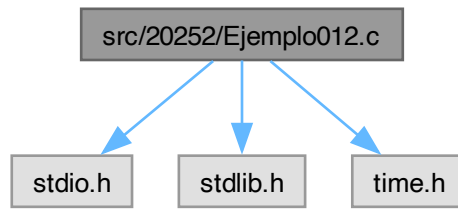
7.126 src/20252/Ejemplo012.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo012.c:



Functions

- int [main](#) (int argc, char *argv[])

7.126.1 Function Documentation

7.126.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 5 of file [Ejemplo012.c](#).

```

00006 {
00007     int i, n;
00008     float max, min, x;
00009     srand(time(NULL));
00010     do{
00011         printf("Ingrese el numero de elementos: ");
00012         scanf("%d", &n);
00013     }while(n<1);
00014     printf("Ingrese el valor maximo: ");
00015     scanf("%f", &max);
00016     printf("Ingrese el valor minimo: ");
00017     scanf("%f", &min);
00018     if(min>max)
00019     {
00020         if(max)
00021         {
00022             if(min)
00023             {
00024                 min*=max;
00025                 max=min/max;
00026                 min/=max;
00027             }
00028             else
00029             {
00030                 min = max;
00031                 max = 0;
00032             }
00033         }
00034         else
00035         {
00036             max = min;
00037             min = 0;
00038         }
00039     }
00040     for(i=0; i<n; i++)
00041     {
00042         x = ((max-min)*rand())/RAND_MAX+min;
00043         printf("%d. %f\n", i+1, x);
00044     }
00045     return 0;
00046 }
```


7.127 Ejemplo012.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 int main(int argc, char*argv[])
00006 {
00007     int i, n;
00008     float max, min, x;
00009     srand(time(NULL));
00010     do{
00011         printf("Ingrese el numero de elementos: ");
00012         scanf("%d", &n);
00013     }while(n<1);
00014     printf("Ingrese el valor maximo: ");
00015     scanf("%f", &max);
00016     printf("Ingrese el valor minimo: ");
00017     scanf("%f", &min);
00018     if(min>max)
00019     {
00020         if(max)
00021         {
00022             if(min)
00023             {
00024                 min*=max;
00025                 max=min/max;
00026                 min/=max;
00027             }
00028             else
00029             {
00030                 min = max;
00031                 max = 0;
00032             }
00033         }
00034         else
00035         {
00036             max = min;
00037             min = 0;
00038         }
00039     }
00040     for(i=0; i<n; i++)
00041     {
00042         x = ((max-min)*rand())/RAND_MAX+min;
00043         printf("%d. %f\n", i+1, x);
00044     }
00045     return 0;
00046 }

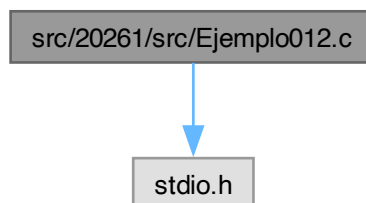
```

7.128 src/20261/src/Ejemplo012.c File Reference

Aproximación de $\ln(x)$ con serie tipo atanh calculando potencias desde cero.

#include <stdio.h>

Include dependency graph for Ejemplo012.c:



Functions

- `int main (int argc, char *argv[ ])`

7.128.1 Detailed Description

Aproximación de $\ln(x)$ con serie tipo atanh calculando potencias desde cero.

Para $x > 0$ se usa la identidad: $\ln(x) = 2 * [f + f^3/3 + f^5/5 + \dots]$ donde: $f = (x - 1) / (x + 1)$

Entonces: $\ln(x) = 2 * \sum_{i=0}^{n-1} f^{2i+1} / (2i+1)$

Este programa:

- Lee n (número de términos) y x (valor real positivo).
- Calcula $f = (x-1)/(x+1)$.
- Para cada término i calcula explícitamente f^{2i+1} multiplicando f varias veces.

Entrada

- Un entero $n \geq 1$ (número de términos).
- Un real $x > 0$.

Salida

Imprime la aproximación:

`ln(x) = valor`

Precondiciones

- x debe ser positivo ($\ln(x)$ solo está definida para $x > 0$ en los reales).

Complejidad

Tiempo: $O(n^2)$ (por el doble ciclo al calcular potencias). Memoria: $O(1)$.

Note

La convergencia es mejor cuando x está cerca de 1 (porque $|f|$ es pequeño).

Definition in file [Ejemplo012.c](#).

7.128.2 Function Documentation

7.128.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 41 of file [Ejemplo012.c](#).

```
00042 {
00043     (void)argc;
00044     (void)argv;
00045
00046     int n, i, j;
00047     int den;           /* denominador impar: 1,3,5,... */
00048     float x;           /* valor para evaluar ln(x) */
00049     float lnx;         /* acumulador de la serie */
00050     float fct;         /* f = (x-1)/(x+1) */
00051     float num;         /* potencia: f^{den} */
00052
00053     /* Leer n >= 1 */
00054     do {
00055         printf("Ingresa el numero de terminos: ");
00056         scanf("%d", &n);
00057     } while (n < 1);
00058
00059     /* Leer x > 0 */
```

```

00060     do {
00061         printf("Ingrese el valor de x: ");
00062         scanf("%f", &x);
00063     } while (x <= 0);
00064
00065     /*
00066     f = (x - 1) / (x + 1)
00067     y la serie es:
00068     ln(x) = 2 * sum_{i=0}^{n-1} f^{2i+1}/(2i+1)
00069     */
00070     for (i = 0, fct = (x - 1) / (x + 1), lnx = 0; i < n; i++)
00071     {
00072         den = 2*i + 1; /* 1, 3, 5, ... */
00073
00074         /*
00075         Calcular num = fct^den multiplicando fct den veces:
00076         num = fct * fct * ... * fct (den factores)
00077         */
00078         for (j = 0, num = 1; j < den; j++)
00079             num *= fct;
00080
00081         /* Sumar el término: f^{den}/den */
00082         lnx += num / den;
00083     }
00084
00085     /* Multiplicar por 2 según la identidad */
00086     lnx *= 2;
00087
00088     printf("ln(%f) = %f\n", x, lnx);
00089     return 0;
00090 }

```

Here is the call graph for this function:



7.129 Ejemplo012.c

[Go to the documentation of this file.](#)

```

00001
00038
00039 #include <stdio.h>
00040
00041 int main(int argc, char *argv[])
00042 {
00043     (void)argc;
00044     (void)argv;
00045
00046     int n, i, j;
00047     int den; /* denominador impar: 1,3,5,... */
00048     float x; /* valor para evaluar ln(x) */
00049     float lnx; /* acumulador de la serie */
00050     float fct; /* f = (x-1)/(x+1) */
00051     float num; /* potencia: f^{den} */
00052
00053     /* Leer n >= 1 */
00054     do {
00055         printf("Ingres el numero de terminos: ");
00056         scanf("%d", &n);
00057     } while (n < 1);
00058
00059     /* Leer x > 0 */
00060     do {
00061         printf("Ingrese el valor de x: ");
00062         scanf("%f", &x);
00063     } while (x <= 0);
00064
00065     /*

```

```

00066     f = (x - 1) / (x + 1)
00067     y la serie es:
00068     ln(x) = 2 * sum_{i=0}^{n-1} f^{2i+1}/(2i+1)
00069 */
00070 for (i = 0, fct = (x - 1) / (x + 1), lnx = 0; i < n; i++)
00071 {
00072     den = 2*i + 1; /* 1, 3, 5, ... */
00073
00074     /*
00075      * Calcular num = fct^den multiplicando fct den veces:
00076      * num = fct * fct * ... * fct (den factores)
00077      */
00078     for (j = 0, num = 1; j < den; j++)
00079         num *= fct;
00080
00081     /* Sumar el término: f^{den}/den */
00082     lnx += num / den;
00083 }
00084
00085 /* Multiplicar por 2 según la identidad */
00086 lnx *= 2;
00087
00088 printf("ln(%f) = %f\n", x, lnx);
00089 return 0;
00090 }

```

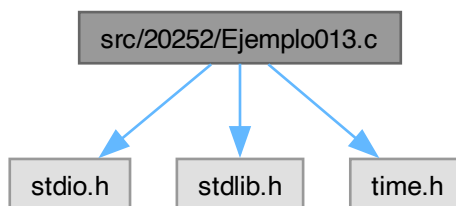
7.130 src/20252/Ejemplo013.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo013.c:



Macros

- #define [N](#) 10000

Functions

- int [main](#) (int argc, char *argv[])

7.130.1 Macro Definition Documentation

7.130.1.1 N

```
#define N 10000
```

Definition at line 5 of file [Ejemplo013.c](#).

7.130.2 Function Documentation

7.130.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file [Ejemplo013.c](#).

```
00008 {
00009     int i, j, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo Burbuja
00049     for(i=0; i<n-1; i++)
00050         for(j=i+1; j<n; j++)
00051             if(x[i]>x[j])
00052             {
00053                 aux = x[i];
00054                 x[i] = x[j];
00055                 x[j] = aux;
00056             }
00057     printf("Ordenado.\n");
00058     for(i=0; i<n; i++)
00059         printf("X[%d] = %f\n", i+1, x[i]);
00060     return 0;
00061 }
```

7.131 Ejemplo013.c

[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 10000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
```

```

00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo Burbuja
00049     for(i=0; i<n-1; i++)
00050         for(j=i+1; j<n; j++)
00051             if(x[i]>x[j])
00052             {
00053                 aux = x[i];
00054                 x[i] = x[j];
00055                 x[j] = aux;
00056             }
00057     printf("Ordenado.\n");
00058     for(i=0; i<n; i++)
00059         printf("X[%d] = %f\n", i+1, x[i]);
00060     return 0;
00061 }

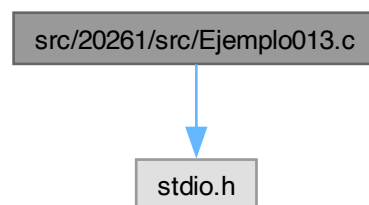
```

7.132 src/20261/src/Ejemplo013.c File Reference

Aproximación de $\ln(x)$ con serie tipo atanh usando recurrencia entre términos.

#include <stdio.h>

Include dependency graph for Ejemplo013.c:



Functions

- int [main](#) (int argc, char *argv[])

7.132.1 Detailed Description

Aproximación de $\ln(x)$ con serie tipo atanh usando recurrencia entre términos.

Se usa la misma identidad que en Ejemplo012: $\ln(x) = 2 * \sum_{i=0}^{\infty} f^{2i+1}/(2i+1)$, $f = (x - 1) / (x + 1)$, $x > 0$.

A diferencia del Ejemplo012, aquí NO se recalcula la potencia desde cero. Se usa la recurrencia de términos: $t_i = f^{2i+1}/(2i+1)$ $t_{i+1} = t_i * f^2 * (2i+1)/(2i+3)$

Entrada

- Un entero $n \geq 1$.
- Un real $x > 0$.

Salida

Imprime:

$\ln(x) = \text{valor}$

Precondiciones

- $x > 0$.

Complejidad

Tiempo: $O(n)$. Memoria: $O(1)$.

Note

Converge más rápido cuando x está cerca de 1 ($|f|$ pequeño).

Definition in file [Ejemplo013.c](#).

7.132.2 Function Documentation

7.132.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 37 of file [Ejemplo013.c](#).

```
00038 {
00039     (void)argc;
00040     (void)argv;
00041
00042     int n, i;
00043     float x;          /* valor para ln(x) */
00044     float lnx;         /* acumulador de la serie */
00045     float fct;         /* f = (x-1)/(x+1) */
00046     float fct2;        /* f^2 */
00047     float suma;        /* término actual t_i = f^{2i+1}/(2i+1) */
00048
00049     /* Leer n >= 1 */
00050     do {
00051         printf("Ingresa el numero de terminos: ");
00052         scanf("%d", &n);
00053     } while (n < 1);
00054
00055     /* Leer x > 0 */
00056     do {
00057         printf("Ingresa el valor de x: ");
00058         scanf("%f", &x);
00059     } while (x <= 0);
00060
00061     /*
00062     Inicialización:
00063         f = (x-1)/(x+1)
00064         f2 = f^2
00065         t0 = f^1/1 = f
00066         lnx = 0
00067     */
00068     for (i = 0, fct = (x - 1) / (x + 1), lnx = 0, fct2 = fct * fct, suma = fct;
00069          i < n;
00070          i++)
```

```

00071     {
00072         /* Acumular término t_i */
00073         lnx += suma;
00074
00075         /*
00076          Actualizar al siguiente término:
00077          t_{i+1} = t_i * f^2 * (2i+1)/(2i+3)
00078          (evita recalcular potencias desde cero)
00079          */
00080         suma *= (fct2 * (2*i + 1)) / (2*i + 3);
00081     }
00082
00083     /* ln(x) = 2 * serie */
00084     lnx *= 2;
00085
00086     printf("ln(%f) = %f\n", x, lnx);
00087     return 0;
00088 }

```

Here is the call graph for this function:



7.133 Ejemplo013.c

[Go to the documentation of this file.](#)

```

00001
00034
00035 #include <stdio.h>
00036
00037 int main(int argc, char *argv[])
00038 {
00039     (void)argc;
00040     (void)argv;
00041
00042     int n, i;
00043     float x;          /* valor para ln(x) */
00044     float lnx;        /* acumulador de la serie */
00045     float fct;        /* f = (x-1)/(x+1) */
00046     float fct2;       /* f^2 */
00047     float suma;       /* término actual t_i = f^{2i+1}/(2i+1) */
00048
00049     /* Leer n >= 1 */
00050     do {
00051         printf("Ingresa el numero de terminos: ");
00052         scanf("%d", &n);
00053     } while (n < 1);
00054
00055     /* Leer x > 0 */
00056     do {
00057         printf("Ingresa el valor de x: ");
00058         scanf("%f", &x);
00059     } while (x <= 0);
00060
00061     /*
00062      Inicialización:
00063      f = (x-1)/(x+1)
00064      f2 = f^2
00065      t0 = f^(1)/1 = f
00066      lnx = 0
00067      */
00068     for (i = 0, fct = (x - 1) / (x + 1), lnx = 0, fct2 = fct * fct, suma = fct;
00069          i < n;
00070          i++)
00071     {
00072         /* Acumular término t_i */
00073         lnx += suma;
00074

```



```

00075      /*
00076      Actualizar al siguiente término:
00077      t_{i+1} = t_i * f^2 * (2i+1)/(2i+3)
00078      (evita recalcular potencias desde cero)
00079      */
00080      suma *= (fct2 * (2*i + 1)) / (2*i + 3);
00081  }
00082
00083  /* ln(x) = 2 * serie */
00084  lnx *= 2;
00085
00086  printf("ln(%f) = %f\n", x, lnx);
00087  return 0;
00088 }

```

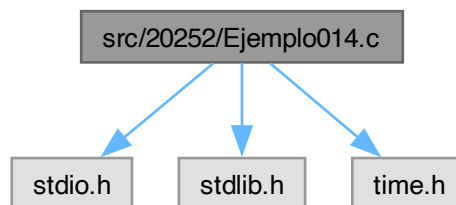
7.134 src/20252/Ejemplo014.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo014.c:



Macros

- `#define N 10000`

Functions

- `int main (int argc, char *argv[ ])`

7.134.1 Macro Definition Documentation

7.134.1.1 N

```
#define N 10000
```

Definition at line 5 of file [Ejemplo014.c](#).

7.134.2 Function Documentation

7.134.2.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 7 of file [Ejemplo014.c](#).

```

00008 {
00009     int i, j, n;
00010     float max, min, x[N], aux;

```

```

00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo Pares y Nones
00049     for(i=0; i<n; i++)
00050     {
00051         for(j=i%2; j<(n+i%2)/2; j++)
00052             if(x[2*j-i%2]>x[2*j+1-i%2])
00053             {
00054                 aux = x[2*j-i%2];
00055                 x[2*j-i%2] = x[2*j+1-i%2];
00056                 x[2*j+1-i%2] = aux;
00057             }
00058     /*
00059     if(i%2)
00060         for(j=1; j<n/2; j++)
00061             x[2*i-1] x[2*i]
00062             x[1] x[2]
00063             x[3] x[4]
00064             x[5] x[6]
00065     else
00066         for(j=0; j<n/2; j++)
00067             x[2*i] x[2*i+1]
00068             x[0] x[1]
00069             x[2] x[3]
00070             x[4] x[5]
00071     */
00072     }
00073     printf("Ordenado.\n");
00074     for(i=0; i<n; i++)
00075         printf("X[%d] = %f\n", i+1, x[i]);
00076     return 0;
00077 }

```

7.135 Ejemplo014.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 10000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");

```

```

00014     scanf("%d", &n);
00015 }while(n<1||n>N);
00016 printf("Ingresa el valor maximo: ");
00017 scanf("%f", &max);
00018 printf("Ingresa el valor minimo: ");
00019 scanf("%f", &min);
00020 if(min>max)
00021 {
00022     if(max)
00023     {
00024         if(min)
00025         {
00026             min*=max;
00027             max=min/max;
00028             min/=max;
00029         }
00030         else
00031         {
00032             min = max;
00033             max = 0;
00034         }
00035     }
00036     else
00037     {
00038         max = min;
00039         min = 0;
00040     }
00041 }
00042 printf("Desordenado.\n");
00043 for(i=0; i<n; i++)
00044 {
00045     x[i] = ((max-min)*rand())/RAND_MAX+min;
00046     printf("X[%d] = %f\n", i+1, x[i]);
00047 }
00048 // Algoritmo Pares y Nones
00049 for(i=0; i<n; i++)
00050 {
00051     for(j=i%2; j<(n+i%2)/2; j++)
00052         if(x[2*j-i%2]>x[2*j+1-i%2])
00053         {
00054             aux = x[2*j-i%2];
00055             x[2*j-i%2] = x[2*j+1-i%2];
00056             x[2*j+1-i%2] = aux;
00057         }
00058     /*
00059     if(i%2)
00060         for(j=1; j<n/2; j++)
00061             x[2*i-1] x[2*i]
00062             x[1] x[2]
00063             x[3] x[4]
00064             x[5] x[6]
00065     else
00066         for(j=0; j<n/2; j++)
00067             x[2*i] x[2*i+1]
00068             x[0] x[1]
00069             x[2] x[3]
00070             x[4] x[5]
00071     */
00072 }
00073 printf("Ordenado.\n");
00074 for(i=0; i<n; i++)
00075     printf("X[%d] = %f\n", i+1, x[i]);
00076 return 0;
00077 }

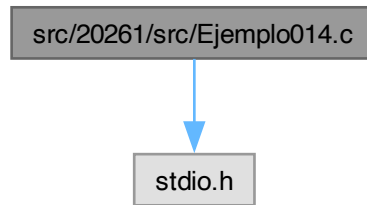
```

7.136 src/20261/src/Ejemplo014.c File Reference

Aproximación de $\sin(x)$ con serie de Taylor calculando potencia y factorial desde cero.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo014.c:



Functions

- int [main](#) (int argc, char *argv[])

7.136.1 Detailed Description

Aproximación de $\sin(x)$ con serie de Taylor calculando potencia y factorial desde cero.

Serie de Taylor: $\sin(x) = \sum_{i=0}^{\infty} \frac{(-1)^i * x^{2i+1}}{(2i+1)!}$

Este programa aproxima con n términos: $\sin(x) \approx \sum_{i=0}^{n-1} \frac{(-1)^i * x^{2i+1}}{(2i+1)!}$

Implementación:

- Para cada i calcula $\text{num} = x^{2i+1}$ y $\text{den} = (2i+1)!$ con un ciclo interno.

Entrada

- Un entero $n \geq 1$.
- Un real x.

Salida

Imprime:

```
sen(x) = valor
```

Complejidad

Tiempo: $O(n^2)$. Memoria: $O(1)$.

Warning

Para $|x|$ grande, puede haber pérdida de precisión (no se hace reducción de rango). Además, el factorial crece rápido (aquí se guarda en int).

Definition in file [Ejemplo014.c](#).

7.136.2 Function Documentation

7.136.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 35 of file [Ejemplo014.c](#).

```

00036 {
00037     (void)argc;
00038     (void)argv;
00039
00040     int n, i, j;
00041     int den;      /* (2i+1)! */
00042     int signo;    /* +1 o -1 */
00043     float x;
00044     float sx;     /* acumulador de sen(x) */
00045     float num;    /* x^(2i+1) */
00046
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     printf("Ingrese el valor de x: ");
00053     scanf("%f", &x);
00054
00055     for (i = 0, sx = 0; i < n; i++)
00056     {
00057         /* signo = +1 si i par, -1 si i impar */
00058         signo = 1 - 2*(i % 2);
00059
00060         /*
00061          * Calcular:
00062          *   num = x^(2i+1)
00063          *   den = (2i+1)!
00064          * multiplicando desde cero en cada término.
00065          */
00066         for (j = 0, num = 1, den = 1; j < (2*i + 1); j++)
00067         {
00068             num *= x;      /* acumula potencia */
00069             den *= (j + 1); /* acumula factorial */
00070         }
00071
00072         /* Sumar el término: (-1)^i * num/den */
00073         sx += (signo * num / den);
00074     }
00075
00076     printf("sen(%f) = %f\n", x, sx);
00077     return 0;
00078 }

```

7.137 Ejemplo014.c

[Go to the documentation of this file.](#)

```

00001
00032
00033 #include <stdio.h>
00034
00035 int main(int argc, char *argv[])
00036 {
00037     (void)argc;
00038     (void)argv;
00039
00040     int n, i, j;
00041     int den;      /* (2i+1)! */
00042     int signo;    /* +1 o -1 */
00043     float x;
00044     float sx;     /* acumulador de sen(x) */
00045     float num;    /* x^(2i+1) */
00046
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     printf("Ingrese el valor de x: ");
00053     scanf("%f", &x);
00054
00055     for (i = 0, sx = 0; i < n; i++)
00056     {
00057         /* signo = +1 si i par, -1 si i impar */
00058         signo = 1 - 2*(i % 2);
00059
00060         /*
00061          * Calcular:
00062          *   num = x^(2i+1)
00063          *   den = (2i+1)!
00064          * multiplicando desde cero en cada término.
00065          */
00066         for (j = 0, num = 1, den = 1; j < (2*i + 1); j++)
00067         {

```

```

00068         num *= x;          /* acumula potencia */
00069         den *= (j + 1);     /* acumula factorial */
00070     }
00071
00072     /* Sumar el término: (-1)^i * num/den */
00073     sx += (signo * num / den);
00074 }
00075
00076 printf("sen(%f) = %f\n", x, sx);
00077 return 0;
00078 }

```

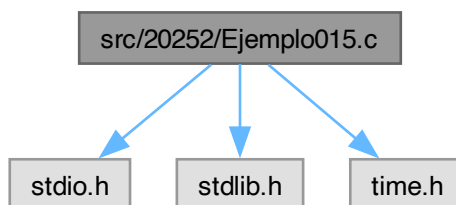
7.138 src/20252/Ejemplo015.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo015.c:



Macros

- `#define N 10000`

Functions

- `int main (int argc, char *argv[ ])`

7.138.1 Macro Definition Documentation

7.138.1.1 N

```
#define N 10000
```

Definition at line 5 of file [Ejemplo015.c](#).

7.138.2 Function Documentation

7.138.2.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 7 of file [Ejemplo015.c](#).

```

00008 {
00009     int i, j, k, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");

```

```

00014     scanf("%d", &n);
00015 }while (n<1||n>N);
00016 printf("Ingrese el valor maximo: ");
00017 scanf("%f", &max);
00018 printf("Ingrese el valor minimo: ");
00019 scanf("%f", &min);
00020 if(min>max)
00021 {
00022     if(max)
00023     {
00024         if(min)
00025         {
00026             min*=max;
00027             max=min/max;
00028             min/=max;
00029         }
00030         else
00031         {
00032             min = max;
00033             max = 0;
00034         }
00035     }
00036     else
00037     {
00038         max = min;
00039         min = 0;
00040     }
00041 }
00042 printf("Desordenado.\n");
00043 for(i=0; i<n; i++)
00044 {
00045     x[i] = ((max-min)*rand())/RAND_MAX+min;
00046     printf("X[%d] = %f\n", i+1, x[i]);
00047 }
00048 // Algoritmo de Seleccion
00049 for(i=0; i<n-1; i++)
00050 {
00051     for(j=i+1, k=i; j<n; j++)
00052         if(x[k]>x[j])
00053             k = j;
00054     if(k!=i)
00055     {
00056         aux = x[i];
00057         x[i] = x[k];
00058         x[k] = aux;
00059     }
00060 }
00061 printf("Ordenado.\n");
00062 for(i=0; i<n; i++)
00063     printf("X[%d] = %f\n", i+1, x[i]);
00064 return 0;
00065 }

```

7.139 Ejemplo015.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 10000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, k, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while (n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;

```

```

00029         }
00030         else
00031         {
00032             min = max;
00033             max = 0;
00034         }
00035     }
00036     else
00037     {
00038         max = min;
00039         min = 0;
00040     }
00041 }
00042 printf("Desordenado.\n");
00043 for(i=0; i<n; i++)
00044 {
00045     x[i] = ((max-min)*rand())/RAND_MAX+min;
00046     printf("X[%d] = %f\n", i+1, x[i]);
00047 }
00048 // Algoritmo de Selecccion
00049 for(i=0; i<n-1; i++)
00050 {
00051     for(j=i+1, k=i; j<n; j++)
00052         if(x[k]>x[j])
00053             k = j;
00054     if(k!=i)
00055     {
00056         aux = x[i];
00057         x[i] = x[k];
00058         x[k] = aux;
00059     }
00060 }
00061 printf("Ordenado.\n");
00062 for(i=0; i<n; i++)
00063     printf("X[%d] = %f\n", i+1, x[i]);
00064 return 0;
00065 }

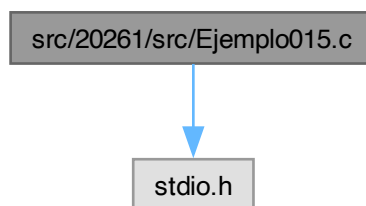
```

7.140 src/20261/src/Ejemplo015.c File Reference

Aproximación de $\sin(x)$ con Taylor usando producto por término (sin factorial entero explícito).

#include <stdio.h>

Include dependency graph for Ejemplo015.c:



Functions

- int [main](#) (int argc, char *argv[])

7.140.1 Detailed Description

Aproximación de $\sin(x)$ con Taylor usando producto por término (sin factorial entero explícito).

Serie: $\sin(x) = \sum_{i=0}^{\infty} \frac{(-1)^i * x^{2i+1}}{(2i+1)!}$

Para cada i , el término: $x^{2i+1}/(2i+1)! = \frac{1}{k!} x^{2i+1}$ (x/k)

Este programa calcula cada término como un producto: $fct = (x/1)*(x/2)*...*(x/(2i+1))$ y alterna el signo con $(-1)^i$.

Entrada

- Un entero $n \geq 1$.
- Un real x .

Salida

Imprime:

`sen(x) = valor`

Complejidad

Tiempo: $O(n^2)$. Memoria: $O(1)$.

Note

Evita factorial como entero, pero sigue recalculando cada término desde cero.

Definition in file [Ejemplo015.c](#).

7.140.2 Function Documentation**7.140.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 35 of file [Ejemplo015.c](#).

```
00036 {
00037     (void)argc;
00038     (void)argv;
00039
00040     int n, i, j;
00041     int signo;
00042     float x;
00043     float sx; /* acumulador */
00044     float fct; /* término positivo: x^(2i+1)/(2i+1)! */
00045
00046     do {
00047         printf("Ingrese el numero de terminos: ");
00048         scanf("%d", &n);
00049     } while (n < 1);
00050
00051     printf("Ingrese el valor de x: ");
00052     scanf("%f", &x);
00053
00054     for (i = 0, sx = 0; i < n; i++)
00055     {
00056         signo = 1 - 2*(i % 2); /* +1, -1, +1, -1... */
00057
00058         /*
00059          fct = _{k=1}^{2i+1} (x/k)
00060          Se implementa con j=0...2i:
00061          multiplica x/(j+1)
00062          */
00063         for (j = 0, fct = 1; j < (2*i + 1); j++)
00064             fct *= (x / (j + 1));
00065
00066         sx += (signo * fct);
00067     }
00068
00069     printf("sen(%f) = %f\n", x, sx);
00070     return 0;
00071 }
```

Here is the call graph for this function:



7.141 Ejemplo015.c

[Go to the documentation of this file.](#)

```

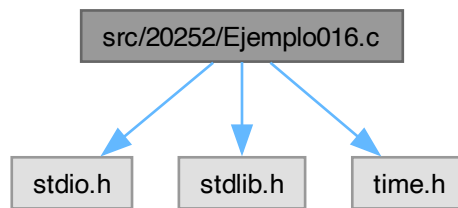
00001
00032
00033 #include <stdio.h>
00034
00035 int main(int argc, char *argv[])
00036 {
00037     (void)argc;
00038     (void)argv;
00039
00040     int n, i, j;
00041     int signo;
00042     float x;
00043     float sx; /* acumulador */
00044     float fct; /* término positivo: x^(2i+1)/(2i+1)! */
00045
00046     do {
00047         printf("Ingrese el numero de terminos: ");
00048         scanf("%d", &n);
00049     } while (n < 1);
00050
00051     printf("Ingrese el valor de x: ");
00052     scanf("%f", &x);
00053
00054     for (i = 0, sx = 0; i < n; i++)
00055     {
00056         signo = 1 - 2*(i % 2); /* +1, -1, +1, -1... */
00057
00058         /*
00059          fct = _{k=1}^{2i+1} (x/k)
00060          Se implementa con j=0..2i:
00061          multiplica x/(j+1)
00062          */
00063         for (j = 0, fct = 1; j < (2*i + 1); j++)
00064             fct *= (x / (j + 1));
00065
00066         sx += (signo * fct);
00067     }
00068
00069     printf("sen(%f) = %f\n", x, sx);
00070     return 0;
00071 }
  
```

7.142 src/20252/Ejemplo016.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
  
```

Include dependency graph for Ejemplo016.c:



Macros

- `#define N 10000`

Functions

- `int main (int argc, char *argv[ ])`

7.142.1 Macro Definition Documentation

7.142.1.1 N

```
#define N 10000
```

Definition at line 5 of file [Ejemplo016.c](#).

7.142.2 Function Documentation

7.142.2.1 main()

```
int main (  
    int argc,  
    char * argv[ ])
```

Definition at line 7 of file [Ejemplo016.c](#).

```
00008 {  
00009     int i, j, k, n;  
00010     float max, min, x[N], aux;  
00011     srand(time(NULL));  
00012     do{  
00013         printf("Ingrese el numero de elementos: ");  
00014         scanf("%d", &n);  
00015     }while(n<1||n>N);  
00016     printf("Ingrese el valor maximo: ");  
00017     scanf("%f", &max);  
00018     printf("Ingrese el valor minimo: ");  
00019     scanf("%f", &min);  
00020     if(min>max)  
00021     {  
00022         if(max)  
00023         {  
00024             if(min)  
00025             {  
00026                 min*=max;  
00027                 max=min/max;  
00028                 min/=max;  
00029             }  
00030             else  
00031             {  
00032                 min = max;  
00033                 max = 0;  
00034             }  
00035         }  
00036     }
```

```

00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo de Insercion
00049     for(i=1; i<n; i++)
00050     {
00051         aux = x[i];
00052         j=i-1;
00053         while(aux<x[j])
00054         {
00055             x[j+1] = x[j];
00056             j--;
00057             if(j<0)
00058                 break;
00059         }
00060         x[j+1] = aux;
00061     }
00062     printf("Ordenado.\n");
00063     for(i=0; i<n; i++)
00064         printf("X[%d] = %f\n", i+1, x[i]);
00065     return 0;
00066 }

```

7.143 Ejemplo016.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 10000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, k, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo de Insercion
00049     for(i=1; i<n; i++)

```

```

00050     {
00051         aux = x[i];
00052         j=i-1;
00053         while (aux<x[j])
00054         {
00055             x[j+1] = x[j];
00056             j--;
00057             if (j<0)
00058                 break;
00059         }
00060         x[j+1] = aux;
00061     }
00062     printf("Ordenado.\n");
00063     for (i=0; i<n; i++)
00064         printf("X[%d] = %f\n", i+1, x[i]);
00065     return 0;
00066 }

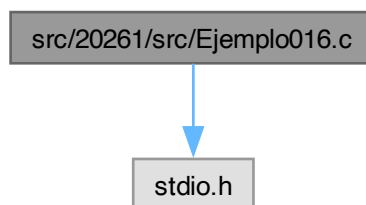
```

7.144 src/20261/src/Ejemplo016.c File Reference

Aproximación de $\sin(x)$ con Taylor usando recurrencia entre términos ($O(n)$).

```
#include <stdio.h>
```

Include dependency graph for Ejemplo016.c:



Functions

- int `main` (int argc, char *argv[])

7.144.1 Detailed Description

Aproximación de $\sin(x)$ con Taylor usando recurrencia entre términos ($O(n)$).

Serie: $\sin(x) = \sum_{i=0}^{\infty} (-1)^i * x^{2i+1} / (2i+1)!$

Definimos el término positivo: $t_i = x^{2i+1} / (2i+1)!$ con recurrencia: $t_0 = x$ $t_{i+1} = t_i * (x^2) / [(2i+2)(2i+3)]$

Luego: $\sin(x) = \sum_{i=0}^{n-1} (-1)^i * t_i$

Entrada

- Un entero $n \geq 1$.
- Un real x .

Salida

Imprime:

```
sen(x) = valor
```

Complejidad

Tiempo: $O(n)$. Memoria: $O(1)$.

Note

Es la versión más eficiente (respecto a Ejemplo014 y Ejemplo015) porque evita recalcular potencias/↔ factoriales en cada término.

Definition in file [Ejemplo016.c](#).

7.144.2 Function Documentation**7.144.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 38 of file [Ejemplo016.c](#).

```
00039 {
00040     (void)argc;
00041     (void)argv;
00042
00043     int n, i;
00044     int signo; /* +1 o -1 alternando */
00045     float x;
00046     float sx; /* acumulador */
00047     float fct; /* término positivo t_i = x^(2i+1)/(2i+1)! */
00048
00049     do {
00050         printf("Ingrese el numero de terminos: ");
00051         scanf("%d", &n);
00052     } while (n < 1);
00053
00054     printf("Ingrese el valor de x: ");
00055     scanf("%f", &x);
00056
00057     /*
00058     Inicialización:
00059     t0 = x
00060     sx = 0
00061     En cada iteración:
00062     sx += (-1)^i * t_i
00063     t_{i+1} = t_i * (x^2) / [(2i+2) (2i+3)]
00064     */
00065     for (i = 0, sx = 0, fct = x; i < n; i++)
00066     {
00067         signo = 1 - 2*(i % 2); /* +1 si i par; -1 si i impar */
00068
00069         sx += (signo * fct);
00070
00071         /* Actualizar al siguiente término positivo */
00072         fct *= (x / (2*i + 2)) * (x / (2*i + 3));
00073     }
00074
00075     printf("sen(%f) = %f\n", x, sx);
00076     return 0;
00077 }
```

Here is the call graph for this function:

**7.145 Ejemplo016.c**

[Go to the documentation of this file.](#)

00001

```

00035
00036 #include <stdio.h>
00037
00038 int main(int argc, char *argv[])
00039 {
00040     (void)argc;
00041     (void)argv;
00042
00043     int n, i;
00044     int signo; /* +1 o -1 alternando */
00045     float x;
00046     float sx; /* acumulador */
00047     float fct; /* término positivo t_i = x^(2i+1)/(2i+1)! */
00048
00049     do {
00050         printf("Ingrese el numero de terminos: ");
00051         scanf("%d", &n);
00052     } while (n < 1);
00053
00054     printf("Ingrese el valor de x: ");
00055     scanf("%f", &x);
00056
00057     /*
00058      Inicialización:
00059      t0 = x
00060      sx = 0
00061      En cada iteración:
00062      sx += (-1)^i * t_i
00063      t_{i+1} = t_i * (x^2)/[(2i+2)(2i+3)]
00064     */
00065     for (i = 0, sx = 0, fct = x; i < n; i++)
00066     {
00067         signo = 1 - 2*(i % 2); /* +1 si i par; -1 si i impar */
00068
00069         sx += (signo * fct);
00070
00071         /* Actualizar al siguiente término positivo */
00072         fct *= (x / (2*i + 2)) * (x / (2*i + 3));
00073     }
00074
00075     printf("sen(%f) = %f\n", x, sx);
00076     return 0;
00077 }

```

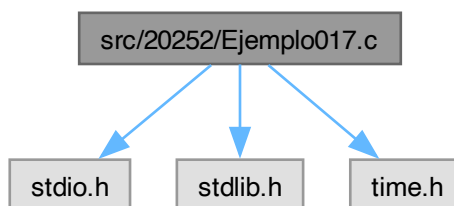
7.146 src/20252/Ejemplo017.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo017.c:



Macros

- #define N 10000

Functions

- int [main](#) (int argc, char *argv[])

7.146.1 Macro Definition Documentation

7.146.1.1 N

#define N 10000

Definition at line 5 of file [Ejemplo017.c](#).

7.146.2 Function Documentation

7.146.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file [Ejemplo017.c](#).

```
00008 {
00009     int i, j, k, l, n, nc, cs[N], id[N], ncs;
00010     float max, min, x[N], xs[N], aux, rng_c;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         cs[i] = 0;
00047         printf("X[%d] = %f\n", i+1, x[i]);
00048     }
00049     // Algoritmo de Bucket
00050     nc = 0;
00051     aux = n;
00052     while(aux>(2*nc+1))
00053     {
00054         aux -= (2*nc+1);
00055         nc++;
00056     }
00057     rng_c = (max-min)/nc;
00058     for(i=0; i<n; i++)
00059         cs[(int)((x[i]-min)/rng_c)]++;
00060     for(i=1, id[0]=0; i<n; i++)
00061         id[i] = id[i-1]+cs[i-1];
00062     for(i=0; i<n; i++)
00063     {
00064         ncs = (int)((x[i]-min)/rng_c);
00065         xs[id[ncs]] = x[i];
00066         id[ncs]++;
```



```

00067     }
00068     printf("%d\t%f\n", nc, rng_c);
00069     for(i=0; i<nc; i++)
00070         printf("CS[%d] = %d\n", i, cs[i]);
00071     for(i=0; i<nc; i++)
00072         printf("%f\t", min+rng_c*(i+1));
00073     printf("Pseudo-ordenado.\n");
00074     for(i=0; i<n; i++)
00075         printf("X[%d] = %f\t%d\t%f\n", i+1, x[i], (int)((x[i]-min)/rng_c), xs[i]);
00076     for(i=1, id[0]=0; i<n; i++)
00077         id[i] = id[i-1]+cs[i-1];
00078     for(l=0; l<nc; l++)
00079     {
00080         // Algoritmo de Seleccion
00081         for(i=id[l]; i<(id[l]+cs[l])-1; i++)
00082         {
00083             for(j=i+1, k=i; j<(id[l]+cs[l]); j++)
00084                 if(xs[k]>xs[j])
00085                     k = j;
00086             if(k!=i)
00087             {
00088                 aux = xs[i];
00089                 xs[i] = xs[k];
00090                 xs[k] = aux;
00091             }
00092         }
00093     }
00094     printf("Ordenado.\n");
00095     for(i=0; i<n; i++)
00096         printf("X[%d] = %f\n", i+1, xs[i]);
00097     return 0;
00098 }

```

7.147 Ejemplo017.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 10000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, k, l, n, nc, cs[N], id[N], ncs;
00010     float max, min, x[N], xs[N], aux, rng_c;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         cs[i] = 0;
00047         printf("X[%d] = %f\n", i+1, x[i]);
00048     }

```

```

00049 // Algoritmo de Bucket
00050 nc = 0;
00051 aux = n;
00052 while(aux > (2*nc+1))
00053 {
00054     aux -= (2*nc+1);
00055     nc++;
00056 }
00057 rng_c = (max-min)/nc;
00058 for(i=0; i<n; i++)
00059     cs[(int)((x[i]-min)/rng_c)]++;
00060 for(i=1, id[0]=0; i<n; i++)
00061     id[i] = id[i-1]+cs[i-1];
00062 for(i=0; i<n; i++)
00063 {
00064     ncs = (int)((x[i]-min)/rng_c);
00065     xs[id[ncs]] = x[i];
00066     id[ncs]++;
00067 }
00068 printf("%d\t%f\n", nc, rng_c);
00069 for(i=0; i<nc; i++)
00070     printf("CS[%d] = %d\n", i, cs[i]);
00071 for(i=0; i<nc; i++)
00072     printf("%f\t", min+rng_c*(i+1));
00073 printf("Pseudo-ordenado.\n");
00074 for(i=0; i<n; i++)
00075     printf("X[%d] = %f\t%d\t%f\n", i+1, x[i], (int)((x[i]-min)/rng_c), xs[i]);
00076 for(i=1, id[0]=0; i<n; i++)
00077     id[i] = id[i-1]+cs[i-1];
00078 for(l=0; l<nc; l++)
00079 {
00080     // Algoritmo de Seleccion
00081     for(i=id[l]; i<(id[l]+cs[l])-1; i++)
00082     {
00083         for(j=i+1, k=i; j<(id[l]+cs[l]); j++)
00084             if(xs[k]>xs[j])
00085                 k = j;
00086         if(k!=i)
00087         {
00088             aux = xs[i];
00089             xs[i] = xs[k];
00090             xs[k] = aux;
00091         }
00092     }
00093 }
00094 printf("Ordenado.\n");
00095 for(i=0; i<n; i++)
00096     printf("X[%d] = %f\n", i+1, xs[i]);
00097 return 0;
00098 }

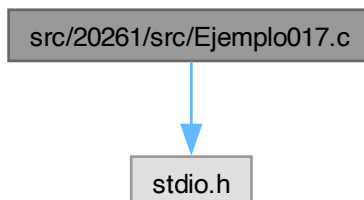
```

7.148 src/20261/src/Ejemplo017.c File Reference

Aproximación de $\arcsin(x)$ mediante serie de Taylor (Maclaurin) para $|x| < 1$.

#include <stdio.h>

Include dependency graph for Ejemplo017.c:



Functions

- `int main (int argc, char *argv[ ])`

Punto de entrada. Aproxima $\arcsin(x)$ con n términos de la serie.

7.148.1 Detailed Description

Aproximación de $\arcsin(x)$ mediante serie de Taylor (Maclaurin) para $|x| < 1$.

Este programa aproxima el seno inverso (arcoseno):

$$\arcsin(x)$$

usando la serie de Maclaurin (Taylor alrededor de 0), válida para $|x| < 1$:

$$\arcsin(x) = \sum_{i=0}^{\infty} \frac{(2i)!}{4^i (i!)^2 (2i+1)} x^{2i+1}, \quad |x| < 1.$$

En lugar de calcular factoriales grandes, la implementación construye cada término de forma incremental:

- `num` acumula la potencia x^{2i+1} a partir de la anterior ($\times x^2$).
- `fact` acumula el coeficiente $C_i = \frac{(2i)!}{4^i (i!)^2}$ mediante la recurrencia:

$$C_i = C_{i-1} \cdot \frac{2i-1}{2i}.$$

Cada iteración suma:

$$\frac{C_i x^{2i+1}}{2i+1}.$$

Entrada estándar

- 1) Un entero n (número de términos), con $n \geq 1$
- 2) Un real x con $-1 < x < 1$

Salida estándar

Imprime la aproximación:

`asen(x) = valor`

Precondiciones

- Se requiere $-1 < x < 1$. La serie converge más lentamente cuando $|x| \rightarrow 1$.

Complejidad

Tiempo: $O(n)$. Memoria: $O(1)$.

Note

- El programa usa `float`; si se desea mayor precisión, usar `double`.
- El nombre impreso es `asen(x)` (arcoseno).

Definition in file [Ejemplo017.c](#).

7.148.2 Function Documentation

7.148.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Punto de entrada. Aproxima $\arcsin(x)$ con n términos de la serie.

Parameters

<code>argc</code>	No usado.
-------------------	-----------

<code>argv</code>	No usado.
-------------------	-----------

Returns

0 si finaliza correctamente.

Definition at line 59 of file [Ejemplo017.c](#).

```

00060 {
00061     (void)argc;
00062     (void)argv;
00063
00064     /*
00065      n   : número de términos de la serie (n >= 1)
00066      i   : índice del término (inicia en 1 porque el término i=0 se inicializa con asx=x)
00067      den : denominador impar (2i+1) del término
00068      x   : valor para evaluar arcsin(x), con -1 < x < 1
00069      asx : acumulador de la suma (aproximación final)
00070      num : potencia acumulada x^(2i+1)
00071      fct : coeficiente acumulado C_i = (2i)! / (4^i (i!)^2)
00072     */
00073     int n, i, den;
00074     float x, asx, num, fct;
00075
00076     /* Leer n >= 1 */
00077     do {
00078         printf("Ingrese el numero de terminos: ");
00079         scanf("%d", &n);
00080     } while (n < 1);
00081
00082     /* Leer x con -1 < x < 1 */
00083     do {
00084         printf("Ingrese el valor de x: ");
00085         scanf("%f", &x);
00086     } while (-1 >= x || x >= 1);
00087
00088     /*
00089      Término inicial (i = 0):
00090      C_0 = 1
00091      x^(2*0+1) = x
00092      (2*0+1) = 1
00093      => arcsin(x)  x
00094     */
00095     for (i = 1, asx = x, num = x, fct = 1; i < n; i++)
00096     {
00097         /*
00098          Actualizar coeficiente:
00099          C_i = C_{i-1} * (2i-1)/(2i)
00100          Se usa 2*i - 1.0f para forzar división en punto flotante.
00101         */
00102         fct *= ((2*i - 1.0f) / (2*i));
00103
00104         /*
00105          Actualizar potencia:
00106          x^(2i+1) = x^(2i-1) * x^2
00107         */
00108         num *= (x * x);
00109
00110         /* Denominador del término: (2i + 1) */
00111         den = (2*i + 1);
00112
00113         /* Sumar término: (C_i * x^(2i+1)) / (2i+1) */
00114         asx += (fct * num / den);
00115     }
00116
00117     printf("asen(%f) = %f\n", x, asx);
00118     return 0;
00119 }
```

Here is the call graph for this function:



7.149 Ejemplo017.c

[Go to the documentation of this file.](#)

```

00001
00050
00051 #include <stdio.h>
00052
00059 int main(int argc, char *argv[])
00060 {
00061     (void)argc;
00062     (void)argv;
00063
00064     /*
00065      n   : número de términos de la serie (n >= 1)
00066      i   : índice del término (inicia en 1 porque el término i=0 se inicializa con asx=x)
00067      den : denominador impar (2i+1) del término
00068      x   : valor para evaluar arcsin(x), con -1 < x < 1
00069      asx : acumulador de la suma (aproximación final)
00070      num : potencia acumulada x^(2i+1)
00071      fct : coeficiente acumulado C_i = (2i)! / (4^i (i!)^2)
00072     */
00073     int n, i, den;
00074     float x, asx, num, fct;
00075
00076     /* Leer n >= 1 */
00077     do {
00078         printf("Ingrese el numero de terminos: ");
00079         scanf("%d", &n);
00080     } while (n < 1);
00081
00082     /* Leer x con -1 < x < 1 */
00083     do {
00084         printf("Ingrese el valor de x: ");
00085         scanf("%f", &x);
00086     } while (-1 >= x || x >= 1);
00087
00088     /*
00089      Término inicial (i = 0):
00090      C_0 = 1
00091      x^(2*0+1) = x
00092      (2*0+1) = 1
00093      => arcsin(x)  x
00094     */
00095     for (i = 1, asx = x, num = x, fct = 1; i < n; i++)
00096     {
00097         /*
00098          Actualizar coeficiente:
00099          C_i = C_{i-1} * (2i-1)/(2i)
00100          Se usa 2*i - 1.0f para forzar división en punto flotante.
00101         */
00102         fct *= ((2*i - 1.0f) / (2*i));
00103
00104         /*
00105          Actualizar potencia:
00106          x^(2i+1) = x^(2i-1) * x^2
00107         */
00108         num *= (x * x);
00109
00110         /* Denominador del término: (2i + 1) */
00111         den = (2*i + 1);
00112
00113         /* Sumar término: (C_i * x^(2i+1)) / (2i+1) */
00114         asx += (fct * num / den);
00115     }

```

```

00116
00117     printf("asen(%f) = %f\n", x, asx);
00118     return 0;
00119 }

```

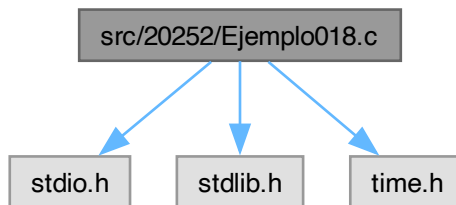
7.150 src/20252/Ejemplo018.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo018.c:



Macros

- #define [N](#) 1000000

Functions

- int [main](#) (int argc, char *argv[])

7.150.1 Macro Definition Documentation

7.150.1.1 N

```
#define N 1000000
```

Definition at line 5 of file [Ejemplo018.c](#).

7.150.2 Function Documentation

7.150.2.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 7 of file [Ejemplo018.c](#).

```

00008 {
00009     int i, j, k, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)

```

```

00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo de Bidireccional
00049     for(i=0; i<n-1; i++)
00050     {
00051         for(j=(i%2?n-1-i/2:i/2+1);
00052             (i%2?j>=(i/2+1):(j<=n-1-i/2));
00053             i%2?j--:j++)
00054         {
00055             if(i%2?(x[i%2?n-i/2-1:i/2]<x[j])
00056                 :(x[i%2?n-i/2-1:i/2]>x[j]))
00057             {
00058                 aux = x[j];
00059                 x[j] = x[i%2?n-i/2-1:i/2];
00060                 x[i%2?n-i/2-1:i/2] = aux;
00061             }
00062             // printf("i=%d\tx[k], k=%d\tj=%d\n", i,
00063             //         i%2?n-i/2-1:i/2, j);
00064         }
00065     }
00066     printf("Ordenado.\n");
00067     for(i=0; i<n; i++)
00068         printf("X[%d] = %f\n", i+1, x[i]);
00069     return 0;
00070 }

```

7.151 Ejemplo018.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, k, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else

```

```

00031         {
00032             min = max;
00033             max = 0;
00034         }
00035     }
00036     else
00037     {
00038         max = min;
00039         min = 0;
00040     }
00041 }
00042 printf("Desordenado.\n");
00043 for(i=0; i<n; i++)
00044 {
00045     x[i] = ((max-min)*rand())/RAND_MAX+min;
00046     printf("X[%d] = %f\n", i+1, x[i]);
00047 }
00048 // Algoritmo de Bidireccional
00049 for(i=0; i<n-1; i++)
00050 {
00051     for(j=(i%2?n-1-i/2:i/2+1);
00052         (i%2?j>=(i/2+1):(j<=n-1-i/2));
00053         i%2?j--:j++)
00054     {
00055         if(i%2?(x[i%2?n-i/2-1:i/2]<x[j])
00056             :(x[i%2?n-i/2-1:i/2]>x[j]))
00057         {
00058             aux = x[j];
00059             x[j] = x[i%2?n-i/2-1:i/2];
00060             x[i%2?n-i/2-1:i/2] = aux;
00061         }
00062         printf("i=%d\tx[k], k=%d\tj=%d\n", i,
00063             i%2?n-i/2-1:i/2, j);
00064     }
00065 }
00066 printf("Ordenado.\n");
00067 for(i=0; i<n; i++)
00068     printf("X[%d] = %f\n", i+1, x[i]);
00069 return 0;
00070 }

```

7.152 src/20261/src/Ejemplo018.c File Reference

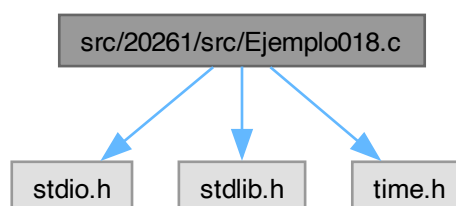
Generación de un arreglo de enteros aleatorios en el rango [min, max].

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo018.c:



Macros

- #define [N](#) 100000

Functions

- int [main](#) (int argc, char *argv[])

7.152.1 Detailed Description

Generación de un arreglo de enteros aleatorios en el rango $[\text{min}, \text{max}]$.

El programa: 1) Solicita al usuario el número de elementos n ($1 \leq n \leq N$). 2) Solicita un valor máximo y un valor mínimo. 3) Si el usuario ingresa $\text{min} > \text{max}$, los intercambia. 4) Inicializa el generador pseudoaleatorio con `srand(time(NULL))`. 5) Genera n enteros aleatorios uniformemente distribuidos (aprox.) en el intervalo $[\text{min}, \text{max}]$ usando: `rand() % (max - min + 1) + min` 6) Imprime todos los elementos generados.

Entrada estándar

- `n` (long int): número de elementos (1..N)
- `max` (int): valor máximo permitido
- `min` (int): valor mínimo permitido

Salida estándar

Imprime el arreglo generado:

```
x[1] = ...
x[2] = ...
...
x[n] = ...
```

Restricciones

- `N` es el tamaño máximo del arreglo (100000).
- Se asume que `max - min + 1` cabe en `int`.

Complejidad

Tiempo: $O(n)$. Memoria: $O(N)$ por el arreglo estático.

Note

- La operación `rand() % rango` puede introducir “sesgo de módulo” si el rango no divide a `RAND_MAX`. Para fines didácticos suele ser suficiente.

```
gcc Ejemplo018.c -o Ejemplo018
./Ejemplo018
```

Definition in file [Ejemplo018.c](#).

7.152.2 Macro Definition Documentation

7.152.2.1 N

```
#define N 100000
```

Tamaño máximo permitido del arreglo.

Definition at line 52 of file [Ejemplo018.c](#).

7.152.3 Function Documentation

7.152.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 54 of file [Ejemplo018.c](#).

```
00055 {
00056     (void)argc;
00057     (void)argv;
00058
00059     long int n, i;          /* n: número de elementos, i: índice de recorrido */
00060     int min, max;          /* límites del rango */
00061     int x[N];              /* arreglo con capacidad máxima N */
00062 }
```

```

00063     /* Leer n y validar que esté en el rango [1, N]. */
00064     do {
00065         printf("Ingrese el numero de elementos: ");
00066         scanf("%ld", &n);
00067     } while (n < 1 || n > N);
00068
00069     /* Leer límites del rango. */
00070     printf("Ingrese el valor maximo: ");
00071     scanf("%d", &max);
00072
00073     printf("Ingrese el valor minimo: ");
00074     scanf("%d", &min);
00075
00076     /*
00077      Si el usuario capturó min > max, se intercambian.
00078      Se usa intercambio sin variable temporal (suma/resta).
00079      Nota: para valores extremos podría haber overflow; en uso típico es OK.
00080     */
00081     if (min > max)
00082     {
00083         max += min;
00084         min = max - min;
00085         max -= min;
00086     }
00087
00088     /*
00089      Inicializar semilla del generador pseudoaleatorio.
00090      time(NULL) cambia cada segundo, así que los valores cambian en cada ejecución.
00091     */
00092     srand(time(NULL));
00093
00094     /*
00095      Generación:
00096      rand() % (max - min + 1) produce valores en [0, max-min]
00097      luego + min desplaza al intervalo [min, max].
00098     */
00099     for (i = 0; i < n; i++)
00100     {
00101         x[i] = rand() % (max - min + 1) + min;
00102         printf("x[%ld] = %d\n", i + 1, x[i]); /* se imprime desde 1 por estética */
00103     }
00104
00105     return 0;
00106 }

```

7.153 Ejemplo018.c

[Go to the documentation of this file.](#)

```

00001
00046
00047 #include <stdio.h>
00048 #include <stdlib.h>
00049 #include <time.h>
00050
00052 #define N 100000
00053
00054 int main(int argc, char *argv[])
00055 {
00056     (void)argc;
00057     (void)argv;
00058
00059     long int n, i;          /* n: número de elementos, i: índice de recorrido */
00060     int min, max;          /* límites del rango */
00061     int x[N];              /* arreglo con capacidad máxima N */
00062
00063     /* Leer n y validar que esté en el rango [1, N]. */
00064     do {
00065         printf("Ingrese el numero de elementos: ");
00066         scanf("%ld", &n);
00067     } while (n < 1 || n > N);
00068
00069     /* Leer límites del rango. */
00070     printf("Ingrese el valor maximo: ");
00071     scanf("%d", &max);
00072
00073     printf("Ingrese el valor minimo: ");
00074     scanf("%d", &min);
00075
00076     /*
00077      Si el usuario capturó min > max, se intercambian.
00078      Se usa intercambio sin variable temporal (suma/resta).
00079      Nota: para valores extremos podría haber overflow; en uso típico es OK.
00080     */
00081     if (min > max)

```

```

00082     {
00083         max += min;
00084         min = max - min;
00085         max -= min;
00086     }
00087
00088     /*
00089     Inicializar semilla del generador pseudoaleatorio.
00090     time(NULL) cambia cada segundo, así que los valores cambian en cada ejecución.
00091     */
00092     srand(time(NULL));
00093
00094     /*
00095     Generación:
00096     rand() % (max - min + 1) produce valores en [0, max-min]
00097     luego + min desplaza al intervalo [min, max].
00098     */
00099     for (i = 0; i < n; i++)
00100     {
00101         x[i] = rand() % (max - min + 1) + min;
00102         printf("x[%ld] = %d\n", i + 1, x[i]); /* se imprime desde 1 por estética */
00103     }
00104
00105     return 0;
00106 }

```

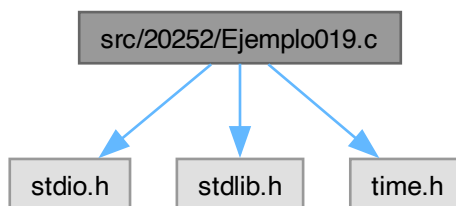
7.154 src/20252/Ejemplo019.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo019.c:



Macros

- #define `N` 1000000

Functions

- int `main` (int argc, char *argv[])

7.154.1 Macro Definition Documentation

7.154.1.1 N

```
#define N 1000000
```

Definition at line 5 of file [Ejemplo019.c](#).

7.154.2 Function Documentation

7.154.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file [Ejemplo019.c](#).

```
00008 {
00009     int i, j, k, n;
00010     int max, min, x[N], h[N];
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%d", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%d", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min+=max;
00027                 max=min-max;
00028                 min-=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = rand()%(max-min+1)+min;
00046         printf("X[%d] = %d\n", i+1, x[i]);
00047     }
00048     // Algoritmo de Conteo
00049     for(i=0; i<(max-min+1); i++)
00050         h[i]=0;
00051     for(i=0; i<n; i++)
00052         h[x[i]-min]++;
00053     for(j=0, i=0; j<(max-min+1); j++)
00054     {
00055         while(h[j])
00056         {
00057             x[i] = j+min;
00058             h[j]--;
00059             i++;
00060         }
00061         if(i==n)
00062             break;
00063     }
00064     printf("Ordenado.\n");
00065     for(i=0; i<n; i++)
00066         printf("X[%d] = %d\n", i+1, x[i]);
00067     return 0;
00068 }
```

7.155 Ejemplo019.c

[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char*argv[])
00008 {
```

```

00009     int i, j, k, n;
00010     int max, min, x[N], h[N];
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%d", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%d", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min+=max;
00027                 max=min-max;
00028                 min-=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = rand()%(max-min+1)+min;
00046         printf("X[%d] = %d\n", i+1, x[i]);
00047     }
00048     // Algoritmo de Conteo
00049     for(i=0; i<(max-min+1); i++)
00050         h[i]=0;
00051     for(i=0; i<n; i++)
00052         h[x[i]-min]++;
00053     for(j=0, i=0; j<(max-min+1); j++)
00054     {
00055         while (h[j])
00056         {
00057             x[i] = j+min;
00058             h[j]--;
00059             i++;
00060         }
00061         if(i==n)
00062             break;
00063     }
00064     printf("Ordenado.\n");
00065     for(i=0; i<n; i++)
00066         printf("X[%d] = %d\n", i+1, x[i]);
00067     return 0;
00068 }

```

7.156 src/20261/src/Ejemplo019.c File Reference

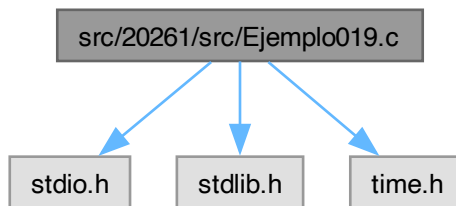
Generación y ordenamiento de un arreglo de enteros aleatorios (orden ascendente).

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo019.c:



Macros

- `#define N 100000`

Functions

- `int main (int argc, char *argv[ ])`

7.156.1 Detailed Description

Generación y ordenamiento de un arreglo de enteros aleatorios (orden ascendente).

El programa: 1) Solicita `n` ($1 \leq n \leq N$), y los límites `min` y `max`. 2) Si `min > max`, intercambia los valores. 3) Genera un arreglo `x` de tamaño `n` con enteros aleatorios en `[min, max]`. 4) Muestra el arreglo “Desordenado”. 5) Ordena el arreglo en orden ascendente mediante un algoritmo $O(n^2)$ basado en comparación e intercambio (similar a selección/intercambio):

- Para cada `i`, compara `x[i]` con `x[j]` ($j > i$) y si `x[i] > x[j]`, intercambia. 6) Muestra el arreglo “Ordenado”.

Entrada estándar

- `n` (long int): número de elementos ($1..N$)
- `max` (int): valor máximo permitido
- `min` (int): valor mínimo permitido

Salida estándar

- Arreglo desordenado (generado).
- Arreglo ordenado en forma ascendente.

Restricciones

- `N = 100000` (capacidad máxima del arreglo).

Complejidad

- Generación + despliegue: $O(n)$
- Ordenamiento: $O(n^2)$ por los dos ciclos anidados
- Memoria: $O(N)$

Note

Para `n` grande (cerca de 100000), $O(n^2)$ será muy lento. Este ejemplo se usa con propósitos didácticos para mostrar un ordenamiento básico.

```
gcc Ejemplo019.c -o Ejemplo019
./Ejemplo019
```

Definition in file [Ejemplo019.c](#).

7.156.2 Macro Definition Documentation

7.156.2.1 N

```
#define N 100000
```

Tamaño máximo permitido del arreglo.

Definition at line 48 of file [Ejemplo019.c](#).

7.156.3 Function Documentation

7.156.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 50 of file [Ejemplo019.c](#).

```
00051 {
00052     (void)argc;
00053     (void)argv;
00054
00055     long int n, i, j; /* n: tamaño; i,j: índices para recorrido/ordenamiento */
00056     int min, max; /* límites del rango */
00057     int x[N]; /* arreglo */
00058     int aux; /* auxiliar para intercambio */
00059
00060     /* Leer n y validar. */
00061     do {
00062         printf("Ingrese el numero de elementos: ");
00063         scanf("%ld", &n);
00064     } while (n < 1 || n > N);
00065
00066     /* Leer límites (max y min). */
00067     printf("Ingrese el valor maximo: ");
00068     scanf("%d", &max);
00069
00070     printf("Ingrese el valor minimo: ");
00071     scanf("%d", &min);
00072
00073     /* Asegurar min <= max intercambiando si es necesario. */
00074     if (min > max)
00075     {
00076         max += min;
00077         min = max - min;
00078         max -= min;
00079     }
00080
00081     /* Semilla para valores aleatorios distintos por ejecución. */
00082     srand(time(NULL));
00083
00084     /* Generar e imprimir arreglo inicial (sin ordenar). */
00085     printf("Desordenado.\n");
00086     for (i = 0; i < n; i++)
00087     {
00088         x[i] = rand() % (max - min + 1) + min;
00089         printf("x[%ld] = %d\n", i + 1, x[i]);
00090     }
00091
00092     /*
00093      Ordenamiento ascendente por comparación e intercambio:
00094      - Para cada i, se busca si existe algún x[j] (j>i) menor que x[i].
00095      - Si x[i] > x[j], se intercambian.
00096      Este patrón es similar a una selección por intercambios y tiene costo O(n^2).
00097     */
00098     for (i = 0; i < n - 1; i++)
00099         for (j = i + 1; j < n; j++)
00100             if (x[i] > x[j])
00101             {
00102                 aux = x[i];
00103                 x[i] = x[j];
00104                 x[j] = aux;
00105             }
00106
00107     /* Imprimir arreglo ordenado. */
00108     printf("Ordenado.\n");
00109     for (i = 0; i < n; i++)
00110         printf("x[%ld] = %d\n", i + 1, x[i]);
00111
00112     return 0;
00113 }
```

7.157 Ejemplo019.c

[Go to the documentation of this file.](#)

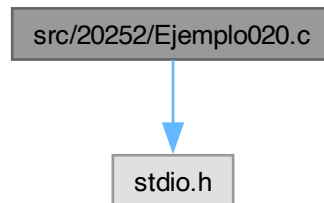
```

00001
00042
00043 #include <stdio.h>
00044 #include <stdlib.h>
00045 #include <time.h>
00046
00048 #define N 100000
00049
00050 int main(int argc, char *argv[])
00051 {
00052     (void)argc;
00053     (void)argv;
00054
00055     long int n, i, j; /* n: tamaño; i,j: índices para recorrido/ordenamiento */
00056     int min, max; /* límites del rango */
00057     int x[N]; /* arreglo */
00058     int aux; /* auxiliar para intercambio */
00059
00060     /* Leer n y validar. */
00061     do {
00062         printf("Ingrese el numero de elementos: ");
00063         scanf("%ld", &n);
00064     } while (n < 1 || n > N);
00065
00066     /* Leer límites (max y min). */
00067     printf("Ingrese el valor maximo: ");
00068     scanf("%d", &max);
00069
00070     printf("Ingrese el valor minimo: ");
00071     scanf("%d", &min);
00072
00073     /* Asegurar min <= max intercambiando si es necesario. */
00074     if (min > max)
00075     {
00076         max += min;
00077         min = max - min;
00078         max -= min;
00079     }
00080
00081     /* Semilla para valores aleatorios distintos por ejecución. */
00082     srand(time(NULL));
00083
00084     /* Generar e imprimir arreglo inicial (sin ordenar). */
00085     printf("Desordenado.\n");
00086     for (i = 0; i < n; i++)
00087     {
00088         x[i] = rand() % (max - min + 1) + min;
00089         printf("x[%ld] = %d\n", i + 1, x[i]);
00090     }
00091
00092     /*
00093     Ordenamiento ascendente por comparación e intercambio:
00094     - Para cada i, se busca si existe algún x[j] (j>i) menor que x[i].
00095     - Si x[i] > x[j], se intercambian.
00096     Este patrón es similar a una selección por intercambios y tiene costo O(n^2).
00097     */
00098     for (i = 0; i < n - 1; i++)
00099         for (j = i + 1; j < n; j++)
00100             if (x[i] > x[j])
00101             {
00102                 aux = x[i];
00103                 x[i] = x[j];
00104                 x[j] = aux;
00105             }
00106
00107     /* Imprimir arreglo ordenado. */
00108     printf("Ordenado.\n");
00109     for (i = 0; i < n; i++)
00110         printf("x[%ld] = %d\n", i + 1, x[i]);
00111
00112     return 0;
00113 }
```


7.158 src/20252/Ejemplo020.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo020.c:



Macros

- `#define N 100`
- `#define M 100`

Functions

- `int main (int argc, char *argv[ ])`

7.158.1 Macro Definition Documentation

7.158.1.1 M

```
#define M 100
```

Definition at line 4 of file [Ejemplo020.c](#).

7.158.1.2 N

```
#define N 100
```

Definition at line 3 of file [Ejemplo020.c](#).

7.158.2 Function Documentation

7.158.2.1 main()

```
int main (  
    int argc,  
    char * argv[ ])
```

Definition at line 6 of file [Ejemplo020.c](#).

```
00007 {  
00008     float A[N][M], B[N][M], C[N][M];  
00009     int n, m, i, j;  
00010     do{  
00011         printf("Ingrese el numero de filas: ");  
00012         scanf("%d", &n);  
00013     }while(n<1||n>N);  
00014     do{  
00015         printf("Ingrese el numero de columnas: ");  
00016         scanf("%d", &m);  
00017     }while(m<1||m>M);  
00018     for(i=0; i<n; i++)  
00019         for(j=0; j<m; j++)  
00020     {
```

```

00021         printf("A[%d][%d] = ", i+1, j+1);
00022         scanf("%f", &(A[i][j]));
00023     }
00024     for(i=0; i<n; i++)
00025         for(j=0; j<m; j++)
00026         {
00027             printf("B[%d][%d] = ", i+1, j+1);
00028             scanf("%f", &(B[i][j]));
00029         }
00030     for(i=0; i<n; i++)
00031         for(j=0; j<m; j++)
00032         {
00033             C[i][j] = A[i][j] + B[i][j];
00034             printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00035         }
00036     return 0;
00037 }

```

7.159 Ejemplo020.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002
00003 #define N 100
00004 #define M 100
00005
00006 int main(int argc, char *argv[])
00007 {
00008     float A[N][M], B[N][M], C[N][M];
00009     int n, m, i, j;
00010     do{
00011         printf("Ingrese el numero de filas: ");
00012         scanf("%d", &n);
00013     }while(n<1||n>N);
00014     do{
00015         printf("Ingrese el numero de columnas: ");
00016         scanf("%d", &m);
00017     }while(m<1||m>M);
00018     for(i=0; i<n; i++)
00019         for(j=0; j<m; j++)
00020         {
00021             printf("A[%d][%d] = ", i+1, j+1);
00022             scanf("%f", &(A[i][j]));
00023         }
00024     for(i=0; i<n; i++)
00025         for(j=0; j<m; j++)
00026         {
00027             printf("B[%d][%d] = ", i+1, j+1);
00028             scanf("%f", &(B[i][j]));
00029         }
00030     for(i=0; i<n; i++)
00031         for(j=0; j<m; j++)
00032         {
00033             C[i][j] = A[i][j] + B[i][j];
00034             printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00035         }
00036     return 0;
00037 }

```

7.160 src/20261/src/Ejemplo020.c File Reference

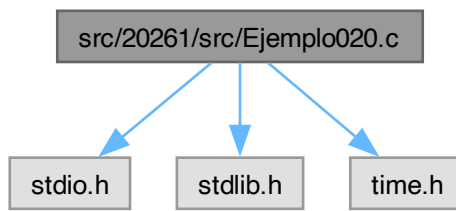
Genera un arreglo de enteros aleatorios y lo ordena con Odd–Even Sort (Brick Sort).

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo020.c:



Macros

- `#define N 100000`

Functions

- `int main (int argc, char *argv[ ])`

7.160.1 Detailed Description

Genera un arreglo de enteros aleatorios y lo ordena con Odd–Even Sort (Brick Sort).

El programa: 1) Lee el tamaño n ($1 \leq n \leq N$) y un rango entero $[\text{min}, \text{max}]$. 2) Genera n enteros pseudoaleatorios uniformes (aprox.) en $[\text{min}, \text{max}]$. 3) Imprime el arreglo "Desordenado". 4) Ordena el arreglo con **Odd–Even Transposition Sort** (también llamado Brick Sort):

- En pasadas alternadas, compara e intercambia pares adyacentes:
 - pasada par: (0,1), (2,3), (4,5), ...
 - pasada impar: (1,2), (3,4), (5,6), ...
- Se realizan aproximadamente $n-1$ pasadas. 5) Imprime el arreglo "Ordenado".

Entrada estándar

- n (long int): número de elementos ($1..N$)
- `max` (int): límite superior del rango
- `min` (int): límite inferior del rango

Salida estándar

- Lista de valores "Desordenado"
- Lista de valores "Ordenado"

Complejidad

- Generación/impresión: $O(n)$
- Ordenamiento (Odd–Even Sort): $O(n^2)$
- Memoria: $O(N)$ (arreglo estático en pila)

Warning

- El intercambio usa suma/resta (sin variable temporal). Puede desbordar si los enteros son muy grandes. Para un uso robusto conviene usar una variable auxiliar.
- `rand()` % `rango` puede introducir sesgo de módulo; suficiente para fines didácticos.

```
gcc Ejemplo020.c -o Ejemplo020
./Ejemplo020
```

Definition in file [Ejemplo020.c](#).

7.160.2 Macro Definition Documentation

7.160.2.1 N

```
#define N 100000
```

Definition at line 46 of file [Ejemplo020.c](#).

7.160.3 Function Documentation

7.160.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 48 of file [Ejemplo020.c](#).

```
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     long int n, i, j;
00054     long int i0, i1;    /* índices del par a comparar */
00055     int min, max;
00056     int x[N];          /* arreglo de enteros */
00057
00058     /* Leer n y validar rango [1, N] */
00059     do {
00060         printf("Ingrese el numero de elementos: ");
00061         scanf("%ld", &n);
00062     } while (n < 1 || n > N);
00063
00064     /* Leer rango [min, max] (si viene invertido, se corrige) */
00065     printf("Ingrese el valor maximo: ");
00066     scanf("%d", &max);
00067     printf("Ingrese el valor minimo: ");
00068     scanf("%d", &min);
00069
00070     if (min > max)
00071     {
00072         /* Intercambio max <-> min sin variable auxiliar (puede desbordar en casos extremos) */
00073         max += min;
00074         min = max - min;
00075         max -= min;
00076     }
00077
00078     /* Semilla del generador pseudoaleatorio */
00079     srand(time(NULL));
00080
00081     /* Generar e imprimir arreglo desordenado */
00082     printf("Desordenado.\n");
00083     for (i = 0; i < n; i++)
00084     {
00085         x[i] = rand() % (max - min + 1) + min;
00086         printf("x[%ld] = %d\n", i + 1, x[i]);
00087     }
00088
00089     /*
00090     ODD-EVEN SORT (Brick Sort):
00091     - i representa la pasada (0..n-2).
00092     - si i es par: compara (0,1), (2,3), ...
00093     - si i es impar: compara (1,2), (3,4), ...
00094
00095     Aquí se implementa mapeando un índice j de pares:
00096     i0 = 2*j - (i%2)
00097     i1 = i0 + 1
00098     */
00099     for (i = 0; i < n - 1; i++)
00100     {
```

```

00101         for (j = i % 2; j < (n + i % 2) / 2; j++)
00102         {
00103             i0 = 2 * j - i % 2;
00104             i1 = 2 * j + 1 - i % 2;
00105
00106             /* Compare-exchange: si están invertidos, intercambia */
00107             if (x[i0] > x[i1])
00108             {
00109                 /* Swap por suma/resta (riesgo de overflow si valores grandes) */
00110                 x[i0] += x[i1];
00111                 x[i1] = x[i0] - x[i1];
00112                 x[i0] -= x[i1];
00113             }
00114         }
00115     }
00116
00117     /* Imprimir arreglo ordenado */
00118     printf("Ordenado.\n");
00119     for (i = 0; i < n; i++)
00120         printf("x[%ld] = %d\n", i + 1, x[i]);
00121
00122     return 0;
00123 }

```

7.161 Ejemplo020.c

[Go to the documentation of this file.](#)

```

00001
00041
00042 #include <stdio.h>
00043 #include <stdlib.h>
00044 #include <time.h>
00045
00046 #define N 100000
00047
00048 int main(int argc, char *argv[])
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     long int n, i, j;
00054     long int i0, i1; /* índices del par a comparar */
00055     int min, max;
00056     int x[N]; /* arreglo de enteros */
00057
00058     /* Leer n y validar rango [1, N] */
00059     do {
00060         printf("Ingrese el numero de elementos: ");
00061         scanf("%ld", &n);
00062     } while (n < 1 || n > N);
00063
00064     /* Leer rango [min, max] (si viene invertido, se corrige) */
00065     printf("Ingrese el valor maximo: ");
00066     scanf("%d", &max);
00067     printf("Ingrese el valor minimo: ");
00068     scanf("%d", &min);
00069
00070     if (min > max)
00071     {
00072         /* Intercambio max <-> min sin variable auxiliar (puede desbordar en casos extremos) */
00073         max += min;
00074         min = max - min;
00075         max -= min;
00076     }
00077
00078     /* Semilla del generador pseudoaleatorio */
00079     srand(time(NULL));
00080
00081     /* Generar e imprimir arreglo desordenado */
00082     printf("Desordenado.\n");
00083     for (i = 0; i < n; i++)
00084     {
00085         x[i] = rand() % (max - min + 1) + min;
00086         printf("x[%ld] = %d\n", i + 1, x[i]);
00087     }
00088
00089     /*
00090     ODD-EVEN SORT (Brick Sort):
00091     - i representa la pasada (0..n-2).
00092     - si i es par: compara (0,1), (2,3), ...
00093     - si i es impar: compara (1,2), (3,4), ...
00094
00095     Aquí se implementa mapeando un índice j de pares:
00096     i0 = 2*j - (i%2)

```

```

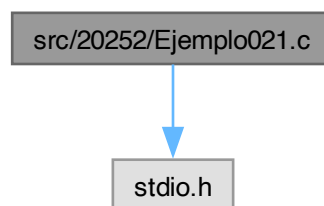
00097     i1 = i0 + 1
00098     */
00099     for (i = 0; i < n - 1; i++)
00100     {
00101         for (j = i % 2; j < (n + i % 2) / 2; j++)
00102         {
00103             i0 = 2 * j - i % 2;
00104             i1 = 2 * j + 1 - i % 2;
00105
00106             /* Compare-exchange: si están invertidos, intercambia */
00107             if (x[i0] > x[i1])
00108             {
00109                 /* Swap por suma/resta (riesgo de overflow si valores grandes) */
00110                 x[i0] += x[i1];
00111                 x[i1] = x[i0] - x[i1];
00112                 x[i0] -= x[i1];
00113             }
00114         }
00115     }
00116
00117     /* Imprimir arreglo ordenado */
00118     printf("Ordenado.\n");
00119     for (i = 0; i < n; i++)
00120         printf("x[%ld] = %d\n", i + 1, x[i]);
00121
00122     return 0;
00123 }

```

7.162 src/20252/Ejemplo021.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo021.c:



Macros

- #define [N](#) 100
- #define [M](#) 100

Functions

- int [main](#) (int argc, char *argv[])

7.162.1 Macro Definition Documentation

7.162.1.1 M

#define M 100

Definition at line 4 of file [Ejemplo021.c](#).

7.162.1.2 N

```
#define N 100
```

Definition at line 3 of file [Ejemplo021.c](#).

7.162.2 Function Documentation

7.162.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 6 of file [Ejemplo021.c](#).

```
00007 {
00008     float A[N][M], B[N][M], C[N][M];
00009     int nA, mA, nB, mB, i, j, k;
00010     do{
00011         printf("Ingrese el numero de filas de A: ");
00012         scanf("%d", &nA);
00013     }while(nA<1||nA>N);
00014     do{
00015         printf("Ingrese el numero de columnas de A: ");
00016         scanf("%d", &mA);
00017     }while(mA<1||mA>M);
00018     do{
00019         printf("Ingrese el numero de filas de B: ");
00020         scanf("%d", &nB);
00021     }while(nB!=mA);
00022     do{
00023         printf("Ingrese el numero de columnas de B: ");
00024         scanf("%d", &mB);
00025     }while(mB<1||mB>M);
00026     for(i=0; i<nA; i++)
00027         for(j=0; j<mA; j++)
00028         {
00029             printf("A[%d][%d] = ", i+1, j+1);
00030             scanf("%f", &(A[i][j]));
00031         }
00032     for(i=0; i<nB; i++)
00033         for(j=0; j<mB; j++)
00034         {
00035             printf("B[%d][%d] = ", i+1, j+1);
00036             scanf("%f", &(B[i][j]));
00037         }
00038     for(i=0; i<nA; i++)
00039         for(j=0; j<mB; j++)
00040         {
00041             for(k=0; C[i][j] = 0; k<mA; k++)
00042                 C[i][j] += (A[i][k]*B[k][j]);
00043             printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00044         }
00045     return 0;
00046 }
```

7.163 Ejemplo021.c

[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002
00003 #define N 100
00004 #define M 100
00005
00006 int main(int argc, char *argv[])
00007 {
00008     float A[N][M], B[N][M], C[N][M];
00009     int nA, mA, nB, mB, i, j, k;
00010     do{
00011         printf("Ingrese el numero de filas de A: ");
00012         scanf("%d", &nA);
00013     }while(nA<1||nA>N);
00014     do{
00015         printf("Ingrese el numero de columnas de A: ");
00016         scanf("%d", &mA);
00017     }while(mA<1||mA>M);
00018     do{
00019         printf("Ingrese el numero de filas de B: ");
00020         scanf("%d", &nB);
00021     }while(nB!=mA);
```

```

00022     do{
00023         printf("Ingrese el numero de columnas de B: ");
00024         scanf("%d", &mB);
00025     }while(mB<1||mB>M);
00026     for(i=0; i<nA; i++)
00027         for(j=0; j<mA; j++)
00028     {
00029         printf("A[%d][%d] = ", i+1, j+1);
00030         scanf("%f", &(A[i][j]));
00031     }
00032     for(i=0; i<nB; i++)
00033         for(j=0; j<mB; j++)
00034     {
00035         printf("B[%d][%d] = ", i+1, j+1);
00036         scanf("%f", &(B[i][j]));
00037     }
00038     for(i=0; i<nA; i++)
00039         for(j=0; j<mB; j++)
00040     {
00041         for(k=0, C[i][j] = 0; k<mA; k++)
00042             C[i][j] += (A[i][k]*B[k][j]);
00043         printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00044     }
00045     return 0;
00046 }

```

7.164 src/20261/src/Ejemplo021.c File Reference

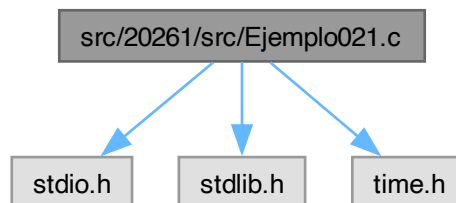
Genera un arreglo de flotantes aleatorios y lo ordena con Selection Sort.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo021.c:



Macros

- #define `N` 100000

Functions

- int `main` (int argc, char *argv[])

7.164.1 Detailed Description

Genera un arreglo de flotantes aleatorios y lo ordena con Selection Sort.

El programa: 1) Lee `n` (1..N) y un rango real [min, max]. 2) Genera `n` flotantes uniformes (aprox.) en [min, max] usando: $x[i] = ((\text{max}-\text{min}) * \text{rand}()) / \text{RAND_MAX} + \text{min}$ 3) Imprime el arreglo "Desordenado". 4) Ordena con **Selection Sort**:

- Primero ubica el mínimo global en `x[0]`.
- Después, para cada posición `i`, selecciona el mínimo del subarreglo [`i..n-1`] y lo intercambia con `x[i]`.

Entrada estándar

- `n` (long int): número de elementos (1..N)
- `max` (float): límite superior
- `min` (float): límite inferior

Salida estándar

- Arreglo desordenado
- Arreglo ordenado (ascendente)

Complejidad

- Generación/impresión: $O(n)$
- Selection Sort: $O(n^2)$
- Memoria: $O(N)$

Warning

- El intercambio de flotantes se hace con multiplicación/división. Si alguno de los valores involucrados es 0, puede provocar división entre 0 o NaN/Inf. Para una versión robusta, usa una variable auxiliar (`aux`).
- También el intercambio inicial de min/max usa multiplicación/división y falla si `min=0`.

```
gcc Ejemplo021.c -o Ejemplo021
./Ejemplo021
```

Definition in file [Ejemplo021.c](#).

7.164.2 Macro Definition Documentation**7.164.2.1 N**

```
#define N 100000
```

Definition at line 46 of file [Ejemplo021.c](#).

7.164.3 Function Documentation**7.164.3.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 48 of file [Ejemplo021.c](#).

```
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     long int n, i, j, i_min;
00054     float min, max;
00055     float x[N];
00056
00057     /* Leer n y validar */
00058     do {
00059         printf("Ingrese el numero de elementos: ");
00060         scanf("%ld", &n);
00061     } while (n < 1 || n > N);
00062
00063     /* Leer rango [min,max] */
00064     printf("Ingrese el valor maximo: ");
00065     scanf("%f", &max);
00066     printf("Ingrese el valor minimo: ");
00067     scanf("%f", &min);
00068
00069     if (min > max)
00070     {
00071         /*
00072          * Intercambio sin variable auxiliar usando multiplicación/división.
00073          * WARNING: falla si min==0 o max==0 (división entre 0).
00074          */
00075     }
```

```

00074         */
00075         max *= min;
00076         min = max / min;
00077         max /= min;
00078     }
00079
00080     srand(time(NULL));
00081
00082     /* Generar e imprimir arreglo desordenado */
00083     printf("Desordenado.\n");
00084     for (i = 0; i < n; i++)
00085     {
00086         /* Escalamiento lineal a [min, max] */
00087         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00088         printf("x[%ld] = %f\n", i + 1, x[i]);
00089     }
00090
00091     /*
00092     Paso 1: localizar el mínimo global y ponerlo en x[0].
00093     Esto no es estrictamente necesario para Selection Sort, pero el código lo hace.
00094     */
00095     for (i = 1, i_min = 0; i < n; i++)
00096         if (x[i] < x[i_min])
00097             i_min = i;
00098
00099     if (i_min)
00100     {
00101         /* Swap por multiplicación/división (WARNING: falla si x[0]==0 o x[i_min]==0) */
00102         x[i_min] *= x[0];
00103         x[0] = x[i_min] / x[0];
00104         x[i_min] /= x[0];
00105     }
00106
00107     /*
00108     Paso 2: Selection Sort desde i=1:
00109     - en cada i, encontrar el mínimo en [i..n-1]
00110     - intercambiarlo con x[i]
00111     */
00112     for (i = 1; i < n - 1; i++)
00113     {
00114         for (j = i + 1, i_min = i; j < n; j++)
00115             if (x[j] < x[i_min])
00116                 i_min = j;
00117
00118         if (i_min != i)
00119         {
00120             /* Swap por multiplicación/división (mismas advertencias) */
00121             x[i_min] *= x[i];
00122             x[i] = x[i_min] / x[i];
00123             x[i_min] /= x[i];
00124         }
00125     }
00126
00127     /* Imprimir arreglo ordenado */
00128     printf("Ordenado.\n");
00129     for (i = 0; i < n; i++)
00130         printf("x[%ld] = %f\n", i + 1, x[i]);
00131
00132     return 0;
00133 }

```

7.165 Ejemplo021.c

[Go to the documentation of this file.](#)

```

00001
00041
00042 #include <stdio.h>
00043 #include <stdlib.h>
00044 #include <time.h>
00045
00046 #define N 100000
00047
00048 int main(int argc, char *argv[])
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     long int n, i, j, i_min;
00054     float min, max;
00055     float x[N];
00056
00057     /* Leer n y validar */
00058     do {
00059         printf("Ingrese el numero de elementos: ");

```

```

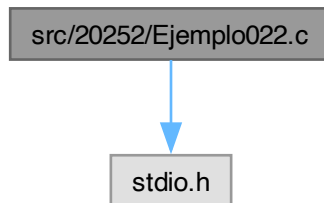
00060     scanf("%ld", &n);
00061 } while (n < 1 || n > N);
00062
00063 /* Leer rango [min,max] */
00064 printf("Ingrese el valor maximo: ");
00065 scanf("%f", &max);
00066 printf("Ingrese el valor minimo: ");
00067 scanf("%f", &min);
00068
00069 if (min > max)
00070 {
00071     /*
00072      * Intercambio sin variable auxiliar usando multiplicación/división.
00073      * WARNING: falla si min==0 o max==0 (división entre 0).
00074      */
00075     max *= min;
00076     min = max / min;
00077     max /= min;
00078 }
00079
00080 srand(time(NULL));
00081
00082 /* Generar e imprimir arreglo desordenado */
00083 printf("Desordenado.\n");
00084 for (i = 0; i < n; i++)
00085 {
00086     /* Escalamiento lineal a [min, max] */
00087     x[i] = ((max - min) * rand()) / RAND_MAX + min;
00088     printf("x[%ld] = %f\n", i + 1, x[i]);
00089 }
00090
00091 /*
00092  * Paso 1: localizar el mínimo global y ponerlo en x[0].
00093  * Esto no es estrictamente necesario para Selection Sort, pero el código lo hace.
00094  */
00095 for (i = 1, i_min = 0; i < n; i++)
00096     if (x[i] < x[i_min])
00097         i_min = i;
00098
00099 if (i_min)
00100 {
00101     /* Swap por multiplicación/división (WARNING: falla si x[0]==0 o x[i_min]==0) */
00102     x[i_min] *= x[0];
00103     x[0] = x[i_min] / x[0];
00104     x[i_min] /= x[0];
00105 }
00106
00107 /*
00108  * Paso 2: Selection Sort desde i=1:
00109  * - en cada i, encontrar el mínimo en [i..n-1]
00110  * - intercambiarlo con x[i]
00111  */
00112 for (i = 1; i < n - 1; i++)
00113 {
00114     for (j = i + 1, i_min = i; j < n; j++)
00115         if (x[j] < x[i_min])
00116             i_min = j;
00117
00118     if (i_min != i)
00119     {
00120         /* Swap por multiplicación/división (mismas advertencias) */
00121         x[i_min] *= x[i];
00122         x[i] = x[i_min] / x[i];
00123         x[i_min] /= x[i];
00124     }
00125 }
00126
00127 /* Imprimir arreglo ordenado */
00128 printf("Ordenado.\n");
00129 for (i = 0; i < n; i++)
00130     printf("x[%ld] = %f\n", i + 1, x[i]);
00131
00132 return 0;
00133 }

```

7.166 src/20252/Ejemplo022.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo022.c:



Macros

- `#define N 100`

Functions

- `int main (int argc, char *argv[ ])`

7.166.1 Macro Definition Documentation

7.166.1.1 N

```
#define N 100
```

Definition at line 3 of file [Ejemplo022.c](#).

7.166.2 Function Documentation

7.166.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 5 of file [Ejemplo022.c](#).

```

00006 {
00007     int n, i, j, k;
00008     float A[N][N], b[N], fct, x[N];
00009     do{
00010         printf("Ingrese el numero de incognitas: ");
00011         scanf("%d", &n);
00012     }while(n<3||n>N);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {
00017             printf("A[%d][%d] = ", i+1, j+1);
00018             scanf("%f", &(A[i][j]));
00019         }
00020         printf("b[%d] = ", i+1);
00021         scanf("%f", &b[i]);
00022         x[i] = 0;
00023     }
00024     for(i=0; i<n; i++)
00025     {
00026         printf("%.2fx1", A[i][0]);
00027         for(j=1; j<n; j++)
00028         {
```

```

00029         printf("%.2fx%d", A[i][j], j+1);
00030     }
00031     printf("=%.2f\n", b[i]);
00032 }
00033 for(i=1; i<n; i++)
00034     for(j=i; j<n; j++)
00035     {
00036         for(k=0, fct=A[j][i-1]/A[i-1][i-1]; k<n; k++)
00037             A[j][k] -= (fct*A[i-1][k]);
00038         b[j] -= (fct*b[i-1]);
00039     }
00040 /*
00041 for(i=0; i<n; i++)
00042 {
00043     for(j=0; j<n; j++)
00044         printf("%.2f\t", A[i][j]);
00045     printf("%.2f\n", b[i]);
00046 }
00047 */
00048 for(i=n-1; i>-1; i--)
00049 {
00050     x[i] = b[i];
00051     for(j=i+1; j<n; j++)
00052         x[i] -= A[i][j]*x[j];
00053     x[i] /= A[i][i];
00054 }
00055 for(i=0; i<n; i++)
00056     printf("X[%d] = %f\n", i+1, x[i]);
00057 return 0;
00058 }

```

Here is the call graph for this function:



7.167 Ejemplo022.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define N 100
00004
00005 int main(int argc, char *argv[])
00006 {
00007     int n, i, j, k;
00008     float A[N][N], b[N], fct, x[N];
00009     do{
00010         printf("Ingrese el numero de incognitas: ");
00011         scanf("%d", &n);
00012     }while(n<3||n>N);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {
00017             printf("A[%d][%d] = ", i+1, j+1);
00018             scanf("%f", &(A[i][j]));
00019         }
00020         printf("b[%d] = ", i+1);
00021         scanf("%f", &b[i]);
00022         x[i] = 0;
00023     }
00024     for(i=0; i<n; i++)
00025     {
00026         printf("%.2fx1", A[i][0]);
00027         for(j=1; j<n; j++)
00028         {
00029             printf("%.2fx%d", A[i][j], j+1);
00030         }

```

```

00031     printf("%.2f\n", b[i]);
00032 }
00033 for(i=1; i<n; i++)
00034     for(j=i; j<n; j++)
00035     {
00036         for(k=0, fct=A[j][i-1]/A[i-1][i-1]; k<n; k++)
00037             A[j][k] -= (fct*A[i-1][k]);
00038         b[j] -= (fct*b[i-1]);
00039     }
00040 /*
00041 for(i=0; i<n; i++)
00042 {
00043     for(j=0; j<n; j++)
00044         printf("%.2f\t", A[i][j]);
00045     printf("%.2f\n", b[i]);
00046 }
00047 */
00048 for(i=n-1; i>-1; i--)
00049 {
00050     x[i] = b[i];
00051     for(j=i+1; j<n; j++)
00052         x[i] -= A[i][j]*x[j];
00053     x[i] /= A[i][i];
00054 }
00055 for(i=0; i<n; i++)
00056     printf("X[%d] = %f\n", i+1, x[i]);
00057 return 0;
00058 }

```

7.168 src/20261/src/Ejemplo022.c File Reference

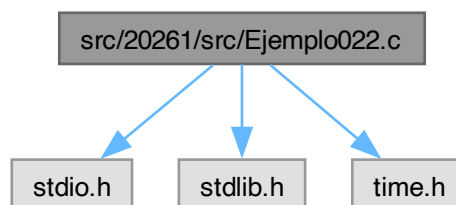
Genera un arreglo de flotantes aleatorios y lo ordena con Insertion Sort.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo022.c:



Macros

- #define [N](#) 100000

Functions

- int [main](#) (int argc, char *argv[])

7.168.1 Detailed Description

Genera un arreglo de flotantes aleatorios y lo ordena con Insertion Sort.

El programa: 1) Lee n (1.. N) y un rango real $[\text{min}, \text{max}]$. 2) Genera n flotantes en $[\text{min}, \text{max}]$. 3) Imprime el arreglo "Desordenado". 4) Ordena el arreglo con **Insertion Sort** (orden ascendente):

- Recorre $i = 1..n-1$, toma $x[i]$ como *aux*,

- desplaza hacia la derecha todos los elementos mayores que `aux`,
- inserta `aux` en la posición correcta. 5) Imprime el arreglo "Ordenado".

Entrada estándar

- `n` (long int): número de elementos (1..N)
- `max` (float): límite superior
- `min` (float): límite inferior

Salida estándar

- Arreglo desordenado
- Arreglo ordenado (ascendente)

Complejidad

- Generación/impresión: $O(n)$
- Insertion Sort: $O(n^2)$ en el peor caso; $O(n)$ si ya está casi ordenado.
- Memoria: $O(N)$

Warning

- El intercambio de min/max se hace con multiplicación/división; falla si `min==0`.
- El arreglo `x[N]` vive en la pila; en algunos entornos con pila pequeña puede fallar.

```
gcc Ejemplo022.c -o Ejemplo022
./Ejemplo022
```

Definition in file [Ejemplo022.c](#).

7.168.2 Macro Definition Documentation

7.168.2.1 N

```
#define N 100000
```

Definition at line 44 of file [Ejemplo022.c](#).

7.168.3 Function Documentation

7.168.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 46 of file [Ejemplo022.c](#).

```
00047 {
00048     (void)argc;
00049     (void)argv;
00050
00051     long int n, i, j;
00052     float min, max;
00053     float x[N], aux;
00054
00055     /* Leer n y validar */
00056     do {
00057         printf("Ingrese el numero de elementos: ");
00058         scanf("%ld", &n);
00059     } while (n < 1 || n > N);
00060
00061     /* Leer rango [min,max] */
00062     printf("Ingrese el valor maximo: ");
00063     scanf("%f", &max);
00064     printf("Ingrese el valor minimo: ");
00065     scanf("%f", &min);
00066 }
```

```

00067     if (min > max)
00068     {
00069         /* Swap por multiplicación/división (WARNING: falla si min==0) */
00070         max *= min;
00071         min = max / min;
00072         max /= min;
00073     }
00074
00075     srand(time(NULL));
00076
00077     /* Generar e imprimir arreglo desordenado */
00078     printf("Desordenado.\n");
00079     for (i = 0; i < n; i++)
00080     {
00081         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00082         printf("x[%ld] = %f\n", i + 1, x[i]);
00083     }
00084
00085     /*
00086     INSERTION SORT:
00087     - i recorre desde 1 hasta n-1.
00088     - aux almacena el elemento actual a insertar.
00089     - j recorre hacia atrás desplazando elementos mayores que aux.
00090     */
00091     for (i = 1; i < n; i++)
00092     {
00093         j = i - 1;
00094         aux = x[i];
00095
00096         /* Desplazar hacia la derecha mientras x[j] > aux */
00097         while (x[j] > aux)
00098         {
00099             x[j + 1] = x[j];
00100             j--;
00101
00102             /* Si j se vuelve negativo, se detiene para evitar acceder fuera de rango */
00103             if (j < 0)
00104                 break;
00105         }
00106
00107         /* Insertar aux en la posición correcta */
00108         x[j + 1] = aux;
00109     }
00110
00111     /* Imprimir arreglo ordenado */
00112     printf("Ordenado.\n");
00113     for (i = 0; i < n; i++)
00114         printf("x[%ld] = %f\n", i + 1, x[i]);
00115
00116     return 0;
00117 }

```

7.169 Ejemplo022.c

[Go to the documentation of this file.](#)

```

00001
00039
00040 #include <stdio.h>
00041 #include <stdlib.h>
00042 #include <time.h>
00043
00044 #define N 100000
00045
00046 int main(int argc, char *argv[])
00047 {
00048     (void)argc;
00049     (void)argv;
00050
00051     long int n, i, j;
00052     float min, max;
00053     float x[N], aux;
00054
00055     /* Leer n y validar */
00056     do {
00057         printf("Ingrese el numero de elementos: ");
00058         scanf("%ld", &n);
00059     } while (n < 1 || n > N);
00060
00061     /* Leer rango [min,max] */
00062     printf("Ingrese el valor maximo: ");
00063     scanf("%f", &max);
00064     printf("Ingrese el valor minimo: ");
00065     scanf("%f", &min);
00066

```



```

00067     if (min > max)
00068     {
00069         /* Swap por multiplicación/división (WARNING: falla si min==0) */
00070         max *= min;
00071         min = max / min;
00072         max /= min;
00073     }
00074
00075     srand(time(NULL));
00076
00077     /* Generar e imprimir arreglo desordenado */
00078     printf("Desordenado.\n");
00079     for (i = 0; i < n; i++)
00080     {
00081         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00082         printf("x[%ld] = %f\n", i + 1, x[i]);
00083     }
00084
00085     /*
00086     INSERTION SORT:
00087     - i recorre desde 1 hasta n-1.
00088     - aux almacena el elemento actual a insertar.
00089     - j recorre hacia atrás desplazando elementos mayores que aux.
00090     */
00091     for (i = 1; i < n; i++)
00092     {
00093         j = i - 1;
00094         aux = x[i];
00095
00096         /* Desplazar hacia la derecha mientras x[j] > aux */
00097         while (x[j] > aux)
00098         {
00099             x[j + 1] = x[j];
00100             j--;
00101
00102             /* Si j se vuelve negativo, se detiene para evitar acceder fuera de rango */
00103             if (j < 0)
00104                 break;
00105         }
00106
00107         /* Insertar aux en la posición correcta */
00108         x[j + 1] = aux;
00109     }
00110
00111     /* Imprimir arreglo ordenado */
00112     printf("Ordenado.\n");
00113     for (i = 0; i < n; i++)
00114         printf("x[%ld] = %f\n", i + 1, x[i]);
00115
00116     return 0;
00117 }

```

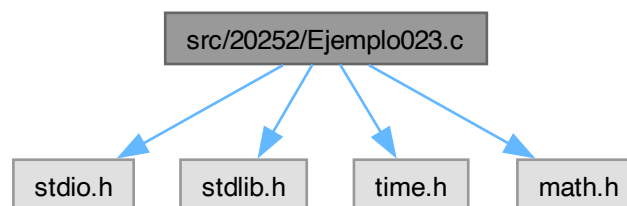
7.170 src/20252/Ejemplo023.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

```

Include dependency graph for Ejemplo023.c:



Macros

- `#define N 1000`

Functions

- `int main (int argc, char *argv[])`

7.170.1 Macro Definition Documentation

7.170.1.1 N

`#define N 1000`

Definition at line 6 of file [Ejemplo023.c](#).

7.170.2 Function Documentation

7.170.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 8 of file [Ejemplo023.c](#).

```
00009 {
00010     int n, i, flag, op;
00011     float max, min, aux, x[N], vmx, vmn, md, st, z[N], cn, nr;
00012     srand(time(NULL));
00013     do{
00014         printf("Ingrese el numero de datos: ");
00015         scanf("%d", &n);
00016     }while(n<1||n>N);
00017     printf("Ingrese el numero maximo: ");
00018     scanf("%f", &max);
00019     printf("Ingrese el numero minimo: ");
00020     scanf("%f", &min);
00021     if(max<min)
00022     {
00023         aux = max;
00024         max = min;
00025         min = aux;
00026     }
00027     for(i=0, vmn=max, vmx=min, md=0, st=0; i<n; i++)
00028     {
00029         x[i] = ((max-min)*rand())/RAND_MAX+min;
00030         printf("X[%d] = %f\n", i+1, x[i]);
00031         if(vmn>x[i])
00032             vmn = x[i];
00033         if(vmx<x[i])
00034             vmx = x[i];
00035         md+=x[i];
00036         st+=(x[i]*x[i]);
00037     }
00038     md/=n;
00039     st/=n;
00040     st=(md*md);
00041     st=sqrt(st);
00042     printf("Max: %f\n Min = %f\n Media = %f\n D. E. = %f\n",
00043         vmx, vmn, md, st);
00044     flag = 0;
00045     printf("Desea centrar: ");
00046     scanf("%d", &op);
00047     if(op)
00048     {
00049         printf("Por rango: ");
00050         scanf("%d", &op);
00051         flag|=op?1:4;
00052     }
00053     printf("Desea normalizar: ");
00054     scanf("%d", &op);
00055     if(op)
00056     {
00057         printf("Por rango: ");
00058         scanf("%d", &op);
00059         flag|=op?2:8;
00060     }
00061     cn = (flag&1)?vmn:(flag&4?md:0);
00062     nr = (flag&2)?vmx-vmn:(flag&8?st:1);
```

```

00063     printf("%d\t%f\t%f\n", flag, cn, nr);
00064     for(i=0; i<n; i++)
00065     {
00066         z[i] = (x[i]-cn)/nr;
00067         printf("Z[%d] = %f\n", i+1, z[i]);
00068     }
00069     return 0;
00070 }

```

7.171 Ejemplo023.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004 #include <math.h>
00005
00006 #define N 1000
00007
00008 int main(int argc, char *argv[])
00009 {
00010     int n, i, flag, op;
00011     float max, min, aux, x[N], vmx, vmn, md, st, z[N], cn, nr;
00012     srand(time(NULL));
00013     do{
00014         printf("Ingrese el numero de datos: ");
00015         scanf("%d", &n);
00016     }while(n<1||n>N);
00017     printf("Ingrese el numero maximo: ");
00018     scanf("%f", &max);
00019     printf("Ingrese el numero minimo: ");
00020     scanf("%f", &min);
00021     if(max<min)
00022     {
00023         aux = max;
00024         max = min;
00025         min = aux;
00026     }
00027     for(i=0, vmn=max, vmx=min, md=0, st=0; i<n; i++)
00028     {
00029         x[i] = ((max-min)*rand())/RAND_MAX+min;
00030         printf("X[%d] = %f\n", i+1, x[i]);
00031         if(vmn>x[i])
00032             vmn = x[i];
00033         if(vmx<x[i])
00034             vmx = x[i];
00035         md+=x[i];
00036         st+=(x[i]*x[i]);
00037     }
00038     md/=n;
00039     st/=n;
00040     st-=(md*md);
00041     st=sqrt(st);
00042     printf("Max: %f\n Min = %f\n Media = %f\n D. E. = %f\n",
00043         vmx, vmn, md, st);
00044     flag = 0;
00045     printf("Desea centrar: ");
00046     scanf("%d", &op);
00047     if(op)
00048     {
00049         printf("Por rango: ");
00050         scanf("%d", &op);
00051         flag|=(op?1:4);
00052     }
00053     printf("Desea normalizar: ");
00054     scanf("%d", &op);
00055     if(op)
00056     {
00057         printf("Por rango: ");
00058         scanf("%d", &op);
00059         flag|=(op?2:8);
00060     }
00061     cn = (flag&1)?vmn:(flag&4?md:0);
00062     nr = (flag&2)?vmx-vmn:(flag&8?st:1);
00063     printf("%d\t%f\t%f\n", flag, cn, nr);
00064     for(i=0; i<n; i++)
00065     {
00066         z[i] = (x[i]-cn)/nr;
00067         printf("Z[%d] = %f\n", i+1, z[i]);
00068     }
00069     return 0;
00070 }

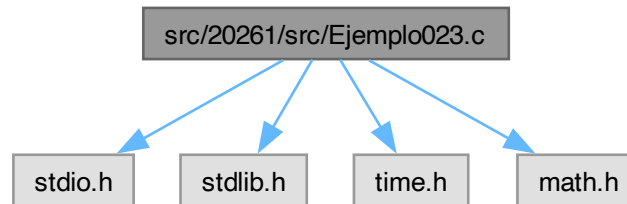
```

7.172 src/20261/src/Ejemplo023.c File Reference

Ordenamiento tipo Bucket Sort (distribución por casilleros) + ordenamiento interno por selección.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
```

Include dependency graph for Ejemplo023.c:



Macros

- `#define N 100000`

Functions

- `int main (int argc, char *argv[ ])`

7.172.1 Detailed Description

Ordenamiento tipo Bucket Sort (distribución por casilleros) + ordenamiento interno por selección.

Este programa genera n números aleatorios (float) en el rango $[min, max]$ y los ordena usando una estrategia tipo

Bucket Sort:

- 1) Define el número de casilleros (buckets) como:

$$nd = \lfloor \sqrt{n} \rfloor$$

- 2) Divide el rango $[min, max]$ en nd intervalos de ancho:

$$Dx = \frac{max - min}{nd}$$

- 3) Para cada valor $x[i]$, calcula el casillero:

$$j = \left\lfloor \frac{x[i] - min}{Dx} \right\rfloor$$

y se hace un ajuste para el caso $x[i] == max$ para evitar $j == nd$ (fuera de rango).

- 4) Usa un arreglo auxiliar $c[]$ como estructura compacta:

- $c[0 \dots nd-1]$: conteo de elementos por casillero
- $c[nd \dots 2*nd-1]$: posiciones de escritura (inician como prefijos y luego se incrementan)
- $c[2*nd \dots 3*nd-1]$: copia de los prefijos (inicio fijo de cada casillero en xc)

- 5) Distribuye los datos en un arreglo $xc[]$ agrupado por casilleros.

- 6) Finalmente **ordena cada casillero internamente**:

- 0 o 1 elemento: no hace nada
- 2 elementos: una comparación e intercambio
- ≥ 3 elementos: Selection Sort dentro del segmento del casillero

El resultado `xc[]` queda totalmente ordenado (ascendente) si cada casillero se ordena.

Entrada estándar

- `n` (int): número de elementos, $1 \leq n \leq N$
- `max` (float): límite superior del rango
- `min` (float): límite inferior del rango

Salida estándar

Imprime (modo “didáctico”, muy verboso):

- Arreglo desordenado.
- Configuración de buckets (`nd`, `rango`, `Dx`).
- Intervalos por casillero.
- Elementos con su casillero.
- Conteos/prefijos.
- Arreglo `xc` tras distribución.
- Arreglo `xc` ya ordenado.

Precondiciones

- Idealmente `min` $\leq$ `max`. Si `min` $>$ `max`, el código los intercambia.

Complejidad

- Generación + distribución: $O(n)$
- Prefijos: $O(nd)$
- Ordenamiento por casillero (Selection Sort): peor caso $O(n^2)$ si todo cae en un solo bucket, típico mejor si los datos se reparten.
- Memoria: $O(N)$

Warning

- `x[i] == max` se fuerza a `j = nd-1` para evitar `j == nd`.
- `c[]` se declara con tamaño `N` y se indexa hasta $\sim 3*nd$; como `nd = sqrt(n)`, se mantiene seguro para `n` ≤ 100000 .

```
gcc Ejemplo023.c -o Ejemplo023 -lm
./Ejemplo023
```

Definition in file [Ejemplo023.c](#).

7.172.2 Macro Definition Documentation

7.172.2.1 N

```
#define N 100000
```

Definition at line 80 of file [Ejemplo023.c](#).

7.172.3 Function Documentation

7.172.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 82 of file [Ejemplo023.c](#).

```
00083 {
00084     (void)argc;
00085     (void)argv;
00086
00087     /* n: cantidad de datos, nd: número de casilleros */
00088     int n, i, j, k, id_min, nd, c[N];
00089
00090     /* Rango [min, max] */
00091     float min, max;
00092
00093     /* x[]: arreglo original, xc[]: arreglo tras distribución y ordenamiento */
00094     float x[N], rango, Dx, xc[N], aux;
00095
00096     /* Leer n con validación */
00097     do {
00098         printf("Ingrese el numero de elementos: ");
00099         scanf("%d", &n);
00100     } while (n < 1 || n > N);
00101
00102     /* Leer rango */
00103     printf("Ingrese el valor maximo: ");
00104     scanf("%f", &max);
00105     printf("Ingrese el valor minimo: ");
00106     scanf("%f", &min);
00107
00108     /*
00109      Asegurar min <= max. El código contempla casos con cero para evitar división entre cero
00110      en el intercambio por multiplicación/división.
00111     */
00112     if (min > max)
00113     {
00114         if (!min)           /* min == 0 */
00115         {
00116             min = max;
00117             max = 0;
00118         }
00119         else if (!max)      /* max == 0 */
00120         {
00121             max = min;
00122             min = 0;
00123         }
00124         else
00125         {
00126             /* Intercambio por multiplicación/división */
00127             max *= min;
00128             min = max / min;
00129             max /= min;
00130         }
00131     }
00132
00133     srand(time(NULL));
00134
00135     /* Generar datos aleatorios y limpiar c[i] (solo una parte; nd <= n así que alcanza) */
00136     printf("Desordenado.\n");
00137     for (i = 0; i < n; i++)
00138     {
00139         /* Escalamiento lineal de rand() a [min, max] */
00140         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00141
00142         /* Inicializar contadores a cero (al menos hasta n) */
00143         c[i] = 0;
00144
00145         printf("x[%02d] = %f\n", i + 1, x[i]);
00146     }
00147
00148     /* Definir buckets como sqrt(n) */
00149     nd = (int)sqrt((double)n);
00150
00151     /* Tamaño de bucket */
00152     rango = max - min;
00153     Dx = rango / nd;
00154
00155     printf("ND = %d\tRango = %f\tDx = %f\n", nd, rango, Dx);
00156
00157     /* Mostrar intervalos por bucket (solo informativo) */
00158     for (i = 0; i < nd; i++)
```

```

00159         printf("%d. [%f, %f]\n", i, min + i*Dx, min + (i+1)*Dx);
00160
00161     /*
00162     1) Conteo por casillero (histograma):
00163         j = floor((x[i]-min)/Dx), pero si x[i]==max forzamos j=nd-1.
00164     */
00165     printf("Desordenado con numero de casillero.\n");
00166     for (i = 0; i < n; i++)
00167     {
00168         j = (x[i] == max ? nd - 1 : (int)((x[i] - min) / Dx));
00169         printf("x[%02d] = %f\t%d\n", i + 1, x[i], j);
00170         c[j]++; /* c[j] = cantidad de elementos del bucket j */
00171     }
00172
00173     /*
00174     2) Prefijos:
00175         Construye posiciones iniciales del bucket i en xc[].
00176         - c[i + nd] guarda la posición "corriente" de escritura del bucket i.
00177         - c[i + 2*nd] guarda una copia del prefijo (inicio fijo del bucket i).
00178     */
00179     for (i = 1; i < nd; i++)
00180     {
00181         c[i + nd] = c[i + nd - 1] + c[i - 1]; /* prefijo */
00182         c[i + 2*nd] = c[i + nd]; /* copia del prefijo */
00183     }
00184
00185     /* Mostrar resumen por bucket: intervalo, conteo, prefijo */
00186     for (i = 0; i < nd; i++)
00187         printf("%d. [%f, %f]\t%d\t%d\n",
00188             i, min + i*Dx, min + (i+1)*Dx, c[i], c[i + nd]);
00189
00190     /*
00191     3) Distribución a xc:
00192         Se inserta cada x[i] en xc según su prefijo, incrementando la posición de escritura.
00193     */
00194     for (i = 0; i < n; i++)
00195     {
00196         j = (x[i] == max ? nd - 1 : (int)((x[i] - min) / Dx));
00197         xc[c[j + nd]] = x[i];
00198         c[j + nd]++; /* avanzar posición de escritura del bucket */
00199     }
00200
00201     /* Imprimir xc con su bucket (informativo) */
00202     for (i = 0; i < n; i++)
00203     {
00204         j = (int)((xc[i] - min) / Dx);
00205         printf("x[%02d] = %f\t%d\n", i + 1, xc[i], j);
00206     }
00207
00208     /*
00209     4) Ordenar internamente cada bucket.
00210         El inicio del bucket i está en c[i + 2*nd].
00211         El tamaño del bucket i está en c[i].
00212     */
00213     for (i = 0; i < nd; i++)
00214     {
00215         switch (c[i])
00216         {
00217             case 0:
00218             case 1:
00219                 /* 0 o 1 elemento: ya está ordenado */
00220                 break;
00221
00222             case 2:
00223                 /* 2 elementos: una sola comparación */
00224                 if (xc[c[i + 2*nd]] > xc[c[i + 2*nd] + 1])
00225                 {
00226                     aux = xc[c[i + 2*nd]];
00227                     xc[c[i + 2*nd]] = xc[c[i + 2*nd] + 1];
00228                     xc[c[i + 2*nd] + 1] = aux;
00229                 }
00230                 break;
00231
00232             default:
00233                 /*
00234                 >= 3 elementos: Selection Sort dentro del segmento del bucket
00235                 [start, start + c[i] - 1], con start = c[i + 2*nd].
00236                 */
00237                 for (j = 0; j < (c[i] - 1); j++)
00238                 {
00239                     for (k = j + 1, id_min = j; k < c[i]; k++)
00240                         if (xc[id_min + c[i + 2*nd]] > xc[k + c[i + 2*nd]])
00241                             id_min = k;
00242
00243                     if (id_min != j)
00244                     {
00245                         aux = xc[j + c[i + 2*nd]];

```

```

00246             xc[j + c[i + 2*nd]] = xc[id_min + c[i + 2*nd]];
00247             xc[id_min + c[i + 2*nd]] = aux;
00248         }
00249     }
00250     break;
00251 }
00252 }
00253
00254 /* Resultado final */
00255 printf("Ordenado.\n");
00256 for (i = 0; i < n; i++)
00257     printf("x[%02d] = %f\n", i + 1, xc[i]);
00258
00259 return 0;
00260 }

```

7.173 Ejemplo023.c

[Go to the documentation of this file.](#)

```

00001
00074
00075 #include <stdio.h>
00076 #include <stdlib.h>
00077 #include <time.h>
00078 #include <math.h>
00079
00080 #define N 100000
00081
00082 int main(int argc, char *argv[])
00083 {
00084     (void)argc;
00085     (void)argv;
00086
00087     /* n: cantidad de datos, nd: número de casilleros */
00088     int n, i, j, k, id_min, nd, c[N];
00089
00090     /* Rango [min, max] */
00091     float min, max;
00092
00093     /* x[]: arreglo original, xc[]: arreglo tras distribución y ordenamiento */
00094     float x[N], rango, Dx, xc[N], aux;
00095
00096     /* Leer n con validación */
00097     do {
00098         printf("Ingrese el numero de elementos: ");
00099         scanf("%d", &n);
00100     } while (n < 1 || n > N);
00101
00102     /* Leer rango */
00103     printf("Ingrese el valor maximo: ");
00104     scanf("%f", &max);
00105     printf("Ingrese el valor minimo: ");
00106     scanf("%f", &min);
00107
00108     /*
00109      Asegurar min <= max. El código contempla casos con cero para evitar división entre cero
00110      en el intercambio por multiplicación/división.
00111     */
00112     if (min > max)
00113     {
00114         if (!min) /* min == 0 */
00115         {
00116             min = max;
00117             max = 0;
00118         }
00119         else if (!max) /* max == 0 */
00120         {
00121             max = min;
00122             min = 0;
00123         }
00124         else
00125         {
00126             /* Intercambio por multiplicación/división */
00127             max *= min;
00128             min = max / min;
00129             max /= min;
00130         }
00131     }
00132
00133     srand(time(NULL));
00134
00135     /* Generar datos aleatorios y limpiar c[i] (solo una parte; nd <= n así que alcanza) */
00136     printf("Desordenado.\n");
00137     for (i = 0; i < n; i++)

```



```

00138     {
00139         /* Escalamiento lineal de rand() a [min, max] */
00140         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00141
00142         /* Inicializar contadores a cero (al menos hasta n) */
00143         c[i] = 0;
00144
00145         printf("x[%02d] = %f\n", i + 1, x[i]);
00146     }
00147
00148     /* Definir buckets como sqrt(n) */
00149     nd = (int)sqrt((double)n);
00150
00151     /* Tamaño de bucket */
00152     rango = max - min;
00153     Dx = rango / nd;
00154
00155     printf("ND = %d\tRango = %f\tDx = %f\n", nd, rango, Dx);
00156
00157     /* Mostrar intervalos por bucket (solo informativo) */
00158     for (i = 0; i < nd; i++)
00159         printf("%d. [%f, %f]\n", i, min + i*Dx, min + (i+1)*Dx);
00160
00161     /*
00162     1) Conteo por casillero (histograma):
00163         j = floor((x[i]-min)/Dx), pero si x[i]==max forzamos j=nd-1.
00164     */
00165     printf("Desordenado con numero de casillero.\n");
00166     for (i = 0; i < n; i++)
00167     {
00168         j = (x[i] == max ? nd - 1 : (int)((x[i] - min) / Dx));
00169         printf("x[%02d] = %f\t%d\n", i + 1, x[i], j);
00170         c[j]++; /* c[j] = cantidad de elementos del bucket j */
00171     }
00172
00173     /*
00174     2) Prefijos:
00175         Construye posiciones iniciales del bucket i en xc[].
00176         - c[i + nd] guarda la posición "corriente" de escritura del bucket i.
00177         - c[i + 2*nd] guarda una copia del prefijo (inicio fijo del bucket i).
00178     */
00179     for (i = 1; i < nd; i++)
00180     {
00181         c[i + nd] = c[i + nd - 1] + c[i - 1]; /* prefijo */
00182         c[i + 2*nd] = c[i + nd]; /* copia del prefijo */
00183     }
00184
00185     /* Mostrar resumen por bucket: intervalo, conteo, prefijo */
00186     for (i = 0; i < nd; i++)
00187         printf("%d. [%f, %f]\t%d\t%d\n",
00188             i, min + i*Dx, min + (i+1)*Dx, c[i], c[i + nd]);
00189
00190     /*
00191     3) Distribución a xc:
00192         Se inserta cada x[i] en xc según su prefijo, incrementando la posición de escritura.
00193     */
00194     for (i = 0; i < n; i++)
00195     {
00196         j = (x[i] == max ? nd - 1 : (int)((x[i] - min) / Dx));
00197         xc[c[j] + nd] = x[i];
00198         c[j + nd]++; /* avanzar posición de escritura del bucket */
00199     }
00200
00201     /* Imprimir xc con su bucket (informativo) */
00202     for (i = 0; i < n; i++)
00203     {
00204         j = (int)((xc[i] - min) / Dx);
00205         printf("x[%02d] = %f\t%d\n", i + 1, xc[i], j);
00206     }
00207
00208     /*
00209     4) Ordenar internamente cada bucket.
00210         El inicio del bucket i está en c[i + 2*nd].
00211         El tamaño del bucket i está en c[i].
00212     */
00213     for (i = 0; i < nd; i++)
00214     {
00215         switch (c[i])
00216         {
00217             case 0:
00218             case 1:
00219                 /* 0 o 1 elemento: ya está ordenado */
00220                 break;
00221
00222             case 2:
00223                 /* 2 elementos: una sola comparación */
00224                 if (xc[c[i + 2*nd]] > xc[c[i + 2*nd] + 1])

```

```

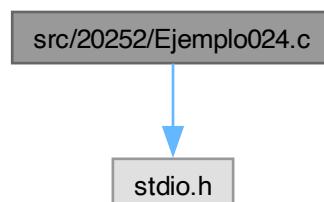
00225         {
00226             aux = xc[c[i + 2*nd]];
00227             xc[c[i + 2*nd]] = xc[c[i + 2*nd] + 1];
00228             xc[c[i + 2*nd] + 1] = aux;
00229         }
00230         break;
00231     }
00232     default:
00233         /*
00234          * >= 3 elementos: Selection Sort dentro del segmento del bucket
00235          * [start, start + c[i] - 1], con start = c[i + 2*nd].
00236          */
00237         for (j = 0; j < (c[i] - 1); j++)
00238         {
00239             for (k = j + 1, id_min = j; k < c[i]; k++)
00240                 if (xc[id_min + c[i + 2*nd]] > xc[k + c[i + 2*nd]])
00241                     id_min = k;
00242
00243             if (id_min != j)
00244             {
00245                 aux = xc[j + c[i + 2*nd]];
00246                 xc[j + c[i + 2*nd]] = xc[id_min + c[i + 2*nd]];
00247                 xc[id_min + c[i + 2*nd]] = aux;
00248             }
00249         }
00250         break;
00251     }
00252 }
00253
00254 /* Resultado final */
00255 printf("Ordenado.\n");
00256 for (i = 0; i < n; i++)
00257     printf("x[%02d] = %f\n", i + 1, xc[i]);
00258
00259 return 0;
00260 }

```

7.174 src/20252/Ejemplo024.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo024.c:



Macros

- #define [BIT](#)(n)
- #define [BIT_GET](#)(x, n)
- #define [BIT_SET](#)(x, n)
- #define [BIT_CLEAR](#)(x, n)
- #define [BIT_TOGGLE](#)(x, n)
- #define [BIT_WRITE](#)(x, n, v)
- #define [ES_PAR](#)(x)

Functions

- int [main](#) (int argc, char *argv[])

7.174.1 Macro Definition Documentation

7.174.1.1 BIT

```
#define BIT(  
    n)
```

Value:

`(1<<(n))`

Definition at line 3 of file [Ejemplo024.c](#).

7.174.1.2 BIT_CLEAR

```
#define BIT_CLEAR(  
    x,  
    n)
```

Value:

`((x) &= ~BIT(n))`

Definition at line 6 of file [Ejemplo024.c](#).

7.174.1.3 BIT_GET

```
#define BIT_GET(  
    x,  
    n)
```

Value:

`((x) & BIT(n))`

Definition at line 4 of file [Ejemplo024.c](#).

7.174.1.4 BIT_SET

```
#define BIT_SET(  
    x,  
    n)
```

Value:

`((x) |= BIT(n))`

Definition at line 5 of file [Ejemplo024.c](#).

7.174.1.5 BIT_TOGGLE

```
#define BIT_TOGGLE(  
    x,  
    n)
```

Value:

`((x) ^= BIT(n))`

Definition at line 7 of file [Ejemplo024.c](#).

7.174.1.6 BIT_WRITE

```
#define BIT_WRITE(  
    x,  
    n,  
    v)
```

Value:

`((v) ? BIT_SET(x,n) : BIT_CLEAR(x,n))`

Definition at line 8 of file [Ejemplo024.c](#).

7.174.1.7 ES_PAR

```
#define ES_PAR(  
    x)
```

Value:

```
(!BIT_GET(x,0))
```

Definition at line 9 of file [Ejemplo024.c](#).

7.174.2 Function Documentation**7.174.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 11 of file [Ejemplo024.c](#).

```
00012 {
00013     int x, nb, i;
00014     printf("x = ");
00015     scanf("%d", &x);
00016     printf("b = ");
00017     scanf("%d", &nb);
00018     for(i=7; i>-1; i--)
00019         printf("%d", BIT_GET(x,i)?1:0);
00020     printf("\n");
00021     BIT_SET(x, nb);
00022     printf("%d\n", x);
00023     return 0;
00024 }
```

7.175 Ejemplo024.c

[Go to the documentation of this file.](#)

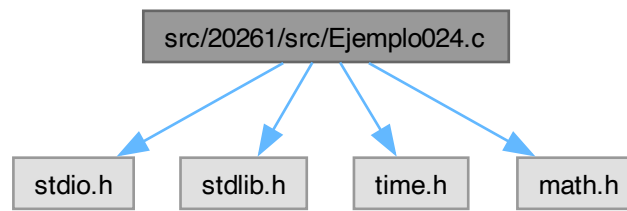
```
00001 #include <stdio.h>
00002
00003 #define BIT(n)          (1<<(n))
00004 #define BIT_GET(x,n)    ((x) & BIT(n))
00005 #define BIT_SET(x,n)    ((x) |= BIT(n))
00006 #define BIT_CLEAR(x,n)  ((x) &= ~BIT(n))
00007 #define BIT_TOGGLE(x,n) ((x) ^= BIT(n))
00008 #define BIT_WRITE(x,n,v) ((v)?BIT_SET(x,n):BIT_CLEAR(x,n))
00009 #define ES_PAR(x)       (!BIT_GET(x,0))
00010
00011 int main(int argc, char *argv[])
00012 {
00013     int x, nb, i;
00014     printf("x = ");
00015     scanf("%d", &x);
00016     printf("b = ");
00017     scanf("%d", &nb);
00018     for(i=7; i>-1; i--)
00019         printf("%d", BIT_GET(x,i)?1:0);
00020     printf("\n");
00021     BIT_SET(x, nb);
00022     printf("%d\n", x);
00023     return 0;
00024 }
```

7.176 src/20261/src/Ejemplo024.c File Reference

Bucket Sort (distribución por casilleros) + ordenamiento interno por selección (versión compacta).

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
```

Include dependency graph for Ejemplo024.c:



Macros

- `#define N 100000`

Functions

- `int main (int argc, char *argv[ ])`

7.176.1 Detailed Description

Bucket Sort (distribución por casilleros) + ordenamiento interno por selección (versión compacta).
Versión menos verbosa que Ejemplo023:

- Genera n números aleatorios en $[\text{min}, \text{max}]$.
- Define $\text{nd} = \text{floor}(\text{sqrt}(n))$ buckets y $\text{Dx} = (\text{max} - \text{min}) / \text{nd}$.
- Cuenta por bucket, calcula prefijos y distribuye a x .
- Ordena internamente cada bucket con Selection Sort (casos 0/1/2 optimizados).

A diferencia de Ejemplo023, aquí no se imprimen intervalos por bucket ni depuración detallada; solo muestra el arreglo final ordenado.

Entrada estándar

- n (int): $1 \leq n \leq N$
- `max` (float)
- `min` (float)

Salida estándar

- Arreglo desordenado.
- Arreglo ordenado.

Complejidad

Distribución: $O(n) + O(nd)$. Ordenamiento interno: depende de la distribución; peor caso $O(n^2)$. Memoria: $O(N)$.

```
gcc Ejemplo024.c -o Ejemplo024 -lm
./Ejemplo024
```

Definition in file [Ejemplo024.c](#).

7.176.2 Macro Definition Documentation

7.176.2.1 N

#define N 100000

Definition at line 39 of file [Ejemplo024.c](#).

7.176.3 Function Documentation

7.176.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 41 of file [Ejemplo024.c](#).

```
00042 {
00043     (void)argc;
00044     (void)argv;
00045
00046     int n, i, j, k, id_min, nd, c[N];
00047     float min, max;
00048     float x[N], rango, Dx, xc[N], aux;
00049
00050     do {
00051         printf("Ingrese el numero de elementos: ");
00052         scanf("%d", &n);
00053     } while (n < 1 || n > N);
00054
00055     printf("Ingrese el valor maximo: ");
00056     scanf("%f", &max);
00057     printf("Ingrese el valor minimo: ");
00058     scanf("%f", &min);
00059
00060     /* Asegurar min <= max (con manejo de ceros) */
00061     if (min > max)
00062     {
00063         if (!min) { min = max; max = 0; }
00064         else if (!max) { max = min; min = 0; }
00065         else { max *= min; min = max / min; max /= min; }
00066     }
00067
00068     srand(time(NULL));
00069
00070     printf("Desordenado.\n");
00071     for (i = 0; i < n; i++)
00072     {
00073         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00074         c[i] = 0;
00075         printf("x[%02d] = %f\n", i + 1, x[i]);
00076     }
00077
00078     /* Configuración de buckets */
00079     nd = (int)sqrt((double)n);
00080     rango = max - min;
00081     Dx = rango / nd;
00082
00083     /* Conteo por bucket */
00084     for (i = 0; i < n; i++)
00085     {
00086         j = (x[i] == max ? nd - 1 : (int)((x[i] - min) / Dx));
00087         c[j]++;
00088     }
00089
00090     /* Prefijos: c[nd..] y copia de prefijos en c[2*nd..] */
00091     for (i = 1; i < nd; i++)
00092     {
00093         c[i + nd] = c[i + nd - 1] + c[i - 1];
00094         c[i + 2*nd] = c[i + nd];
00095     }
00096
00097     /* Distribución a xc */
00098     for (i = 0; i < n; i++)
00099     {
00100         j = (x[i] == max ? nd - 1 : (int)((x[i] - min) / Dx));
00101         xc[c[j + nd]] = x[i];
00102         c[j + nd]++;
00103     }
00104
00105     /* Ordenar internamente cada bucket */
00106     for (i = 0; i < nd; i++)
00107     {
00108         switch (c[i])
```

```

00109     {
00110         case 0:
00111         case 1:
00112             break;
00113
00114         case 2:
00115             if (xc[c[i + 2*nd]] > xc[c[i + 2*nd] + 1])
00116             {
00117                 aux = xc[c[i + 2*nd]];
00118                 xc[c[i + 2*nd]] = xc[c[i + 2*nd] + 1];
00119                 xc[c[i + 2*nd] + 1] = aux;
00120             }
00121             break;
00122
00123         default:
00124             for (j = 0; j < (c[i] - 1); j++)
00125             {
00126                 for (k = j + 1, id_min = j; k < c[i]; k++)
00127                     if (xc[id_min + c[i + 2*nd]] > xc[k + c[i + 2*nd]])
00128                         id_min = k;
00129
00130                 if (id_min != j)
00131                 {
00132                     aux = xc[j + c[i + 2*nd]];
00133                     xc[j + c[i + 2*nd]] = xc[id_min + c[i + 2*nd]];
00134                     xc[id_min + c[i + 2*nd]] = aux;
00135                 }
00136             }
00137             break;
00138     }
00139 }
00140
00141 /* Resultado */
00142 printf("Ordenado.\n");
00143 for (i = 0; i < n; i++)
00144     printf("x[%02d] = %f\n", i + 1, xc[i]);
00145
00146 return 0;
00147 }

```

7.177 Ejemplo024.c

[Go to the documentation of this file.](#)

```

00001
00033
00034 #include <stdio.h>
00035 #include <stdlib.h>
00036 #include <time.h>
00037 #include <math.h>
00038
00039 #define N 100000
00040
00041 int main(int argc, char *argv[])
00042 {
00043     (void)argc;
00044     (void)argv;
00045
00046     int n, i, j, k, id_min, nd, c[N];
00047     float min, max;
00048     float x[N], rango, Dx, xc[N], aux;
00049
00050     do {
00051         printf("Ingrese el numero de elementos: ");
00052         scanf("%d", &n);
00053     } while (n < 1 || n > N);
00054
00055     printf("Ingrese el valor maximo: ");
00056     scanf("%f", &max);
00057     printf("Ingrese el valor minimo: ");
00058     scanf("%f", &min);
00059
00060     /* Asegurar min <= max (con manejo de ceros) */
00061     if (min > max)
00062     {
00063         if (!min) { min = max; max = 0; }
00064         else if (!max) { max = min; min = 0; }
00065         else { max *= min; min = max / min; max /= min; }
00066     }
00067
00068     srand(time(NULL));
00069
00070     printf("Desordenado.\n");
00071     for (i = 0; i < n; i++)
00072     {

```

```

00073         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00074         c[i] = 0;
00075         printf("x[%02d] = %f\n", i + 1, x[i]);
00076     }
00077
00078     /* Configuración de buckets */
00079     nd = (int)sqrt((double)n);
00080     rango = max - min;
00081     Dx = rango / nd;
00082
00083     /* Conteo por bucket */
00084     for (i = 0; i < n; i++)
00085     {
00086         j = (x[i] == max ? nd - 1 : (int)((x[i] - min) / Dx));
00087         c[j]++;
00088     }
00089
00090     /* Prefijos: c[nd..] y copia de prefijos en c[2*nd..] */
00091     for (i = 1; i < nd; i++)
00092     {
00093         c[i + nd] = c[i + nd - 1] + c[i - 1];
00094         c[i + 2*nd] = c[i + nd];
00095     }
00096
00097     /* Distribución a xc */
00098     for (i = 0; i < n; i++)
00099     {
00100         j = (x[i] == max ? nd - 1 : (int)((x[i] - min) / Dx));
00101         xc[c[j + nd]] = x[i];
00102         c[j + nd]++;
00103     }
00104
00105     /* Ordenar internamente cada bucket */
00106     for (i = 0; i < nd; i++)
00107     {
00108         switch (c[i])
00109         {
00110             case 0:
00111             case 1:
00112                 break;
00113
00114             case 2:
00115                 if (xc[c[i + 2*nd]] > xc[c[i + 2*nd] + 1])
00116                 {
00117                     aux = xc[c[i + 2*nd]];
00118                     xc[c[i + 2*nd]] = xc[c[i + 2*nd] + 1];
00119                     xc[c[i + 2*nd] + 1] = aux;
00120                 }
00121                 break;
00122
00123             default:
00124                 for (j = 0; j < (c[i] - 1); j++)
00125                 {
00126                     for (k = j + 1, id_min = j; k < c[i]; k++)
00127                         if (xc[id_min + c[i + 2*nd]] > xc[k + c[i + 2*nd]])
00128                             id_min = k;
00129
00130                     if (id_min != j)
00131                     {
00132                         aux = xc[j + c[i + 2*nd]];
00133                         xc[j + c[i + 2*nd]] = xc[id_min + c[i + 2*nd]];
00134                         xc[id_min + c[i + 2*nd]] = aux;
00135                     }
00136                 }
00137                 break;
00138         }
00139     }
00140
00141     /* Resultado */
00142     printf("Ordenado.\n");
00143     for (i = 0; i < n; i++)
00144         printf("x[%02d] = %f\n", i + 1, xc[i]);
00145
00146     return 0;
00147 }

```

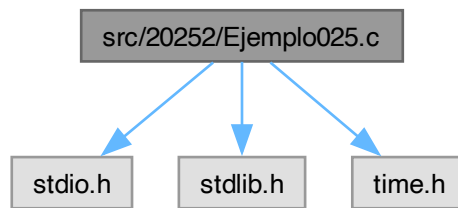
7.178 src/20252/Ejemplo025.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```


Include dependency graph for Ejemplo025.c:



Macros

- `#define BIT(n)`
- `#define BIT_GET(x, n)`
- `#define BIT_SET(x, n)`
- `#define BIT_CLEAR(x, n)`
- `#define BIT_TOGGLE(x, n)`
- `#define BIT_WRITE(x, n, v)`
- `#define ES_PAR(x)`
- `#define N 50`

Functions

- `int main (int argc, char *argv[])`

7.178.1 Macro Definition Documentation

7.178.1.1 BIT

```
#define BIT(
    n)
```

Value:

```
((1<<(n))
```

Definition at line 5 of file [Ejemplo025.c](#).

7.178.1.2 BIT_CLEAR

```
#define BIT_CLEAR(
    x,
    n)
```

Value:

```
((x) &= ~BIT(n))
```

Definition at line 8 of file [Ejemplo025.c](#).

7.178.1.3 BIT_GET

```
#define BIT_GET(
    x,
    n)
```

Value:

```
((x) & BIT(n))
```

Definition at line 6 of file [Ejemplo025.c](#).

7.178.1.4 BIT_SET

```
#define BIT_SET(
    x,
    n)
```

Value:

```
((x) |= BIT(n))
```

Definition at line 7 of file [Ejemplo025.c](#).

7.178.1.5 BIT_TOGGLE

```
#define BIT_TOGGLE(
    x,
    n)
```

Value:

```
((x) ^= BIT(n))
```

Definition at line 9 of file [Ejemplo025.c](#).

7.178.1.6 BIT_WRITE

```
#define BIT_WRITE(
    x,
    n,
    v)
```

Value:

```
((v) ? BIT_SET(x,n) : BIT_CLEAR(x,n))
```

Definition at line 10 of file [Ejemplo025.c](#).

7.178.1.7 ES_PAR

```
#define ES_PAR(
    x)
```

Value:

```
(!BIT_GET(x,0))
```

Definition at line 11 of file [Ejemplo025.c](#).

7.178.1.8 N

```
#define N 50
```

Definition at line 12 of file [Ejemplo025.c](#).

7.178.2 Function Documentation

7.178.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 14 of file [Ejemplo025.c](#).

```
00015 {
00016     char str[N], c, p, v;
00017     int i, nc, cc, cb;
00018     srand(time(NULL));
00019     i = 0;
00020     do{
00021         c = getchar();
00022         str[i] = c;
00023         i++;
00024     }while(c!=10&i<(N-1));
00025     nc = i-1;
00026     str[nc] = '\0';
00027     p = 0;
00028     i = 0;
00029     while(str[i]!='\0')
00030     {
```

```

00031         p^=str[i];
00032         i++;
00033     }
00034     printf("%s (%d)\n", str, p);
00035     cc = (rand()%nc);
00036     cb = (rand()%8);
00037     BIT_TOGGLE(str[cc],cb);
00038     v = p;
00039     i = 0;
00040     while(str[i]!='\0')
00041     {
00042         v^=str[i];
00043         i++;
00044     }
00045     printf("%s (%d, %d, %d)\n", str, v, cc, cb);
00046     return 0;
00047 }

```

7.179 Ejemplo025.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define BIT(n)                (1<<(n))
00006 #define BIT_GET(x,n)          ((x) & BIT(n))
00007 #define BIT_SET(x,n)          ((x) |= BIT(n))
00008 #define BIT_CLEAR(x,n)        ((x) &= ~BIT(n))
00009 #define BIT_TOGGLE(x,n)       ((x) ^= BIT(n))
00010 #define BIT_WRITE(x,n,v)      ((v)?BIT_SET(x,n):BIT_CLEAR(x,n))
00011 #define ES_PAR(x)              (!BIT_GET(x,0))
00012 #define N 50
00013
00014 int main(int argc, char *argv[])
00015 {
00016     char str[N], c, p, v;
00017     int i, nc, cc, cb;
00018     srand(time(NULL));
00019     i = 0;
00020     do{
00021         c = getchar();
00022         str[i] = c;
00023         i++;
00024     }while(c!=10&&i<(N-1));
00025     nc = i-1;
00026     str[nc] = '\0';
00027     p = 0;
00028     i = 0;
00029     while(str[i]!='\0')
00030     {
00031         p^=str[i];
00032         i++;
00033     }
00034     printf("%s (%d)\n", str, p);
00035     cc = (rand()%nc);
00036     cb = (rand()%8);
00037     BIT_TOGGLE(str[cc],cb);
00038     v = p;
00039     i = 0;
00040     while(str[i]!='\0')
00041     {
00042         v^=str[i];
00043         i++;
00044     }
00045     printf("%s (%d, %d, %d)\n", str, v, cc, cb);
00046     return 0;
00047 }

```

7.180 src/20261/src/Ejemplo025.c File Reference

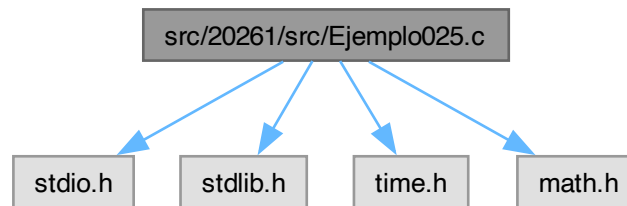
Ordenamiento alternado por selección: mínimo a la izquierda, máximo a la derecha (floats).

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

```

Include dependency graph for Ejemplo025.c:



Macros

- `#define N 100000`

Functions

- `int main (int argc, char *argv[ ])`

7.180.1 Detailed Description

Ordenamiento alternado por selección: mínimo a la izquierda, máximo a la derecha (floats).

El programa genera n flotantes aleatorios en $[\text{min}, \text{max}]$ y los ordena con un método tipo **selección bidireccional alternada**:

- En pasadas pares (i par): selecciona el **mínimo** del subarreglo actual y lo coloca al inicio (izquierda).
- En pasadas impares (i impar): selecciona el **máximo** del subarreglo actual y lo coloca al final (derecha).

Los límites del subarreglo se van cerrando conforme avanza:

- $\text{izquierda} = i/2$
- $\text{derecha} = n-1 - i/2$

Implementación:

- Se usa una sola variable i para alternar entre mínimo y máximo (según $i\%2$).
- j recorre el rango activo en dirección apropiada.

Entrada estándar

- n (int): $1 \leq n \leq N$
- `max` (float)
- `min` (float)

Salida estándar

- Arreglo desordenado.
- Arreglo ordenado.

Complejidad

Tiempo: $O(n^2)$. Memoria: $O(N)$.

Warning

El intercambio de [min/max](#) se hace con multiplicación/división y puede fallar si `min==0` o `max==0`. Para una versión robusta, usar una variable auxiliar.

```
gcc Ejemplo025.c -o Ejemplo025
./Ejemplo025
```

Definition in file [Ejemplo025.c](#).

7.180.2 Macro Definition Documentation

7.180.2.1 N

```
#define N 100000
```

Definition at line 49 of file [Ejemplo025.c](#).

7.180.3 Function Documentation

7.180.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 51 of file [Ejemplo025.c](#).

```
00052 {
00053     (void)argc;
00054     (void)argv;
00055
00056     int n, i, j;
00057     float min, max;
00058     float x[N], aux;
00059
00060     /* Leer n */
00061     do {
00062         printf("Ingrese el numero de elementos: ");
00063         scanf("%d", &n);
00064     } while (n < 1 || n > N);
00065
00066     /* Leer rango */
00067     printf("Ingrese el valor maximo: ");
00068     scanf("%f", &max);
00069     printf("Ingrese el valor minimo: ");
00070     scanf("%f", &min);
00071
00072     /* Asegurar min <= max (swap por mult/div: no robusto si hay ceros) */
00073     if (min > max)
00074     {
00075         max *= min;
00076         min = max / min;
00077         max /= min;
00078     }
00079
00080     srand(time(NULL));
00081
00082     /* Generar datos */
00083     printf("Desordenado.\n");
00084     for (i = 0; i < n; i++)
00085     {
00086         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00087         printf("x[%02d] = %f\n", i + 1, x[i]);
00088     }
00089
00090     /*
00091     Ordenamiento alternado:
00092     - i recorre 0..n-2
00093     - si i es par:
00094         fija posición izquierda = i/2 y busca el mínimo en [izq..der]
00095     - si i es impar:
00096         fija posición derecha = n-1-i/2 y busca el máximo en [izq..der]
00097
00098     Las expresiones ternarias compactan el índice objetivo:
00099     objetivo = (i%2 ? n-1-i/2 : i/2)
00100
00101     y recorren j en dirección adecuada según el caso.
00102     */
00103     for (i = 0; i < (n - 1); i++)
00104         for (j = (i % 2 ? n - 2 - i/2 : i/2 + 1);
00105              (i % 2 ? j > (i/2) : j < (n - i/2));
```

```

00106         (i % 2 ? j--          : j++))
00107     {
00108         /*
00109         Comparación:
00110         - i par (mínimo):    si x[j] < x[i/2] -> swap con x[i/2]
00111         - i impar (máximo):  si x[n-1-i/2] < x[j] -> swap con x[n-1-i/2]
00112         */
00113         if (x[i%2 ? n - 1 - i/2 : j] < x[i%2 ? j : i/2])
00114         {
00115             /* Swap clásico con auxiliar */
00116             aux = x[j];
00117             x[j] = x[i%2 ? n - 1 - i/2 : i/2];
00118             x[i%2 ? n - 1 - i/2 : i/2] = aux;
00119         }
00120     }
00121
00122     /* Imprimir resultado */
00123     printf("Ordenado.\n");
00124     for (i = 0; i < n; i++)
00125         printf("x[%02d] = %f\n", i + 1, x[i]);
00126
00127     return 0;
00128 }

```

7.181 Ejemplo025.c

[Go to the documentation of this file.](#)

```

00001
00043
00044 #include <stdio.h>
00045 #include <stdlib.h>
00046 #include <time.h>
00047 #include <math.h>
00048
00049 #define N 100000
00050
00051 int main(int argc, char *argv[])
00052 {
00053     (void)argc;
00054     (void)argv;
00055
00056     int n, i, j;
00057     float min, max;
00058     float x[N], aux;
00059
00060     /* Leer n */
00061     do {
00062         printf("Ingrese el numero de elementos: ");
00063         scanf("%d", &n);
00064     } while (n < 1 || n > N);
00065
00066     /* Leer rango */
00067     printf("Ingrese el valor maximo: ");
00068     scanf("%f", &max);
00069     printf("Ingrese el valor minimo: ");
00070     scanf("%f", &min);
00071
00072     /* Asegurar min <= max (swap por mult/div: no robusto si hay ceros) */
00073     if (min > max)
00074     {
00075         max *= min;
00076         min = max / min;
00077         max /= min;
00078     }
00079
00080     srand(time(NULL));
00081
00082     /* Generar datos */
00083     printf("Desordenado.\n");
00084     for (i = 0; i < n; i++)
00085     {
00086         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00087         printf("x[%02d] = %f\n", i + 1, x[i]);
00088     }
00089
00090     /*
00091     Ordenamiento alternado:
00092     - i recorre 0..n-2
00093     - si i es par:
00094         fija posición izquierda = i/2 y busca el mínimo en [izq..der]
00095     - si i es impar:
00096         fija posición derecha = n-1-i/2 y busca el máximo en [izq..der]
00097
00098     Las expresiones ternarias compactan el índice objetivo:

```

```

00099         objetivo = (i%2 ? n-1-i/2 : i/2)
00100
00101     y recorren j en dirección adecuada según el caso.
00102     */
00103     for (i = 0; i < (n - 1); i++)
00104         for (j = (i % 2 ? n - 2 - i/2 : i/2 + 1);
00105              (i % 2 ? j > (i/2) : j < (n - i/2));
00106              (i % 2 ? j-- : j++))
00107         {
00108             /*
00109              Comparación:
00110              - i par (mínimo):    si x[j] < x[i/2] -> swap con x[i/2]
00111              - i impar (máximo): si x[n-1-i/2] < x[j] -> swap con x[n-1-i/2]
00112             */
00113             if (x[i%2 ? n - 1 - i/2 : j] < x[i%2 ? j : i/2])
00114             {
00115                 /* Swap clásico con auxiliar */
00116                 aux = x[j];
00117                 x[j] = x[i%2 ? n - 1 - i/2 : i/2];
00118                 x[i%2 ? n - 1 - i/2 : i/2] = aux;
00119             }
00120         }
00121
00122     /* Imprimir resultado */
00123     printf("Ordenado.\n");
00124     for (i = 0; i < n; i++)
00125         printf("x[%02d] = %f\n", i + 1, x[i]);
00126
00127     return 0;
00128 }

```

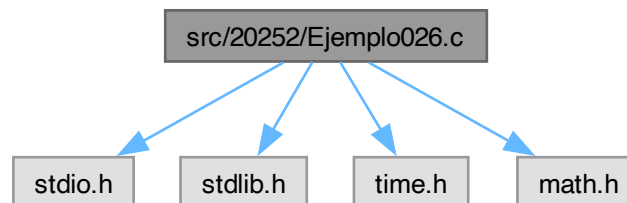
7.182 src/20252/Ejemplo026.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

```

Include dependency graph for Ejemplo026.c:



Macros

- #define `BIT(n)`
- #define `BIT_GET(x, n)`
- #define `BIT_SET(x, n)`
- #define `BIT_CLEAR(x, n)`
- #define `BIT_TOGGLE(x, n)`
- #define `BIT_WRITE(x, n, v)`
- #define `ES_PAR(x)`
- #define `N 50`

Functions

- int [main](#) (int argc, char *argv[])

7.182.1 Macro Definition Documentation

7.182.1.1 BIT

```
#define BIT(  
        n)
```

Value:

$(1 \ll (n))$

Definition at line 6 of file [Ejemplo026.c](#).

7.182.1.2 BIT_CLEAR

```
#define BIT_CLEAR(  
        x,  
        n)
```

Value:

$((x) \&= \sim \text{BIT}(n))$

Definition at line 9 of file [Ejemplo026.c](#).

7.182.1.3 BIT_GET

```
#define BIT_GET(  
        x,  
        n)
```

Value:

$((x) \& \text{BIT}(n))$

Definition at line 7 of file [Ejemplo026.c](#).

7.182.1.4 BIT_SET

```
#define BIT_SET(  
        x,  
        n)
```

Value:

$((x) \mid= \text{BIT}(n))$

Definition at line 8 of file [Ejemplo026.c](#).

7.182.1.5 BIT_TOGGLE

```
#define BIT_TOGGLE(  
        x,  
        n)
```

Value:

$((x) \wedge= \text{BIT}(n))$

Definition at line 10 of file [Ejemplo026.c](#).

7.182.1.6 BIT_WRITE

```
#define BIT_WRITE(  
        x,  
        n,  
        v)
```

Value:

$((v) ? \text{BIT_SET}(x, n) : \text{BIT_CLEAR}(x, n))$

Definition at line 11 of file [Ejemplo026.c](#).

7.182.1.7 ES_PAR

```
#define ES_PAR(  
    x)
```

Value:

```
(!BIT_GET(x,0))
```

Definition at line 12 of file [Ejemplo026.c](#).

7.182.1.8 N

```
#define N 50
```

Definition at line 13 of file [Ejemplo026.c](#).

7.182.2 Function Documentation

7.182.2.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 15 of file [Ejemplo026.c](#).

```
00016 {  
00017     char str[N], strP[N], strR[N], c, p, v;  
00018     int i, j, k, l, nc, cc, cb, np, p2, p2f, cc2, cb2;  
00019     srand(time(NULL));  
00020     i = 0;  
00021     do{  
00022         c = getchar();  
00023         str[i] = c;  
00024         i++;  
00025     }while(c!=10&& i<(N-1));  
00026     nc = i-1;  
00027     str[nc] = '\0';  
00028     np = (int)(log2(nc+log2(nc)+1)+1);  
00029     printf("NP = %d\n", np);  
00030     printf("%s (%d, %d)\n", str, nc, np);  
00031     for(i=0, j=0, p2=1; i<(nc+np); i++)  
00032     {  
00033         if((i+1)==p2)  
00034         {  
00035             strP[i] = 0;  
00036             p2<<=1;  
00037         }  
00038         else  
00039         {  
00040             c = str[j++];  
00041             strP[i] = c;  
00042             for(k=0, p2f=1; k<np; k++)  
00043             {  
00044                 if(p2f&(i+1))  
00045                     strP[p2f-1]^=c;  
00046                 p2f<<=1;  
00047             }  
00048         }  
00049     }  
00050     strP[nc+np] = '\0';  
00051     cc = rand()%(nc+np);  
00052     cb = rand()%8;  
00053     BIT_TOGGLE(strP[cc],cb);  
00054     printf("%s\n", strP);  
00055     for(i=0, j=0, l=0, p2=1; i<(nc+np); i++)  
00056     {  
00057         if((i+1)==p2)  
00058         {  
00059             strR[nc+1+l] = strP[i];  
00060             l++;  
00061             p2 <= 1;  
00062         }  
00063         else  
00064         {  
00065             c = strP[i];  
00066             strR[j++] = c;  
00067             for(k=0, p2f=1; k<np; k++)  
00068             {  
00069                 if(p2f&(i+1))  
00070                     strR[nc+1+k]^=c;  
00071                 p2f<<=1;  
00072             }  
00073         }  
00074     }  
00075 }
```

```

00073     }
00074 }
00075 strR[nc] = '\0';
00076 strR[nc+np+1] = '\0';
00077 printf("%s (%d, %d)\n", strR, cc, cb);
00078 for(i=0, cc2=0; i<np; i++)
00079 {
00080     printf("%d\t", strR[nc+1+i]);
00081     BIT_WRITE(cc2,i,strR[nc+1+i]);
00082     if(strR[nc+1+i])
00083         cb2 = log2(strR[nc+1+i]);
00084 }
00085 printf("(%d, %d)\n", cc2-1, cb2);
00086 BIT_TOGGLE(strP[cc2-1],cb2);
00087 printf("%s\n", strP);
00088 return 0;
00089 }

```

7.183 Ejemplo026.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004 #include <math.h>
00005
00006 #define BIT(n) (1<<(n))
00007 #define BIT_GET(x,n) ((x) & BIT(n))
00008 #define BIT_SET(x,n) ((x) |= BIT(n))
00009 #define BIT_CLEAR(x,n) ((x) &= ~BIT(n))
00010 #define BIT_TOGGLE(x,n) ((x) ^= BIT(n))
00011 #define BIT_WRITE(x,n,v) ((v)?BIT_SET(x,n):BIT_CLEAR(x,n))
00012 #define ES_PAR(x) (!BIT_GET(x,0))
00013 #define N 50
00014
00015 int main(int argc, char *argv[])
00016 {
00017     char str[N], strP[N], strR[N], c, p, v;
00018     int i, j, k, l, nc, cc, cb, np, p2, p2f, cc2, cb2;
00019     srand(time(NULL));
00020     i = 0;
00021     do{
00022         c = getchar();
00023         str[i] = c;
00024         i++;
00025     }while(c!=10&& i<(N-1));
00026     nc = i-1;
00027     str[nc] = '\0';
00028     np = (int)(log2(nc+log2(nc)+1)+1);
00029     printf("NP = %d\n", np);
00030     printf("%s (%d, %d)\n", str, nc, np);
00031     for(i=0, j=0, p2=1; i<(nc+np); i++)
00032     {
00033         if((i+1)==p2)
00034         {
00035             strP[i] = 0;
00036             p2<<=1;
00037         }
00038         else
00039         {
00040             c = str[j++];
00041             strP[i] = c;
00042             for(k=0, p2f=1; k<np; k++)
00043             {
00044                 if(p2f&(i+1))
00045                     strP[p2f-1]^=c;
00046                 p2f<<=1;
00047             }
00048         }
00049     }
00050     strP[nc+np] = '\0';
00051     cc = rand()%(nc+np);
00052     cb = rand()%8;
00053     BIT_TOGGLE(strP[cc],cb);
00054     printf("%s\n", strP);
00055     for(i=0, j=0, l=0, p2=1; i<(nc+np); i++)
00056     {
00057         if((i+1)==p2)
00058         {
00059             strR[nc+1+l] = strP[i];
00060             l++;
00061             p2 <= 1;
00062         }
00063         else

```

```

00064     {
00065         c = strP[i];
00066         strR[j++] = c;
00067         for(k=0, p2f=1; k<np; k++)
00068         {
00069             if(p2f&(i+1))
00070                 strR[nc+1+k]^=c;
00071             p2f<<=1;
00072         }
00073     }
00074 }
00075 strR[nc] = '\0';
00076 strR[nc+np+1] = '\0';
00077 printf("%s (%d, %d)\n", strR, cc, cb);
00078 for(i=0, cc2=0; i<np; i++)
00079 {
00080     printf("%d\t", strR[nc+1+i]);
00081     BIT_WRITE(cc2,i,strR[nc+1+i]);
00082     if(strR[nc+1+i])
00083         cb2 = log2(strR[nc+1+i]);
00084 }
00085 printf("(%d, %d)\n", cc2-1, cb2);
00086 BIT_TOGGLE(strP[cc2-1],cb2);
00087 printf("%s\n", strP);
00088 return 0;
00089 }

```

7.184 src/20261/src/Ejemplo026.c File Reference

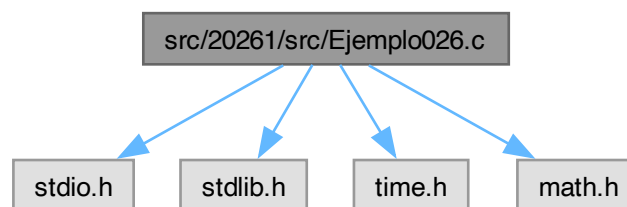
Ordenamiento por conteo (Counting Sort) para enteros en el rango [0, max].

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

```

Include dependency graph for Ejemplo026.c:



Macros

- #define `N` 300000
- #define `mx` 16384

Functions

- int `main` (int argc, char *argv[])

7.184.1 Detailed Description

Ordenamiento por conteo (Counting Sort) para enteros en el rango [0, max].

Este programa genera un arreglo de `n` enteros aleatorios en el rango [0, max], construye un histograma de frecuencias `h[]` y reconstruye el arreglo ordenado.

Algoritmo (Counting Sort): 1) Generar datos: $x[i] \in [0, \text{max}]$ 2) Inicializar histograma $h[0..\text{max}] = 0$ 3) Contar frecuencias: $h[x[i]]++$ 4) Reconstruir ordenado:

- para cada valor j de $0..\text{max}$, escribirlo $h[j]$ veces en $x[]$

Entrada estándar

- n (long int): número de elementos, $1 \leq n \leq N$
- max (int): valor máximo permitido ($1 \leq \text{max} \leq \text{mx}$)

Salida estándar

- Arreglo desordenado
- Arreglo ordenado

Restricciones

- $N = 300000$ (máximo tamaño del arreglo)
- $\text{mx} = 16384$ (máximo rango permitido para histograma)
- Los valores generados están en $[0, \text{max}]$.

Complejidad

- Tiempo: $O(n + \text{max})$
- Memoria: $O(N + \text{mx})$

Note

Se calcula $\text{nd} = \text{ceil}(\log_{10}(n))$ para imprimir índices con ceros a la izquierda, por ejemplo $x[0001]$, $x[0123]$, etc.

```
gcc Ejemplo026.c -o Ejemplo026 -lm
./Ejemplo026
```

Definition in file [Ejemplo026.c](#).

7.184.2 Macro Definition Documentation

7.184.2.1 mx

```
#define mx 16384
```

Definition at line 49 of file [Ejemplo026.c](#).

7.184.2.2 N

```
#define N 300000
```

Definition at line 48 of file [Ejemplo026.c](#).

7.184.3 Function Documentation

7.184.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 51 of file [Ejemplo026.c](#).

```
00052 {
00053     (void)argc;
00054     (void)argv;
00055
00056     int max, x[N];
00057     long int n, h[mx], i, j, k;
```

```

00058     float nd;
00059
00060     /* Leer n con validación */
00061     do {
00062         printf("Ingrese el numero de elementos: ");
00063         scanf("%ld", &n);
00064     } while (n < 1 || n > N);
00065
00066     /* Semilla para números pseudoaleatorios */
00067     srand((unsigned)time(NULL));
00068
00069     /* nd = número de dígitos aproximado para dar formato a los índices */
00070     nd = ceil(log10(n));
00071
00072     /* Leer max con validación */
00073     do {
00074         printf("Ingrese l valor maximo: ");
00075         scanf("%d", &max);
00076     } while (max < 1 || max > mx);
00077
00078     /* Generar e imprimir arreglo desordenado */
00079     printf("Desordenado\n");
00080     for (i = 0; i < n; i++)
00081     {
00082         /* Valores en [0, max] */
00083         x[i] = rand() % (max + 1);
00084         printf("x[%0*ld] = %d\n", (int)nd, i + 1, x[i]);
00085     }
00086
00087     /* Inicializar histograma */
00088     for (i = 0; i <= max; i++)
00089         h[i] = 0;
00090
00091     /* Contar frecuencias */
00092     for (i = 0; i < n; i++)
00093         h[x[i]]++;
00094
00095     /*
00096      Reconstrucción del arreglo ordenado:
00097      - j recorre el dominio 0..max buscando valores con frecuencia > 0
00098      - escribe j h[j] veces
00099     */
00100     i = 0;
00101     j = 0;
00102     while (i < n)
00103     {
00104         while (!h[j]) /* saltar valores con frecuencia 0 */
00105             j++;
00106
00107         for (k = 0; k < h[j]; k++)
00108             x[i++] = (int)j;
00109
00110         j++;
00111     }
00112
00113     /* Imprimir arreglo ordenado */
00114     printf("Ordenado\n");
00115     for (i = 0; i < n; i++)
00116         printf("x[%0*ld] = %d\n", (int)nd, i + 1, x[i]);
00117
00118     return 0;
00119 }

```

7.185 Ejemplo026.c

[Go to the documentation of this file.](#)

```

00001
00042
00043 #include <stdio.h>
00044 #include <stdlib.h>
00045 #include <time.h>
00046 #include <math.h>
00047
00048 #define N 300000
00049 #define mx 16384
00050
00051 int main(int argc, char *argv[])
00052 {
00053     (void)argc;
00054     (void)argv;
00055
00056     int max, x[N];
00057     long int n, h[mx], i, j, k;
00058     float nd;

```

```

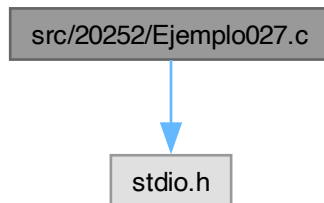
00059
00060     /* Leer n con validación */
00061     do {
00062         printf("Ingrese el numero de elementos: ");
00063         scanf("%ld", &n);
00064     } while (n < 1 || n > N);
00065
00066     /* Semilla para números pseudoaleatorios */
00067     srand((unsigned)time(NULL));
00068
00069     /* nd = número de dígitos aproximado para dar formato a los índices */
00070     nd = ceil(log10(n));
00071
00072     /* Leer max con validación */
00073     do {
00074         printf("Ingrese l valor maximo: ");
00075         scanf("%d", &max);
00076     } while (max < 1 || max > mx);
00077
00078     /* Generar e imprimir arreglo desordenado */
00079     printf("Desordenado\n");
00080     for (i = 0; i < n; i++)
00081     {
00082         /* Valores en [0, max] */
00083         x[i] = rand() % (max + 1);
00084         printf("x[%0*ld] = %d\n", (int)nd, i + 1, x[i]);
00085     }
00086
00087     /* Inicializar histograma */
00088     for (i = 0; i <= max; i++)
00089         h[i] = 0;
00090
00091     /* Contar frecuencias */
00092     for (i = 0; i < n; i++)
00093         h[x[i]]++;
00094
00095     /*
00096     Reconstrucción del arreglo ordenado:
00097     - j recorre el dominio 0..max buscando valores con frecuencia > 0
00098     - escribe j h[j] veces
00099     */
00100     i = 0;
00101     j = 0;
00102     while (i < n)
00103     {
00104         while (!h[j]) /* saltar valores con frecuencia 0 */
00105             j++;
00106
00107         for (k = 0; k < h[j]; k++)
00108             x[i++] = (int)j;
00109
00110         j++;
00111     }
00112
00113     /* Imprimir arreglo ordenado */
00114     printf("Ordenado\n");
00115     for (i = 0; i < n; i++)
00116         printf("x[%0*ld] = %d\n", (int)nd, i + 1, x[i]);
00117
00118     return 0;
00119 }

```

7.186 src/20252/Ejemplo027.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo027.c:



Functions

- `int main (int argc, char *argv[])`

7.186.1 Function Documentation

7.186.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo027.c](#).

```

00004 {
00005     int n, a, ai, b, m, c, mr, i;
00006     printf("n = ");
00007     scanf("%d", &n);
00008     printf("a = ");
00009     scanf("%d", &a);
00010     printf("b = ");
00011     scanf("%d", &b);
00012     for(i=0; i<n; i++)
00013         if(((a*i)%n)==1)
00014         {
00015             ai = i;
00016             printf("ai = %d\n", ai);
00017         }
00018     printf("m = ");
00019     scanf("%d", &m);
00020     c = (a*m+b)%n;
00021     printf("c = %d\n", c);
00022     if(c>=b)
00023         mr = c-b;
00024     else
00025         mr = n+c-b;
00026     mr = (ai*mr)%n;
00027     printf("m = %d\n", mr);
00028     return 0;
00029 }
```

7.187 Ejemplo027.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int n, a, ai, b, m, c, mr, i;
```

```

00006     printf("n = ");
00007     scanf("%d", &n);
00008     printf("a = ");
00009     scanf("%d", &a);
00010     printf("b = ");
00011     scanf("%d", &b);
00012     for(i=0; i<n; i++)
00013         if(((a*i)%n)==1)
00014         {
00015             ai = i;
00016             printf("ai = %d\n", ai);
00017         }
00018     printf("m = ");
00019     scanf("%d", &m);
00020     c = (a*m+b)%n;
00021     printf("c = %d\n", c);
00022     if(c>=b)
00023         mr = c-b;
00024     else
00025         mr = n+c-b;
00026     mr = (ai*mr)%n;
00027     printf("m = %d\n", mr);
00028     return 0;
00029 }

```

7.188 src/20261/src/Ejemplo027.c File Reference

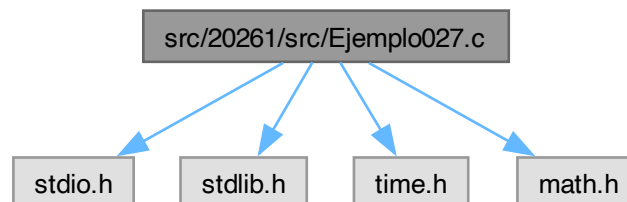
Ordenamiento por conteo (Counting Sort) para enteros en el rango [min, max].

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

```

Include dependency graph for Ejemplo027.c:



Macros

- #define [N](#) 300000
- #define [mx](#) 32768

Functions

- int [main](#) (int argc, char *argv[])

7.188.1 Detailed Description

Ordenamiento por conteo (Counting Sort) para enteros en el rango [min, max].

Este programa genera un arreglo de n enteros aleatorios en el rango [min, max], usa un histograma $h[]$ de tamaño $(\text{max}-\text{min}+1)$ y reconstruye el arreglo ordenado.

Se utiliza el desplazamiento: $\text{índice_hist} = x[i] - \text{min}$ para mapear [min, max] a [0, max-min].

Entrada estándar

- `n` (long int): número de elementos, $1 \leq n \leq N$
- `max` (int)
- `min` (int)

Salida estándar

- Arreglo desordenado
- Arreglo ordenado

Restricciones

- `N` = 300000
- `mx` = 32768 (capacidad del histograma)
- Se requiere que $(\text{max} - \text{min}) \leq \text{mx} - 1$ para no desbordar `h[]`.

Complejidad

- Tiempo: $O(n + (\text{max} - \text{min}))$
- Memoria: $O(N + \text{mx})$

```
gcc Ejemplo027.c -o Ejemplo027 -lm
./Ejemplo027
```

Definition in file [Ejemplo027.c](#).

7.188.2 Macro Definition Documentation**7.188.2.1 `mx`**

```
#define mx 32768
```

Definition at line 43 of file [Ejemplo027.c](#).

7.188.2.2 `N`

```
#define N 300000
```

Definition at line 42 of file [Ejemplo027.c](#).

7.188.3 Function Documentation**7.188.3.1 `main()`**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 45 of file [Ejemplo027.c](#).

```
00046 {
00047     (void)argc;
00048     (void)argv;
00049
00050     int min, max, x[N], aux;
00051     long int n, h[mx], i, j, k;
00052     float nd;
00053
00054     /* Leer n con validación */
00055     do {
00056         printf("Ingrese el numero de elementos: ");
00057         scanf("%ld", &n);
00058     } while (n < 1 || n > N);
00059
00060     srand((unsigned)time(NULL));
00061
00062     nd = ceil(log10(n));
00063
00064     /* Leer rango */
```

```

00065     printf("Ingrese el valor maximo: ");
00066     scanf("%d", &max);
00067     printf("Ingrese el valor minimo: ");
00068     scanf("%d", &min);
00069
00070     /* Asegurar min <= max */
00071     if (max < min)
00072     {
00073         aux = min;
00074         min = max;
00075         max = aux;
00076     }
00077
00078     /* Generar e imprimir */
00079     printf("Desordenado\n");
00080     for (i = 0; i < n; i++)
00081     {
00082         x[i] = rand() % (max - min + 1) + min;
00083         printf("x[%0*ld] = %d\n", (int)nd, i + 1, x[i]);
00084     }
00085
00086     /* Inicializar histograma de tamaño (max-min+1) */
00087     for (i = 0; i <= (max - min); i++)
00088         h[i] = 0;
00089
00090     /* Contar (con desplazamiento por min) */
00091     for (i = 0; i < n; i++)
00092         h[x[i] - min]++;
00093
00094     /* Reconstrucción */
00095     i = 0;
00096     j = 0;
00097     while (i < n)
00098     {
00099         while (!h[j])
00100             j++;
00101
00102         for (k = 0; k < h[j]; k++)
00103             x[i++] = (int)(j + min);
00104
00105         j++;
00106     }
00107
00108     /* Imprimir ordenado */
00109     printf("Ordenado\n");
00110     for (i = 0; i < n; i++)
00111         printf("x[%0*ld] = %d\n", (int)nd, i + 1, x[i]);
00112
00113     return 0;
00114 }

```

7.189 Ejemplo027.c

[Go to the documentation of this file.](#)

```

00001
00036
00037 #include <stdio.h>
00038 #include <stdlib.h>
00039 #include <time.h>
00040 #include <math.h>
00041
00042 #define N 300000
00043 #define mx 32768
00044
00045 int main(int argc, char *argv[])
00046 {
00047     (void)argc;
00048     (void)argv;
00049
00050     int min, max, x[N], aux;
00051     long int n, h[mx], i, j, k;
00052     float nd;
00053
00054     /* Leer n con validación */
00055     do {
00056         printf("Ingrese el numero de elementos: ");
00057         scanf("%ld", &n);
00058     } while (n < 1 || n > N);
00059
00060     srand((unsigned)time(NULL));
00061
00062     nd = ceil(log10(n));
00063
00064     /* Leer rango */

```

```

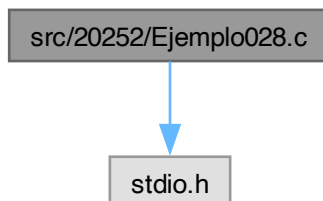
00065     printf("Ingrese el valor maximo: ");
00066     scanf("%d", &max);
00067     printf("Ingrese el valor minimo: ");
00068     scanf("%d", &min);
00069
00070     /* Asegurar min <= max */
00071     if (max < min)
00072     {
00073         aux = min;
00074         min = max;
00075         max = aux;
00076     }
00077
00078     /* Generar e imprimir */
00079     printf("Desordenado\n");
00080     for (i = 0; i < n; i++)
00081     {
00082         x[i] = rand() % (max - min + 1) + min;
00083         printf("x[%0*ld] = %d\n", (int)nd, i + 1, x[i]);
00084     }
00085
00086     /* Inicializar histograma de tamaño (max-min+1) */
00087     for (i = 0; i <= (max - min); i++)
00088         h[i] = 0;
00089
00090     /* Contar (con desplazamiento por min) */
00091     for (i = 0; i < n; i++)
00092         h[x[i] - min]++;
00093
00094     /* Reconstrucción */
00095     i = 0;
00096     j = 0;
00097     while (i < n)
00098     {
00099         while (!h[j])
00100             j++;
00101
00102         for (k = 0; k < h[j]; k++)
00103             x[i++] = (int)(j + min);
00104
00105         j++;
00106     }
00107
00108     /* Imprimir ordenado */
00109     printf("Ordenado\n");
00110     for (i = 0; i < n; i++)
00111         printf("x[%0*ld] = %d\n", (int)nd, i + 1, x[i]);
00112
00113     return 0;
00114 }

```

7.190 src/20252/Ejemplo028.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo028.c:



Macros

- #define N 50

- `#define NC 52`

Functions

- `int main (int argc, char *argv[])`

7.190.1 Macro Definition Documentation

7.190.1.1 N

`#define N 50`

Definition at line 3 of file [Ejemplo028.c](#).

7.190.1.2 NC

`#define NC 52`

Definition at line 4 of file [Ejemplo028.c](#).

7.190.2 Function Documentation

7.190.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 6 of file [Ejemplo028.c](#).

```
00007 {
00008     char str[N], strC[N], strD[N], c;
00009     int i, a, b, ai, nc;
00010     ai = -1;
00011     do{
00012         printf("a = ");
00013         scanf("%d", &a);
00014         for(i=0; i<NC; i++)
00015             if(((a*i)%NC)==1)
00016                 {
00017                     ai = i;
00018                     break;
00019                 }
00020     }while(ai<0);
00021     printf("b = ");
00022     scanf("%d", &b);
00023     while ((c = getchar()) != '\n' && c != EOF) {}
00024     i = 0;
00025     do{
00026         c = getchar();
00027         if ((c>='A' && c<='Z') || ((c>='a' && c<='z') || c==' '))
00028             {
00029                 str[i] = c;
00030                 i++;
00031             }
00032     }while(c!=10&&i<(N-1));
00033     nc = i;
00034     str[nc] = '\0';
00035     printf("MSG: %s\n", str);
00036     for(i=0; i<nc; i++)
00037     {
00038         if(str[i]>='A' && str[i]<='Z')
00039             strC[i] = str[i]-'A';
00040         else if(str[i]>='a' && str[i]<='z')
00041             strC[i] = str[i]-'a'+26;
00042         else
00043             strC[i] = NC;
00044     }
00045     strC[nc] = '\0';
00046     for(i=0; i<nc; i++)
00047         if(strC[i]<NC)
00048             strC[i] = (a*strC[i]+b)%NC;
00049     for(i=0; i<nc; i++)
00050     {
00051         if(strC[i]<NC/2)
00052             strC[i] = strC[i]+'A';
00053         else if(strC[i]<NC)
00054             strC[i] = strC[i]+'a'-26;
00055         else
```

```

00056         strC[i] = ' ';
00057     }
00058     printf("MSG ENCRIPTADO: %s\n", strC);
00059     for(i=0; i<nc; i++)
00060     {
00061         if(strC[i]>='A' && strC[i]<='Z')
00062             strD[i] = strC[i]-'A';
00063         else if(strC[i]>='a' && strC[i]<='z')
00064             strD[i] = strC[i]-'a'+26;
00065         else
00066             strD[i] = NC;
00067     }
00068     strD[nc] = '\0';
00069     for(i=0; i<nc; i++)
00070         if(strD[i]<NC)
00071         {
00072             if(strD[i]>=b)
00073                 strD[i] = (ai*(strD[i]-b))%NC;
00074             else
00075                 strD[i] = (ai*(NC+strD[i]-b))%NC;
00076         }
00077     for(i=0; i<nc; i++)
00078     {
00079         if(strD[i]<NC/2)
00080             strD[i] = strD[i]+'A';
00081         else if(strD[i]<NC)
00082             strD[i] = strD[i]+'a'-26;
00083         else
00084             strD[i] = ' ';
00085     }
00086     printf("MSG DESENCRIPTADO: %s\n", strD);
00087     return 0;
00088 }

```

7.191 Ejemplo028.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define N 50
00004 #define NC 52
00005
00006 int main(int argc, char *argv[])
00007 {
00008     char str[N], strC[N], strD[N], c;
00009     int i, a, b, ai, nc;
00010     ai = -1;
00011     do{
00012         printf("a = ");
00013         scanf("%d", &a);
00014         for(i=0; i<NC; i++)
00015             if(((a*i)%NC)==1)
00016             {
00017                 ai = i;
00018                 break;
00019             }
00020     }while(ai<0);
00021     printf("b = ");
00022     scanf("%d", &b);
00023     while ((c = getchar()) != '\n' && c != EOF) {}
00024     i = 0;
00025     do{
00026         c = getchar();
00027         if((c>='A' && c<='Z') || ((c>='a' && c<='z') || c==' '))
00028         {
00029             str[i] = c;
00030             i++;
00031         }
00032     }while(c!=10&&i<(N-1));
00033     nc = i;
00034     str[nc] = '\0';
00035     printf("MSG: %s\n", str);
00036     for(i=0; i<nc; i++)
00037     {
00038         if(str[i]>='A' && str[i]<='Z')
00039             strC[i] = str[i]-'A';
00040         else if(str[i]>='a' && str[i]<='z')
00041             strC[i] = str[i]-'a'+26;
00042         else
00043             strC[i] = NC;
00044     }
00045     strC[nc] = '\0';
00046     for(i=0; i<nc; i++)
00047         if(strC[i]<NC)

```

```

00048         strC[i] = (a*strC[i]+b)%NC;
00049     for(i=0; i<nc; i++)
00050     {
00051         if(strC[i]<NC/2)
00052             strC[i] = strC[i]+'A';
00053         else if(strC[i]<NC)
00054             strC[i] = strC[i]+'a'-26;
00055         else
00056             strC[i] = ' ';
00057     }
00058     printf("MSG ENCRYPTADO: %s\n", strC);
00059     for(i=0; i<nc; i++)
00060     {
00061         if(strC[i]>='A' && strC[i]<='Z')
00062             strD[i] = strC[i]-'A';
00063         else if(strC[i]>='a' && strC[i]<='z')
00064             strD[i] = strC[i]-'a'+26;
00065         else
00066             strD[i] = NC;
00067     }
00068     strD[nc] = '\0';
00069     for(i=0; i<nc; i++)
00070         if(strD[i]<NC)
00071         {
00072             if(strD[i]>=b)
00073                 strD[i] = (ai*(strD[i]-b))%NC;
00074             else
00075                 strD[i] = (ai*(NC+strD[i]-b))%NC;
00076         }
00077     for(i=0; i<nc; i++)
00078     {
00079         if(strD[i]<NC/2)
00080             strD[i] = strD[i]+'A';
00081         else if(strD[i]<NC)
00082             strD[i] = strD[i]+'a'-26;
00083         else
00084             strD[i] = ' ';
00085     }
00086     printf("MSG DESENCRIPTADO: %s\n", strD);
00087     return 0;
00088 }

```

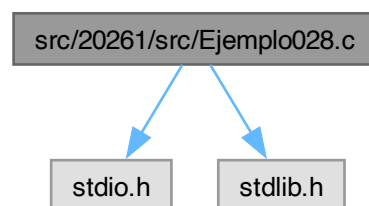
7.192 src/20261/src/Ejemplo028.c File Reference

Resta de matrices: $C = A - B$ para matrices reales de tamaño $n \times m$.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo028.c:



Macros

- #define **N** 10
- #define **M** 10

Functions

- `int main (int argc, char *argv[ ])`

7.192.1 Detailed Description

Resta de matrices: $C = A - B$ para matrices reales de tamaño $n \times m$.

El programa: 1) Lee las dimensiones n (renglones) y m (columnas), limitadas por N y M . 2) Captura los elementos de A y B . 3) Calcula $C[i][j] = A[i][j] - B[i][j]$. 4) Imprime C .

Entrada estándar

- n (int): renglones, $1 \leq n \leq N$
- m (int): columnas, $1 \leq m \leq M$
- $A[i][j]$ (float): $n \times m$ valores
- $B[i][j]$ (float): $n \times m$ valores

Salida estándar

- $C[i][j]$ (float): $n \times m$ valores

Complejidad

Tiempo: $O(n \times m)$. Memoria: $O(N \times M)$.

```
gcc Ejemplo028.c -o Ejemplo028
./Ejemplo028
```

Definition in file [Ejemplo028.c](#).

7.192.2 Macro Definition Documentation

7.192.2.1 M

```
#define M 10
```

Definition at line 34 of file [Ejemplo028.c](#).

7.192.2.2 N

```
#define N 10
```

Definition at line 33 of file [Ejemplo028.c](#).

7.192.3 Function Documentation

7.192.3.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 36 of file [Ejemplo028.c](#).

```
00037 {
00038     (void)argc;
00039     (void)argv;
00040
00041     int n, m, i, j;
00042     float A[N][M], B[N][M], C[N][M];
00043
00044     /* Leer dimensiones */
00045     do {
00046         printf("Ingrese el numero de rengles: ");
00047         scanf("%d", &n);
00048     } while (n < 1 || n > N);
00049
00050     do {
00051         printf("Ingrese el numero de columnas: ");
00052         scanf("%d", &m);
00053     } while (m < 1 || m > M);
```

```

00054
00055     /* Capturar A */
00056     for (i = 0; i < n; i++)
00057         for (j = 0; j < m; j++)
00058         {
00059             printf("A[%d][%d] = ", i + 1, j + 1);
00060             scanf("%f", &A[i][j]);
00061         }
00062
00063     /* Capturar B */
00064     for (i = 0; i < n; i++)
00065         for (j = 0; j < m; j++)
00066         {
00067             printf("B[%d][%d] = ", i + 1, j + 1);
00068             scanf("%f", &B[i][j]);
00069         }
00070
00071     /* C = A - B */
00072     for (i = 0; i < n; i++)
00073         for (j = 0; j < m; j++)
00074             C[i][j] = A[i][j] - B[i][j];
00075
00076     /* Imprimir C */
00077     for (i = 0; i < n; i++)
00078         for (j = 0; j < m; j++)
00079             printf("C[%d][%d] = %f\n", i + 1, j + 1, C[i][j]);
00080
00081     return 0;
00082 }

```

7.193 Ejemplo028.c

[Go to the documentation of this file.](#)

```

00001
00029
00030 #include <stdio.h>
00031 #include <stdlib.h>
00032
00033 #define N 10
00034 #define M 10
00035
00036 int main(int argc, char *argv[])
00037 {
00038     (void)argc;
00039     (void)argv;
00040
00041     int n, m, i, j;
00042     float A[N][M], B[N][M], C[N][M];
00043
00044     /* Leer dimensiones */
00045     do {
00046         printf("Ingrese el numero de rengles: ");
00047         scanf("%d", &n);
00048     } while (n < 1 || n > N);
00049
00050     do {
00051         printf("Ingrese el numero de columnas: ");
00052         scanf("%d", &m);
00053     } while (m < 1 || m > M);
00054
00055     /* Capturar A */
00056     for (i = 0; i < n; i++)
00057         for (j = 0; j < m; j++)
00058         {
00059             printf("A[%d][%d] = ", i + 1, j + 1);
00060             scanf("%f", &A[i][j]);
00061         }
00062
00063     /* Capturar B */
00064     for (i = 0; i < n; i++)
00065         for (j = 0; j < m; j++)
00066         {
00067             printf("B[%d][%d] = ", i + 1, j + 1);
00068             scanf("%f", &B[i][j]);
00069         }
00070
00071     /* C = A - B */
00072     for (i = 0; i < n; i++)
00073         for (j = 0; j < m; j++)
00074             C[i][j] = A[i][j] - B[i][j];
00075
00076     /* Imprimir C */
00077     for (i = 0; i < n; i++)
00078         for (j = 0; j < m; j++)

```

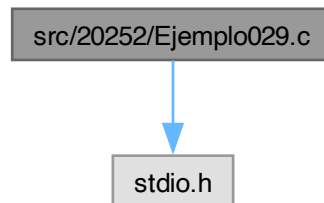


```
00079         printf("C[%d][%d] = %f\n", i + 1, j + 1, C[i][j]);
00080
00081     return 0;
00082 }
```

7.194 src/20252/Ejemplo029.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo029.c:



Functions

- double [exp1](#) (double x, int n)
- double [exp3](#) (double x, int n)
- double [exp4](#) (double x, int n)
- double [exp5](#) (double x, int n)
- double [potencia](#) (double x, int n)
- long int [factorial](#) (int n)
- double [potencia2](#) (double x, int n)
- long int [factorial2](#) (int n)
- double [fct](#) (double x, int n)
- int [main](#) (int argc, char *argv[])

7.194.1 Function Documentation

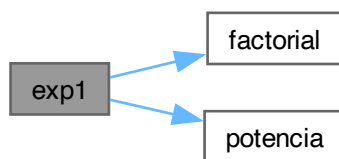
7.194.1.1 [exp1\(\)](#)

```
double exp1 (  
    double x,  
    int n)
```

Definition at line 62 of file [Ejemplo029.c](#).

```
00063 {  
00064     int i;  
00065     double ex;  
00066     for(i=0, ex=0; i<n; i++)  
00067         ex += potencia(x, i)/factorial(i);  
00068     return ex;  
00069 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.194.1.2 exp3()

```
double exp3 (  
    double x,  
    int n)
```

Definition at line 50 of file [Ejemplo029.c](#).

```
00051 {  
00052     int i;  
00053     double ex, fct;  
00054     for(i=0, ex=0, fct=1; i<n; i++)  
00055     {  
00056         ex += fct;  
00057         fct *= x/(i+1);  
00058     }  
00059     return ex;  
00060 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



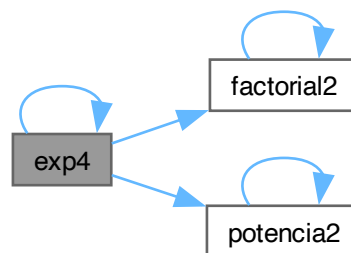
7.194.1.3 exp4()

```
double exp4 (  
    double x,  
    int n)
```

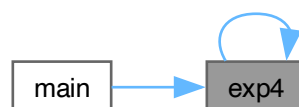
Definition at line 89 of file [Ejemplo029.c](#).

```
00090 {  
00091     if (n>0)  
00092         return exp4(x, n-1)+potencia2(x,n)/factorial2(n);  
00093     else  
00094         return 1;  
00095 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.194.1.4 exp5()

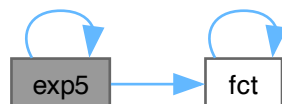
```
double exp5 (  
    double x,
```

```
double x,
int n)
```

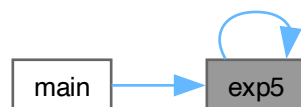
Definition at line 42 of file [Ejemplo029.c](#).

```
00043 {
00044     if(n)
00045         return exp5(x, n-1)+fct(x, n);
00046     else
00047         return 1;
00048 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



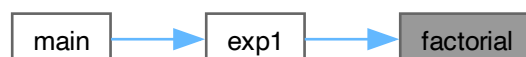
7.194.1.5 factorial()

```
long int factorial (
    int n)
```

Definition at line 80 of file [Ejemplo029.c](#).

```
00081 {
00082     int i;
00083     long int fn;
00084     for(i=0, fn=1; i<n; i++)
00085         fn *= (i+1);
00086     return fn;
00087 }
```

Here is the caller graph for this function:



7.194.1.6 factorial2()

```
long int factorial2 (  
    int n)
```

Definition at line 105 of file [Ejemplo029.c](#).

```
00106 {  
00107     if (n>0)  
00108         return factorial2(n-1)*n;  
00109     else  
00110         return 1;  
00111 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.194.1.7 fct()

```
double fct (  
    double x,  
    int n)
```

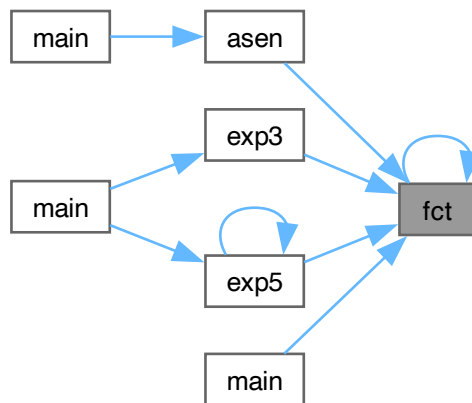
Definition at line 34 of file [Ejemplo029.c](#).

```
00035 {  
00036     if (n!=1)  
00037         return fct(x, n-1) * (x/n);  
00038     else  
00039         return x;  
00040 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



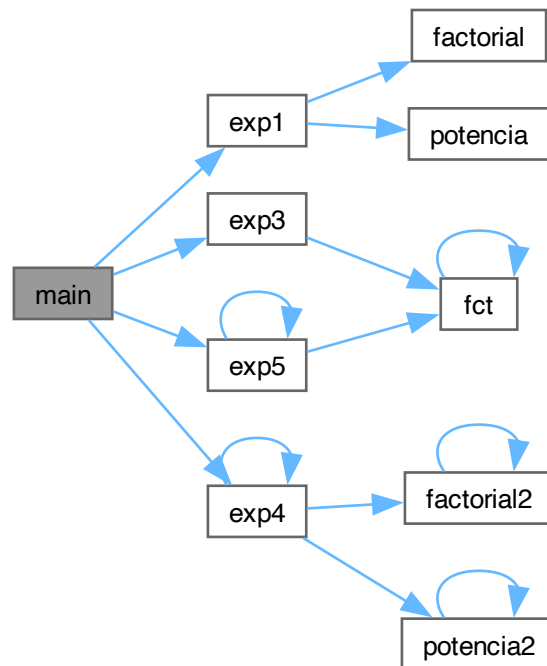
7.194.1.8 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 13 of file [Ejemplo029.c](#).

```
00014 {
00015     double x, ex, ex2, ex3, ex4;
00016     int n;
00017     printf("Ingrese el valor de x: ");
00018     scanf("%lf", &x);
00019     do{
00020         printf("Ingrese le numero de terminos: ");
00021         scanf("%d", &n);
00022     }while(n<1);
00023     ex = exp1(x, n);
00024     ex2 = exp3(x, n);
00025     ex3 = exp4(x, n);
00026     ex4 = exp5(x, n);
00027     printf("exp(%lf) = %lf\n", x, ex);
00028     printf("exp(%lf) = %lf\n", x, ex2);
00029     printf("exp(%lf) = %lf\n", x, ex3);
00030     printf("exp(%lf) = %lf\n", x, ex4);
00031     return 0;
00032 }
```

Here is the call graph for this function:



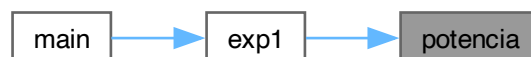
7.194.1.9 potencia()

```
double potencia (
    double x,
    int n)
```

Definition at line 71 of file [Ejemplo029.c](#).

```
00072 {
00073     int i;
00074     double xn;
00075     for(i=0, xn=1; i<n; i++)
00076         xn *= x;
00077     return xn;
00078 }
```

Here is the caller graph for this function:



7.194.1.10 potencia2()

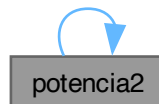
```
double potencia2 (
```

```
double x,
int n)
```

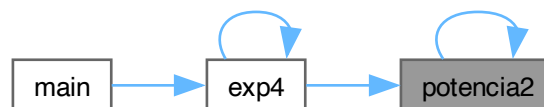
Definition at line 97 of file [Ejemplo029.c](#).

```
00098 {
00099     if (n>0)
00100         return potencia2(x, n-1)*x;
00101     else
00102         return 1;
00103 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.195 Ejemplo029.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 double exp1(double x, int n);
00004 double exp3(double x, int n);
00005 double exp4(double x, int n);
00006 double exp5(double x, int n);
00007 double potencia(double x, int n);
00008 long int factorial(int n);
00009 double potencia2(double x, int n);
00010 long int factorial2(int n);
00011 double fct(double x, int n);
00012
00013 int main(int argc, char *argv[])
00014 {
00015     double x, ex, ex2, ex3, ex4;
00016     int n;
00017     printf("Ingrese el valor de x: ");
00018     scanf("%lf", &x);
00019     do{
00020         printf("Ingrese le numero de terminos: ");
00021         scanf("%d", &n);
00022     }while(n<1);
00023     ex = exp1(x, n);
00024     ex2 = exp3(x, n);
00025     ex3 = exp4(x, n);
00026     ex4 = exp5(x, n);
00027     printf("exp(%lf) = %lf\n", x, ex);
00028     printf("exp(%lf) = %lf\n", x, ex2);
00029     printf("exp(%lf) = %lf\n", x, ex3);
```



```

00030     printf("exp(%lf) = %lf\n", x, ex4);
00031     return 0;
00032 }
00033
00034 double fct(double x, int n)
00035 {
00036     if(n!=1)
00037         return fct(x, n-1)*(x/n);
00038     else
00039         return x;
00040 }
00041
00042 double exp5(double x, int n)
00043 {
00044     if(n)
00045         return exp5(x, n-1)+fct(x, n);
00046     else
00047         return 1;
00048 }
00049
00050 double exp3(double x, int n)
00051 {
00052     int i;
00053     double ex, fct;
00054     for(i=0, ex=0, fct=1; i<n; i++)
00055     {
00056         ex += fct;
00057         fct *= x/(i+1);
00058     }
00059     return ex;
00060 }
00061
00062 double exp1(double x, int n)
00063 {
00064     int i;
00065     double ex;
00066     for(i=0, ex=0; i<n; i++)
00067         ex += potencia(x, i)/factorial(i);
00068     return ex;
00069 }
00070
00071 double potencia(double x, int n)
00072 {
00073     int i;
00074     double xn;
00075     for(i=0, xn=1; i<n; i++)
00076         xn *= x;
00077     return xn;
00078 }
00079
00080 long int factorial(int n)
00081 {
00082     int i;
00083     long int fn;
00084     for(i=0, fn=1; i<n; i++)
00085         fn *= (i+1);
00086     return fn;
00087 }
00088
00089 double exp4(double x, int n)
00090 {
00091     if(n>0)
00092         return exp4(x, n-1)+potencia2(x,n)/factorial2(n);
00093     else
00094         return 1;
00095 }
00096
00097 double potencia2(double x, int n)
00098 {
00099     if(n>0)
00100         return potencia2(x, n-1)*x;
00101     else
00102         return 1;
00103 }
00104
00105 long int factorial2(int n)
00106 {
00107     if(n>0)
00108         return factorial2(n-1)*n;
00109     else
00110         return 1;
00111 }

```

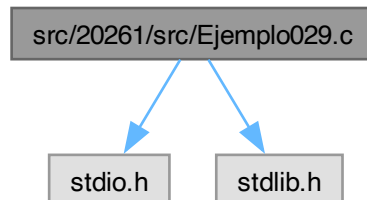
7.196 src/20261/src/Ejemplo029.c File Reference

Multiplicación de matrices: $C = A * B$ con verificación de dimensiones.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo029.c:



Macros

- `#define` [N](#) 10
- `#define` [M](#) 10

Functions

- `int` [main](#) (`int argc`, `char *argv[]`)

7.196.1 Detailed Description

Multiplicación de matrices: $C = A * B$ con verificación de dimensiones.

El programa: 1) Lee dimensiones de A: $nA \times mA$ y de B: $nB \times mB$. 2) Verifica condición de multiplicación: $mA == nB$ Si no se cumple, termina con error. 3) Captura elementos de A y B. 4) Calcula C ($nA \times mB$) usando: $C[i][j] = \sum_{k=0..mA-1} A[i][k] * B[k][j]$ 5) Imprime C.

Entrada estándar

- Dimensiones: nA , mA , nB , mB (1.. N/M)
- Elementos de A y B (float)

Salida estándar

- $C[i][j]$ (float) para $i=1..nA$ y $j=1..mB$

Complejidad

Tiempo: $O(nA * mB * mA)$. Memoria: $O(N * M)$.

```
gcc Ejemplo029.c -o Ejemplo029
./Ejemplo029
```

Definition in file [Ejemplo029.c](#).

7.196.2 Macro Definition Documentation

7.196.2.1 M

```
#define M 10
```

Definition at line [36](#) of file [Ejemplo029.c](#).

7.196.2.2 N

#define N 10

Definition at line 35 of file [Ejemplo029.c](#).

7.196.3 Function Documentation

7.196.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 38 of file [Ejemplo029.c](#).

```
00039 {
00040     (void)argc;
00041     (void)argv;
00042
00043     int nA, mA, nB, mB, i, j, k;
00044     float A[N][M], B[N][M], C[N][M];
00045
00046     /* Dimensiones */
00047     do {
00048         printf("Ingrese el numero de rengles de A: ");
00049         scanf("%d", &nA);
00050     } while (nA < 1 || nA > N);
00051
00052     do {
00053         printf("Ingrese el numero de columnas de A: ");
00054         scanf("%d", &mA);
00055     } while (mA < 1 || mA > M);
00056
00057     do {
00058         printf("Ingrese el numero de rengles de B: ");
00059         scanf("%d", &nB);
00060     } while (nB < 1 || nB > N);
00061
00062     do {
00063         printf("Ingrese el numero de columnas de B: ");
00064         scanf("%d", &mB);
00065     } while (mB < 1 || mB > M);
00066
00067     /* Condición para multiplicación */
00068     if (mA != nB)
00069     {
00070         printf("Error las columnas de A no son iguales a los renglones de B.\n");
00071         return 1;
00072     }
00073
00074     /* Capturar A */
00075     for (i = 0; i < nA; i++)
00076         for (j = 0; j < mA; j++)
00077         {
00078             printf("A[%d][%d] = ", i + 1, j + 1);
00079             scanf("%f", &A[i][j]);
00080         }
00081
00082     /* Capturar B */
00083     for (i = 0; i < nB; i++)
00084         for (j = 0; j < mB; j++)
00085         {
00086             printf("B[%d][%d] = ", i + 1, j + 1);
00087             scanf("%f", &B[i][j]);
00088         }
00089
00090     /* Multiplicación C = A*B */
00091     for (i = 0; i < nA; i++)
00092         for (j = 0; j < mB; j++)
00093             for (k = 0, C[i][j] = 0; k < mA; k++)
00094                 C[i][j] += A[i][k] * B[k][j];
00095
00096     /* Imprimir C */
00097     for (i = 0; i < nA; i++)
00098         for (j = 0; j < mB; j++)
00099             printf("C[%d][%d] = %f\n", i + 1, j + 1, C[i][j]);
00100
00101     return 0;
00102 }
```

7.197 Ejemplo029.c

[Go to the documentation of this file.](#)

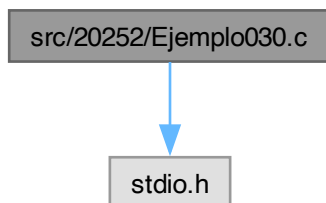
```

00001
00031
00032 #include <stdio.h>
00033 #include <stdlib.h>
00034
00035 #define N 10
00036 #define M 10
00037
00038 int main(int argc, char *argv[])
00039 {
00040     (void)argc;
00041     (void)argv;
00042
00043     int nA, mA, nB, mB, i, j, k;
00044     float A[N][M], B[N][M], C[N][M];
00045
00046     /* Dimensiones */
00047     do {
00048         printf("Ingrese el numero de rengles de A: ");
00049         scanf("%d", &nA);
00050     } while (nA < 1 || nA > N);
00051
00052     do {
00053         printf("Ingrese el numero de columnas de A: ");
00054         scanf("%d", &mA);
00055     } while (mA < 1 || mA > M);
00056
00057     do {
00058         printf("Ingrese el numero de rengles de B: ");
00059         scanf("%d", &nB);
00060     } while (nB < 1 || nB > N);
00061
00062     do {
00063         printf("Ingrese el numero de columnas de B: ");
00064         scanf("%d", &mB);
00065     } while (mB < 1 || mB > M);
00066
00067     /* Condición para multiplicación */
00068     if (mA != nB)
00069     {
00070         printf("Error las columnas de A no son iguales a los renglones de B.\n");
00071         return 1;
00072     }
00073
00074     /* Capturar A */
00075     for (i = 0; i < nA; i++)
00076         for (j = 0; j < mA; j++)
00077         {
00078             printf("A[%d][%d] = ", i + 1, j + 1);
00079             scanf("%f", &A[i][j]);
00080         }
00081
00082     /* Capturar B */
00083     for (i = 0; i < nB; i++)
00084         for (j = 0; j < mB; j++)
00085         {
00086             printf("B[%d][%d] = ", i + 1, j + 1);
00087             scanf("%f", &B[i][j]);
00088         }
00089
00090     /* Multiplicación C = A*B */
00091     for (i = 0; i < nA; i++)
00092         for (j = 0; j < mB; j++)
00093             for (k = 0, C[i][j] = 0; k < mA; k++)
00094                 C[i][j] += A[i][k] * B[k][j];
00095
00096     /* Imprimir C */
00097     for (i = 0; i < nA; i++)
00098         for (j = 0; j < mB; j++)
00099             printf("C[%d][%d] = %f\n", i + 1, j + 1, C[i][j]);
00100
00101     return 0;
00102 }
```

7.198 src/20252/Ejemplo030.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo030.c:



Functions

- int [buscar](#) (int n)
- int [main](#) (int argc, char *argv[])

7.198.1 Function Documentation

7.198.1.1 [buscar\(\)](#)

```
int buscar (  
    int n)
```

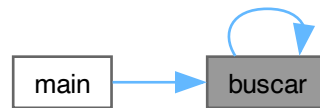
Definition at line 3 of file [Ejemplo030.c](#).

```
00004 {  
00005     return n%2?n:buscar(n/2);  
00006 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



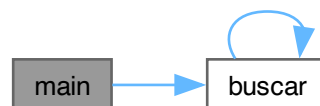
7.198.1.2 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 8 of file [Ejemplo030.c](#).

```
00009 {
00010     int n, a, b, i, j, x;
00011     scanf("%d", &n);
00012     for(i=0; i<n; i++)
00013     {
00014         scanf("%d %d", &a, &b);
00015         for(j=a, x=0; j<=b; j++)
00016         {
00017             printf("%d (%d)\t", j, buscar(j));
00018             x += buscar(j);
00019         }
00020         printf("%d\n", x);
00021     }
00022     return 0;
00023 }
```

Here is the call graph for this function:



7.199 Ejemplo030.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 int buscar(int n)
00004 {
00005     return n%2?n:buscar(n/2);
00006 }
00007
00008 int main(int argc, char *argv[])
00009 {
00010     int n, a, b, i, j, x;
00011     scanf("%d", &n);
00012     for(i=0; i<n; i++)
00013     {
00014         scanf("%d %d", &a, &b);
```

```

00015         for(j=a, x=0; j<=b; j++)
00016         {
00017             printf("%d (%d)\t", j, buscar(j));
00018             x += buscar(j);
00019         }
00020         printf("%d\n", x);
00021     }
00022     return 0;
00023 }

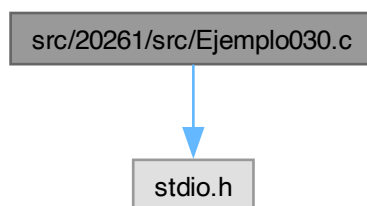
```

7.200 src/20261/src/Ejemplo030.c File Reference

Resolución de un sistema lineal $Ax=b$ por eliminación de Gauss (sin pivoteo) y sustitución hacia atrás.

#include <stdio.h>

Include dependency graph for Ejemplo030.c:



Macros

- #define `N` 10

Functions

- int `main` (int argc, char *argv[])

7.200.1 Detailed Description

Resolución de un sistema lineal $Ax=b$ por eliminación de Gauss (sin pivoteo) y sustitución hacia atrás.

El programa resuelve el sistema: $Ax = b$ para n variables ($n < N$) usando:

- 1) Eliminación hacia adelante (triangularización): Para cada pivote i , elimina $A[j][i]$ para $j > i$: $fct = A[j][i] / A[i][i]$ fila_j = fila_j - fct * fila_i $b[j] = b[j] - fct * b[i]$
- 2) Muestra la matriz aumentada resultante $[A|b]$ (ya triangular superior).
- 3) Sustitución hacia atrás: Recorre $i = n-1 \dots 0$ y calcula $x[i]$.

Entrada estándar

- n (int): número de variables, $1 \leq n < N$
- Matriz A (float): $n \times n$ valores
- Vector b (float): n valores

Salida estándar

- Matriz aumentada triangularizada (impresa con 4 decimales)
- Solución $x[i]$

Complejidad

- Eliminación: $O(n^3)$
- Sustitución: $O(n^2)$
- Memoria: $O(N^2)$

Warning

- No hay pivoteo parcial: si $A[i][i] == 0$ (o muy pequeño), puede haber división por cero o inestabilidad numérica. En implementación robusta se usa pivoteo.

```
gcc Ejemplo030.c -o Ejemplo030
./Ejemplo030
```

Definition in file [Ejemplo030.c](#).

7.200.2 Macro Definition Documentation

7.200.2.1 N

```
#define N 10
```

Definition at line 47 of file [Ejemplo030.c](#).

7.200.3 Function Documentation

7.200.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 49 of file [Ejemplo030.c](#).

```
00050 {
00051     (void)argc;
00052     (void)argv;
00053
00054     int n, i, j, k;
00055     float A[N][N], b[N], fct, x[N];
00056
00057     /* Leer n con validación (nota: aquí exige n < N) */
00058     do {
00059         printf("Ingrese el numero de variables: ");
00060         scanf("%d", &n);
00061     } while (1 > n || n >= N);
00062
00063     /* Capturar A y b; inicializar x */
00064     for (i = 0; i < n; i++)
00065     {
00066         for (j = 0; j < n; j++)
00067         {
00068             printf("A[%d][%d] = ", i + 1, j + 1);
00069             scanf("%f", &A[i][j]);
00070         }
00071         printf("b[%d] = ", i + 1);
00072         scanf("%f", &b[i]);
00073         x[i] = 0;
00074     }
00075
00076     /*
00077     Eliminación hacia adelante:
00078     Convierte A en una matriz triangular superior.
00079     */
00080     for (i = 0; i < n - 1; i++)
00081     for (j = i + 1; j < n; j++)
00082     {
00083         /* factor = A[j][i]/A[i][i] */
00084         for (k = 0, fct = A[j][i] / A[i][i]; k < n; k++)
00085             A[j][k] -= (fct * A[i][k]);
00086
00087         b[j] -= (fct * b[i]);
00088     }
00089
00090     /* Imprimir matriz aumentada [A|b] ya triangularizada */
00091     for (i = 0; i < n; i++)
00092     {
00093         for (j = 0; j < n; j++)
```



```

00094         printf("%.4f\t", A[i][j]);
00095         printf("%.4f\n", b[i]);
00096     }
00097
00098     /*
00099     Sustitución hacia atrás:
00100     Recorre desde la última ecuación hacia la primera.
00101     Nota: el código resta x[j]*A[i][j] para todo j != i.
00102     Como x[j] para j<i aún vale 0 en ese momento, no afecta el resultado.
00103     */
00104     for (i = n - 1; i > -1; i--)
00105     {
00106         x[i] = b[i];
00107
00108         for (j = 0; j < n; j++)
00109             if (i != j)
00110                 x[i] -= x[j] * A[i][j];
00111
00112         x[i] /= A[i][i];
00113     }
00114
00115     /* Imprimir solución */
00116     for (i = 0; i < n; i++)
00117         printf("x[%d] = %f\n", i + 1, x[i]);
00118
00119     return 0;
00120 }

```

Here is the call graph for this function:



7.201 Ejemplo030.c

[Go to the documentation of this file.](#)

```

00001
00044
00045 #include <stdio.h>
00046
00047 #define N 10
00048
00049 int main(int argc, char *argv[])
00050 {
00051     (void)argc;
00052     (void)argv;
00053
00054     int n, i, j, k;
00055     float A[N][N], b[N], fct, x[N];
00056
00057     /* Leer n con validación (nota: aquí exige n < N) */
00058     do {
00059         printf("Ingrese el numero de variables: ");
00060         scanf("%d", &n);
00061     } while (1 > n || n >= N);
00062
00063     /* Capturar A y b; inicializar x */
00064     for (i = 0; i < n; i++)
00065     {
00066         for (j = 0; j < n; j++)
00067         {
00068             printf("A[%d][%d] = ", i + 1, j + 1);
00069             scanf("%f", &A[i][j]);
00070         }
00071         printf("b[%d] = ", i + 1);
00072         scanf("%f", &b[i]);
00073         x[i] = 0;
00074     }
00075

```

```

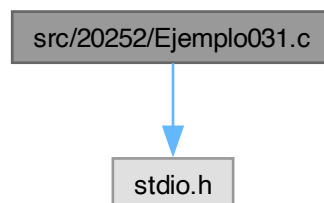
00076      /*
00077      Eliminación hacia adelante:
00078      Convierte A en una matriz triangular superior.
00079      */
00080      for (i = 0; i < n - 1; i++)
00081          for (j = i + 1; j < n; j++)
00082              {
00083                  /* factor = A[j][i]/A[i][i] */
00084                  for (k = 0, fct = A[j][i] / A[i][i]; k < n; k++)
00085                      A[j][k] -= (fct * A[i][k]);
00086
00087                  b[j] -= (fct * b[i]);
00088              }
00089
00090      /* Imprimir matriz aumentada [A|b] ya triangularizada */
00091      for (i = 0; i < n; i++)
00092          {
00093              for (j = 0; j < n; j++)
00094                  printf("%.4f\t", A[i][j]);
00095              printf("%.4f\n", b[i]);
00096          }
00097
00098      /*
00099      Sustitución hacia atrás:
00100      Recorre desde la última ecuación hacia la primera.
00101      Nota: el código resta x[j]*A[i][j] para todo j != i.
00102      Como x[j] para j<i aún vale 0 en ese momento, no afecta el resultado.
00103      */
00104      for (i = n - 1; i > -1; i--)
00105          {
00106              x[i] = b[i];
00107
00108              for (j = 0; j < n; j++)
00109                  if (i != j)
00110                      x[i] -= x[j] * A[i][j];
00111
00112              x[i] /= A[i][i];
00113          }
00114
00115      /* Imprimir solución */
00116      for (i = 0; i < n; i++)
00117          printf("x[%d] = %f\n", i + 1, x[i]);
00118
00119      return 0;
00120 }

```

7.202 src/20252/Ejemplo031.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo031.c:



Functions

- int [main](#) (int argc, char *argv[])

7.202.1 Function Documentation

7.202.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo031.c](#).

```
00004 {
00005     int i;
00006     printf("Numero de parametros de enytrada: %d\n", argc);
00007     for(i=0; i<argc; i++)
00008         printf("%d. %s\n", i+1, argv[i]);
00009     return 0;
00010 }
```

7.203 Ejemplo031.c

[Go to the documentation of this file.](#)

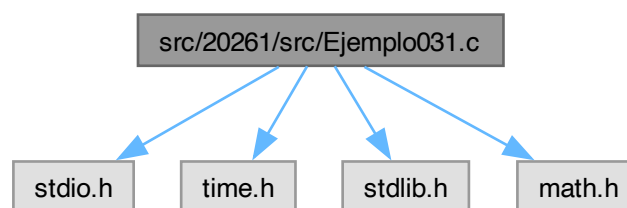
```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int i;
00006     printf("Numero de parametros de enytrada: %d\n", argc);
00007     for(i=0; i<argc; i++)
00008         printf("%d. %s\n", i+1, argv[i]);
00009     return 0;
00010 }
```

7.204 src/20261/src/Ejemplo031.c File Reference

Generación de datos aleatorios y transformación por centrado/normalización (min–max o z-score).

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include <math.h>
```

Include dependency graph for Ejemplo031.c:



Macros

- `#define N 10000`

Functions

- `int main (int argc, char *argv[])`

7.204.1 Detailed Description

Generación de datos aleatorios y transformación por centrado/normalización (min–max o z-score).

Este programa genera un arreglo `x [ ]` con `n` valores aleatorios en el rango `[mn_X, mx_X]` y calcula estadísticas básicas: mínimo, máximo, media y varianza (poblacional).

Posteriormente genera un arreglo `z [ ]` aplicando una transformación opcional:

- **Centrar:** restar un valor de referencia `a`
- **Normalizar:** dividir entre un factor de escala `b`

De modo que:

$$z_i = \frac{x_i - a}{b}$$

Métodos disponibles: 1) **Máximo–Mínimo**

- Centrar: `a = min`
- Normalizar: `b = (max - min)`
- Con ambas opciones: `z = (x - min)/(max-min)`

2) **z-score**

- Centrar: `a = media`
- Normalizar: `b = sqrt(varianza)`
- Con ambas opciones: `z = (x - media)/std`

Entrada estándar

- `n` (int): número de elementos, $1 \leq n \leq N$
- `mx_X` (float): valor máximo del rango de generación
- `mn_X` (float): valor mínimo del rango de generación
- `op1` (int): ¿Quieres centrar? (0/1)
- `op2` (int): ¿Quieres normalizar? (0/1)
- `op3` (int): método (1=Max–Min, 2=z-score) si `op1` u `op2` son 1

Salida estándar

- Máximo, mínimo, media y varianza
- Tabla con: índice, `x[i]` y `z[i]`

Complejidad

Tiempo: $O(n)$. Memoria: $O(N)$.

Warning

- Si `max==min` (rango 0) o `varianza==0`, entonces `b` puede ser 0 y la división no es válida. (Este ejemplo asume datos suficientemente variados; en uso robusto conviene validar `b`.)

```
gcc Ejemplo031.c -o Ejemplo031 -lm
./Ejemplo031
```

Definition in file [Ejemplo031.c](#).

7.204.2 Macro Definition Documentation

7.204.2.1 N

```
#define N 10000
```

Definition at line 60 of file [Ejemplo031.c](#).

7.204.3 Function Documentation

7.204.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 62 of file [Ejemplo031.c](#).

```
00063 {
00064     (void)argc;
00065     (void)argv;
00066
00067     float x[N], z[N];
00068     float mx_X, mn_X;          /* rango de generación */
00069     float min, max, media, varianza;
00070     float a, b;               /* parámetros de la transformación z=(x-a)/b */
00071
00072     int i, n;
00073     int bandera;              /* máscara de bits para elegir a y b */
00074     int op1, op2, op3 = 0;     /* op3 se inicializa para evitar uso sin definir */
00075
00076     srand((unsigned int)time(NULL));
00077
00078     /* Leer n */
00079     do {
00080         printf("Ingrese el numero de elementos: ");
00081         scanf("%d", &n);
00082     } while (n < 1 || n > N);
00083
00084     /* Leer rango [mn_X, mx_X] */
00085     printf("Ingrese el valor maximo: ");
00086     scanf("%f", &mx_X);
00087     printf("Ingrese el valor minimo: ");
00088     scanf("%f", &mn_X);
00089
00090     /*
00091      * Asegurar mn_X <= mx_X.
00092      * El intercambio usa multiplicación/división con casos especiales para ceros.
00093      */
00094     if (mx_X < mn_X)
00095     {
00096         if (mx_X && mn_X)
00097         {
00098             mx_X *= mn_X;
00099             mn_X = mx_X / mn_X;
00100             mx_X /= mn_X;
00101         }
00102         else if (mx_X)
00103         {
00104             mn_X = mx_X;
00105             mx_X = 0;
00106         }
00107         else if (mn_X)
00108         {
00109             mx_X = mn_X;
00110             mn_X = 0;
00111         }
00112         else
00113             mx_X = mn_X + 1; /* ambos eran 0: fuerza un rango no degenerado */
00114     }
00115
00116     /* Opciones del usuario */
00117     printf("¿Quieres centrar?: ");
00118     scanf("%d", &op1);
00119     printf("¿Quieres normalizar?: ");
00120     scanf("%d", &op2);
00121
00122     if (op1 || op2)
00123     {
00124         printf("Menu:\n");
00125         printf("1. Maximo-Minimo.\n");
00126         printf("2. z-score.\n");
00127         printf("Opción: ");
00128         scanf("%d", &op3);
00129     }
00130
00131     /*
00132      * 'bandera' codifica qué usar para (a,b):
00133      * bit 0 (1): a = min
00134      * bit 2 (4): a = media
00135      * bit 1 (2): b = max-min
00136      * bit 3 (8): b = sqrt(varianza)
00137
00138      * Se arma a partir de (op1, op2, op3) con una tabla compacta (switch).
```

```

00139  */
00140  switch (8*op1 + 4*op2 + op3)
00141  {
00142      case 5: bandera = 2; break; /* normalizar con max-min, sin centrar */
00143      case 6: bandera = 8; break; /* normalizar con std, sin centrar */
00144      case 9: bandera = 1; break; /* centrar con min */
00145      case 10: bandera = 4; break; /* centrar con media */
00146      case 13: bandera = 3; break; /* (x-min)/(max-min) */
00147      case 14: bandera = 12; break; /* (x-media)/std */
00148      default: bandera = 0;
00149  }
00150
00151  /* Generar datos aleatorios */
00152  for (i = 0; i < n; i++)
00153      x[i] = ((mx_X - mn_X) * rand()) / RAND_MAX + mn_X;
00154
00155  /* Calcular min y max */
00156  for (i = 1, max = x[0], min = x[0]; i < n; i++)
00157  {
00158      if (max < x[i]) max = x[i];
00159      if (min > x[i]) min = x[i];
00160  }
00161
00162  /* Calcular media y varianza poblacional: Var = E[x^2] - (E[x])^2 */
00163  for (i = 1, media = x[0], varianza = x[0]*x[0]; i < n; i++)
00164  {
00165      media += x[i];
00166      varianza += (x[i] * x[i]);
00167  }
00168  media /= n;
00169  varianza /= n;
00170  varianza -= (media * media);
00171
00172  /* Elegir a (offset) según bandera */
00173  if (bandera & 1) a = min;
00174  else if (bandera & 4) a = media;
00175  else a = 0;
00176
00177  /* Elegir b (escala) según bandera */
00178  if (bandera & 2) b = max - min;
00179  else if (bandera & 8) b = sqrtf(varianza);
00180  else b = 1;
00181
00182  /* Transformación */
00183  for (i = 0; i < n; i++)
00184      z[i] = (x[i] - a) / b;
00185
00186  /* Reporte */
00187  printf("Maximo: %f\n", max);
00188  printf("Minimo: %f\n", min);
00189  printf("Media: %f\n", media);
00190  printf("Varianza: %f\n", varianza);
00191
00192  for (i = 0; i < n; i++)
00193      printf("%d\t%f\t%f\n", i + 1, x[i], z[i]);
00194
00195  return 0;
00196 }

```

Here is the call graph for this function:



7.205 Ejemplo031.c

[Go to the documentation of this file.](#)

```

00001
00054

```

```

00055 #include <stdio.h>
00056 #include <time.h>
00057 #include <stdlib.h>
00058 #include <math.h>
00059
00060 #define N 10000
00061
00062 int main(int argc, char *argv[])
00063 {
00064     (void)argc;
00065     (void)argv;
00066
00067     float x[N], z[N];
00068     float mx_X, mn_X;          /* rango de generación */
00069     float min, max, media, varianza;
00070     float a, b;               /* parámetros de la transformación z=(x-a)/b */
00071
00072     int i, n;
00073     int bandera;              /* máscara de bits para elegir a y b */
00074     int op1, op2, op3 = 0;    /* op3 se inicializa para evitar uso sin definir */
00075
00076     srand((unsigned int)time(NULL));
00077
00078     /* Leer n */
00079     do {
00080         printf("Ingrese el numero de elementos: ");
00081         scanf("%d", &n);
00082     } while (n < 1 || n > N);
00083
00084     /* Leer rango [mn_X, mx_X] */
00085     printf("Ingrese el valor maximo: ");
00086     scanf("%f", &mx_X);
00087     printf("Ingrese el valor minimo: ");
00088     scanf("%f", &mn_X);
00089
00090     /*
00091      Asegurar mn_X <= mx_X.
00092      El intercambio usa multiplicación/división con casos especiales para ceros.
00093     */
00094     if (mx_X < mn_X)
00095     {
00096         if (mx_X && mn_X)
00097         {
00098             mx_X *= mn_X;
00099             mn_X = mx_X / mn_X;
00100             mx_X /= mn_X;
00101         }
00102         else if (mx_X)
00103         {
00104             mn_X = mx_X;
00105             mx_X = 0;
00106         }
00107         else if (mn_X)
00108         {
00109             mx_X = mn_X;
00110             mn_X = 0;
00111         }
00112         else
00113             mx_X = mn_X + 1; /* ambos eran 0: fuerza un rango no degenerado */
00114     }
00115
00116     /* Opciones del usuario */
00117     printf("¿Quieres centrar?: ");
00118     scanf("%d", &op1);
00119     printf("¿Quieres normalizar?: ");
00120     scanf("%d", &op2);
00121
00122     if (op1 || op2)
00123     {
00124         printf("Menu:\n");
00125         printf("1. Maximo-Minimo.\n");
00126         printf("2. z-score.\n");
00127         printf("Opción: ");
00128         scanf("%d", &op3);
00129     }
00130
00131     /*
00132      'bandera' codifica qué usar para (a,b):
00133      bit 0 (1): a = min
00134      bit 2 (4): a = media
00135      bit 1 (2): b = max-min
00136      bit 3 (8): b = sqrt(varianza)
00137
00138      Se arma a partir de (op1, op2, op3) con una tabla compacta (switch).
00139     */
00140     switch (8*op1 + 4*op2 + op3)
00141     {

```

```

00142         case 5: bandera = 2; break; /* normalizar con max-min, sin centrar */
00143         case 6: bandera = 8; break; /* normalizar con std, sin centrar */
00144         case 9: bandera = 1; break; /* centrar con min */
00145         case 10: bandera = 4; break; /* centrar con media */
00146         case 13: bandera = 3; break; /* (x-min)/(max-min) */
00147         case 14: bandera = 12; break; /* (x-media)/std */
00148         default: bandera = 0;
00149     }
00150
00151     /* Generar datos aleatorios */
00152     for (i = 0; i < n; i++)
00153         x[i] = ((mx_X - mn_X) * rand()) / RAND_MAX + mn_X;
00154
00155     /* Calcular min y max */
00156     for (i = 1, max = x[0], min = x[0]; i < n; i++)
00157     {
00158         if (max < x[i]) max = x[i];
00159         if (min > x[i]) min = x[i];
00160     }
00161
00162     /* Calcular media y varianza poblacional: Var = E[x^2] - (E[x])^2 */
00163     for (i = 1, media = x[0], varianza = x[0]*x[0]; i < n; i++)
00164     {
00165         media += x[i];
00166         varianza += (x[i] * x[i]);
00167     }
00168     media /= n;
00169     varianza /= n;
00170     varianza -= (media * media);
00171
00172     /* Elegir a (offset) según bandera */
00173     if (bandera & 1) a = min;
00174     else if (bandera & 4) a = media;
00175     else a = 0;
00176
00177     /* Elegir b (escala) según bandera */
00178     if (bandera & 2) b = max - min;
00179     else if (bandera & 8) b = sqrtf(varianza);
00180     else b = 1;
00181
00182     /* Transformación */
00183     for (i = 0; i < n; i++)
00184         z[i] = (x[i] - a) / b;
00185
00186     /* Reporte */
00187     printf("Maximo: %f\n", max);
00188     printf("Minimo: %f\n", min);
00189     printf("Media: %f\n", media);
00190     printf("Varianza: %f\n", varianza);
00191
00192     for (i = 0; i < n; i++)
00193         printf("%d\t%f\t%f\n", i + 1, x[i], z[i]);
00194
00195     return 0;
00196 }

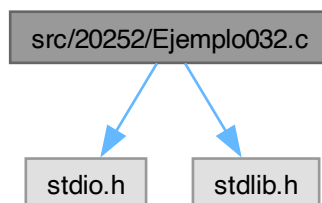
```

7.206 src/20252/Ejemplo032.c File Reference

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo032.c:



Functions

- `int main (int argc, char *argv[])`

7.206.1 Function Documentation

7.206.1.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 4 of file [Ejemplo032.c](#).

```
00005 {  
00006     int i;  
00007     float pi;  
00008     if(argc>1)  
00009     {  
00010         for(i=1, pi=0; i<argc; i++)  
00011             pi+=atof(argv[i]);  
00012         printf("La suma es %f\n", pi);  
00013         return 0;  
00014     }  
00015     else  
00016         return 1;  
00017 }
```

7.207 Ejemplo032.c

[Go to the documentation of this file.](#)

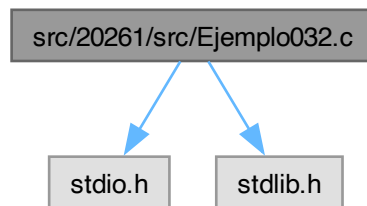
```
00001 #include <stdio.h>  
00002 #include <stdlib.h>  
00003  
00004 int main(int argc, char *argv[])  
00005 {  
00006     int i;  
00007     float pi;  
00008     if(argc>1)  
00009     {  
00010         for(i=1, pi=0; i<argc; i++)  
00011             pi+=atof(argv[i]);  
00012         printf("La suma es %f\n", pi);  
00013         return 0;  
00014     }  
00015     else  
00016         return 1;  
00017 }
```

7.208 src/20261/src/Ejemplo032.c File Reference

Demostración de macros para manipulación de bits en un entero.

```
#include <stdio.h>  
#include <stdlib.h>
```

Include dependency graph for Ejemplo032.c:



Macros

- `#define BIT(n)`
- `#define BIT_GET(x, n)`
- `#define BIT_SET(x, n)`
- `#define BIT_CLEAR(x, n)`
- `#define BIT_TOGGLE(x, n)`
- `#define BIT_WRITE(x, n, v)`

Functions

- `int main (int argc, char *argv[])`

7.208.1 Detailed Description

Demostración de macros para manipulación de bits en un entero.

El programa: 1) Lee un entero `x`. 2) Imprime su representación binaria (desde el bit más significativo). 3) Permite aplicar operaciones sobre un bit `nb`:

- SET: poner el bit en 1
- CLEAR: poner el bit en 0
- TOGGLE: invertir el bit
- WRITE: escribir 0 o 1 según el valor `v`

Entrada estándar

- `x (int)`
- `nb (int)`: índice del bit (0..(bits-1))
- `v (int)`: valor 0/1 (solo para WRITE)

Salida estándar

- Representación binaria después de cada operación y el valor decimal de `x`.

Complejidad

Tiempo: $O(w)$ por impresión, donde w = número de bits del entero. Memoria: $O(1)$.

Warning

- Se valida que `nb` esté dentro del rango permitido para evitar desplazamientos inválidos.

```
gcc Ejemplo032.c -o Ejemplo032
./Ejemplo032
```

Definition in file [Ejemplo032.c](#).

7.208.2 Macro Definition Documentation

7.208.2.1 BIT

```
#define BIT(  
        n)
```

Value:

```
(1U << (n))
```

Definition at line 39 of file [Ejemplo032.c](#).

7.208.2.2 BIT_CLEAR

```
#define BIT_CLEAR(  
        x,  
        n)
```

Value:

```
((x) &= ~(int)BIT(n))
```

Definition at line 42 of file [Ejemplo032.c](#).

7.208.2.3 BIT_GET

```
#define BIT_GET(  
        x,  
        n)
```

Value:

```
((unsigned)(x) & BIT(n)) != 0U)
```

Definition at line 40 of file [Ejemplo032.c](#).

7.208.2.4 BIT_SET

```
#define BIT_SET(  
        x,  
        n)
```

Value:

```
((x) |= (int)BIT(n))
```

Definition at line 41 of file [Ejemplo032.c](#).

7.208.2.5 BIT_TOGGLE

```
#define BIT_TOGGLE(  
        x,  
        n)
```

Value:

```
((x) ^= (int)BIT(n))
```

Definition at line 43 of file [Ejemplo032.c](#).

7.208.2.6 BIT_WRITE

```
#define BIT_WRITE(  
        x,  
        n,  
        v)
```

Value:

```
((v) ? BIT_SET(x,n) : BIT_CLEAR(x,n))
```

Definition at line 44 of file [Ejemplo032.c](#).

7.208.3 Function Documentation

7.208.3.1 main()

```
int main (
```

```

    int argc,
    char * argv[])

```

Definition at line 46 of file [Ejemplo032.c](#).

```

00047 {
00048     (void)argc;
00049     (void)argv;
00050
00051     int x, i, nbits, nb, v;
00052
00053     nbits = 8 * (int)sizeof(x);
00054
00055     printf("X = ");
00056     scanf("%d", &x);
00057
00058     /* Imprimir bits de x */
00059     for (i = nbits - 1; i > -1; i--)
00060         printf("%d ", BIT_GET(x, i) ? 1 : 0);
00061     printf("\n");
00062
00063     /* SET */
00064     printf("Bit set.\n");
00065     do {
00066         printf("Numero de bit (0..%d): ", nbits - 1);
00067         scanf("%d", &nb);
00068     } while (nb < 0 || nb >= nbits);
00069
00070     BIT_SET(x, nb);
00071
00072     for (i = nbits - 1; i > -1; i--)
00073         printf("%d ", BIT_GET(x, i) ? 1 : 0);
00074     printf("\n");
00075     printf("x = %d\n", x);
00076
00077     /* CLEAR */
00078     printf("Bit clear.\n");
00079     do {
00080         printf("Numero de bit (0..%d): ", nbits - 1);
00081         scanf("%d", &nb);
00082     } while (nb < 0 || nb >= nbits);
00083
00084     BIT_CLEAR(x, nb);
00085
00086     for (i = nbits - 1; i > -1; i--)
00087         printf("%d ", BIT_GET(x, i) ? 1 : 0);
00088     printf("\n");
00089     printf("x = %d\n", x);
00090
00091     /* TOGGLE */
00092     printf("Bit toggle.\n");
00093     do {
00094         printf("Numero de bit (0..%d): ", nbits - 1);
00095         scanf("%d", &nb);
00096     } while (nb < 0 || nb >= nbits);
00097
00098     BIT_TOGGLE(x, nb);
00099
00100     for (i = nbits - 1; i > -1; i--)
00101         printf("%d ", BIT_GET(x, i) ? 1 : 0);
00102     printf("\n");
00103     printf("x = %d\n", x);
00104
00105     /* WRITE */
00106     printf("Bit write.\n");
00107     do {
00108         printf("Numero de bit (0..%d): ", nbits - 1);
00109         scanf("%d", &nb);
00110     } while (nb < 0 || nb >= nbits);
00111
00112     do {
00113         printf("Valor del bit (0/1): ");
00114         scanf("%d", &v);
00115     } while (v != 0 && v != 1);
00116
00117     BIT_WRITE(x, nb, v);
00118
00119     for (i = nbits - 1; i > -1; i--)
00120         printf("%d ", BIT_GET(x, i) ? 1 : 0);
00121     printf("\n");
00122     printf("x = %d\n", x);
00123
00124     return 0;
00125 }

```

7.209 Ejemplo032.c

[Go to the documentation of this file.](#)

```

00001
00034
00035 #include <stdio.h>
00036 #include <stdlib.h>
00037
00038 /* Usar lU evita problemas de signo al desplazar */
00039 #define BIT(n) (1U < (n))
00040 #define BIT_GET(x,n) (((unsigned)(x) & BIT(n)) != 0U)
00041 #define BIT_SET(x,n) ((x) |= (int)BIT(n))
00042 #define BIT_CLEAR(x,n) ((x) &= ~(int)BIT(n))
00043 #define BIT_TOGGLE(x,n) ((x) ^= (int)BIT(n))
00044 #define BIT_WRITE(x,n,v) ((v) ? BIT_SET(x,n) : BIT_CLEAR(x,n))
00045
00046 int main(int argc, char *argv[])
00047 {
00048     (void)argc;
00049     (void)argv;
00050
00051     int x, i, nbits, nb, v;
00052
00053     nbits = 8 * (int)sizeof(x);
00054
00055     printf("X = ");
00056     scanf("%d", &x);
00057
00058     /* Imprimir bits de x */
00059     for (i = nbits - 1; i > -1; i--)
00060         printf("%d ", BIT_GET(x, i) ? 1 : 0);
00061     printf("\n");
00062
00063     /* SET */
00064     printf("Bit set.\n");
00065     do {
00066         printf("Numero de bit (0..%d): ", nbits - 1);
00067         scanf("%d", &nb);
00068     } while (nb < 0 || nb >= nbits);
00069
00070     BIT_SET(x, nb);
00071
00072     for (i = nbits - 1; i > -1; i--)
00073         printf("%d ", BIT_GET(x, i) ? 1 : 0);
00074     printf("\n");
00075     printf("x = %d\n", x);
00076
00077     /* CLEAR */
00078     printf("Bit clear.\n");
00079     do {
00080         printf("Numero de bit (0..%d): ", nbits - 1);
00081         scanf("%d", &nb);
00082     } while (nb < 0 || nb >= nbits);
00083
00084     BIT_CLEAR(x, nb);
00085
00086     for (i = nbits - 1; i > -1; i--)
00087         printf("%d ", BIT_GET(x, i) ? 1 : 0);
00088     printf("\n");
00089     printf("x = %d\n", x);
00090
00091     /* TOGGLE */
00092     printf("Bit toggle.\n");
00093     do {
00094         printf("Numero de bit (0..%d): ", nbits - 1);
00095         scanf("%d", &nb);
00096     } while (nb < 0 || nb >= nbits);
00097
00098     BIT_TOGGLE(x, nb);
00099
00100     for (i = nbits - 1; i > -1; i--)
00101         printf("%d ", BIT_GET(x, i) ? 1 : 0);
00102     printf("\n");
00103     printf("x = %d\n", x);
00104
00105     /* WRITE */
00106     printf("Bit write.\n");
00107     do {
00108         printf("Numero de bit (0..%d): ", nbits - 1);
00109         scanf("%d", &nb);
00110     } while (nb < 0 || nb >= nbits);
00111
00112     do {
00113         printf("Valor del bit (0/1): ");
00114         scanf("%d", &v);
00115     } while (v != 0 && v != 1);

```

```

00116
00117     BIT_WRITE(x, nb, v);
00118
00119     for (i = nbits - 1; i > -1; i--)
00120         printf("%d ", BIT_GET(x, i) ? 1 : 0);
00121     printf("\n");
00122     printf("x = %d\n", x);
00123
00124     return 0;
00125 }

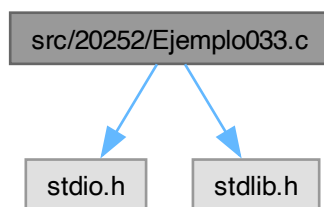
```

7.210 src/20252/Ejemplo033.c File Reference

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo033.c:



Macros

- #define [N](#) 1000

Functions

- float [validar2](#) (float a, float [min](#), float [max](#), char *str)
- float [validar](#) (float [min](#), float [max](#), char *str)
- float [asen](#) (float x, int n)
- int [main](#) (int argc, char *argv[])

7.210.1 Macro Definition Documentation

7.210.1.1 N

```
#define N 1000
```

Definition at line 4 of file [Ejemplo033.c](#).

7.210.2 Function Documentation

7.210.2.1 asen()

```

float asen (
    float x,
    int n)

```

Definition at line 29 of file [Ejemplo033.c](#).

```

00030 {
00031     int i;
00032     float asx, fct;
00033     for(i=0, asx=0, fct=x; i<n; i++)

```

```
00034     {
00035         asx+=fct;
00036         fct*= ((2*i+1)*x*x/(2*i+3));
00037         fct*= ((2*i+1.0)/(2*(i+1)));
00038     }
00039     return asx;
00040 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



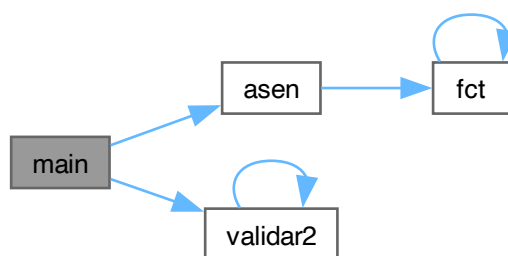
7.210.2.2 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 42 of file [Ejemplo033.c](#).

```
00043 {
00044     int n=0;
00045     float x=-1, asx;
00046     if(argc==3)
00047         x = atof(argv[2]);
00048     if(argc&2)
00049         n = atoi(argv[1]);
00050     n = (int)validar2(n, 1, N, "n");
00051     x = validar2(x, -1, 1, "x");
00052     asx = asen(x, n);
00053     printf("asen(%f) = %f\n", x, asx);
00054     return 0;
00055 }
```

Here is the call graph for this function:



7.210.2.3 validar()

```
float validar (
    float min,
    float max,
    char * str)
```

Definition at line 18 of file [Ejemplo033.c](#).

```
00019 {
00020     float a;
00021     printf("Ingrese el valor de %s: ", str);
00022     scanf("%f", &a);
00023     if(a>min&&a<max)
00024         return a;
00025     else
00026         return validar(min, max, str);
00027 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.210.2.4 validar2()

```
float validar2 (
    float a,
    float min,
    float max,
    char * str)
```

Definition at line 6 of file [Ejemplo033.c](#).

```
00007 {
00008     if(a>min&&a<max)
00009         return a;
00010     printf("Ingrese el valor de %s: ", str);
00011     scanf("%f", &a);
00012     if(a>min&&a<max)
00013         return a;
00014     else
00015         return validar2(a, min, max, str);
00016 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.211 Ejemplo033.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 #define N 1000
00005
00006 float validar2(float a, float min, float max, char *str)
00007 {
00008     if(a>min&&a<max)
00009         return a;
00010     printf("Ingrese el valor de %s: ", str);
00011     scanf("%f", &a);
00012     if(a>min&&a<max)
00013         return a;
00014     else
00015         return validar2(a, min, max, str);
00016 }
00017
00018 float validar(float min, float max, char *str)
```

```

00019 {
00020     float a;
00021     printf("Ingrese el valor de %s: ", str);
00022     scanf("%f", &a);
00023     if(a>min&&a<max)
00024         return a;
00025     else
00026         return validar(min, max, str);
00027 }
00028
00029 float asen(float x, int n)
00030 {
00031     int i;
00032     float asx, fct;
00033     for(i=0, asx=0, fct=x; i<n; i++)
00034     {
00035         asx+=fct;
00036         fct*=((2*i+1)*x*x/(2*i+3));
00037         fct*=((2*i+1.0)/(2*(i+1)));
00038     }
00039     return asx;
00040 }
00041
00042 int main(int argc, char *argv[])
00043 {
00044     int n=0;
00045     float x=-1, asx;
00046     if(argc==3)
00047         x = atof(argv[2]);
00048     if(argc&2)
00049         n = atoi(argv[1]);
00050     n = (int)validar2(n, 1, N, "n");
00051     x = validar2(x, -1, 1, "x");
00052     asx = asen(x, n);
00053     printf("asen(%f) = %f\n", x, asx);
00054     return 0;
00055 }

```

7.212 src/20261/src/Ejemplo033.c File Reference

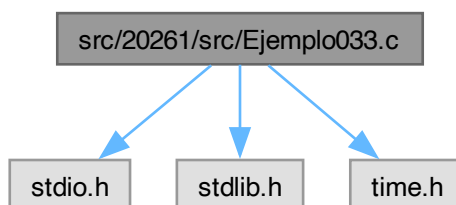
Cálculo de paridad XOR de un mensaje y simulación de error (flip) en un bit aleatorio.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo033.c:



Macros

- #define N 100
- #define BIT(n)
- #define BIT_GET(x, n)
- #define BIT_TOGGLE_BYTE(x, n)

Functions

- int [main](#) (int argc, char *argv[])

7.212.1 Detailed Description

Cálculo de paridad XOR de un mensaje y simulación de error (flip) en un bit aleatorio. El programa lee una línea de texto (hasta N-1 caracteres), calcula su **paridad** como:

$$P = m_0 \oplus m_1 \oplus \dots \oplus m_{n-1}$$

(XOR byte a byte).

Luego simula un error de transmisión:

- selecciona un carácter aleatorio `nc` y un bit aleatorio `nb` (0..7)
- invierte ese bit del mensaje

Después valida recalculando:

$$V = P \oplus m'_0 \oplus m'_1 \oplus \dots \oplus m'_{n-1}$$

Si no hubo error, típicamente $V==0$. Si hubo un flip, V será distinto de 0.

Entrada estándar

- Una línea de texto (puede incluir espacios), terminada en ENTER.

Salida estándar

- Mensaje original, paridad P (en bits)
- Posición y bit alterados
- Mensaje alterado, validación V (en bits)

Complejidad

Tiempo: $O(n)$. Memoria: $O(N)$.

Warning

- Se maneja el caso de mensaje vacío para evitar `rand() % 0`.

```
gcc Ejemplo033.c -o Ejemplo033
./Ejemplo033
```

Definition in file [Ejemplo033.c](#).

7.212.2 Macro Definition Documentation

7.212.2.1 BIT

```
#define BIT(
           n)
```

Value:

```
(1U << (n))
```

Definition at line 48 of file [Ejemplo033.c](#).

7.212.2.2 BIT_GET

```
#define BIT_GET(
           x,
           n)
```

Value:

```
((unsigned)(x) & BIT(n)) != 0U)
```

Definition at line 49 of file [Ejemplo033.c](#).

7.212.2.3 BIT_TOGGLE_BYTE

```
#define BIT_TOGGLE_BYTE(
    x,
    n)
```

Value:

```
((x) ^= (unsigned char)BIT(n))
```

Definition at line 50 of file [Ejemplo033.c](#).

7.212.2.4 N

```
#define N 100
```

Definition at line 46 of file [Ejemplo033.c](#).

7.212.3 Function Documentation

7.212.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 52 of file [Ejemplo033.c](#).

```
00053 {
00054     (void)argc;
00055     (void)argv;
00056
00057     char msg[N];
00058     unsigned char P, V; /* usar unsigned evita problemas de signo */
00059     int i, n, nc, nb;
00060
00061     srand((unsigned)time(NULL));
00062
00063     /* Leer línea hasta ENTER o hasta N-1 caracteres */
00064     i = 0;
00065     do {
00066         msg[i] = (char)getchar();
00067         i++;
00068     } while (i < N && msg[i - 1] != '\n');
00069
00070     /* Reemplazar '\n' por terminador */
00071     msg[i - 1] = '\0';
00072     n = i - 1;
00073
00074     /* Caso especial: mensaje vacío */
00075     if (n <= 0)
00076     {
00077         printf("MSG vacío.\n");
00078         return 0;
00079     }
00080
00081     /* Paridad XOR de todos los bytes */
00082     for (i = 0, P = 0; i < n; i++)
00083         P ^= (unsigned char)msg[i];
00084
00085     printf("MSG: %s\n", msg);
00086
00087     printf("Paridad: %u (", (unsigned)P);
00088     for (i = 7; i > -1; i--)
00089         printf("%d ", BIT_GET(P, i) ? 1 : 0);
00090     printf(")\n");
00091
00092     /* Elegir carácter y bit a voltear */
00093     nc = rand() % n;
00094     nb = rand() % 8;
00095     printf("NC = %d\tNB = %d\n", nc, nb);
00096
00097     /* Simular error: invertir ese bit del carácter seleccionado */
00098     BIT_TOGGLE_BYTE(msg[nc], nb);
00099
00100     printf("MSG: %s\n", msg);
00101
00102     /* Validación: V = P XOR XOR(mensaje_modificado) */
00103     for (i = 0, V = P; i < n; i++)
00104         V ^= (unsigned char)msg[i];
00105
00106     printf("Validacion: %u (", (unsigned)V);
00107     for (i = 7; i > -1; i--)
00108         printf("%d ", BIT_GET(V, i) ? 1 : 0);
```

```

00109     printf("\n");
00110
00111     return 0;
00112 }

```

7.213 Ejemplo033.c

[Go to the documentation of this file.](#)

```

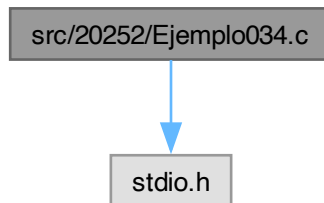
00001
00041
00042 #include <stdio.h>
00043 #include <stdlib.h>
00044 #include <time.h>
00045
00046 #define N 100
00047
00048 #define BIT(n)                (1U << (n))
00049 #define BIT_GET(x,n)          (((unsigned)(x) & BIT(n)) != 0U)
00050 #define BIT_TOGGLE_BYTE(x,n) ((x) ^= (unsigned char)BIT(n))
00051
00052 int main(int argc, char *argv[])
00053 {
00054     (void)argc;
00055     (void)argv;
00056
00057     char msg[N];
00058     unsigned char P, V; /* usar unsigned evita problemas de signo */
00059     int i, n, nc, nb;
00060
00061     srand((unsigned)time(NULL));
00062
00063     /* Leer línea hasta ENTER o hasta N-1 caracteres */
00064     i = 0;
00065     do {
00066         msg[i] = (char)getchar();
00067         i++;
00068     } while (i < N && msg[i - 1] != '\n');
00069
00070     /* Reemplazar '\n' por terminador */
00071     msg[i - 1] = '\0';
00072     n = i - 1;
00073
00074     /* Caso especial: mensaje vacío */
00075     if (n <= 0)
00076     {
00077         printf("MSG vacío.\n");
00078         return 0;
00079     }
00080
00081     /* Paridad XOR de todos los bytes */
00082     for (i = 0, P = 0; i < n; i++)
00083         P ^= (unsigned char)msg[i];
00084
00085     printf("MSG: %s\n", msg);
00086
00087     printf("Paridad: %u (", (unsigned)P);
00088     for (i = 7; i > -1; i--)
00089         printf("%d ", BIT_GET(P, i) ? 1 : 0);
00090     printf("\n");
00091
00092     /* Elegir carácter y bit a voltear */
00093     nc = rand() % n;
00094     nb = rand() % 8;
00095     printf("NC = %d\tNB = %d\n", nc, nb);
00096
00097     /* Simular error: invertir ese bit del carácter seleccionado */
00098     BIT_TOGGLE_BYTE(msg[nc], nb);
00099
00100     printf("MSG: %s\n", msg);
00101
00102     /* Validación: V = P XOR XOR(mensaje_modificado) */
00103     for (i = 0, V = P; i < n; i++)
00104         V ^= (unsigned char)msg[i];
00105
00106     printf("Validación: %u (", (unsigned)V);
00107     for (i = 7; i > -1; i--)
00108         printf("%d ", BIT_GET(V, i) ? 1 : 0);
00109     printf("\n");
00110
00111     return 0;
00112 }

```

7.214 src/20252/Ejemplo034.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo034.c:



Data Structures

- struct [contacto](#)

Macros

- `#define` [NM](#) 50
- `#define` [NE](#) 20
- `#define` [NC](#) 10

Functions

- int [main](#) (int argc, char *argv[])

7.214.1 Macro Definition Documentation

7.214.1.1 NC

```
#define NC 10
```

Definition at line 5 of file [Ejemplo034.c](#).

7.214.1.2 NE

```
#define NE 20
```

Definition at line 4 of file [Ejemplo034.c](#).

7.214.1.3 NM

```
#define NM 50
```

Definition at line 3 of file [Ejemplo034.c](#).

7.214.2 Function Documentation

7.214.2.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 14 of file [Ejemplo034.c](#).

```

00015 {
00016     contacto agenda[NC];
00017     int op, i, n;
00018     char c;
00019     n = 0;
00020     do{
00021         printf("Nombre: ");
00022         i = 0;
00023         do{
00024             c = getchar();
00025             agenda[n].nombre[i++] = c;
00026         }while(c!=10);
00027         agenda[n].nombre[i-1] = '\0';
00028         // scanf("%s", agenda[n].nombre);
00029         // &(agenda[i].nombre[]) -> agenda[i].nombre
00030         printf("Telefono: ");
00031         scanf("%ld", &(agenda[n].telefono));
00032         printf("Correo electronico: ");
00033         scanf("%s", agenda[n].email);
00034         n++;
00035         printf("Ingresar otro contacto: ");
00036         scanf("%d", &op);
00037         do{
00038             c = getchar();
00039         }while(c!=10);
00040     }while(op&&(n<NC));
00041     printf("Agenda.\n");
00042     for(i=0; i<n; i++)
00043         printf("%s, %ld, %s\n", agenda[i].nombre, agenda[i].telefono, agenda[i].email);
00044     return 0;
00045 }

```

7.215 Ejemplo034.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define NM 50
00004 #define NE 20
00005 #define NC 10
00006
00007 typedef struct
00008 {
00009     char nombre[NM];
00010     long int telefono;
00011     char email[NE];
00012 }contacto;
00013
00014 int main(int argc, char *argv[])
00015 {
00016     contacto agenda[NC];
00017     int op, i, n;
00018     char c;
00019     n = 0;
00020     do{
00021         printf("Nombre: ");
00022         i = 0;
00023         do{
00024             c = getchar();
00025             agenda[n].nombre[i++] = c;
00026         }while(c!=10);
00027         agenda[n].nombre[i-1] = '\0';
00028         // scanf("%s", agenda[n].nombre);
00029         // &(agenda[i].nombre[]) -> agenda[i].nombre
00030         printf("Telefono: ");
00031         scanf("%ld", &(agenda[n].telefono));
00032         printf("Correo electronico: ");
00033         scanf("%s", agenda[n].email);
00034         n++;
00035         printf("Ingresar otro contacto: ");
00036         scanf("%d", &op);
00037         do{
00038             c = getchar();
00039         }while(c!=10);
00040     }while(op&&(n<NC));
00041     printf("Agenda.\n");
00042     for(i=0; i<n; i++)
00043         printf("%s, %ld, %s\n", agenda[i].nombre, agenda[i].telefono, agenda[i].email);
00044     return 0;
00045 }

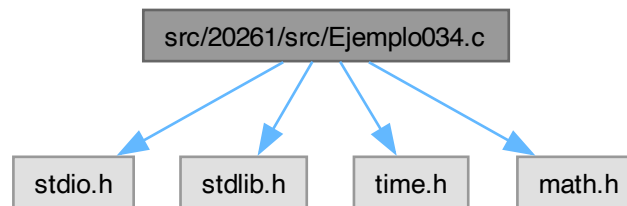
```

7.216 src/20261/src/Ejemplo034.c File Reference

Inserción de paridades (Hamming) en posiciones potencia de 2 y detección de error de 1 bit.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
```

Include dependency graph for Ejemplo034.c:



Macros

- `#define N 100`
- `#define P 7`
- `#define BIT(n)`
- `#define BIT_GET(x, n)`
- `#define BIT_SET(x, n)`
- `#define BIT_TOGGLE(x, n)`
- `#define IS_POW2_POS(pos)`

Functions

- `int main (int argc, char *argv[ ])`

7.216.1 Detailed Description

Inserción de paridades (Hamming) en posiciones potencia de 2 y detección de error de 1 bit.

El programa: 1) Lee un mensaje `msg` (hasta N-1 caracteres, termina al presionar ENTER). 2) Calcula `np` (número de paridades) con la condición mínima de Hamming: $2^{np} \geq n + np + 1$ 3) Construye `msgP` (longitud `n+np`) insertando bytes de paridad en posiciones 1,2,4,8,... (1-based). 4) Calcula las paridades por XOR usando el patrón de "bloques" del código original. 5) Simula un error: invierte un bit aleatorio en un byte aleatorio. 6) Recalcula paridades y forma:

- `NC_V`: posición del byte con error (0-based)
- `NB_V`: índice del bit alterado (0..7) obtenido desde la máscara XOR (potencia de 2)

Entrada estándar

Una línea de texto terminada con ENTER.

Salida estándar

Mantiene la forma de impresión del programa original (`MSG_P/MSG_E` con `s`, `P_i`, `NC_V`, `NB_V`).

Warning

- MSG_P: s puede truncarse si dentro de msgP aparece el byte 0 ('\0'). Se conserva así para no cambiar la forma de impresión original.

Definition in file [Ejemplo034.c](#).

7.216.2 Macro Definition Documentation**7.216.2.1 BIT**

```
#define BIT(  
           n)
```

Value:

```
(1U<<(n))
```

Definition at line 36 of file [Ejemplo034.c](#).

7.216.2.2 BIT_GET

```
#define BIT_GET(  
              x,  
              n)
```

Value:

```
((x) & BIT(n))
```

Definition at line 37 of file [Ejemplo034.c](#).

7.216.2.3 BIT_SET

```
#define BIT_SET(  
              x,  
              n)
```

Value:

```
((x) |= BIT(n))
```

Definition at line 38 of file [Ejemplo034.c](#).

7.216.2.4 BIT_TOGGLE

```
#define BIT_TOGGLE(  
                 x,  
                 n)
```

Value:

```
((x) ^= BIT(n))
```

Definition at line 39 of file [Ejemplo034.c](#).

7.216.2.5 IS_POW2_POS

```
#define IS_POW2_POS(  
                  pos)
```

Value:

```
((pos) && (((pos) & ((pos)-1U)) == 0U))
```

Definition at line 42 of file [Ejemplo034.c](#).

7.216.2.6 N

```
#define N 100
```

Definition at line 33 of file [Ejemplo034.c](#).

7.216.2.7 P

```
#define P 7
```

Definition at line 34 of file [Ejemplo034.c](#).

7.216.3 Function Documentation

7.216.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 44 of file [Ejemplo034.c](#).

```
00045 {
00046     (void)argc;
00047     (void)argv;
00048
00049     /*
00050      msg  : mensaje original (solo datos)
00051      msgP : mensaje con espacios de paridad insertados (datos + paridades)
00052      V    : vector de verificación de paridad por cada bit de paridad i
00053      msgV : reservado (no se usa en esta versión; se conserva por estructura original)
00054     */
00055     char msg[N], msgP[N+P];
00056
00057     /*
00058      ERROR CORREGIDO:
00059      V debe indexarse por i (0..np-1). En el original V era char y se usaba con
00060      índices derivados de j=(2^i-1), lo que podía desbordar o dejar valores sin inicializar.
00061      Se usa unsigned char para evitar problemas de signo en XOR.
00062     */
00063     unsigned char V[P];
00064
00065     int i, j, k, n, np, nc, nb, ncv, nbv;
00066     float aux;
00067
00068     srand((unsigned)time(NULL));
00069
00070     /* ----- Leer mensaje hasta ENTER ----- */
00071     i = 0;
00072     do{
00073         msg[i] = (char)getchar();
00074         i++;
00075     }while(i < N && msg[i-1] != 10); /* 10 == '\n' */
00076
00077     msg[i-1] = '\0';
00078     n = i-1;
00079
00080     /*
00081      ERROR CORREGIDO:
00082      np no se calcula con aproximación logarítmica.
00083      Condición de Hamming (mínimo np):
00084      2^np >= n + np + 1
00085      Esto garantiza suficientes posiciones de paridad para identificar la posición del error.
00086     */
00087     np = 0;
00088     while( ((1U<np) < (unsigned)(n + np + 1)) && np < P )
00089         np++;
00090
00091     printf("n = %d\tnp = %d\n", n, np);
00092     printf("MSG: %s\n", msg);
00093
00094     /*
00095      ----- Construir msgP con paridades -----
00096      Longitud total L = n + np.
00097      Posiciones (1-based) potencia de 2 se reservan para paridad (byte 0).
00098      El resto se llena con los bytes del mensaje original.
00099     */
00100     {
00101         int L = n + np;
00102         int data_idx = 0;
00103
00104         for(i = 0; i < L; i++)
00105         {
00106             int pos = i + 1; /* 1-based */
00107
00108             if(IS_POW2_POS((unsigned)pos))
00109             {
00110                 msgP[i] = 0; /* placeholder de paridad */
00111             }
00112             else
00113             {
00114                 msgP[i] = msg[data_idx++]; /* copiar dato */
00115             }
00116         }
00117         /* Terminador para impresión con %s (se mantiene como en el original) */
00118         msgP[L] = '\0';
00119     }
00120 }
```

```

00121  /*
00122  ----- Cálculo de paridades (codificación) -----
00123  Para cada i:
00124      j = 2^i - 1 (índice 0-based donde vive la paridad i)
00125  Se aplica el patrón de bloques del código original:
00126      incluir (j+1) bytes, saltar (j+1) bytes, incluir, ...
00127  y se acumula XOR en msgP[j].
00128  */
00129  for(i=0; i<np; i++)
00130  {
00131      j = (1<<i)-1;
00132      for(k=j; k<(n+np); k++)
00133          if(((k-j)/(j+1)+1)%2)
00134              msgP[j] ^= msgP[k];
00135  }
00136
00137  /* Impresión original (puede truncarse si hay '\0' internos) */
00138  printf("MSG_P: %s\n", msgP);
00139
00140  /* ----- Simular error: voltear bit nb en byte nc ----- */
00141  nc = rand()% (n+np);
00142  nb = rand()%8;
00143  printf("NC = %d\tNB = %d\n", nc, nb);
00144
00145  BIT_TOGGLE(msgP[nc], nb);
00146
00147  /* Impresión original */
00148  printf("MSG_E: %s\n", msgP);
00149
00150  /*
00151  ----- Recalcular paridades (verificación) -----
00152  V[i] guarda el XOR del grupo de paridad i (debe ser 0 si no hay error).
00153  Se mantiene el mismo patrón de bloques, pero guardando en V[i] (no V[j]).
00154  */
00155  for(i=0; i<np; i++)
00156  {
00157      j = (1<<i)-1;
00158
00159      /* Inicializar con el valor actual del byte de paridad */
00160      V[i] = (unsigned char)msgP[j];
00161
00162      /* XOR sobre los bloques del grupo de paridad i */
00163      for(k=j+1; k<(n+np); k++)
00164      {
00165          if(((k-j)/(j+1)+1)%2)
00166              V[i] ^= (unsigned char)msgP[k];
00167      }
00168
00169      /* Impresión original */
00170      printf("P_%d = %d\n", i, (int)V[i]);
00171  }
00172
00173  /*
00174  ----- Síndrome -----
00175  ncv: se arma con bits i donde V[i] != 0.
00176  Esto da la posición del byte erróneo en 1-based.
00177  nbv: máscara del bit erróneo (potencia de 2), tomada del primer V[i] != 0.
00178  */
00179  for(i=0, ncv=0, nbv=0; i<np; i++)
00180      if(V[i])
00181      {
00182          BIT_SET(ncv, i);
00183          if(!nbv)
00184              nbv = (int)V[i];
00185      }
00186
00187  /* Mantener la conversión original a 0-based */
00188  ncv--;
00189
00190  /*
00191  Convertir máscara nbv (ej. 8) a índice de bit (3) con log2.
00192  ERROR CORREGIDO: si nbv==0, log2 es inválido.
00193  */
00194  if(nbv > 0)
00195  {
00196      aux = (float)log2((double)nbv);
00197      nbv = (int)aux;
00198  }
00199  else
00200  {
00201      aux = 0.0f;
00202      nbv = 0;
00203  }
00204
00205  printf("NC_V = %d\tNB_V = %d\n", ncv, nbv);
00206
00207  /* Validación original (comentada) se conserva tal cual */

```

```

00208     /*
00209     for(i=0, V=P; i<n; i++)
00210         V^=msgP[i];
00211     printf("Validacion: %c (", V);
00212     for(i=7; i>-1; i--)
00213         printf("%d ", BIT_GET(V, i)?1:0);
00214     printf("\n");
00215     */
00216
00217     return 0;
00218 }

```

7.217 Ejemplo034.c

[Go to the documentation of this file.](#)

```

00001
00027
00028 #include <stdio.h>
00029 #include <stdlib.h>
00030 #include <time.h>
00031 #include <math.h>
00032
00033 #define N 100
00034 #define P 7
00035
00036 #define BIT(n)                (1U<<(n))
00037 #define BIT_GET(x,n)         ((x) & BIT(n))
00038 #define BIT_SET(x,n)         ((x) |= BIT(n))
00039 #define BIT_TOGGLE(x,n)      ((x) ^= BIT(n))
00040
00041 /* True si pos (1-based) es potencia de 2: 1,2,4,8,... */
00042 #define IS_POW2_POS(pos)      ((pos) && (((pos) & ((pos)-1U)) == 0U))
00043
00044 int main(int argc, char *argv[])
00045 {
00046     (void)argc;
00047     (void)argv;
00048
00049     /*
00050     msg  : mensaje original (solo datos)
00051     msgP : mensaje con espacios de paridad insertados (datos + paridades)
00052     V    : vector de verificación de paridad por cada bit de paridad i
00053     msgV : reservado (no se usa en esta versión; se conserva por estructura original)
00054     */
00055     char msg[N], msgP[N+P];
00056
00057     /*
00058     ERROR CORREGIDO:
00059     V debe indexarse por i (0..np-1). En el original V era char y se usaba con
00060     índices derivados de j=(2^i-1), lo que podía desbordar o dejar valores sin inicializar.
00061     Se usa unsigned char para evitar problemas de signo en XOR.
00062     */
00063     unsigned char V[P];
00064
00065     int i, j, k, n, np, nc, nb, ncv, nbv;
00066     float aux;
00067
00068     srand((unsigned)time(NULL));
00069
00070     /* ----- Leer mensaje hasta ENTER ----- */
00071     i = 0;
00072     do{
00073         msg[i] = (char)getchar();
00074         i++;
00075     }while(i < N && msg[i-1] != 10); /* 10 == '\n' */
00076
00077     msg[i-1] = '\0';
00078     n = i-1;
00079
00080     /*
00081     ERROR CORREGIDO:
00082     np no se calcula con aproximación logarítmica.
00083     Condición de Hamming (mínimo np):
00084     2^np >= n + np + 1
00085     Esto garantiza suficientes posiciones de paridad para identificar la posición del error.
00086     */
00087     np = 0;
00088     while( ((1U<<np) < (unsigned)(n + np + 1)) && np < P )
00089         np++;
00090
00091     printf("n = %d\tnp = %d\n", n, np);
00092     printf("MSG: %s\n", msg);
00093
00094     /*

```

```

00095     ----- Construir msgP con paridades -----
00096     Longitud total L = n + np.
00097     Posiciones (1-based) potencia de 2 se reservan para paridad (byte 0).
00098     El resto se llena con los bytes del mensaje original.
00099     */
00100     {
00101         int L = n + np;
00102         int data_idx = 0;
00103
00104         for(i = 0; i < L; i++)
00105         {
00106             int pos = i + 1; /* 1-based */
00107
00108             if(IS_POW2_POS((unsigned)pos))
00109             {
00110                 msgP[i] = 0; /* placeholder de paridad */
00111             }
00112             else
00113             {
00114                 msgP[i] = msg[data_idx++]; /* copiar dato */
00115             }
00116         }
00117         /* Terminador para impresión con %s (se mantiene como en el original) */
00118         msgP[L] = '\0';
00119     }
00120
00121     /*
00122     ----- Cálculo de paridades (codificación) -----
00123     Para cada i:
00124         j = 2^i - 1 (índice 0-based donde vive la paridad i)
00125     Se aplica el patrón de bloques del código original:
00126         incluir (j+1) bytes, saltar (j+1) bytes, incluir, ...
00127     y se acumula XOR en msgP[j].
00128     */
00129     for(i=0; i<np; i++)
00130     {
00131         j = (1<<i)-1;
00132         for(k=j; k<(n+np); k++)
00133             if(((k-j)/(j+1)+1)%2)
00134                 msgP[j] ^= msgP[k];
00135     }
00136
00137     /* Impresión original (puede truncarse si hay '\0' internos) */
00138     printf("MSG_P: %s\n", msgP);
00139
00140     /* ----- Simular error: voltear bit nb en byte nc ----- */
00141     nc = rand()%(n+np);
00142     nb = rand()%8;
00143     printf("NC = %d\tNB = %d\n", nc, nb);
00144
00145     BIT_TOGGLE(msgP[nc], nb);
00146
00147     /* Impresión original */
00148     printf("MSG_E: %s\n", msgP);
00149
00150     /*
00151     ----- Recalcular paridades (verificación) -----
00152     V[i] guarda el XOR del grupo de paridad i (debe ser 0 si no hay error).
00153     Se mantiene el mismo patrón de bloques, pero guardando en V[i] (no V[j]).
00154     */
00155     for(i=0; i<np; i++)
00156     {
00157         j = (1<<i)-1;
00158
00159         /* Inicializar con el valor actual del byte de paridad */
00160         V[i] = (unsigned char)msgP[j];
00161
00162         /* XOR sobre los bloques del grupo de paridad i */
00163         for(k=j+1; k<(n+np); k++)
00164         {
00165             if(((k-j)/(j+1)+1)%2)
00166                 V[i] ^= (unsigned char)msgP[k];
00167         }
00168
00169         /* Impresión original */
00170         printf("P_%d = %d\n", i, (int)V[i]);
00171     }
00172
00173     /*
00174     ----- Síndrome -----
00175     ncv: se arma con bits i donde V[i] != 0.
00176     Esto da la posición del byte erróneo en 1-based.
00177     nbv: máscara del bit erróneo (potencia de 2), tomada del primer V[i] != 0.
00178     */
00179     for(i=0, ncv=0, nbv=0; i<np; i++)
00180         if(V[i])
00181             {

```

```

00182         BIT_SET(ncv, i);
00183         if(!nbv)
00184             nbv = (int)V[i];
00185     }
00186
00187     /* Mantener la conversión original a 0-based */
00188     ncv--;
00189
00190     /*
00191     Convertir máscara nbv (ej. 8) a índice de bit (3) con log2.
00192     ERROR CORREGIDO: si nbv==0, log2 es inválido.
00193     */
00194     if(nbv > 0)
00195     {
00196         aux = (float)log2((double)nbv);
00197         nbv = (int)aux;
00198     }
00199     else
00200     {
00201         aux = 0.0f;
00202         nbv = 0;
00203     }
00204
00205     printf("NC_V = %d\tNB_V = %d\n", ncv, nbv);
00206
00207     /* Validación original (comentada) se conserva tal cual */
00208     /*
00209     for(i=0, V=P; i<n; i++)
00210         V^=msgP[i];
00211     printf("Validacion: %c (", V);
00212     for(i=7; i>-1; i--)
00213         printf("%d ", BIT_GET(V, i)?1:0);
00214     printf("\n");
00215     */
00216
00217     return 0;
00218 }

```

7.218 src/20261/src/Ejemplo034_2.c File Reference

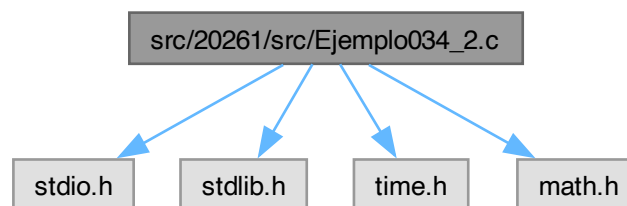
Inserción de bytes de paridad en posiciones potencia de 2 (Hamming por bytes) y detección de un error de 1 bit.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

```

Include dependency graph for Ejemplo034_2.c:



Macros

- #define N 100
- #define P 7
- #define BIT(n)
- #define BIT_GET(x, n)
- #define BIT_SET(x, n)
- #define BIT_TOGGLE(x, n)

- `#define IS_POW2_POS(pos)`

Functions

- `int main (int argc, char *argv[])`

7.218.1 Detailed Description

Inserción de bytes de paridad en posiciones potencia de 2 (Hamming por bytes) y detección de un error de 1 bit. Este programa implementa una codificación tipo Hamming sobre un mensaje ASCII, pero trabajando a nivel de **bytes** (caracteres) y usando XOR (^) como operación de paridad.

Flujo general: 1) Lee un mensaje `msg` hasta ENTER. 2) Calcula el número mínimo de paridades `np` que satisface: $2^{np} \geq n + np + 1$ 3) Construye `msgP` (mensaje extendido) de longitud $L = n + np$:

- en posiciones potencia de 2 (1,2,4,8,...) coloca bytes de paridad (inicialmente 0)
- en el resto coloca los bytes del mensaje original. 4) Calcula las paridades usando el patrón de “bloques” del código original. 5) Simula un error: invierte un bit aleatorio en un byte aleatorio del código. 6) Recalcula las paridades y construye:
- `NC_V`: byte con error (0-based)
- `NB_V`: bit con error (0..7)

Entrada estándar

Una línea de texto (hasta N-1 caracteres) terminada con ENTER.

Salida estándar

Mantiene exactamente la forma de impresión del original:

- `MSG`, `MSG_P`, `NC/NB`, `MSG_E`
- `P_i`
- `NC_V` y `NB_V`

Warning

- La impresión `MSG_P`: s y `MSG_E`: s puede truncarse si aparecen bytes '\0' dentro de `msgP`. Se conserva esta impresión por compatibilidad con el original.

```
gcc Ejemplo034_2.c -o Ejemplo034_2 -lm
./Ejemplo034_2
```

Definition in file [Ejemplo034_2.c](#).

7.218.2 Macro Definition Documentation

7.218.2.1 BIT

```
#define BIT(
    n)
```

Value:

```
(1U<<(n))
```

Definition at line 50 of file [Ejemplo034_2.c](#).

7.218.2.2 BIT_GET

```
#define BIT_GET(
    x,
    n)
```

Value:

```
((x) & BIT(n))
```

Definition at line 51 of file [Ejemplo034_2.c](#).

7.218.2.3 BIT_SET

```
#define BIT_SET(  
    x,  
    n)
```

Value:

```
((x) |= BIT(n))
```

Definition at line 52 of file [Ejemplo034_2.c](#).

7.218.2.4 BIT_TOGGLE

```
#define BIT_TOGGLE(  
    x,  
    n)
```

Value:

```
((x) ^= BIT(n))
```

Definition at line 53 of file [Ejemplo034_2.c](#).

7.218.2.5 IS_POW2_POS

```
#define IS_POW2_POS(  
    pos)
```

Value:

```
((pos) && (((pos) & ((pos)-1U)) == 0U))
```

Definition at line 56 of file [Ejemplo034_2.c](#).

7.218.2.6 N

```
#define N 100
```

Definition at line 46 of file [Ejemplo034_2.c](#).

7.218.2.7 P

```
#define P 7
```

Definition at line 47 of file [Ejemplo034_2.c](#).

7.218.3 Function Documentation

7.218.3.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 58 of file [Ejemplo034_2.c](#).

```
00059 {  
00060     (void)argc;  
00061     (void)argv;  
00062  
00063     /*  
00064         msg  : mensaje original (solo datos)  
00065         msgP : mensaje extendido (datos + bytes de paridad)  
00066         V    : verificaciones de paridad (una por cada bit de paridad i)  
00067                Se usa unsigned char para que XOR sea estable y no haya signos negativos.  
00068     */  
00069     char msg[N], msgP[N+P];  
00070     unsigned char V[P];  
00071  
00072     /*  
00073         n    : longitud del mensaje original (bytes)  
00074         np   : número de paridades  
00075         nc   : índice (0-based) del byte donde se inyecta el error  
00076         nb   : índice del bit (0..7) que se invierte para simular el error  
00077         ncv  : síndrome (posición 1-based del byte con error, luego se convierte a 0-based con ncv--)  
00078         nbv  : máscara del bit alterado (potencia de 2), luego se convierte a índice con log2  
00079     */  
00080     int i, j, k, n, np, nc, nb, ncv, nbv;  
00081     float aux;
```



```

00082
00083     srand(time(NULL));
00084
00085     /* ----- Lectura del mensaje ----- */
00086     i = 0;
00087     do{
00088         msg[i] = getchar();
00089         i++;
00090     }while(i < N && msg[i-1] != 10); /* 10 == '\n' */
00091
00092     msg[i-1] = '\0';
00093     n = i-1;
00094
00095     /* Protección: mensaje vacío (evita rand()%0 y accesos inválidos) */
00096     if (n <= 0)
00097         return 0;
00098
00099     /*
00100     ----- Cálculo correcto de np -----
00101     Condición mínima de Hamming:
00102          $2^{np} \geq n + np + 1$ 
00103     Se acota con P para evitar desbordar el tamaño del arreglo msgP.
00104     */
00105     np = 0;
00106     while (((1U << np) < (unsigned)(n + np + 1)) && np < P)
00107         np++;
00108
00109     printf("n = %d\tnp = %d\n", n, np);
00110     printf("MSG: %s\n", msg);
00111
00112     /*
00113     ----- Construcción de msgP -----
00114     Longitud del código:
00115         L = n + np
00116
00117     Recorremos posiciones 1-based (pos = i+1):
00118     - si pos es potencia de 2 => posición de paridad => msgP[i] = 0
00119     - si no => posición de dato => copiar el siguiente byte del mensaje
00120     */
00121     {
00122         int L = n + np;
00123         int data_idx = 0;
00124
00125         for (i = 0; i < L; i++)
00126         {
00127             int pos = i + 1; /* 1-based */
00128             if (IS_POW2_POS((unsigned)pos))
00129                 msgP[i] = 0; /* paridad */
00130             else
00131                 msgP[i] = msg[data_idx++]; /* dato */
00132         }
00133
00134         /* Terminador para imprimir con %s (misma práctica del original) */
00135         msgP[L] = '\0';
00136     }
00137
00138     /*
00139     ----- Cálculo de paridades (codificación) -----
00140     Para cada bit de paridad i:
00141         j = 2^i - 1 (índice 0-based donde vive esa paridad)
00142     Se aplica el patrón de bloques del código original:
00143         incluir (j+1) bytes, saltar (j+1) bytes, incluir, ...
00144     y se acumula XOR en msgP[j].
00145     */
00146     for (i = 0; i < np; i++)
00147     {
00148         j = (1 << i) - 1;
00149         for (k = j; k < (n + np); k++)
00150             if (((k - j) / (j + 1) + 1) % 2)
00151                 msgP[j] ^= msgP[k];
00152     }
00153
00154     printf("MSG_P: %s\n", msgP);
00155
00156     /* ----- Simulación del error ----- */
00157     nc = rand() % (n + np);
00158     nb = rand() % 8;
00159     printf("NC = %d\tnb = %d\n", nc, nb);
00160
00161     BIT_TOGGLE(msgP[nc], nb);
00162
00163     printf("MSG_E: %s\n", msgP);
00164
00165     /*
00166     ----- Verificación de paridades -----
00167     V[i] acumula el XOR del grupo de paridad i (debe ser 0 si todo es consistente).
00168     Se conserva el mismo patrón de bloques (iniciando V[i] con msgP[j]).

```

```

00169  */
00170  for (i = 0; i < np; i++)
00171  {
00172      j = (1 << i) - 1;
00173
00174      for (k = j + 1, V[i] = (unsigned char)msgP[j]; k < (n + np); k++)
00175      {
00176          if (((k - j) / (j + 1) + 1) % 2)
00177              V[i] ^= (unsigned char)msgP[k];
00178      }
00179
00180      printf("P_%d = %d\n", i, (int)V[i]);
00181  }
00182
00183  /*
00184  ----- Síndrome y bit de error -----
00185  - ncv se arma con los bits i para los cuales V[i] != 0.
00186    Eso da la posición 1-based del byte erróneo.
00187  - nbv toma la máscara del bit (potencia de 2), del primer V[i] no nulo.
00188  */
00189  for (i = 0, ncv = 0, nbv = 0; i < np; i++)
00190      if (V[i])
00191      {
00192          BIT_SET(ncv, i);
00193          if (!nbv)
00194              nbv = (int)V[i];
00195      }
00196
00197  /* Convertir a 0-based (como el original) */
00198  ncv--;
00199
00200  /*
00201  Convertir máscara (potencia de 2) a índice de bit con log2.
00202  Protección: log2(0) no es válido (aunque aquí normalmente hay error inyectado).
00203  */
00204  if (nbv > 0)
00205  {
00206      aux = (float)log2((double)nbv);
00207      nbv = (int)aux;
00208  }
00209  else
00210  {
00211      nbv = 0;
00212  }
00213
00214  printf("NC_V = %d\tNB_V = %d\n", ncv, nbv);
00215
00216  /* Bloque de validación del original se conserva comentado */
00217  /*
00218  for(i=0, V=P; i<n; i++)
00219      V^=msgP[i];
00220  printf("Validacion: %c ", V);
00221  for(i=7; i>=1; i--)
00222      printf("%d ", BIT_GET(V, i)?1:0);
00223  printf("\n");
00224  */
00225
00226  return 0;
00227 }

```

7.219 Ejemplo034_2.c

[Go to the documentation of this file.](#)

```

00001
00040
00041 #include <stdio.h>
00042 #include <stdlib.h>
00043 #include <time.h>
00044 #include <math.h>
00045
00046 #define N 100
00047 #define P 7
00048
00049 /* Macros de bits (se usan para simular el error y construir el síndrome) */
00050 #define BIT(n) (1U<<(n))
00051 #define BIT_GET(x,n) ((x) & BIT(n))
00052 #define BIT_SET(x,n) ((x) |= BIT(n))
00053 #define BIT_TOGGLE(x,n) ((x) ^= BIT(n))
00054
00055 /* True si una posición 1-based es potencia de 2: 1,2,4,8,... */
00056 #define IS_POW2_POS(pos) ((pos) && (((pos) & ((pos)-1U)) == 0U))
00057
00058 int main(int argc, char *argv[])
00059 {

```

```

00060     (void)argc;
00061     (void)argv;
00062
00063     /*
00064         msg  : mensaje original (solo datos)
00065         msgP : mensaje extendido (datos + bytes de paridad)
00066         V    : verificaciones de paridad (una por cada bit de paridad i)
00067                Se usa unsigned char para que XOR sea estable y no haya signos negativos.
00068     */
00069     char msg[N], msgP[N+P];
00070     unsigned char V[P];
00071
00072     /*
00073         n    : longitud del mensaje original (bytes)
00074         np   : número de paridades
00075         nc   : índice (0-based) del byte donde se inyecta el error
00076         nb   : índice del bit (0..7) que se invierte para simular el error
00077         ncv  : síndrome (posición 1-based del byte con error, luego se convierte a 0-based con ncv--)
00078         nbv  : máscara del bit alterado (potencia de 2), luego se convierte a índice con log2
00079     */
00080     int i, j, k, n, np, nc, nb, ncv, nbv;
00081     float aux;
00082
00083     srand(time(NULL));
00084
00085     /* ----- Lectura del mensaje ----- */
00086     i = 0;
00087     do{
00088         msg[i] = getchar();
00089         i++;
00090     }while(i < N && msg[i-1] != 10); /* 10 == '\n' */
00091
00092     msg[i-1] = '\0';
00093     n = i-1;
00094
00095     /* Protección: mensaje vacío (evita rand()%0 y accesos inválidos) */
00096     if (n <= 0)
00097         return 0;
00098
00099     /*
00100         ----- Cálculo correcto de np -----
00101         Condición mínima de Hamming:
00102             2^np >= n + np + 1
00103         Se acota con P para evitar desbordar el tamaño del arreglo msgP.
00104     */
00105     np = 0;
00106     while (((1U << np) < (unsigned)(n + np + 1)) && np < P)
00107         np++;
00108
00109     printf("n = %d\tnp = %d\n", n, np);
00110     printf("MSG: %s\n", msg);
00111
00112     /*
00113         ----- Construcción de msgP -----
00114         Longitud del código:
00115             L = n + np
00116
00117         Recorremos posiciones 1-based (pos = i+1):
00118         - si pos es potencia de 2 => posición de paridad => msgP[i] = 0
00119         - si no => posición de dato => copiar el siguiente byte del mensaje
00120     */
00121     {
00122         int L = n + np;
00123         int data_idx = 0;
00124
00125         for (i = 0; i < L; i++)
00126         {
00127             int pos = i + 1; /* 1-based */
00128             if (IS_POW2_POS((unsigned)pos))
00129                 msgP[i] = 0; /* paridad */
00130             else
00131                 msgP[i] = msg[data_idx++]; /* dato */
00132         }
00133
00134         /* Terminador para imprimir con %s (misma práctica del original) */
00135         msgP[L] = '\0';
00136     }
00137
00138     /*
00139         ----- Cálculo de paridades (codificación) -----
00140         Para cada bit de paridad i:
00141             j = 2^i - 1 (índice 0-based donde vive esa paridad)
00142         Se aplica el patrón de bloques del código original:
00143             incluir (j+1) bytes, saltar (j+1) bytes, incluir, ...
00144         y se acumula XOR en msgP[j].
00145     */
00146     for (i = 0; i < np; i++)

```

```

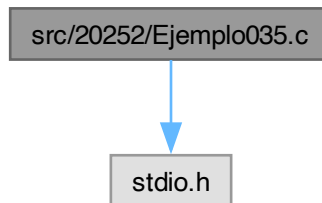
00147     {
00148         j = (1 << i) - 1;
00149         for (k = j; k < (n + np); k++)
00150             if (((k - j) / (j + 1) + 1) % 2)
00151                 msgP[j] ^= msgP[k];
00152     }
00153
00154     printf("MSG_P: %s\n", msgP);
00155
00156     /* ----- Simulación del error ----- */
00157     nc = rand() % (n + np);
00158     nb = rand() % 8;
00159     printf("NC = %d\tNB = %d\n", nc, nb);
00160
00161     BIT_TOGGLE(msgP[nc], nb);
00162
00163     printf("MSG_E: %s\n", msgP);
00164
00165     /*
00166     ----- Verificación de paridades -----
00167     V[i] acumula el XOR del grupo de paridad i (debe ser 0 si todo es consistente).
00168     Se conserva el mismo patrón de bloques (iniciando V[i] con msgP[j]).
00169     */
00170     for (i = 0; i < np; i++)
00171     {
00172         j = (1 << i) - 1;
00173
00174         for (k = j + 1, V[i] = (unsigned char)msgP[j]; k < (n + np); k++)
00175         {
00176             if (((k - j) / (j + 1) + 1) % 2)
00177                 V[i] ^= (unsigned char)msgP[k];
00178         }
00179
00180         printf("P_%d = %d\n", i, (int)V[i]);
00181     }
00182
00183     /*
00184     ----- Síndrome y bit de error -----
00185     - ncv se arma con los bits i para los cuales V[i] != 0.
00186     Eso da la posición 1-based del byte erróneo.
00187     - nbv toma la máscara del bit (potencia de 2), del primer V[i] no nulo.
00188     */
00189     for (i = 0, ncv = 0, nbv = 0; i < np; i++)
00190         if (V[i])
00191         {
00192             BIT_SET(ncv, i);
00193             if (!nbv)
00194                 nbv = (int)V[i];
00195         }
00196
00197     /* Convertir a 0-based (como el original) */
00198     ncv--;
00199
00200     /*
00201     Convertir máscara (potencia de 2) a índice de bit con log2.
00202     Protección: log2(0) no es válido (aunque aquí normalmente hay error inyectado).
00203     */
00204     if (nbv > 0)
00205     {
00206         aux = (float)log2((double)nbv);
00207         nbv = (int)aux;
00208     }
00209     else
00210     {
00211         nbv = 0;
00212     }
00213
00214     printf("NC_V = %d\tNB_V = %d\n", ncv, nbv);
00215
00216     /* Bloque de validación del original se conserva comentado */
00217     /*
00218     for(i=0, V=P; i<n; i++)
00219         V^=msgP[i];
00220     printf("Validacion: %c (", V);
00221     for(i=7; i>=0; i--)
00222         printf("%d ", BIT_GET(V, i)?1:0);
00223     printf("\n");
00224     */
00225
00226     return 0;
00227 }

```

7.220 src/20252/Ejemplo035.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo035.c:



Data Structures

- struct [dato_s](#)
- union [dato_u](#)

Functions

- int [main](#) (int argc, char *argv[])

7.220.1 Function Documentation

7.220.1.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 15 of file [Ejemplo035.c](#).

```

00016 {
00017     dato_s x;
00018     dato_u y;
00019     printf("Sizeof(char) = %ld\n", sizeof(char));
00020     printf("Sizeof(int) = %ld\n", sizeof(int));
00021     printf("Sizeof(float) = %ld\n", sizeof(float));
00022     printf("Sizeof(dato_s) = %ld\n", sizeof(x));
00023     printf("Sizeof(dato_u) = %ld\n", sizeof(y));
00024     x.a = 1;
00025     y.a = 1;
00026     printf("x.a = %d\n", x.a);
00027     printf("y.a = %d\n", y.a);
00028     x.b = 2;
00029     y.b = 2;
00030     printf("x.a = %d\n", x.a);
00031     printf("y.a = %d\n", y.a);
00032     printf("x.b = %d\n", x.b);
00033     printf("y.b = %d\n", y.b);
00034     return 0;
00035 }
```

7.221 Ejemplo035.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 typedef struct{
00004     char a;
```

```

00005     int b;
00006     float c;
00007 }dato_s;
00008
00009 typedef union{
00010     char a;
00011     int b;
00012     float c;
00013 }dato_u;
00014
00015 int main(int argc, char *argv[])
00016 {
00017     dato_s x;
00018     dato_u y;
00019     printf("Sizeof(char) = %ld\n", sizeof(char));
00020     printf("Sizeof(int) = %ld\n", sizeof(int));
00021     printf("Sizeof(float) = %ld\n", sizeof(float));
00022     printf("Sizeof(dato_s) = %ld\n", sizeof(x));
00023     printf("Sizeof(dato_u) = %ld\n", sizeof(y));
00024     x.a = 1;
00025     y.a = 1;
00026     printf("x.a = %d\n", x.a);
00027     printf("y.a = %d\n", y.a);
00028     x.b = 2;
00029     y.b = 2;
00030     printf("x.a = %d\n", x.a);
00031     printf("y.a = %d\n", y.a);
00032     printf("x.b = %d\n", x.b);
00033     printf("y.b = %d\n", y.b);
00034     return 0;
00035 }

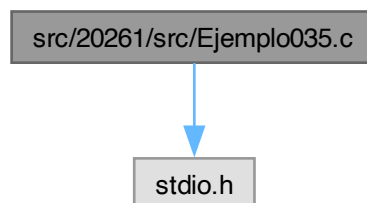
```

7.222 src/20261/src/Ejemplo035.c File Reference

Cifrado/descifrado XOR sobre letras (A–Z, a–z) usando una clave numérica.

#include <stdio.h>

Include dependency graph for Ejemplo035.c:



Macros

- #define `N` 100
- #define `BIT`(n)
- #define `BIT_GET`(x, n)
- #define `BIT_SET`(x, n)
- #define `BIT_CLEAR`(x, n)

Functions

- int `main` (int argc, char *argv[])

7.222.1 Detailed Description

Cifrado/descifrado XOR sobre letras (A–Z, a–z) usando una clave numérica.

Este programa implementa un cifrado didáctico:

- Solo modifica letras del alfabeto:
 - 'A'..'Z' -> valores 0..25
 - 'a'..'z' -> valores 26..51
- A ese valor se le aplica XOR con una clave `cl_c` (un byte).
- Después se “re-mapea” al rango de letras:
 - 0..25 -> 'A'..'Z'
 - 26..51 -> 'a'..'z'

El programa imprime trazas numéricas por carácter para mostrar la transformación.

Entrada estándar

- 1) Una línea de texto (hasta N-1 caracteres) terminada con ENTER.
- 2) Una clave entera (se trunca a 1 byte al convertirla a char).

Salida estándar

- Mensaje original MSG
- Mensaje cifrado MSG E
- Mensaje descifrado MSG D
- Trazas por carácter con valores intermedios.

Note

- Solo letras A–Z / a–z se cifran; otros caracteres se dejan igual.
- Es un ejemplo didáctico; no es criptografía segura.

```
gcc Ejemplo035.c -o Ejemplo035
./Ejemplo035
```

Definition in file [Ejemplo035.c](#).

7.222.2 Macro Definition Documentation

7.222.2.1 BIT

```
#define BIT(
    n)
```

Value:

```
(1<<(n))
```

Definition at line 40 of file [Ejemplo035.c](#).

7.222.2.2 BIT_CLEAR

```
#define BIT_CLEAR(
    x,
    n)
```

Value:

```
((x) &= ~BIT(n))
```

Definition at line 43 of file [Ejemplo035.c](#).

7.222.2.3 BIT_GET

```
#define BIT_GET(
    x,
    n)
```

Value:

```
((x) & BIT(n))
```

Definition at line 41 of file [Ejemplo035.c](#).

7.222.2.4 BIT_SET

```
#define BIT_SET(
    x,
    n)
```

Value:

```
((x) |= BIT(n))
```

Definition at line 42 of file [Ejemplo035.c](#).

7.222.2.5 N

```
#define N 100
```

Definition at line 39 of file [Ejemplo035.c](#).

7.222.3 Function Documentation

7.222.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 45 of file [Ejemplo035.c](#).

```
00046 {
00047     (void)argc;
00048     (void)argv;
00049
00050     char msg[N], msgE[N], msgD[N], cl_c, E;
00051     int i, cl_I, n;
00052
00053     /* Leer mensaje hasta ENTER */
00054     i = 0;
00055     do{
00056         msg[i] = (char)getchar();
00057         i++;
00058     }while(i<N&&msg[i-1]!=10);
00059     msg[i-1] = '\0';
00060     n = i-1;
00061
00062     /* Leer clave como entero y convertirla a un byte (char) */
00063     printf("Ingrese la clave: ");
00064     scanf("%d", &cl_I);
00065     cl_c = (char)cl_I;
00066
00067     printf("MSG: %s\n", msg);
00068
00069     /* ----- CIFRADO ----- */
00070     msgE[n] = '\0';
00071     for(i=0; i<n; i++)
00072     {
00073         printf("%d. %u\t", i, (unsigned int)(unsigned char)msg[i]);
00074
00075         msgE[i] = msg[i];
00076         E = 0;
00077
00078         /* Si es minúscula, mapear a 26..51 */
00079         if(msgE[i]>='a' && msgE[i]<='z')
00080         {
00081             msgE[i] -= 'a';
00082             msgE[i] += 26;
00083             printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00084
00085             msgE[i] ^= cl_c; /* XOR con la clave */
00086             E = 1;
00087             printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00088         }
00089         /* Si es mayúscula, mapear a 0..25 */
00090         else if(msgE[i]>='A' && msgE[i]<='Z')
00091         {
00092             msgE[i] -= 'A';
00093             printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00094
00095             msgE[i] ^= cl_c;
00096             E = 1;
00097             printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00098         }
00099
00100         /* Re-mapeo a letra */
00101         if(E && (unsigned char)msgE[i] < 26)
```



```

00102     {
00103         msgE[i] += 'A';
00104         printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00105     }
00106     else if(E && (unsigned char)msgE[i] < 52)
00107     {
00108         msgE[i] -= 26;
00109         msgE[i] += 'a';
00110         printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00111     }
00112
00113     printf("\n");
00114 }
00115 printf("MSG E: %s\n", msgE);
00116
00117 /* ----- DESCIFRADO ----- */
00118 msgD[n] = '\0';
00119 for(i=0; i<n; i++)
00120 {
00121     printf("%d. %u\t", i, (unsigned int)(unsigned char)msgE[i]);
00122
00123     msgD[i] = msgE[i];
00124     E = 0;
00125
00126     if(msgD[i]>='a' && msgD[i]<='z')
00127     {
00128         msgD[i] -= 'a';
00129         msgD[i] += 26;
00130         printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00131
00132         msgD[i] ^= cl_c;
00133         printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00134         E = 1;
00135     }
00136     else if(msgD[i]>='A' && msgD[i]<='Z')
00137     {
00138         msgD[i] -= 'A';
00139         printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00140
00141         msgD[i] ^= cl_c;
00142         printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00143         E = 1;
00144     }
00145
00146     if(E && (unsigned char)msgD[i] < 26)
00147     {
00148         msgD[i] += 'A';
00149         printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00150     }
00151     else if(E && (unsigned char)msgD[i] < 52)
00152     {
00153         msgD[i] -= 26;
00154         msgD[i] += 'a';
00155         printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00156     }
00157
00158     printf("\n");
00159 }
00160 printf("MSG D: %s\n", msgD);
00161
00162 return 0;
00163 }

```

7.223 Ejemplo035.c

[Go to the documentation of this file.](#)

```

00001
00036
00037 #include <stdio.h>
00038
00039 #define N 100
00040 #define BIT(n) ((1<<(n))
00041 #define BIT_GET(x,n) ((x) & BIT(n))
00042 #define BIT_SET(x,n) ((x) |= BIT(n))
00043 #define BIT_CLEAR(x,n) ((x) &= ~BIT(n))
00044
00045 int main(int argc, char *argv[])
00046 {
00047     (void)argc;
00048     (void)argv;
00049
00050     char msg[N], msgE[N], msgD[N], cl_c, E;
00051     int i, cl_I, n;
00052

```

```

00053      /* Leer mensaje hasta ENTER */
00054      i = 0;
00055      do{
00056          msg[i] = (char)getchar();
00057          i++;
00058      }while(i<N&&msg[i-1]!=10);
00059      msg[i-1] = '\0';
00060      n = i-1;
00061
00062      /* Leer clave como entero y convertirla a un byte (char) */
00063      printf("Ingrese la clave: ");
00064      scanf("%d", &cl_I);
00065      cl_c = (char)cl_I;
00066
00067      printf("MSG: %s\n", msg);
00068
00069      /* ----- CIFRADO ----- */
00070      msgE[n] = '\0';
00071      for(i=0; i<n; i++)
00072      {
00073          printf("%d. %u\t", i, (unsigned int)(unsigned char)msg[i]);
00074
00075          msgE[i] = msg[i];
00076          E = 0;
00077
00078          /* Si es minúscula, mapear a 26..51 */
00079          if(msgE[i]>='a' && msgE[i]<='z')
00080          {
00081              msgE[i] -= 'a';
00082              msgE[i] += 26;
00083              printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00084
00085              msgE[i] ^= cl_c; /* XOR con la clave */
00086              E = 1;
00087              printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00088          }
00089          /* Si es mayúscula, mapear a 0..25 */
00090          else if(msgE[i]>='A' && msgE[i]<='Z')
00091          {
00092              msgE[i] -= 'A';
00093              printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00094
00095              msgE[i] ^= cl_c;
00096              E = 1;
00097              printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00098          }
00099
00100          /* Re-mapeo a letra */
00101          if(E && (unsigned char)msgE[i] < 26)
00102          {
00103              msgE[i] += 'A';
00104              printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00105          }
00106          else if(E && (unsigned char)msgE[i] < 52)
00107          {
00108              msgE[i] -= 26;
00109              msgE[i] += 'a';
00110              printf("%u\t", (unsigned int)(unsigned char)msgE[i]);
00111          }
00112
00113          printf("\n");
00114      }
00115      printf("MSG E: %s\n", msgE);
00116
00117      /* ----- DESCIFRADO ----- */
00118      msgD[n] = '\0';
00119      for(i=0; i<n; i++)
00120      {
00121          printf("%d. %u\t", i, (unsigned int)(unsigned char)msgE[i]);
00122
00123          msgD[i] = msgE[i];
00124          E = 0;
00125
00126          if(msgD[i]>='a' && msgD[i]<='z')
00127          {
00128              msgD[i] -= 'a';
00129              msgD[i] += 26;
00130              printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00131
00132              msgD[i] ^= cl_c;
00133              printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00134              E = 1;
00135          }
00136          else if(msgD[i]>='A' && msgD[i]<='Z')
00137          {
00138              msgD[i] -= 'A';
00139              printf("%u\t", (unsigned int)(unsigned char)msgD[i]);

```

```

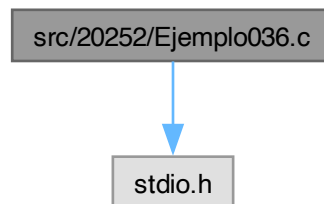
00140
00141     msgD[i] ^= cl_c;
00142     printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00143     E = 1;
00144 }
00145
00146 if(E && (unsigned char)msgD[i] < 26)
00147 {
00148     msgD[i] += 'A';
00149     printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00150 }
00151 else if(E && (unsigned char)msgD[i] < 52)
00152 {
00153     msgD[i] -= 26;
00154     msgD[i] += 'a';
00155     printf("%u\t", (unsigned int)(unsigned char)msgD[i]);
00156 }
00157
00158 printf("\n");
00159 }
00160 printf("MSG D: %s\n", msgD);
00161
00162 return 0;
00163 }

```

7.224 src/20252/Ejemplo036.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo036.c:



Functions

- int [main](#) (int argc, char *argv[])

7.224.1 Function Documentation

7.224.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo036.c](#).

```

00004 {
00005     int x, *px;
00006     char *py;
00007     double *pz;
00008     x = 20;
00009     px = &x;
00010     py = (char*)&x;
00011     pz = (double*)&x;
00012     printf("x = %d\n", x);
00013     printf("&x = %p\n", &x);
00014     printf("px = %p\n", px);
00015     printf("*px = %d\n", *px);

```

```

00016     printf("px = %p\n", px);
00017     printf("py = %p\n", py);
00018     printf("pz = %p\n", pz);
00019     printf("px = %p (%ld)\n", px+1, sizeof(int));
00020     printf("py = %p (%ld)\n", py+1, sizeof(char));
00021     printf("pz = %p (%ld)\n", pz+1, sizeof(double));
00022     (*px)++;
00023     printf("x = %d\n", x);
00024     printf("px = %p\n", px);
00025     (*py)++;
00026     printf("x = %d\n", x);
00027     printf("py = %p\n", py);
00028     py++;
00029     (*py)++;
00030     printf("x = %d\n", x);
00031     printf("py = %p\n", py);
00032     (*py)++;
00033     printf("x = %d\n", x);
00034     printf("py = %p\n", py);
00035     py++;
00036     (*py)++;
00037     printf("x = %d\n", x);
00038     printf("py = %p\n", py);
00039     return 0;
00040 }

```

7.225 Ejemplo036.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int x, *px;
00006     char *py;
00007     double *pz;
00008     x = 20;
00009     px = &x;
00010     py = (char*)&x;
00011     pz = (double*)&x;
00012     printf("x = %d\n", x);
00013     printf("&x = %p\n", &x);
00014     printf("px = %p\n", px);
00015     printf("*px = %d\n", *px);
00016     printf("px = %p\n", px);
00017     printf("py = %p\n", py);
00018     printf("pz = %p\n", pz);
00019     printf("px = %p (%ld)\n", px+1, sizeof(int));
00020     printf("py = %p (%ld)\n", py+1, sizeof(char));
00021     printf("pz = %p (%ld)\n", pz+1, sizeof(double));
00022     (*px)++;
00023     printf("x = %d\n", x);
00024     printf("px = %p\n", px);
00025     (*py)++;
00026     printf("x = %d\n", x);
00027     printf("py = %p\n", py);
00028     py++;
00029     (*py)++;
00030     printf("x = %d\n", x);
00031     printf("py = %p\n", py);
00032     (*py)++;
00033     printf("x = %d\n", x);
00034     printf("py = %p\n", py);
00035     py++;
00036     (*py)++;
00037     printf("x = %d\n", x);
00038     printf("py = %p\n", py);
00039     return 0;
00040 }

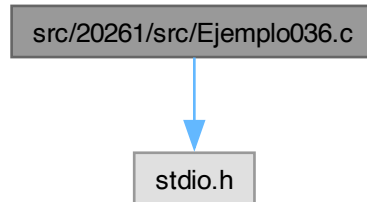
```

7.226 src/20261/src/Ejemplo036.c File Reference

Comparación de implementaciones de potencia, factorial y aproximación de exp(x).

```
#include <stdio.h>
```

Include dependency graph for Ejemplo036.c:



Functions

- float `exp1` (float x, int n)
Aproxima exp(x) con n términos de la serie de Taylor (k=0..n-1).
- float `exp3` (float x, int n)
Aproxima exp(x) sumando términos desde 0 hasta n (incluye el término n).
- float `exp4` (float x, int n)
Aproxima exp(x) sumando términos desde 0 hasta n (incluye el término n).
- float `pot` (float x, int n)
Calcula x^n de forma iterativa.
- float `pot2` (float x, int n)
Calcula x^n de forma recursiva.
- float `facto` (int n)
Calcula n! de forma iterativa.
- float `fct` (float x, int n)
Calcula el término $\frac{x^n}{n!}$ de forma recursiva.
- int `main` (int argc, char *argv[])
Punto de entrada. Lee x y n, calcula y muestra potencia, factorial y aproximaciones de exp(x).

7.226.1 Detailed Description

Comparación de implementaciones de potencia, factorial y aproximación de exp(x).

Este programa ilustra distintas formas de calcular:

- Potencia: x^n
 - `pot()` : iterativa (for)
 - `pot2()` : recursiva
- Factorial: $n!$
 - `facto()` : iterativa
- Aproximación de exp(x) con serie de Taylor (Maclaurin):

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$$

- `exp1(x, n)` : suma n términos (k=0..n-1) usando potencia iterativa y factorial iterativo
- `exp3(x, n)` : suma n+1 términos (k=0..n) de forma recursiva usando `pot2()` y `facto()`
- `exp4(x, n)` : suma n+1 términos (k=0..n) recursiva usando `fct()` que genera $x^n/n!$

Entrada estándar

- `x` (float): valor real
- `n` (int): número de términos / exponente ($n \geq 0$)

Salida estándar

- x^n calculado por dos métodos
- `n!` (factorial)
- `exp(x)` aproximado por tres métodos

Complejidad

- `pot`: $O(n)$, `pot2`: $O(n)$
- `facto`: $O(n)$
- `exp1`: $O(n^2)$ (por potencia+factorial dentro del ciclo)
- `exp3`: $O(n^2)$ (recursión + llamadas repetidas a `pot2/facto`)
- `exp4`: $O(n^2)$ (recursión + `fct` recursivo)

Warning

- Las funciones recursivas pueden desbordar pila si `n` es muy grande.
- Para aproximar `exp(x)` con buena precisión se requieren más términos, especialmente si $|x|$ es grande.

```
gcc Ejemplo036.c -o Ejemplo036
./Ejemplo036
```

Definition in file [Ejemplo036.c](#).

7.226.2 Function Documentation**7.226.2.1 `exp1()`**

```
float exp1 (
    float x,
    int n)
```

Aproxima `exp(x)` con `n` términos de la serie de Taylor ($k=0..n-1$).
Implementa:

$$e^x \approx \sum_{k=0}^{n-1} \frac{x^k}{k!}$$

Parameters

<code>x</code>	Valor real.
<code>n</code>	Número de términos ($n \geq 0$). Si $n=0$, la suma queda 0.

Returns

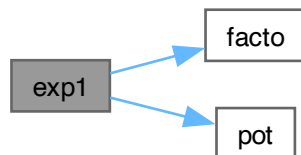
Aproximación de `exp(x)`.

Definition at line 174 of file [Ejemplo036.c](#).

```
00175 {
00176     float ex;
00177     int i;
00178
00179     for(i = 0, ex = 0.0f; i < n; i++)
00180         ex += pot(x, i) / facto(i);
00181
00182     return ex;
```

```
00183 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.226.2.2 exp3()

```
float exp3 (
    float x,
    int n)
```

Aproxima $\exp(x)$ sumando términos desde 0 hasta n (incluye el término n).

Implementa:

$$e^x \approx \sum_{k=0}^n \frac{x^k}{k!}$$

usando recursión:

$$\exp3(x, n) = \exp3(x, n-1) + \frac{x^n}{n!}, \quad \exp3(x, 0) = 1$$

Parameters

x	Valor real.
n	Entero $n \geq 0$.

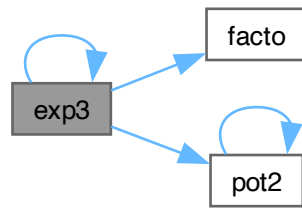
Returns

Aproximación de $\exp(x)$.

Definition at line 156 of file [Ejemplo036.c](#).

```
00157 {
00158     return n ? exp3(x, n-1) + pot2(x, n) / facto(n) : 1.0f;
00159 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.226.2.3 exp4()

```
float exp4 (
    float x,
    int n)
```

Aproxima exp(x) sumando términos desde 0 hasta n (incluye el término n).

Implementa:

$$e^x \approx \sum_{k=0}^n \frac{x^k}{k!}$$

usando recursión:

$$\exp4(x, n) = \exp4(x, n-1) + \text{fct}(x, n), \quad \exp4(x, 0) = 1$$

Parameters

<i>x</i>	Valor real.
<i>n</i>	Entero n>=0.

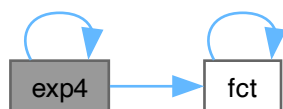
Returns

Aproximación de exp(x).

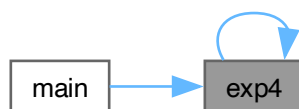
Definition at line 134 of file [Ejemplo036.c](#).

```
00135 {
00136     return n ? exp4(x, n-1) + fct(x, n) : 1.0f;
00137 }
```


Here is the call graph for this function:



Here is the caller graph for this function:



7.226.2.4 `facto()`

```
float facto (  
    int n)
```

Calcula $n!$ de forma iterativa.

Parameters

n	Entero $n \geq 0$.
-----	---------------------

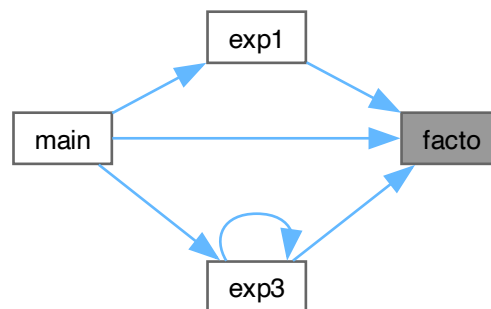
Returns

$n!$ como float.

Definition at line 221 of file [Ejemplo036.c](#).

```
00222 {
00223     float fn;
00224     int i;
00225
00226     for(i = 0, fn = 1.0f; i < n; i++)
00227         fn *= (i + 1);
00228
00229     return fn;
00230 }
```

Here is the caller graph for this function:

**7.226.2.5 fct()**

```
float fct (
    float x,
    int n)
```

Calcula el término $\frac{x^n}{n!}$ de forma recursiva.

Se usa la recurrencia:

$$fct(x, n) = fct(x, n - 1) \cdot \frac{x}{n}, \quad fct(x, 0) = 1$$

Parameters

x	Valor real.
n	Entero $n \geq 0$.

Returns

Valor de $x^n/n!$.

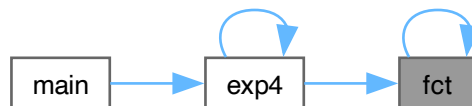
Definition at line 112 of file [Ejemplo036.c](#).

```
00113 {
00114     return n ? fct(x, n-1) * (x / n) : 1.0f;
00115 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.226.2.6 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Lee x y n, calcula y muestra potencia, factorial y aproximaciones de exp(x).

Definition at line 60 of file [Ejemplo036.c](#).

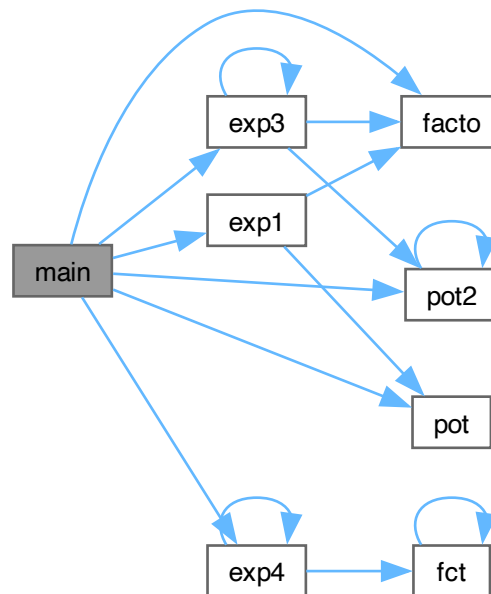
```
00061 {
00062     (void)argc;
00063     (void)argv;
00064
00065     float x, xn, xn2, fn, ex, ex2, ex3;
00066     int n;
00067
00068     printf("Ingrese el valor de x: ");
00069     scanf("%f", &x);
00070
00071     /* n debe ser >= 0 para potencia, factorial y serie */
00072     do {
00073         printf("Ingrese el valor de n: ");
00074         scanf("%d", &n);
00075     } while (n < 0);
00076
00077     /* x^n por dos métodos */
00078     xn = pot(x, n); /* iterativo */
00079     xn2 = pot2(x, n); /* recursivo */
00080
00081     /* n! */
00082     fn = facto(n);
00083
00084     /* exp(x) por tres métodos */
00085     ex = exp1(x, n); /* suma k=0..n-1 (n términos) */
```

```

00086     ex2 = exp3(x, n); /* suma k=0..n   (n+1 términos) */
00087     ex3 = exp4(x, n); /* suma k=0..n   (n+1 términos) */
00088
00089     printf("%f^%d = %f\n", x, n, xn);
00090     printf("%f^%d = %f\n", x, n, xn2);
00091     printf("%d! = %f\n", n, fn);
00092     printf("exp(%f) = %f\n", x, ex);
00093     printf("exp(%f) = %f\n", x, ex2);
00094     printf("exp(%f) = %f\n", x, ex3);
00095
00096     return 0;
00097 }

```

Here is the call graph for this function:



7.226.2.7 pot()

```

float pot (
    float x,
    int n)

```

Calcula x^n de forma iterativa.

Parameters

x	Base.
n	Exponente entero $n \geq 0$.

Returns

x^n .

Definition at line 204 of file [Ejemplo036.c](#).

```

00205 {
00206     float px;
00207     int i;

```

```

00208
00209     for (i = 0, px = 1.0f; i < n; i++)
00210         px *= x;
00211
00212     return px;
00213 }

```

Here is the caller graph for this function:



7.226.2.8 pot2()

```

float pot2 (
    float x,
    int n)

```

Calcula x^n de forma recursiva.

Parameters

x	Base.
n	Exponente entero $n \geq 0$.

Returns

x^n .

Definition at line 192 of file [Ejemplo036.c](#).

```

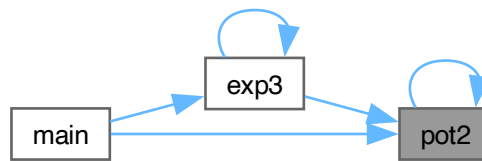
00193 {
00194     return (n ? pot2(x, n-1) * x : 1.0f);
00195 }

```

Here is the call graph for this function:



Here is the caller graph for this function:



7.227 Ejemplo036.c

[Go to the documentation of this file.](#)

```

00001
00045
00046 #include <stdio.h>
00047
00048 /* Prototipos */
00049 float exp1(float x, int n);
00050 float exp3(float x, int n);
00051 float exp4(float x, int n);
00052 float pot(float x, int n);
00053 float pot2(float x, int n);
00054 float facto(int n);
00055 float fct(float x, int n);
00056
00060 int main(int argc, char* argv[])
00061 {
00062     (void)argc;
00063     (void)argv;
00064
00065     float x, xn, xn2, fn, ex, ex2, ex3;
00066     int n;
00067
00068     printf("Ingrese el valor de x: ");
00069     scanf("%f", &x);
00070
00071     /* n debe ser >= 0 para potencia, factorial y serie */
00072     do {
00073         printf("Ingrese el valor de n: ");
00074         scanf("%d", &n);
00075     } while (n < 0);
00076
00077     /* x^n por dos métodos */
00078     xn = pot(x, n); /* iterativo */
00079     xn2 = pot2(x, n); /* recursivo */
00080
00081     /* n! */
00082     fn = facto(n);
00083
00084     /* exp(x) por tres métodos */
00085     ex = exp1(x, n); /* suma k=0..n-1 (n términos) */
00086     ex2 = exp3(x, n); /* suma k=0..n (n+1 términos) */
00087     ex3 = exp4(x, n); /* suma k=0..n (n+1 términos) */
00088
00089     printf("%f^%d = %f\n", x, n, xn);
00090     printf("%f^%d = %f\n", x, n, xn2);
00091     printf("%d! = %f\n", n, fn);
00092     printf("exp(%f) = %f\n", x, ex);
00093     printf("exp(%f) = %f\n", x, ex2);
00094     printf("exp(%f) = %f\n", x, ex3);
00095
00096     return 0;
00097 }
00098
00112 float fct(float x, int n)
00113 {
00114     return n ? fct(x, n-1) * (x / n) : 1.0f;
00115 }
00116
00134 float exp4(float x, int n)
00135 {

```

```

00136     return n ? exp4(x, n-1) + fct(x, n) : 1.0f;
00137 }
00138
00156 float exp3(float x, int n)
00157 {
00158     return n ? exp3(x, n-1) + pot2(x, n) / facto(n) : 1.0f;
00159 }
00160
00174 float expl(float x, int n)
00175 {
00176     float ex;
00177     int i;
00178
00179     for(i = 0, ex = 0.0f; i < n; i++)
00180         ex += pot(x, i) / facto(i);
00181
00182     return ex;
00183 }
00184
00192 float pot2(float x, int n)
00193 {
00194     return (n ? pot2(x, n-1) * x : 1.0f);
00195 }
00196
00204 float pot(float x, int n)
00205 {
00206     float px;
00207     int i;
00208
00209     for(i = 0, px = 1.0f; i < n; i++)
00210         px *= x;
00211
00212     return px;
00213 }
00214
00221 float facto(int n)
00222 {
00223     float fn;
00224     int i;
00225
00226     for(i = 0, fn = 1.0f; i < n; i++)
00227         fn *= (i + 1);
00228
00229     return fn;
00230 }

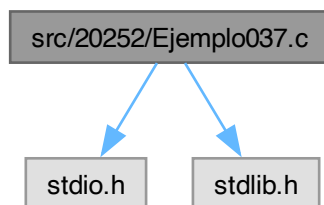
```

7.228 src/20252/Ejemplo037.c File Reference

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo037.c:



Functions

- int `main` (int argc, char *argv[])

7.228.1 Function Documentation

7.228.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 4 of file [Ejemplo037.c](#).

```
00005 {
00006     double *x, m;
00007     int n, i;
00008     do{
00009         printf("Ingrese el numero de elementos: ");
00010         scanf("%d", &n);
00011     }while(n<1);
00012     x = (double*)malloc(n*sizeof(double));
00013     if(x==NULL)
00014         return 1;
00015     for(i=0, m=0; i<n; i++)
00016     {
00017         printf("x[%d] = ", i+1);
00018         scanf("%lf", x+i);
00019         m += x[i];
00020     }
00021     m/=n;
00022     printf("Media = %lf\n", m);
00023     free(x);
00024     return 0;
00025 }
```

7.229 Ejemplo037.c

[Go to the documentation of this file.](#)

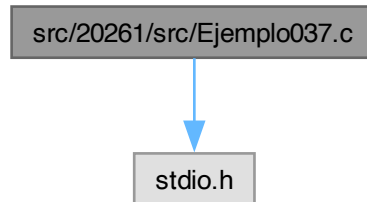
```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 int main(int argc, char *argv[])
00005 {
00006     double *x, m;
00007     int n, i;
00008     do{
00009         printf("Ingrese el numero de elementos: ");
00010         scanf("%d", &n);
00011     }while(n<1);
00012     x = (double*)malloc(n*sizeof(double));
00013     if(x==NULL)
00014         return 1;
00015     for(i=0, m=0; i<n; i++)
00016     {
00017         printf("x[%d] = ", i+1);
00018         scanf("%lf", x+i);
00019         m += x[i];
00020     }
00021     m/=n;
00022     printf("Media = %lf\n", m);
00023     free(x);
00024     return 0;
00025 }
```

7.230 src/20261/src/Ejemplo037.c File Reference

Calcula la suma de OddD(k) en un intervalo [A,B] usando recursión.


```
#include <stdio.h>
```

Include dependency graph for Ejemplo037.c:



Functions

- int `OddD` (int X)
Devuelve el "componente impar" de X.
- int `SoddD` (int A, int B)
Suma `OddD(k)` para k en $[A,B]$ (intervalo inclusivo).
- int `main` (int argc, char *argv[])

7.230.1 Detailed Description

Calcula la suma de `OddD(k)` en un intervalo $[A,B]$ usando recursión.

El programa lee un entero n (número de casos) y, por cada caso, lee dos enteros A y B . Luego imprime: Suma = `SoddD(A,B)`

Donde:

- `OddD (X)` devuelve el "componente impar" de X : divide X entre 2 repetidamente hasta que sea impar.
Ejemplo: `OddD(12)=3`, `OddD(40)=5`, `OddD(8)=1`.
- `SoddD (A, B)` suma `OddD(k)` para k desde A hasta B (intervalo inclusivo).

Entrada estándar

```

n
A1 B1
A2 B2
...
An Bn
  
```

Salida estándar

Por cada caso:

```
Suma = <valor>
```

Precondiciones

- Se asume A y B enteros. Si $A > B$, se intercambian para evitar recursión infinita.
- `OddD (X)` está definido en este ejemplo solo para $X > 0$ (para $X \leq 0$ devuelve 0).

Complejidad

- `OddD (X)` : $O(\log_2(X))$ en promedio (divisiones entre 2).
- `SoddD (A, B)` : $O(|B-A|)$ llamadas recursivas (puede desbordar la pila si el rango es grande).

Warning

Para rangos muy grandes, esta versión recursiva puede provocar stack overflow.

```
gcc Ejemplo037.c -o Ejemplo037
./Ejemplo037
```

Definition in file [Ejemplo037.c](#).

7.230.2 Function Documentation**7.230.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 53 of file [Ejemplo037.c](#).

```
00054 {
00055     (void)argc;
00056     (void)argv;
00057
00058     int n, A, B, i, S;
00059
00060     /* Lee el número de casos */
00061     scanf("%d", &n);
00062
00063     for (i = 0; i < n; i++)
00064     {
00065         /* Lee el intervalo [A,B] */
00066         scanf("%d", &A);
00067         scanf("%d", &B);
00068
00069         /* Calcula suma de OddD en el intervalo */
00070         S = SoddD(A, B);
00071
00072         printf("Suma = %d\n", S);
00073     }
00074
00075     return 0;
00076 }
```

Here is the call graph for this function:

**7.230.2.2 OddD()**

```
int OddD (
    int X)
```

Devuelve el “componente impar” de X.

- Si $X \leq 0$: devuelve 0 (convención de este ejemplo).
- Si X es potencia de 2: devuelve 1 rápidamente.
- Si X es impar: devuelve X.
- Si X es par: devuelve $\text{OddD}(X/2)$.

Parameters

X	Entero.
---	---------

Returns

Componente impar de X (o 0 si $X \leq 0$).

Definition at line 117 of file [Ejemplo037.c](#).

```

00118 {
00119     /* Evita recursión infinita y define comportamiento para no-positivos */
00120     if (X <= 0) return 0;
00121
00122     /* Si X es potencia de 2 (1,2,4,8,...) su componente impar es 1 */
00123     if ((X & (X - 1)) == 0)
00124         return 1;
00125
00126     /* Si es impar, termina; si es par, divide entre 2 y continúa */
00127     return (X % 2) ? X : OddD(X / 2);
00128 }

```

Here is the call graph for this function:



Here is the caller graph for this function:

**7.230.2.3 SoddD()**

```

int SoddD (
    int A,
    int B)

```

Suma OddD(k) para k en [A,B] (intervalo inclusivo).

Implementación recursiva: $SoddD(A,B) = OddD(A) + SoddD(A+1,B)$, si $A < B$ $SoddD(A,B) = OddD(A)$, si $A == B$

Se agrega una corrección mínima:

- si $A > B$, se intercambian para evitar recursión infinita.

Parameters

<i>A</i>	Límite inferior (puede venir mayor que B, se corrige).
<i>B</i>	Límite superior.

Returns

Suma de OddD(k) para k en [A,B].

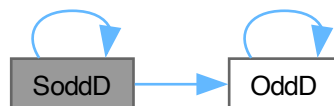
Definition at line 93 of file [Ejemplo037.c](#).

```

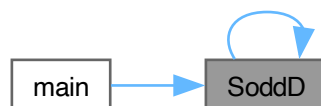
00094 {
00095     /* Corrección mínima: si el usuario da A>B, se intercambia */
00096     if (A > B) {
00097         int tmp = A;
00098         A = B;
00099         B = tmp;
00100     }
00101
00102     return OddD(A) + (A == B ? 0 : SoddD(A + 1, B));
00103 }

```

Here is the call graph for this function:



Here is the caller graph for this function:



7.231 Ejemplo037.c

[Go to the documentation of this file.](#)

```

00001
00047
00048 #include <stdio.h>
00049
00050 int OddD(int X);
00051 int SoddD(int A, int B);
00052
00053 int main(int argc, char *argv[])
00054 {
00055     (void)argc;
00056     (void)argv;
00057
00058     int n, A, B, i, S;
00059

```

```

00060      /* Lee el número de casos */
00061      scanf("%d", &n);
00062
00063      for (i = 0; i < n; i++)
00064      {
00065          /* Lee el intervalo [A,B] */
00066          scanf("%d", &A);
00067          scanf("%d", &B);
00068
00069          /* Calcula suma de OddD en el intervalo */
00070          S = SoddD(A, B);
00071
00072          printf("Suma = %d\n", S);
00073      }
00074
00075      return 0;
00076 }
00077
00093 int SoddD(int A, int B)
00094 {
00095     /* Corrección mínima: si el usuario da A>B, se intercambia */
00096     if (A > B) {
00097         int tmp = A;
00098         A = B;
00099         B = tmp;
00100     }
00101
00102     return OddD(A) + (A == B ? 0 : SoddD(A + 1, B));
00103 }
00104
00117 int OddD(int X)
00118 {
00119     /* Evita recursión infinita y define comportamiento para no-positivos */
00120     if (X <= 0) return 0;
00121
00122     /* Si X es potencia de 2 (1,2,4,8,...) su componente impar es 1 */
00123     if ((X & (X - 1)) == 0)
00124         return 1;
00125
00126     /* Si es impar, termina; si es par, divide entre 2 y continúa */
00127     return (X % 2) ? X : OddD(X / 2);
00128 }

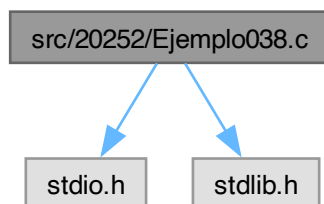
```

7.232 src/20252/Ejemplo038.c File Reference

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo038.c:



Functions

- int [main](#) (int argc, char *argv[])

7.232.1 Function Documentation

7.232.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 4 of file [Ejemplo038.c](#).

```
00005 {
00006     int r, c, i, j;
00007     double *A;
00008     do{
00009         printf("Ingrese el numero de renglones: ");
00010         scanf("%d", &r);
00011     }while(r<1);
00012     do{
00013         printf("Ingrese el numero de columnas: ");
00014         scanf("%d", &c);
00015     }while(c<1);
00016     A = (double*)malloc(r*c*sizeof(double));
00017     if(A==NULL)
00018         return 1;
00019     for(i=0; i<r; i++)
00020         for(j=0; j<c; j++)
00021         {
00022             printf("A[%d][%d] = ", i+1, j+1);
00023             scanf("%lf", &A[c*i+j]);
00024             scanf("%lf", &*(A+c*i+j));
00025             scanf("%lf", A+c*i+j);
00026         }
00027     for(i=0; i<r; i++)
00028     {
00029         for(j=0; j<c; j++)
00030             printf("%.4lf\t", A[c*i+j]);
00031         printf("\n");
00032     }
00033     free(A);
00034     return 0;
00035 }
```

7.233 Ejemplo038.c

[Go to the documentation of this file.](#)

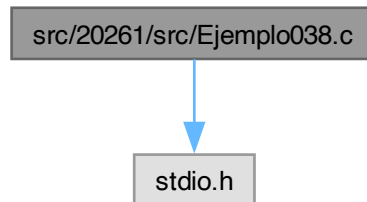
```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 int main(int argc, char *argv[])
00005 {
00006     int r, c, i, j;
00007     double *A;
00008     do{
00009         printf("Ingrese el numero de renglones: ");
00010         scanf("%d", &r);
00011     }while(r<1);
00012     do{
00013         printf("Ingrese el numero de columnas: ");
00014         scanf("%d", &c);
00015     }while(c<1);
00016     A = (double*)malloc(r*c*sizeof(double));
00017     if(A==NULL)
00018         return 1;
00019     for(i=0; i<r; i++)
00020         for(j=0; j<c; j++)
00021         {
00022             printf("A[%d][%d] = ", i+1, j+1);
00023             scanf("%lf", &A[c*i+j]);
00024             scanf("%lf", &*(A+c*i+j));
00025             scanf("%lf", A+c*i+j);
00026         }
00027     for(i=0; i<r; i++)
00028     {
00029         for(j=0; j<c; j++)
00030             printf("%.4lf\t", A[c*i+j]);
00031         printf("\n");
00032     }
00033     free(A);
00034     return 0;
00035 }
```

7.234 src/20261/src/Ejemplo038.c File Reference

Imprime la cantidad de argumentos y lista argv[i].

```
#include <stdio.h>
```

Include dependency graph for Ejemplo038.c:



Functions

- int [main](#) (int argc, char *argv[])

7.234.1 Detailed Description

Imprime la cantidad de argumentos y lista argv[i].

Programa demostrativo del uso de parámetros de línea de comandos:

- `argc`: número de argumentos (incluye el nombre del programa).
- `argv`: arreglo de cadenas con cada argumento.

Entrada

Argumentos en la línea de comandos.

Salida

Imprime `argc` y luego cada `argv[i]`.

```
gcc Ejemplo038.c -o Ejemplo038
./Ejemplo038 uno dos tres
```

Definition in file [Ejemplo038.c](#).

7.234.2 Function Documentation

7.234.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 24 of file [Ejemplo038.c](#).

```
00025 {
00026     int i;
00027
00028     printf("# argumentos: %d\n", argc);
00029
00030     for (i = 0; i < argc; i++)
00031         printf("%d. %s\n", i, argv[i]);
00032
00033     return 0;
00034 }
```

7.235 Ejemplo038.c

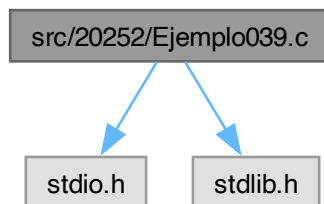
[Go to the documentation of this file.](#)

```
00001
00021
00022 #include <stdio.h>
00023
00024 int main(int argc, char *argv[])
00025 {
00026     int i;
00027
00028     printf("# argumentos: %d\n", argc);
00029
00030     for (i = 0; i < argc; i++)
00031         printf("%d. %s\n", i, argv[i]);
00032
00033     return 0;
00034 }
```

7.236 src/20252/Ejemplo039.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for Ejemplo039.c:



Functions

- double * [crearMat](#) (int r, int c)
- int [capturar](#) (double *M, int r, int c, char *str)
- int [imprimir](#) (double *M, int r, int c)
- double * [multiplicar](#) (double *M1, int r1, int c1, double *M2, int r2, int c2)
- int [main](#) (int argc, char *argv[])

7.236.1 Function Documentation

7.236.1.1 capturar()

```
int capturar (
    double * M,
    int r,
    int c,
    char * str)
```

Definition at line 13 of file [Ejemplo039.c](#).

```
00014 {
00015     int i, j;
00016     for(i=0; i<r; i++)
00017         for(j=0; j<c; j++)
00018     {
```

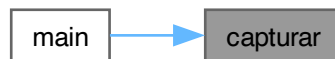


```

00019         printf("%s[%d][%d] = ", str, i+1, j+1);
00020         scanf("%lf", M+c*i+j);
00021     }
00022     return 0;
00023 }

```

Here is the caller graph for this function:



7.236.1.2 crearMat()

```

double * crearMat (
    int r,
    int c)

```

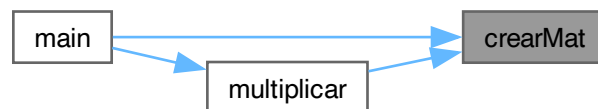
Definition at line 4 of file [Ejemplo039.c](#).

```

00005 {
00006     double *M;
00007     M = (double*)malloc(r*c*sizeof(double));
00008     if(M==NULL)
00009         return NULL;
00010     return M;
00011 }

```

Here is the caller graph for this function:



7.236.1.3 imprimir()

```

int imprimir (
    double * M,
    int r,
    int c)

```

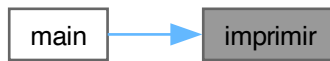
Definition at line 25 of file [Ejemplo039.c](#).

```

00026 {
00027     int i, j;
00028     for(i=0; i<r; i++)
00029     {
00030         for(j=0; j<c; j++)
00031             printf("%.4lf\t", M[c*i+j]);
00032         printf("\n");
00033     }
00034     return 0;
00035 }

```

Here is the caller graph for this function:



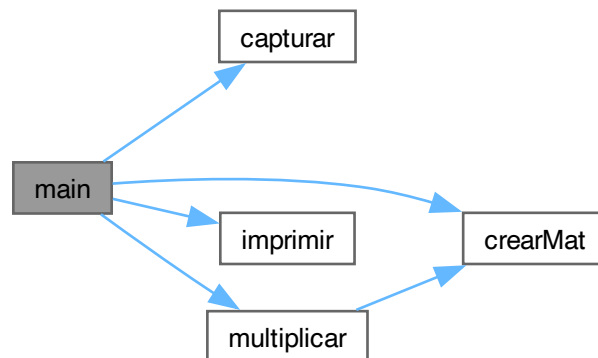
7.236.1.4 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 59 of file [Ejemplo039.c](#).

```
00060 {
00061     int rA, rB, cA, cB;
00062     double *A, *B, *C;
00063     do{
00064         printf("Ingrese el numero de renglones de A: ");
00065         scanf("%d", &rA);
00066     }while(rA<1);
00067     do{
00068         printf("Ingrese el numero de columnas de A: ");
00069         scanf("%d", &cA);
00070     }while(cA<1);
00071     A = crearMat(rA, cA);
00072     if(A==NULL)
00073         return 1;
00074     capturar(A, rA, cA, "A");
00075     do{
00076         printf("Ingrese el numero de renglones de B: ");
00077         scanf("%d", &rB);
00078     }while(rB<1);
00079     do{
00080         printf("Ingrese el numero de columnas de B: ");
00081         scanf("%d", &cB);
00082     }while(cB<1);
00083     B = crearMat(rB, cB);
00084     if(B==NULL)
00085     {
00086         free(A);
00087         return 2;
00088     }
00089     capturar(B, rB, cB, "B");
00090     C = multiplicar(A, rA, cA, B, rB, cB);
00091     if(C==NULL)
00092     {
00093         free(A);
00094         free(B);
00095         return 4;
00096     }
00097     printf("A\n");
00098     imprimir(A, rA, cA);
00099     printf("B\n");
00100     imprimir(B, rB, cB);
00101     printf("C\n");
00102     imprimir(C, rA, cB);
00103     free(A);
00104     free(B);
00105     free(C);
00106     return 0;
00107 }
```

Here is the call graph for this function:



7.236.1.5 multiplicar()

```
double * multiplicar (
    double * M1,
    int r1,
    int c1,
    double * M2,
    int r2,
    int c2)
```

Definition at line 37 of file [Ejemplo039.c](#).

```
00038 {
00039     int i, j, k;
00040     double *M3;
00041     if(c1!=r2)
00042     {
00043         printf("Error: En el tamaño de las matrices.\n");
00044         return NULL;
00045     }
00046     M3 = crearMat(r1, c2);
00047     if(M3==NULL)
00048         return NULL;
00049     for(i=0; i<r1; i++)
00050         for(j=0; j<c2; j++)
00051             for(k=0; M3[i*c1+j]=0; k<c1; k++)
00052                 M3[i*c2+j] += M1[i*c1+k]*M2[k*c2+j];
00053     // M3[i][j] = M3[i][j]+M1[i][k]*M2[k][j]
00054     // M3[i][j] += M1[i][k]*M2[k][j]
00055     // M3[i*c2+j] += M1[i*c1+k]*M2[k*r2+j]
00056     return M3;
00057 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.237 Ejemplo039.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 double *crearMat(int r, int c)
00005 {
00006     double *M;
00007     M = (double*)malloc(r*c*sizeof(double));
00008     if(M==NULL)
00009         return NULL;
00010     return M;
00011 }
00012
00013 int capturar(double *M, int r, int c, char *str)
00014 {
00015     int i, j;
00016     for(i=0; i<r; i++)
00017         for(j=0; j<c; j++)
00018         {
00019             printf("%s[%d][%d] = ", str, i+1, j+1);
00020             scanf("%lf", M+c*i+j);
00021         }
00022     return 0;
00023 }
00024
00025 int imprimir(double *M, int r, int c)
00026 {
00027     int i, j;
00028     for(i=0; i<r; i++)
00029     {
00030         for(j=0; j<c; j++)
00031             printf("%.4lf\t", M[c*i+j]);
00032         printf("\n");
00033     }
00034     return 0;
00035 }
00036
00037 double *multiplicar(double *M1, int r1, int c1, double *M2, int r2, int c2)
00038 {
00039     int i, j, k;
00040     double *M3;
00041     if(c1!=r2)
00042     {
00043         printf("Error: En el tamaño de las matrices.\n");
00044         return NULL;
00045     }
00046     M3 = crearMat(r1, c2);
00047     if(M3==NULL)
00048         return NULL;
00049     for(i=0; i<r1; i++)
00050         for(j=0; j<c2; j++)
00051             for(k=0, M3[i*c1+j]=0; k<c1; k++)
00052                 M3[i*c2+j] += M1[i*c1+k]*M2[k*c2+j];
00053     // M3[i][j] = M3[i][j]+M1[i][k]*M2[k][j]
00054     // M3[i][j] += M1[i][k]*M2[k][j]
00055     // M3[i*c2+j] += M1[i*c1+k]*M2[k*r2+j]
00056     return M3;
00057 }
00058
00059 int main(int argc, char *argv[])
00060 {
00061     int rA, rB, cA, cB;
00062     double *A, *B, *C;
00063     do{

```

```

00064     printf("Ingrese el numero de renglones de A: ");
00065     scanf("%d", &rA);
00066 }while (rA<1);
00067 do{
00068     printf("Ingrese el numero de columnas de A: ");
00069     scanf("%d", &cA);
00070 }while (cA<1);
00071 A = crearMat (rA, cA);
00072 if (A==NULL)
00073     return 1;
00074 capturar (A, rA, cA, "A");
00075 do{
00076     printf("Ingrese el numero de renglones de B: ");
00077     scanf("%d", &rB);
00078 }while (rB<1);
00079 do{
00080     printf("Ingrese el numero de columnas de B: ");
00081     scanf("%d", &cB);
00082 }while (cB<1);
00083 B = crearMat (rB, cB);
00084 if (B==NULL)
00085 {
00086     free(A);
00087     return 2;
00088 }
00089 capturar (B, rB, cB, "B");
00090 C = multiplicar (A, rA, cA, B, rB, cB);
00091 if (C==NULL)
00092 {
00093     free(A);
00094     free(B);
00095     return 4;
00096 }
00097 printf("A\n");
00098 imprimir (A, rA, cA);
00099 printf("B\n");
00100 imprimir (B, rB, cB);
00101 printf("C\n");
00102 imprimir (C, rA, cB);
00103 free(A);
00104 free(B);
00105 free(C);
00106 return 0;
00107 }

```

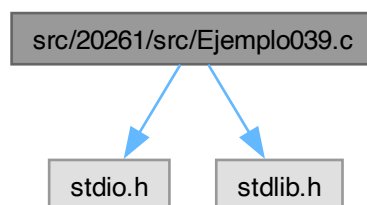
7.238 src/20261/src/Ejemplo039.c File Reference

Suma dos números reales recibidos por línea de comandos.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo039.c:



Functions

- int [main](#) (int argc, char *argv[])

7.238.1 Detailed Description

Suma dos números reales recibidos por línea de comandos.

- Requiere exactamente 2 argumentos numéricos: `argv[1] = A` `argv[2] = B`
- Convierte con `atof()` y calcula $C = A + B$.
- Imprime con formato: $A(+/-)B=C$

Entrada

```
./Ejemplo039 A B
```

Salida

```
1.00+2.00=3.00
```

Warning

- Si `argc != 3` el programa termina con código 1 sin imprimir mensaje (se mantiene así).
- `atof()` no valida errores de conversión; para robustez se recomienda `strtod()`.

```
gcc Ejemplo039.c -o Ejemplo039
./Ejemplo039 1.25 -3
```

Definition in file [Ejemplo039.c](#).

7.238.2 Function Documentation

7.238.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 36 of file [Ejemplo039.c](#).

```
00037 {
00038     float A, B, C;
00039
00040     /* Se esperan 2 argumentos numéricos => argc debe ser 3 (incluye nombre del programa) */
00041     if (argc != 3)
00042         return 1;
00043
00044     /* Convertir cadenas a float */
00045     A = (float)atof(argv[1]);
00046     B = (float)atof(argv[2]);
00047
00048     /* Sumar */
00049     C = A + B;
00050
00051     /* Mostrar: A(+/-)B=C */
00052     printf("%.2f%+.2f=%.2f\n", A, B, C);
00053
00054     return 0;
00055 }
```

7.239 Ejemplo039.c

[Go to the documentation of this file.](#)

```
00001
00002
00003 #include <stdio.h>
00004 #include <stdlib.h>
00005
00006 int main(int argc, char *argv[])
00007 {
00008     float A, B, C;
00009
00010     /* Se esperan 2 argumentos numéricos => argc debe ser 3 (incluye nombre del programa) */
00011     if (argc != 3)
00012         return 1;
```

```

00043
00044     /* Convertir cadenas a float */
00045     A = (float)atof(argv[1]);
00046     B = (float)atof(argv[2]);
00047
00048     /* Sumar */
00049     C = A + B;
00050
00051     /* Mostrar: A(+/-)B=C */
00052     printf("%.2f%+.2f=%+.2f\n", A, B, C);
00053
00054     return 0;
00055 }

```

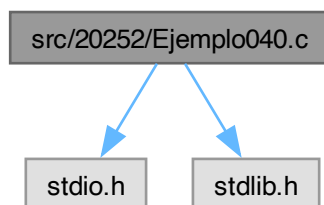
7.240 src/20252/Ejemplo040.c File Reference

```

#include <stdio.h>
#include <stdlib.h>

```

Include dependency graph for Ejemplo040.c:



Functions

- double * [crearMat](#) (int r, int c)
- double ** [crearMatC](#) (int r, int c)
- double ** [crearMatD](#) (int r, int c)
- int [capturar](#) (double *M, int r, int c, char *str)
- int [imprimir](#) (double *M, int r, int c)
- int [imprimirM](#) (double **M, int r, int c)
- double * [multiplicar](#) (double *M1, int r1, int c1, double *M2, int r2, int c2)
- double ** [multiplicarM](#) (double **M1, int r1, int c1, double **M2, int r2, int c2)
- int [capturarM](#) (double **M, int r, int c, char *str)
- int [liberar](#) (double **M, int r)
- int [main](#) (int argc, char *argv[])

7.240.1 Function Documentation

7.240.1.1 [capturar\(\)](#)

```

int capturar (
    double * M,
    int r,
    int c,
    char * str)

```

Definition at line [52](#) of file [Ejemplo040.c](#).

```

00053 {
00054     int i, j;

```

```

00055     for(i=0; i<r; i++)
00056         for(j=0; j<c; j++)
00057         {
00058             printf("%s[%d][%d] = ", str, i+1, j+1);
00059             scanf("%lf", M+c*i+j);
00060         }
00061     return 0;
00062 }

```

7.240.1.2 capturarM()

```

int capturarM (
    double ** M,
    int r,
    int c,
    char * str)

```

Definition at line 129 of file [Ejemplo040.c](#).

```

00130 {
00131     int i, j;
00132     for(i=0; i<r; i++)
00133         for(j=0; j<c; j++)
00134         {
00135             printf("%s[%d][%d] = ", str, i+1, j+1);
00136             scanf("%lf", M[i]+j);
00137         }
00138     return 0;
00139 }

```

Here is the caller graph for this function:



7.240.1.3 crearMat()

```

double * crearMat (
    int r,
    int c)

```

Definition at line 4 of file [Ejemplo040.c](#).

```

00005 {
00006     double *M;
00007     M = (double*) malloc(r*c*sizeof(double));
00008     if(M==NULL)
00009         return NULL;
00010     return M;
00011 }

```

Here is the caller graph for this function:



7.240.1.4 crearMatC()

```
double ** crearMatC (
    int r,
    int c)
```

Definition at line 13 of file [Ejemplo040.c](#).

```
00014 {
00015     double **M;
00016     int i;
00017     M = (double**)malloc(r*sizeof(double));
00018     if(M==NULL)
00019         return NULL;
00020     M[0] = (double*)malloc(r*c*sizeof(double));
00021     if(M[0]==NULL)
00022     {
00023         free(M);
00024         return NULL;
00025     }
00026     for(i=1; i<r; i++)
00027         M[i] = M[i-1]+c;
00028     return M;
00029 }
```

Here is the caller graph for this function:

**7.240.1.5 crearMatD()**

```
double ** crearMatD (
    int r,
    int c)
```

Definition at line 31 of file [Ejemplo040.c](#).

```
00032 {
00033     double **M;
00034     int i;
00035     M = (double**)malloc(r*sizeof(double));
00036     if(M==NULL)
00037         return NULL;
00038     for(i=0; i<r; i++)
00039     {
00040         M[i] = (double*)malloc(c*sizeof(double));
00041         if(M[i]==NULL)
00042         {
00043             for(--i; i>-1; i--)
00044                 free(M[i]);
00045             free(M);
00046             return NULL;
00047         }
00048     }
00049     return M;
00050 }
```

Here is the caller graph for this function:



7.240.1.6 imprimir()

```

int imprimir (
    double * M,
    int r,
    int c)
  
```

Definition at line 64 of file [Ejemplo040.c](#).

```

00065 {
00066     int i, j;
00067     for(i=0; i<r; i++)
00068     {
00069         for(j=0; j<c; j++)
00070             printf("%.4lf\t", M[c*i+j]);
00071         printf("\n");
00072     }
00073     return 0;
00074 }
  
```

7.240.1.7 imprimirM()

```

int imprimirM (
    double ** M,
    int r,
    int c)
  
```

Definition at line 76 of file [Ejemplo040.c](#).

```

00077 {
00078     int i, j;
00079     for(i=0; i<r; i++)
00080     {
00081         for(j=0; j<c; j++)
00082             printf("%.4lf\t", M[i][j]);
00083         printf("\n");
00084     }
00085     return 0;
00086 }
  
```

Here is the caller graph for this function:



7.240.1.8 liberar()

```

int liberar (
  
```

```
double ** M,
int r)
```

Definition at line 141 of file [Ejemplo040.c](#).

```
00142 {
00143     int i;
00144     for(i=0; i<r; i++)
00145         free(M[i]);
00146     free(M);
00147     return 0;
00148 }
```

Here is the caller graph for this function:



7.240.1.9 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 150 of file [Ejemplo040.c](#).

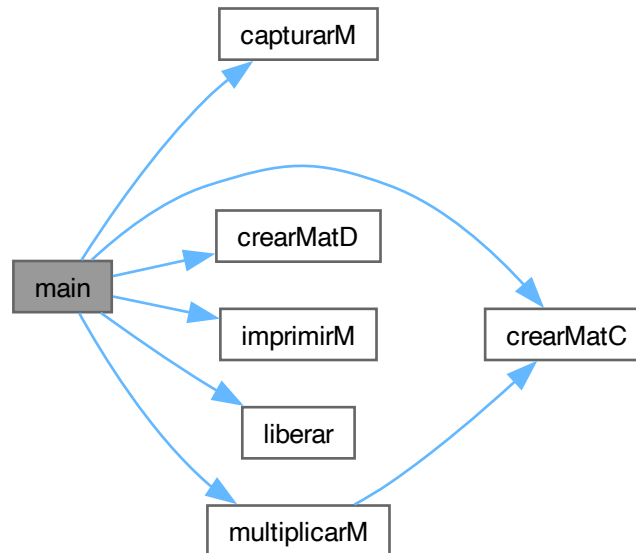
```
00151 {
00152     int rA, rB, cA, cB;
00153     double **A, **B, **C;
00154     do{
00155         printf("Ingrese el numero de renglones de A: ");
00156         scanf("%d", &rA);
00157     }while(rA<1);
00158     do{
00159         printf("Ingrese el numero de columnas de A: ");
00160         scanf("%d", &cA);
00161     }while(cA<1);
00162     A = crearMatC(rA, cA);
00163     if(A==NULL)
00164         return 1;
00165     capturarM(A, rA, cA, "A");
00166     do{
00167         printf("Ingrese el numero de renglones de B: ");
00168         scanf("%d", &rB);
00169     }while(rB<1);
00170     do{
00171         printf("Ingrese el numero de columnas de B: ");
00172         scanf("%d", &cB);
00173     }while(cB<1);
00174     B = crearMatD(rB, cB);
00175     if(B==NULL)
00176     {
00177         free(A[0]);
00178         free(A);
00179         return 2;
00180     }
00181     capturarM(B, rB, cB, "B");
00182     C = multiplicarM(A, rA, cA, B, rB, cB);
00183     if(C==NULL)
00184     {
00185         free(A[0]);
00186         free(A);
00187         liberar(B, rB);
00188         return 4;
00189     }
00190     printf("A\n");
00191     imprimirM(A, rA, cA);
00192     printf("B\n");
00193     imprimirM(B, rB, cB);
00194     printf("C\n");
00195     imprimirM(C, rA, cB);
00196     free(A[0]);
00197     free(A);
```

```

00198     liberar(B, rB);
00199     free(C);
00200     return 0;
00201 }

```

Here is the call graph for this function:



7.240.1.10 multiplicar()

```

double * multiplicar (
    double * M1,
    int r1,
    int c1,
    double * M2,
    int r2,
    int c2)

```

Definition at line 88 of file [Ejemplo040.c](#).

```

00089 {
00090     int i, j, k;
00091     double *M3;
00092     if(c1!=r2)
00093     {
00094         printf("Error: En el tamaño de las matrices.\n");
00095         return NULL;
00096     }
00097     M3 = crearMat(r1, c2);
00098     if(M3==NULL)
00099         return NULL;
00100     for(i=0; i<r1; i++)
00101         for(j=0; j<c2; j++)
00102             for(k=0; M3[i*c1+j]=0; k<c1; k++)
00103                 M3[i*c2+j] += M1[i*c1+k]*M2[k*c2+j];
00104     // M3[i][j] = M3[i][j]+M1[i][k]*M2[k][j]
00105     // M3[i][j] += M1[i][k]*M2[k][j]
00106     // M3[i*c2+j] += M1[i*c1+k]*M2[k*r2+j]
00107     return M3;
00108 }

```

Here is the call graph for this function:



7.240.1.11 multiplicarM()

```
double ** multiplicarM (  
    double ** M1,  
    int r1,  
    int c1,  
    double ** M2,  
    int r2,  
    int c2)
```

Definition at line 110 of file [Ejemplo040.c](#).

```
00111 {  
00112     int i, j, k;  
00113     double **M3;  
00114     if(c1!=r2)  
00115     {  
00116         printf("Error: En el tamaño de las matrices.\n");  
00117         return NULL;  
00118     }  
00119     M3 = crearMatC(r1, c2);  
00120     if(M3==NULL)  
00121         return NULL;  
00122     for(i=0; i<r1; i++)  
00123         for(j=0; j<c2; j++)  
00124             for(k=0, M3[i][j]=0; k<c1; k++)  
00125                 M3[i][j] += M1[i][k]*M2[k][j];  
00126     return M3;  
00127 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.241 Ejemplo040.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 double *creaMat(int r, int c)
00005 {
00006     double *M;
00007     M = (double*)malloc(r*c*sizeof(double));
00008     if(M==NULL)
00009         return NULL;
00010     return M;
00011 }
00012
00013 double **creaMatC(int r, int c)
00014 {
00015     double **M;
00016     int i;
00017     M = (double**)malloc(r*sizeof(double*));
00018     if(M==NULL)
00019         return NULL;
00020     M[0] = (double*)malloc(r*c*sizeof(double));
00021     if(M[0]==NULL)
00022     {
00023         free(M);
00024         return NULL;
00025     }
00026     for(i=1; i<r; i++)
00027         M[i] = M[i-1]+c;
00028     return M;
00029 }
00030
00031 double **creaMatD(int r, int c)
00032 {
00033     double **M;
00034     int i;
00035     M = (double**)malloc(r*sizeof(double*));
00036     if(M==NULL)
00037         return NULL;
00038     for(i=0; i<r; i++)
00039     {
00040         M[i] = (double*)malloc(c*sizeof(double));
00041         if(M[i]==NULL)
00042         {
00043             for(--i; i>-1; i--)
00044                 free(M[i]);
00045             free(M);
00046             return NULL;
00047         }
00048     }
00049     return M;
00050 }
00051
00052 int capturar(double *M, int r, int c, char *str)
00053 {
00054     int i, j;
00055     for(i=0; i<r; i++)
00056         for(j=0; j<c; j++)
00057         {
00058             printf("%s[%d][%d] = ", str, i+1, j+1);
00059             scanf("%lf", M+c*i+j);
00060         }
00061     return 0;
00062 }
00063
00064 int imprimir(double *M, int r, int c)
00065 {
00066     int i, j;
00067     for(i=0; i<r; i++)
00068     {
00069         for(j=0; j<c; j++)
00070             printf("%.4lf\t", M[c*i+j]);
00071         printf("\n");
00072     }
00073     return 0;
00074 }
00075
00076 int imprimirM(double **M, int r, int c)
00077 {
00078     int i, j;
00079     for(i=0; i<r; i++)
00080     {
00081         for(j=0; j<c; j++)
00082             printf("%.4lf\t", M[i][j]);
00083         printf("\n");

```

```

00084     }
00085     return 0;
00086 }
00087
00088 double *multiplicar(double *M1, int r1, int c1, double *M2, int r2, int c2)
00089 {
00090     int i, j, k;
00091     double *M3;
00092     if(c1!=r2)
00093     {
00094         printf("Error: En el tamaño de las matrices.\n");
00095         return NULL;
00096     }
00097     M3 = crearMat(r1, c2);
00098     if(M3==NULL)
00099         return NULL;
00100     for(i=0; i<r1; i++)
00101         for(j=0; j<c2; j++)
00102             for(k=0, M3[i*c1+j]=0; k<c1; k++)
00103                 M3[i*c2+j] += M1[i*c1+k]*M2[k*c2+j];
00104     // M3[i][j] = M3[i][j]+M1[i][k]*M2[k][j]
00105     // M3[i][j] += M1[i][k]*M2[k][j]
00106     // M3[i*c2+j] += M1[i*c1+k]*M2[k*r2+j]
00107     return M3;
00108 }
00109
00110 double **multiplicarM(double **M1, int r1, int c1, double **M2, int r2, int c2)
00111 {
00112     int i, j, k;
00113     double **M3;
00114     if(c1!=r2)
00115     {
00116         printf("Error: En el tamaño de las matrices.\n");
00117         return NULL;
00118     }
00119     M3 = crearMatC(r1, c2);
00120     if(M3==NULL)
00121         return NULL;
00122     for(i=0; i<r1; i++)
00123         for(j=0; j<c2; j++)
00124             for(k=0, M3[i][j]=0; k<c1; k++)
00125                 M3[i][j] += M1[i][k]*M2[k][j];
00126     return M3;
00127 }
00128
00129 int capturarM(double **M, int r, int c, char *str)
00130 {
00131     int i, j;
00132     for(i=0; i<r; i++)
00133         for(j=0; j<c; j++)
00134         {
00135             printf("%s[%d][%d] = ", str, i+1, j+1);
00136             scanf("%lf", &M[i+j]);
00137         }
00138     return 0;
00139 }
00140
00141 int liberar(double **M, int r)
00142 {
00143     int i;
00144     for(i=0; i<r; i++)
00145         free(M[i]);
00146     free(M);
00147     return 0;
00148 }
00149
00150 int main(int argc, char *argv[])
00151 {
00152     int rA, rB, cA, cB;
00153     double **A, **B, **C;
00154     do{
00155         printf("Ingrese el numero de renglones de A: ");
00156         scanf("%d", &rA);
00157     }while(rA<1);
00158     do{
00159         printf("Ingrese el numero de columnas de A: ");
00160         scanf("%d", &cA);
00161     }while(cA<1);
00162     A = crearMatC(rA, cA);
00163     if(A==NULL)
00164         return 1;
00165     capturarM(A, rA, cA, "A");
00166     do{
00167         printf("Ingrese el numero de renglones de B: ");
00168         scanf("%d", &rB);
00169     }while(rB<1);
00170     do{

```

```

00171         printf("Ingrese el numero de columnas de B: ");
00172         scanf("%d", &cB);
00173     }while(cB<1);
00174     B = crearMatD(rB, cB);
00175     if (B==NULL)
00176     {
00177         free(A[0]);
00178         free(A);
00179         return 2;
00180     }
00181     capturarM(B, rB, cB, "B");
00182     C = multiplicarM(A, rA, cA, B, rB, cB);
00183     if (C==NULL)
00184     {
00185         free(A[0]);
00186         free(A);
00187         liberar(B, rB);
00188         return 4;
00189     }
00190     printf("A\n");
00191     imprimirM(A, rA, cA);
00192     printf("B\n");
00193     imprimirM(B, rB, cB);
00194     printf("C\n");
00195     imprimirM(C, rA, cB);
00196     free(A[0]);
00197     free(A);
00198     liberar(B, rB);
00199     free(C);
00200     return 0;
00201 }

```

7.242 src/20261/src/Ejemplo040.c File Reference

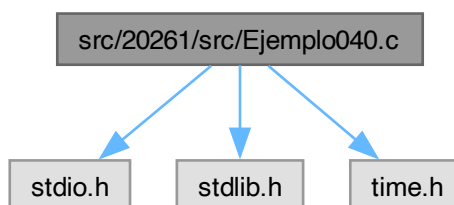
Genera n números aleatorios en [min,max] y calcula la media.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo040.c:



Macros

- #define N 1000000

Functions

- int main (int argc, char *argv[])

7.242.1 Detailed Description

Genera n números aleatorios en [min,max] y calcula la media.

Uso: ./Ejemplo040 n min max

Donde:

- n es el número de muestras ($1 \leq n \leq N$).
- `min` y `max` definen el intervalo; si vienen invertidos, el programa los intercambia.

El programa: 1) Genera n valores: $x[i] = ((\text{max}-\text{min}) * \text{rand}()) / \text{RAND_MAX} + \text{min}$ 2) Imprime cada valor. 3) Calcula e imprime la media.

Entrada

Argumentos por línea de comandos:

```
./Ejemplo040 n min max
```

Salida

- Imprime cada `X[i]`
- Imprime `Media = ...`

Complejidad

Tiempo: $O(n)$. Memoria: $O(N)$.

Note

Revisión mínima aplicada:

- `x[N]` se declara `static` para evitar problemas de stack por su tamaño (no cambia la salida).

```
gcc Ejemplo040.c -o Ejemplo040
./Ejemplo040 5 0 10
```

Definition in file [Ejemplo040.c](#).

7.242.2 Macro Definition Documentation

7.242.2.1 N

```
#define N 1000000
```

Definition at line 46 of file [Ejemplo040.c](#).

7.242.3 Function Documentation

7.242.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 48 of file [Ejemplo040.c](#).

```
00049 {
00050     /* Arreglo grande: se deja en memoria estática para evitar stack overflow */
00051     static float x[N];
00052
00053     float min, max, aux, media;
00054     int n, i;
00055
00056     /* Se esperan 3 argumentos: n min max => argc = 4 */
00057     if (argc != 4)
00058         return 1;
00059
00060     n = atoi(argv[1]);
00061     min = (float)atof(argv[2]);
00062     max = (float)atof(argv[3]);
00063
00064     if (n < 1 || n > N)
00065         return 2;
00066
00067     /* Asegurar min <= max */
00068     if (max < min)
00069     {
00070         aux = max;
00071         max = min;
00072         min = aux;
```

```

00073     }
00074
00075     srand((unsigned)time(NULL));
00076
00077     /* Generar valores y acumular media */
00078     for (i = 0, media = 0.0f; i < n; i++)
00079     {
00080         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00081         media += x[i];
00082         printf("X[%d] = %f\n", i + 1, x[i]);
00083     }
00084
00085     media /= n;
00086     printf("Media = %f\n", media);
00087
00088     return 0;
00089 }

```

Here is the call graph for this function:



7.243 Ejemplo040.c

[Go to the documentation of this file.](#)

```

00001
00041
00042 #include <stdio.h>
00043 #include <stdlib.h>
00044 #include <time.h>
00045
00046 #define N 1000000
00047
00048 int main(int argc, char *argv[])
00049 {
00050     /* Arreglo grande: se deja en memoria estática para evitar stack overflow */
00051     static float x[N];
00052
00053     float min, max, aux, media;
00054     int n, i;
00055
00056     /* Se esperan 3 argumentos: n min max => argc = 4 */
00057     if (argc != 4)
00058         return 1;
00059
00060     n = atoi(argv[1]);
00061     min = (float)atof(argv[2]);
00062     max = (float)atof(argv[3]);
00063
00064     if (n < 1 || n > N)
00065         return 2;
00066
00067     /* Asegurar min <= max */
00068     if (max < min)
00069     {
00070         aux = max;
00071         max = min;
00072         min = aux;
00073     }
00074
00075     srand((unsigned)time(NULL));
00076
00077     /* Generar valores y acumular media */
00078     for (i = 0, media = 0.0f; i < n; i++)
00079     {
00080         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00081         media += x[i];
00082         printf("X[%d] = %f\n", i + 1, x[i]);

```

```
00083     }
00084
00085     media /= n;
00086     printf("Media = %f\n", media);
00087
00088     return 0;
00089 }
```

7.244 src/template/Template.c File Reference

Functions

- int [main](#) (int argc, char *argv[])

7.244.1 Function Documentation

7.244.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 24 of file [Template.c](#).

```
00025 {
00026     // (Opcional) Evitar advertencias de compilación si no usamos argc/argv
00027     (void)argc;
00028     (void)argv;
00029
00030     // Termina el programa indicando "éxito".
00031     return 0;
00032 }
```

7.245 Template.c

[Go to the documentation of this file.](#)

```
00001
00023
00024 int main(int argc, char *argv[])
00025 {
00026     // (Opcional) Evitar advertencias de compilación si no usamos argc/argv
00027     (void)argc;
00028     (void)argv;
00029
00030     // Termina el programa indicando "éxito".
00031     return 0;
00032 }
```

